MACHINE LEARNING

AI · ML · DS / REVISION NOTES

COURSERA · DEEPLEARNING.AI & STANFORD ONLINE · ANDREW NG

Advanced Learning Algorithms

Machine Learning Specialization, Course 2 — neural networks, ML system design, decision trees

Volume 2 of 2 · Sections 48–63

16 SECTIONS

122 FIGURES

127 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 48 Rare Disease Classification Example 3
LECTURE 49 Trading Off Precision and Recall 14
LECTURE 50 Decision Tree 28
LECTURE 51 Decision Tree Learning 34
LECTURE 52 Entropy as a Measure of Impurity 49
LECTURE 53 Choosing a Split 58
LECTURE 54 Recursive Splitting 67
LECTURE 55 One Hot Encoding 74
LECTURE 56 Not Cat 79
LECTURE 57 Regression with Decision Trees 84
LECTURE 58 Tree Ensemble 93
LECTURE 59 Sampling with Replacement 97
LECTURE 60 Randomizing the Feature Choice 101
LECTURE 61 Boosted Trees Intuition 106
LECTURE 62 Decision Trees vs Neural Networks 112
LECTURE 63 Language 116

LECTURE 48

Rare Disease Classification Example

If you're working on a machine learning application where the ratio of positive to negative examples is very skewed, very far from 50-50, then it turns out that the usual error metrics like accuracy don't work that well. Let's start with an example. Let's say you're training a binary classifier to detect a rare disease in patients based on lab tests or based on other data from the patients. Y is equal to 1 if the disease is present and y is equal to 0 otherwise. Suppose you find that you've achieved one percent error on the test set, so you have a 99 percent correct diagnosis. This seems like a great outcome. But it turns out that if this is a rare disease, so y is equal to 1, very rarely, then this may not be as impressive as it sounds.

Specifically, if it is a rare disease and if only 0.5 percent of the patients in your population have the disease, then if instead you wrote the program, that just said, print y equals 0.

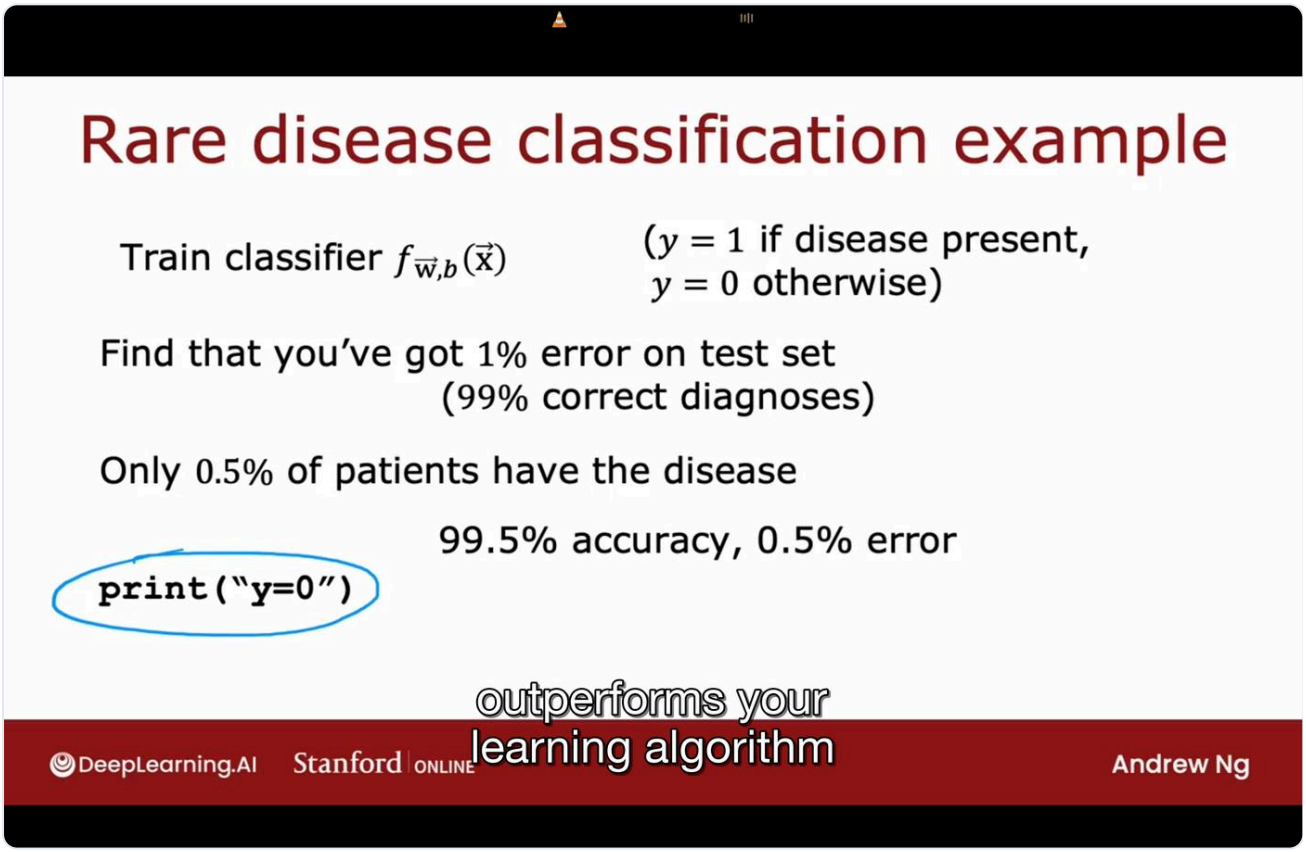


FIGURE 1.1 | Rare Disease Classification Example

It predicts y equals 0 all the time. This very simple even non-learning algorithm, because it just says y equals 0 all the time, this will actually have 99.5 percent accuracy or 0.5 percent error. This really dumb algorithm outperforms your learning algorithm which had one percent error, much worse than 0.5 percent error. But I think a piece of software that just prints y equals 0, is not a very useful diagnostic tool. What this really means is that you can't tell if getting one percent er-

ror is actually a good result or a bad result. In particular, if you have one algorithm that achieves 99.5 percent accuracy, different one that achieves 99.2 percent accuracy, different one that achieves 99.6 percent accuracy.

It's difficult to know which of these is actually the best algorithm. Because if you have an algorithm that achieve 0.5 percent error and a different one that achieves one percent error and a different one that achieves 1.2 percent error, it's difficult to know which of these is the best algorithm.

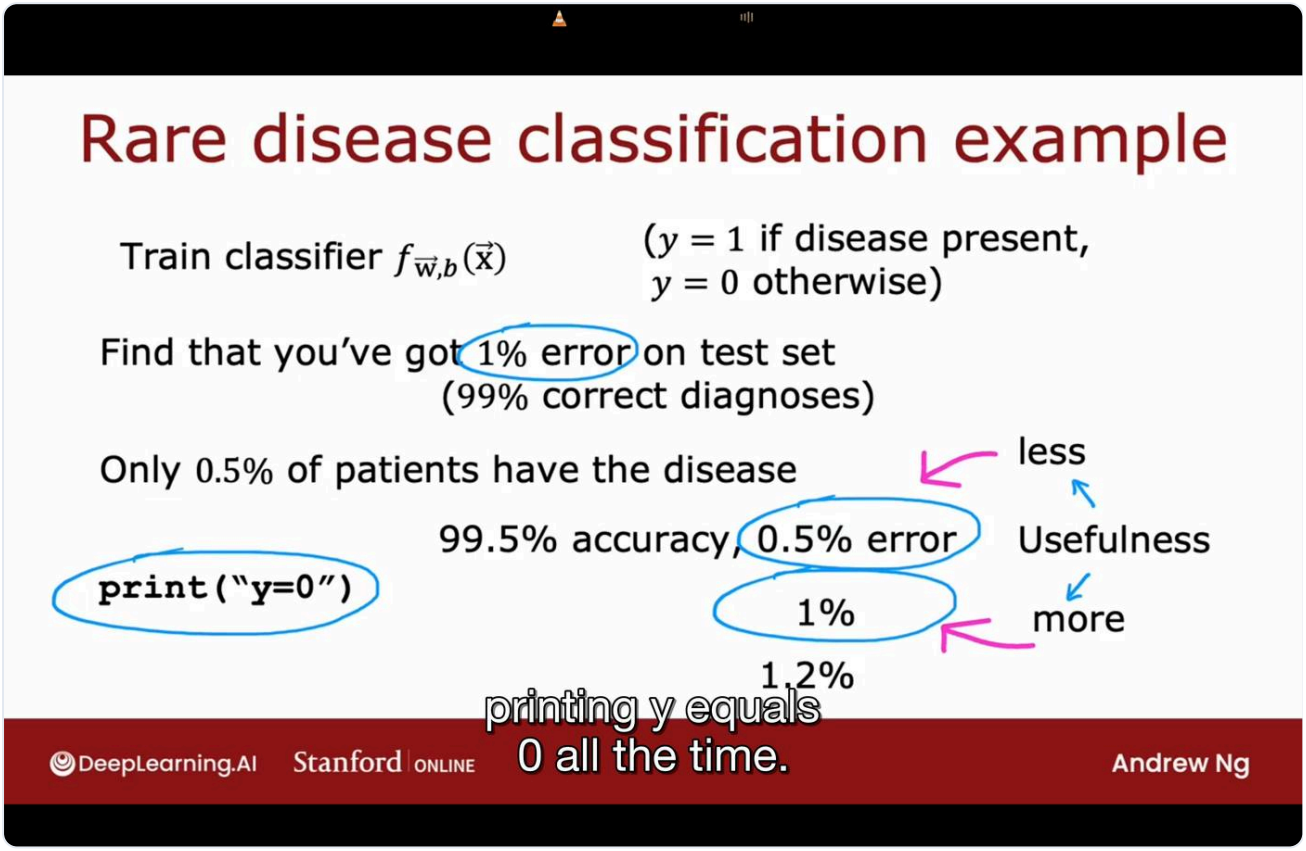


FIGURE 1.2 | Rare Disease Classification Example

Because the one with the lowest error may be is not particularly useful prediction like this that always predicts y equals 0 and never ever diagnose any patient as having this disease. Quite possibly an algorithm that has one percent error, but that at least diagnosis some patients as having the disease could be more useful than just printing y equals 0 all the time. When working on problems with skewed data sets, we usually use a different error metric rather than just classification error to figure out how well your learning algorithm is doing. In particular, a common pair of error metrics are precision and recall, which we'll define on the slide.

In this example, y equals one will be the rare class, such as the rare disease that we may want to detect. In particular, to evaluate a learning algorithm's performance with one rare class it's useful to construct what's called a confusion matrix, which is a two-by-two matrix or a two-by-two table that looks like this. On the axis on top, I'm going to write the actual class, which could be one or zero. On the vertical axis, I'm going to write the predicted class, which is what did your learning algorithm predicts on a given example, one or zero?

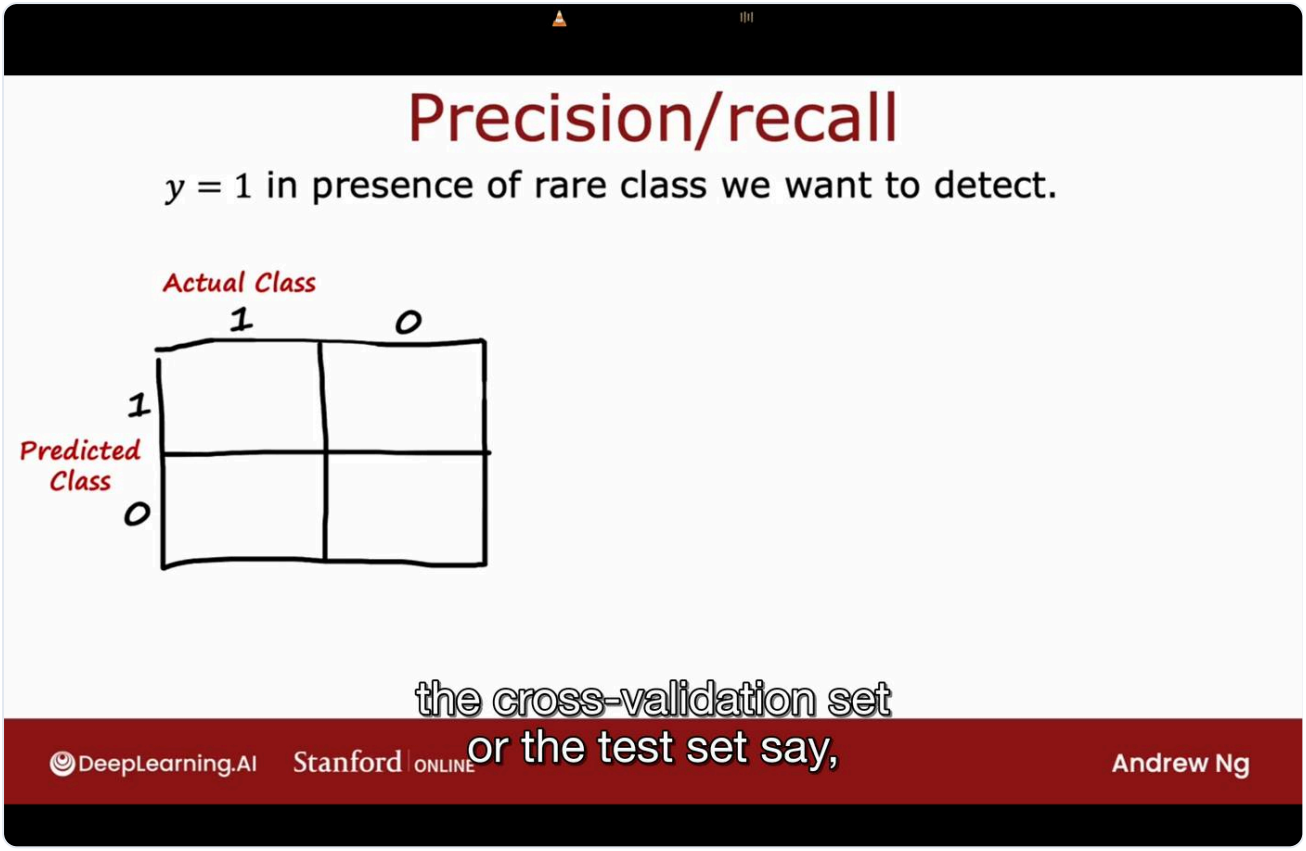


FIGURE 1.3 | Precision/recall

To evaluate your algorithm's performance on the cross-validation set or the test set say, we will then count up how many examples? Was the actual class, 1, and the predicted class 1? Maybe you have 100 cross-validation examples and on 15 of them, the learning algorithm had predicted one and the actual label was also one. Over here you would count up the number of examples in C or cross-validation set where the actual class was zero and your algorithm predicted one. Maybe you've five examples there and here predicted Class 0, actual Class 1.

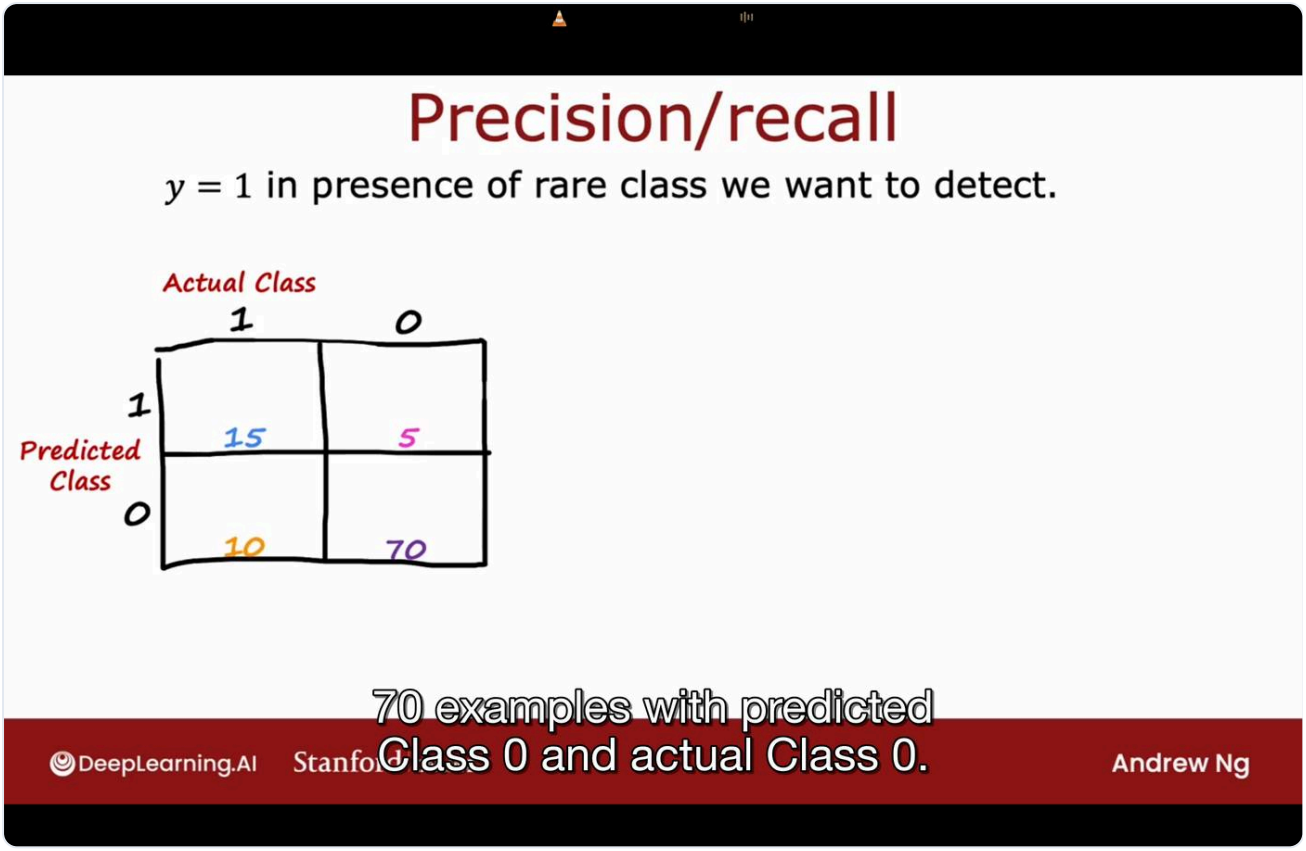


FIGURE 1.4 | Precision/recall

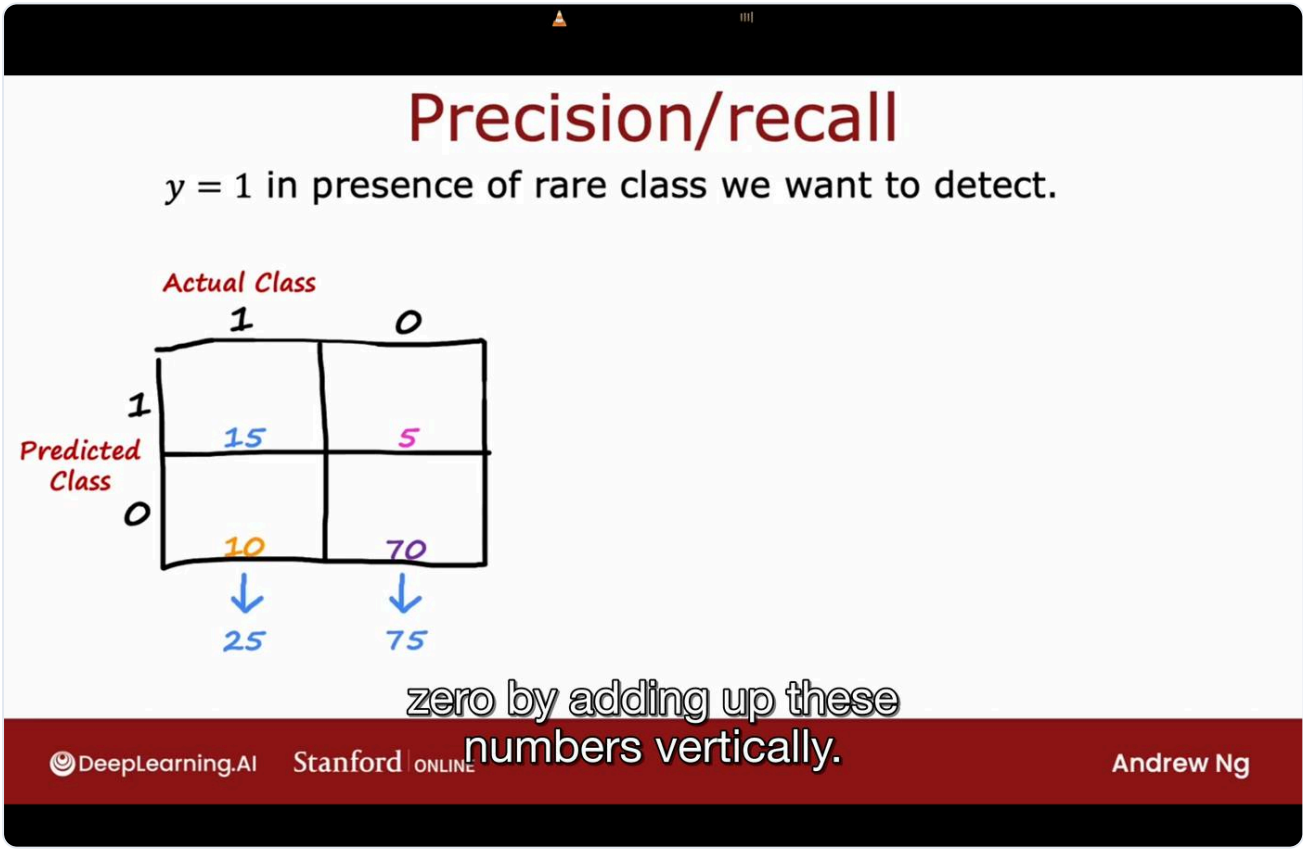


FIGURE 1.5 | Precision/recall

You have 10 examples and let's say 70 examples with predicted Class 0 and actual Class 0.

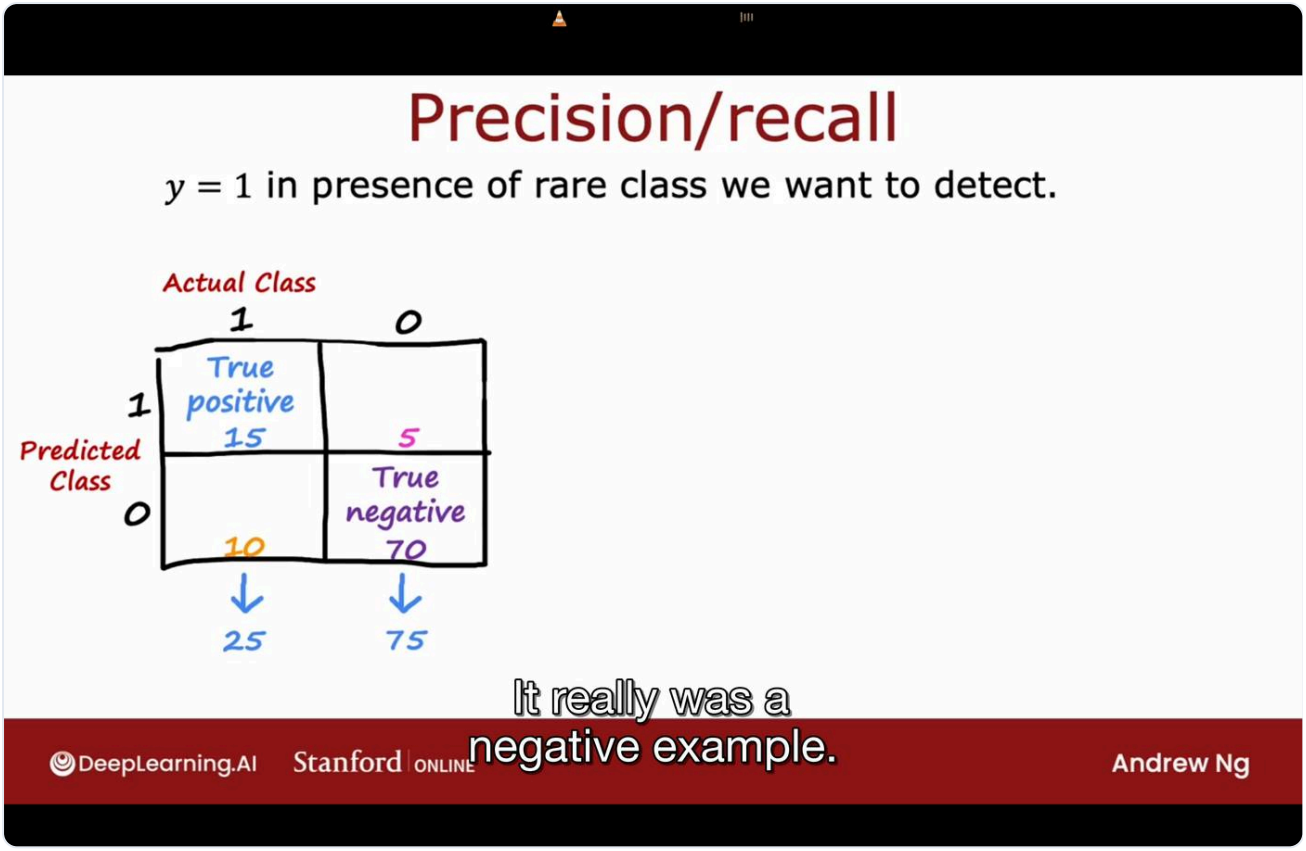


FIGURE 1.6 | Precision/recall

In this example, the skew isn't as extreme as what I had on the previous slide.

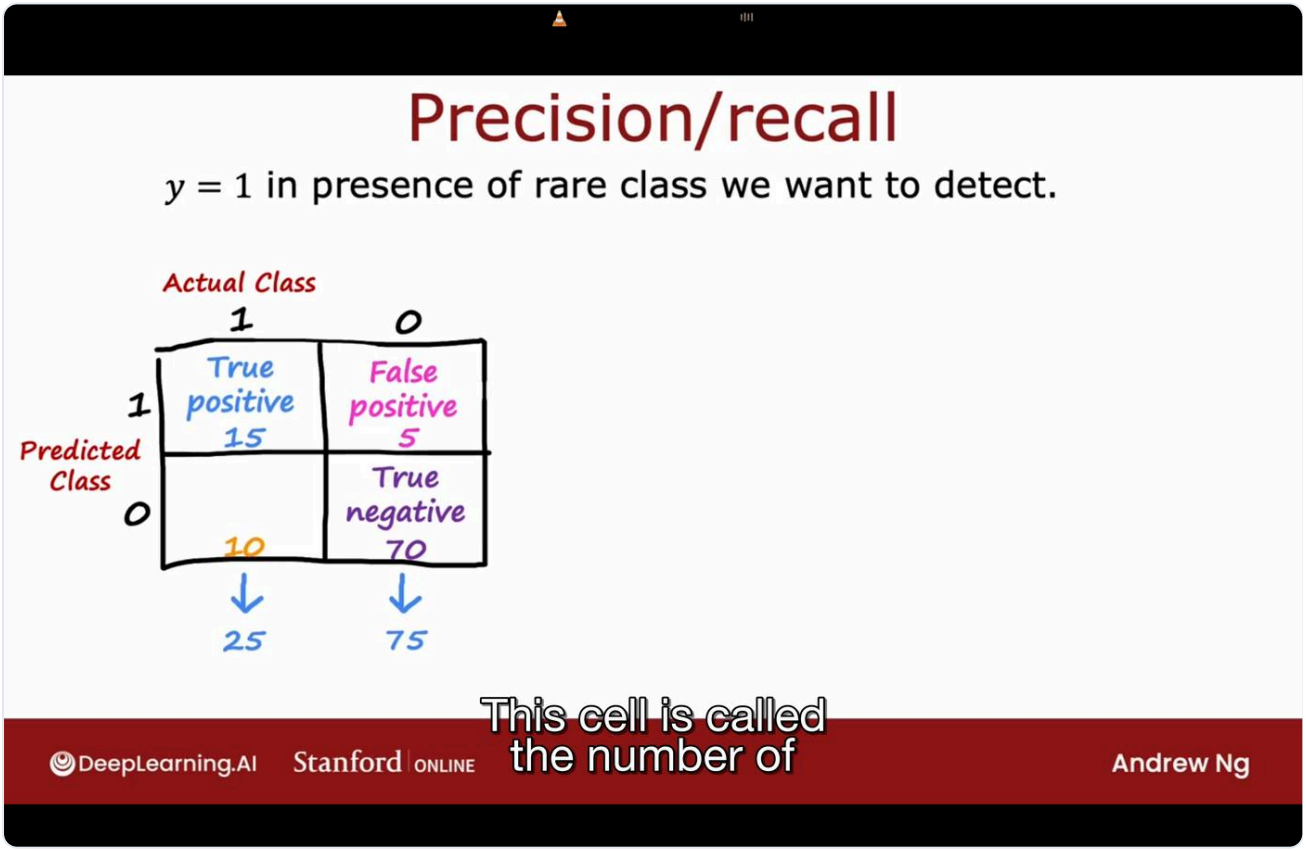


FIGURE 1.7 | Precision/recall

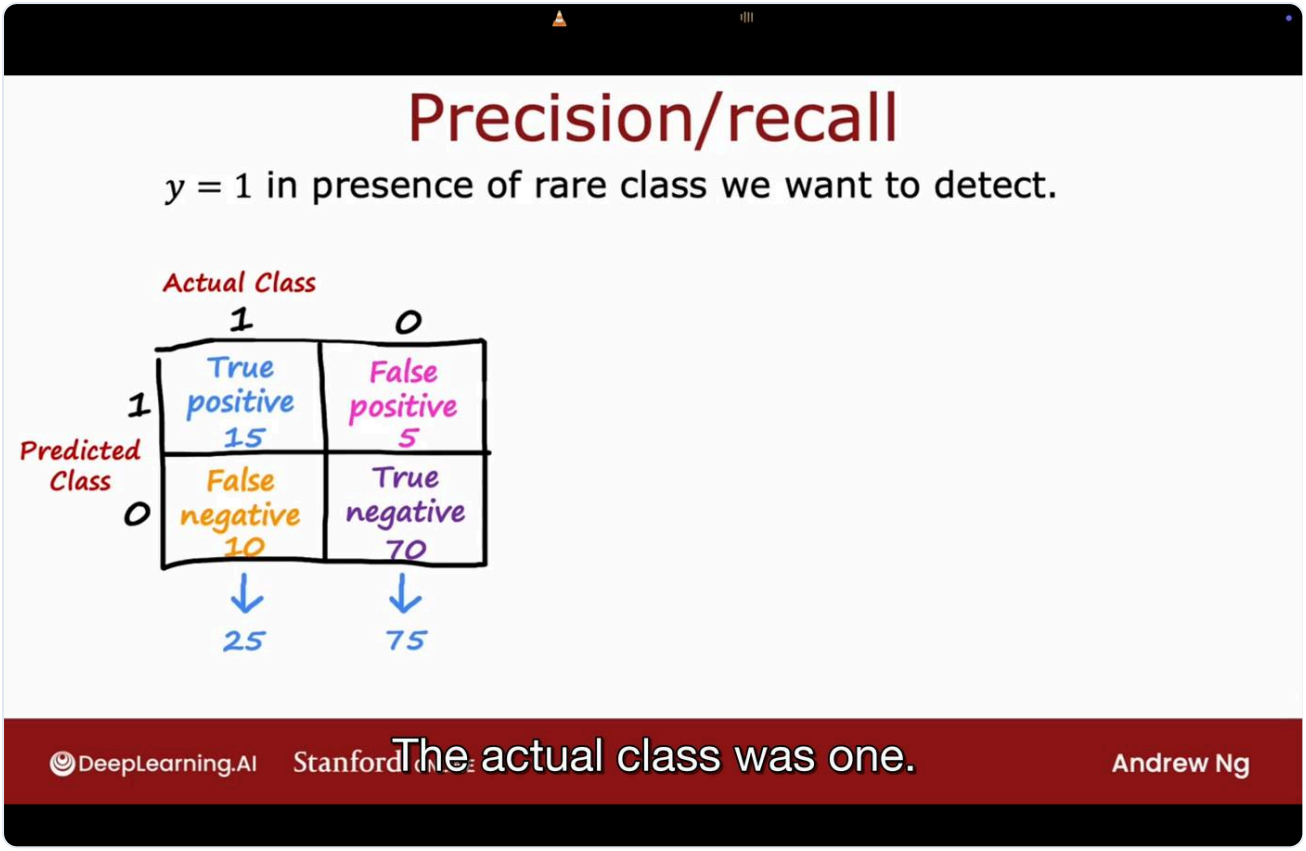


FIGURE 1.8 | Precision/recall

Because in these 100 examples in your cross-validation set, we have a total of 25 examples where the actual class was one and 75 where the actual class was zero by adding up these numbers vertically. You'll notice also that I'm using different colors to indicate these four cells in the table. I'm actually going to give names to these four cells. When the actual class is one and the predicted class is one, we're going to call that a true positive because you predicted positive and it was true there's a positive example. In this cell on the lower right, where the actual class is zero and the predicted class is zero, we will call that a true negative because you predicted negative and it was true.

It really was a negative example. This cell on the upper right is called a false positive because the algorithm predicted positive, but it was false. It's not actually positive, so this is called a false positive. This cell is called the number of false negatives because the algorithm predicted zero, but it was false. It wasn't actually negative. The actual class was one. Having divided the classifications into these four cells, two common metrics you might compute are then the precision and recall. Here's what they mean. The precision of the learning algorithm computes of all the patients where we predicted y is equal to 1, what fraction actually has the rare disease.

In other words, precision is defined as the number of true positives divided by the number classified as positive. In other words, of all the examples you predicted as positive, what fraction did we actually get right.

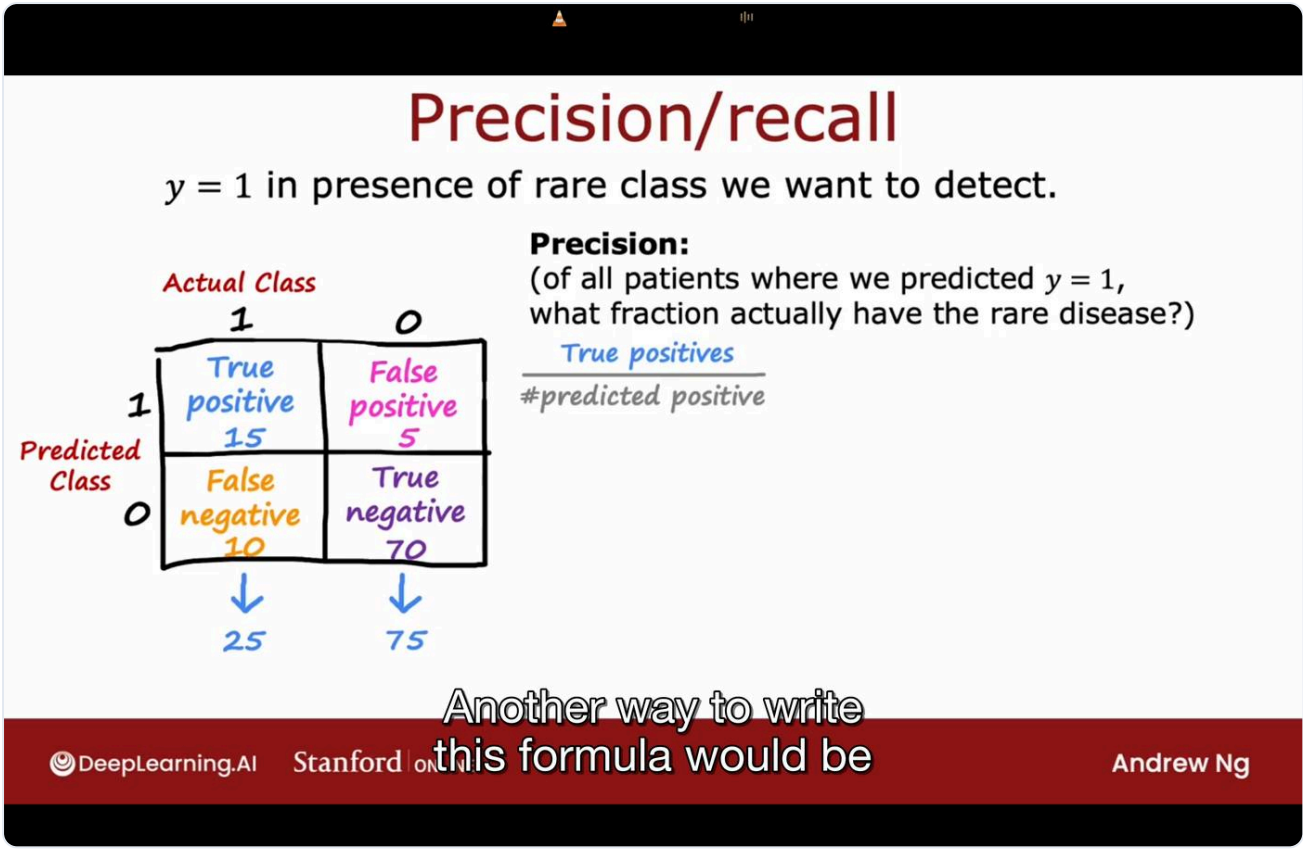


FIGURE 1.9 | Precision/recall

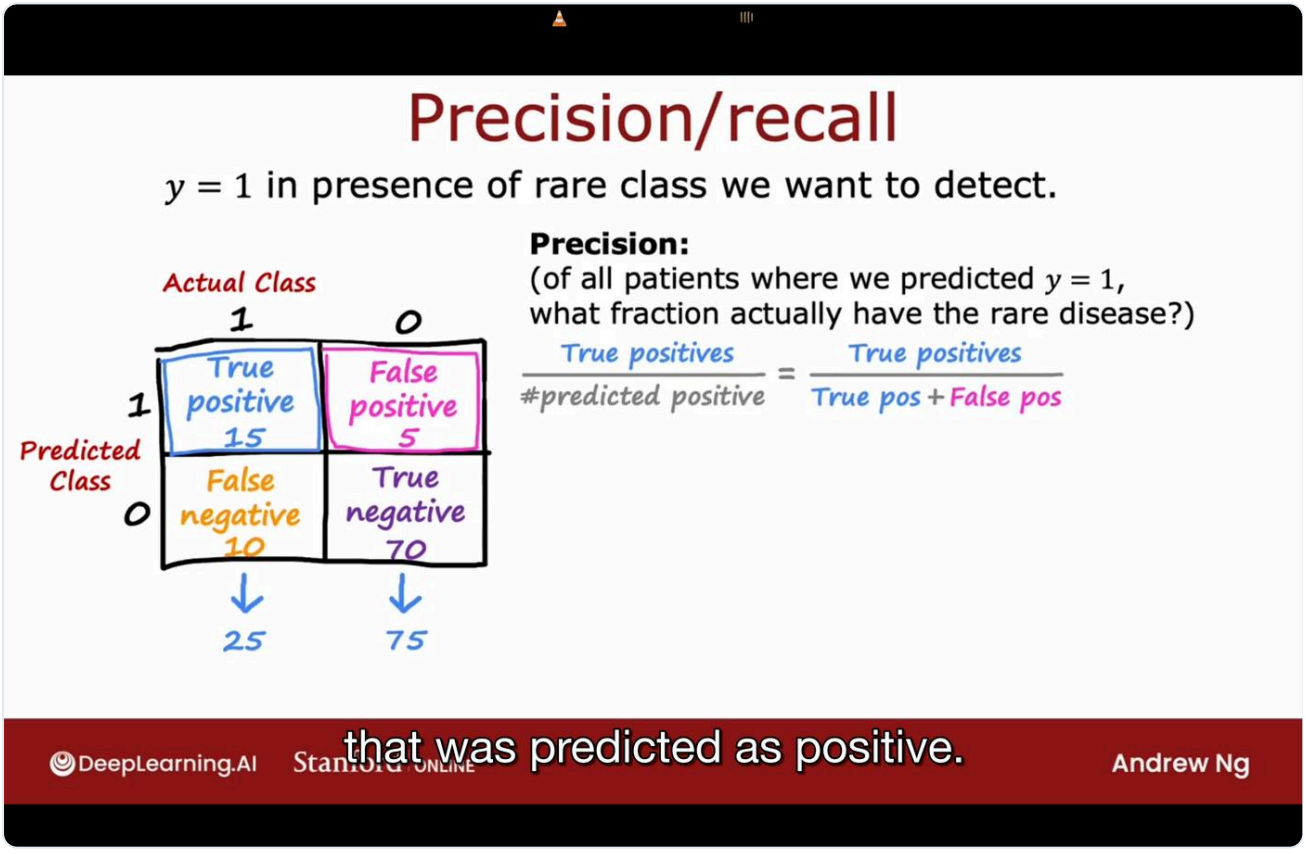


FIGURE 1.10 | Precision/recall

Another way to write this formula would be true positives divided by true positives plus false positives because it is by summing this cell and this cell that you end up with the total number that was predicted as positive.

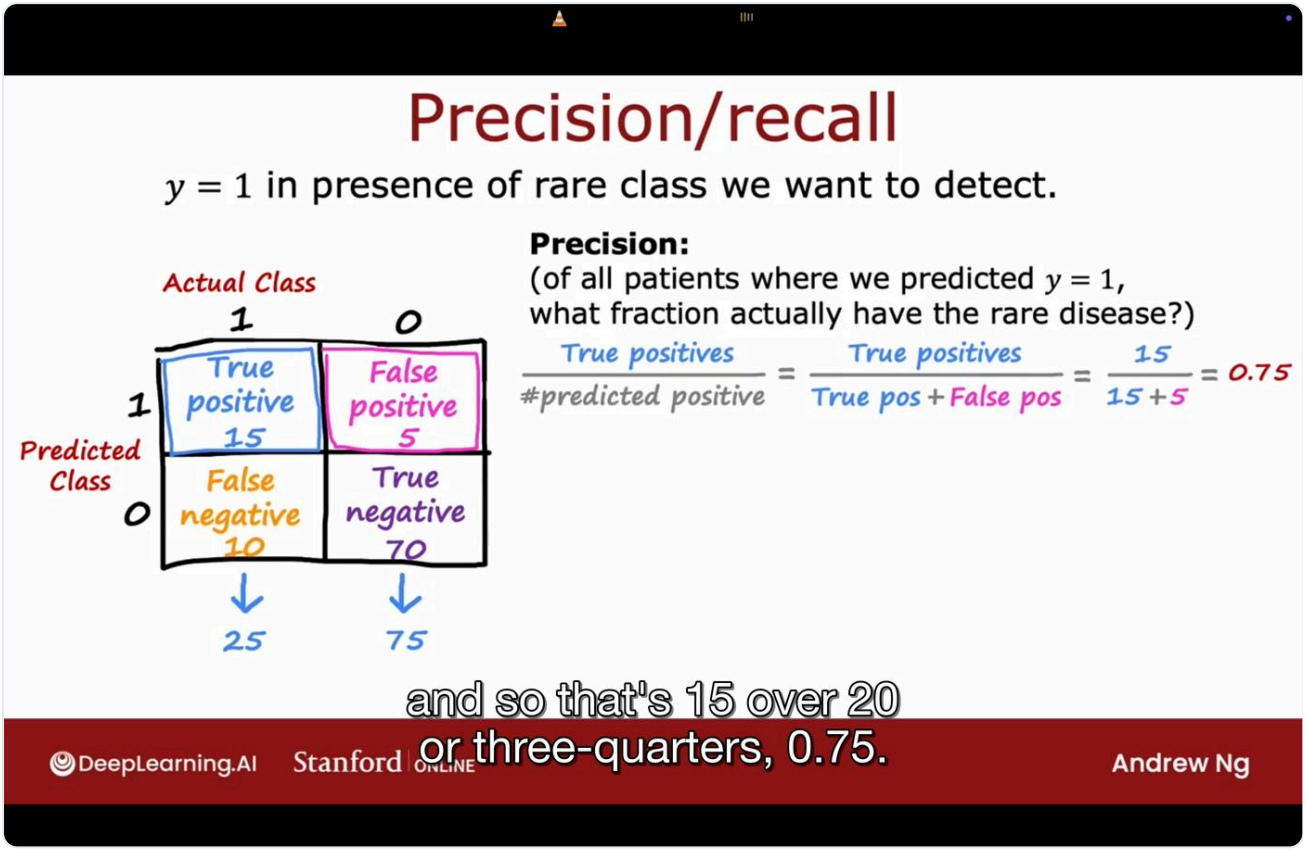


FIGURE 1.11 | Precision/recall

In this example, the numerator, true positives, would be 15 and divided by 15 plus 5, and so that's 15 over 20 or three-quarters, 0.75. So we say that this algorithm has a precision of 75 percent because of all the things it predicted as positive, of all the patients that it thought has this rare disease, it was right 75 percent of the time. The second metric that is useful to compute is recall.

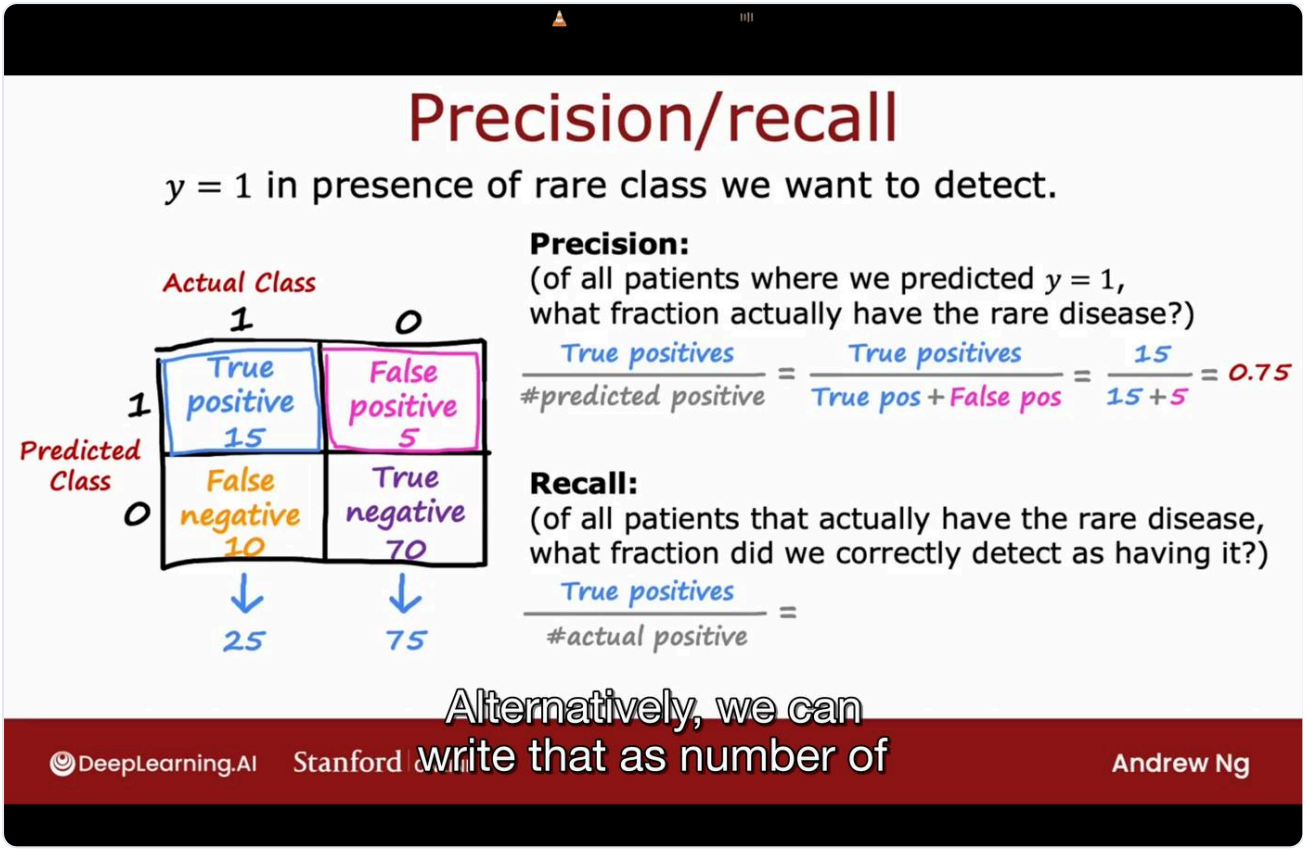


FIGURE 1.12 | Precision/recall

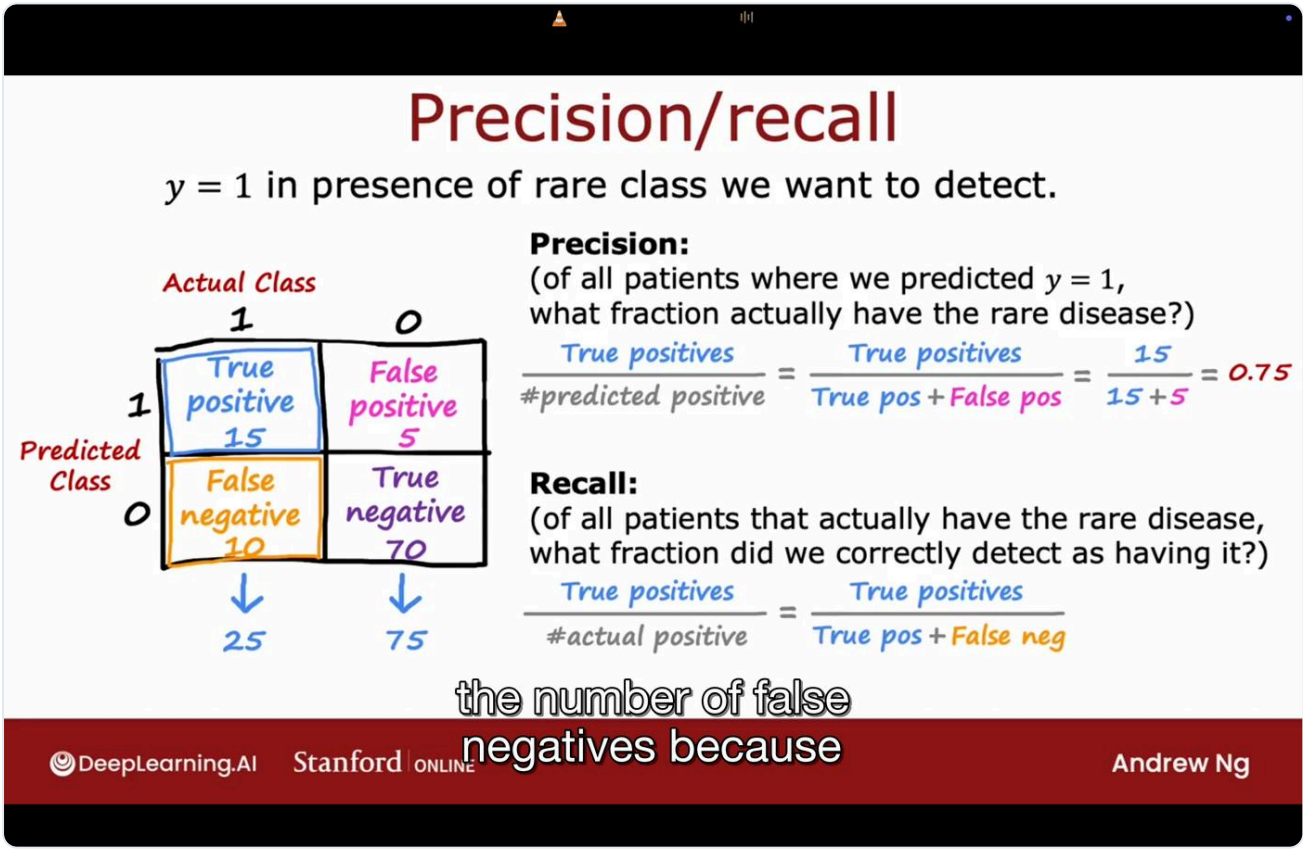


FIGURE 1.13 | Precision/recall

And recall asks: Of all the patients that actually have the rare disease, what fraction did we correctly detect as having it? Recall is defined as the number of true positives divided by the number of actual positives. Alternatively, we can write that as number of true positives divided by the number of actual positives. Well, it's this cell plus this cell. So it's actually the number of true positives plus the number of false negatives because it's by summing up this upper-left cell and this lower-left cell that you get the number of actual positive examples.

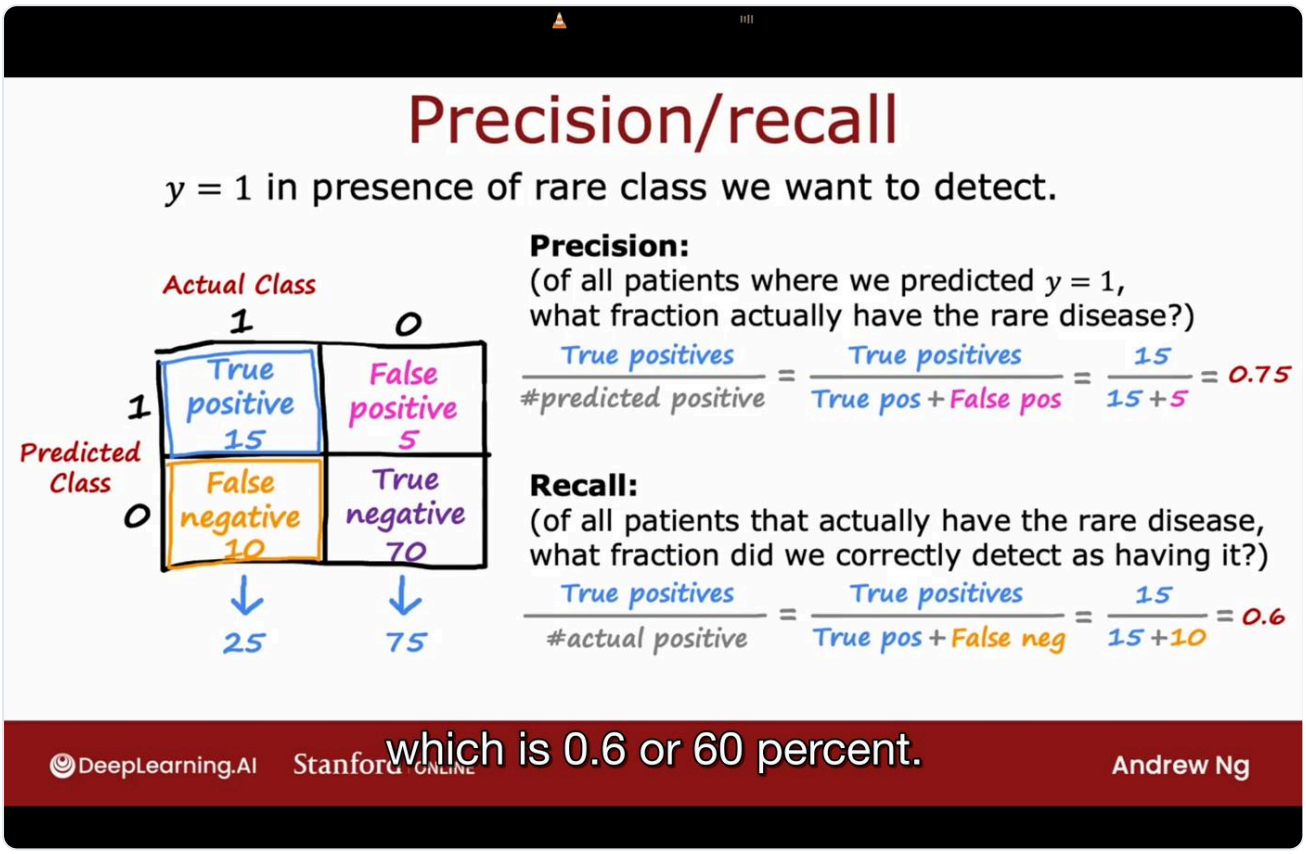


FIGURE 1.14 | Precision/recall

In our example, this would be 15 divided by 15 plus 10, which is 15 over 25, which is 0.6 or 60 percent. This learning algorithm would have 0.75 precision and 0.60 recall. You notice that this will help you detect if the learning algorithm is just printing y equals 0 all the time. Because if it predicts zero all the time, then the numerator of both of these quantities would be zero. It has no true positives. The recall metric in particular helps you detect if the learning algorithm is predicting zero all the time. Because if your learning algorithm just prints y equals 0, then the number of true positives will be zero because it never predicts positive, and so the recall will be equal to zero divided by the number of actual positives, which is equal to zero.

In general, a learning algorithm with either zero precision or zero recall is not a useful algorithm. But just as a side note, if an algorithm actually predicts zero all the time, precision actually becomes undefined because it's actually zero over. zero. But in practice, if an algorithm doesn't predict even a single positive, we just say that precision is also equal to zero. But we'll find that computing both precision and recall makes it easier to spot if an algorithm is both reasonably accurate, in that, when it says a patient has a disease, there's a good chance the patient

has a disease, such as 0.75 chance in this example, and also making sure that of all the patients that have the disease, it's helping to diagnose a reasonable fraction of them, such as here it's finding 60 percent of them.

When you have a rare class, looking at precision and recall and making sure that both numbers are decently high, that hopefully helps reassure you that your learning algorithm is actually useful. The term recall was motivated by this observation that if you have a group of patients or population of patients, then recall measures, of all the patients that have the disease, how many would you have accurately diagnosed as having it. So when you have skewed classes or a rare class that you want to detect, precision and recall helps you tell if your learning algorithm is making good predictions or useful predictions. Now that we have these metrics for telling how well your learning algorithm is doing, in the next video, let's take a look at how to trade-off between precision and recall to try to optimize the performance of your learning algorithm.

LECTURE 49

Trading Off Precision and Recall

In the ideal case, we like for learning algorithms that have high precision and high recall. High precision would mean that if a diagnosis of patients have that rare disease, probably the patient does have it and it's an accurate diagnosis. High recall means that if there's a patient with that rare disease, probably the algorithm will correctly identify that they do have that disease. But it turns out that in practice there's often a trade-off between precision and recall. In this video, we'll take a look at that trade-off and how you can pick a good point along that trade-off. Here are the definitions from the last video on precision and recall, I'll just write them here.

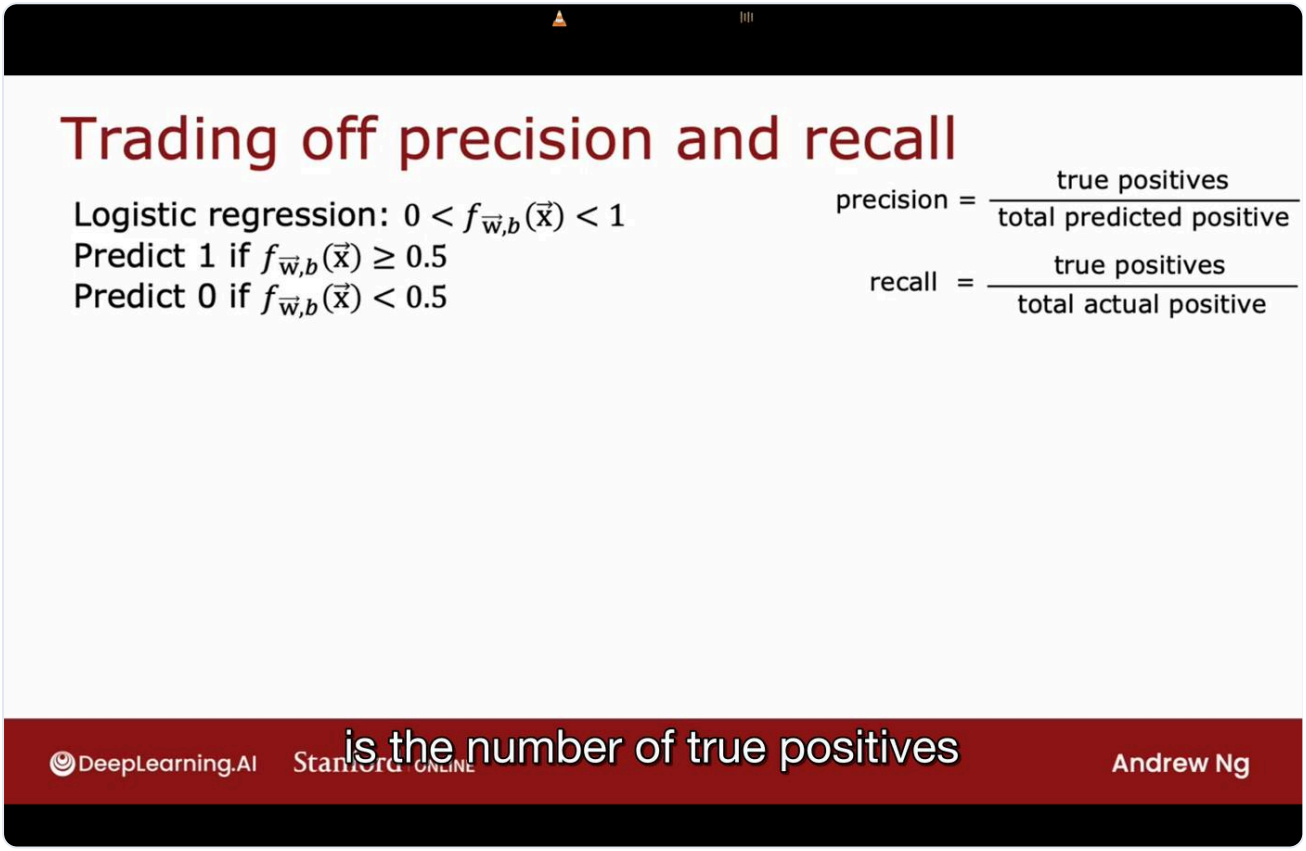


FIGURE 2.1 | Trading Off Precision and Recall

Well, you recall precision is the number of true positives divided by the total number that was predicted positive, and recall is the number of true positives divided by the total actual number of positives. If you're using logistic regression to make predictions, then the logistic regression model will output numbers between 0 and 1. We would typically threshold the output of logistic regression at 0.5 and predict 1 if f of x is greater than equal to 0.5 and predict 0 if it's less than 0.5. But suppose we want to predict that y is equal to 1. That is, the rare disease is present only if we're very confident. If our philosophy is, whenever we predict that the patient has a disease, we may have to send them for possibly invasive and expensive treatment.

If the consequences of the disease aren't that bad, even if left not treated aggressively, then we may want to predict y equals 1 only if we're very confident.

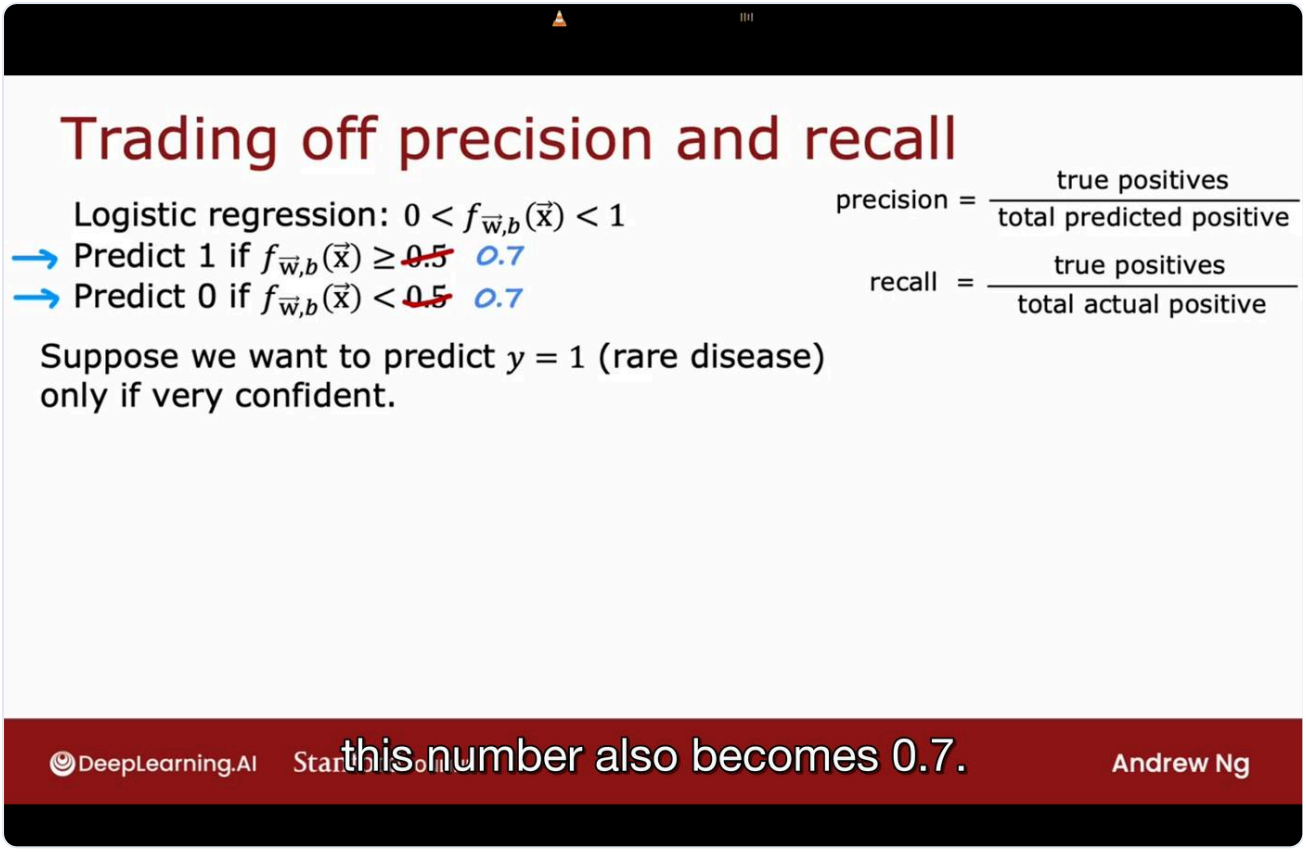


FIGURE 2.2 | Trading Off Precision and Recall

In that case, we may choose to set a higher threshold where we will predict y is 1 only if f of x is greater than or equal to 0.7, so this is saying we'll predict y equals 1 only we're at least 70 percent sure, rather than just 50 percent sure and so this number also becomes 0.7. Notice that these two numbers have to be the same because it's just depending on whether it's greater than or equal to or less than this number that you predict 1 or 0. By raising this threshold, you predict y equals 1 only if you're pretty confident and what that means is that precision will increase because whenever you predict one, you're more likely to be right so raising the thresholds will result in higher precision, but it also results in lower recall because we're now predicting one less often and so of the total number of patients with the disease, we're going to correctly diagnose fewer of them.

By raising this threshold to 0.7, you end up with higher precision, but lower recall.

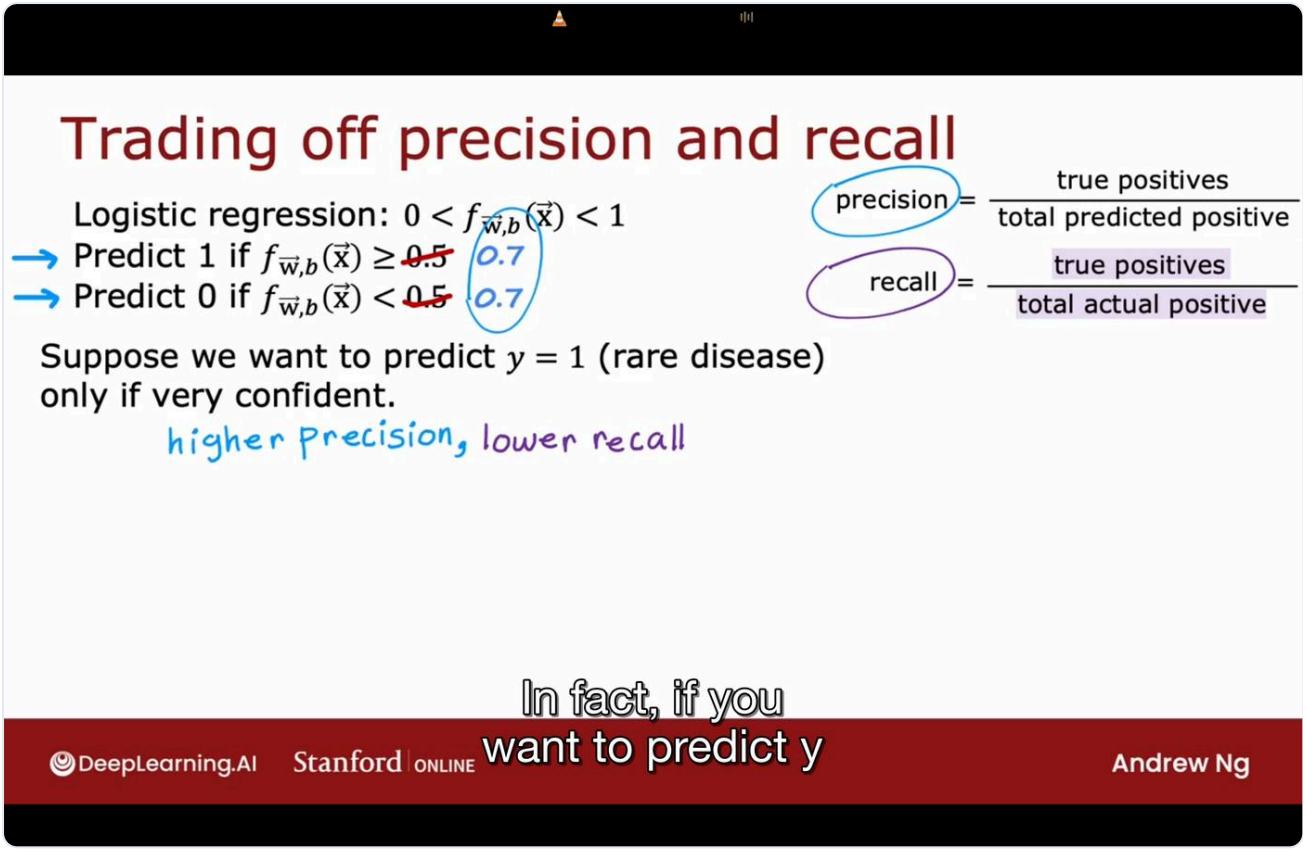


FIGURE 2.3 | Trading Off Precision and Recall

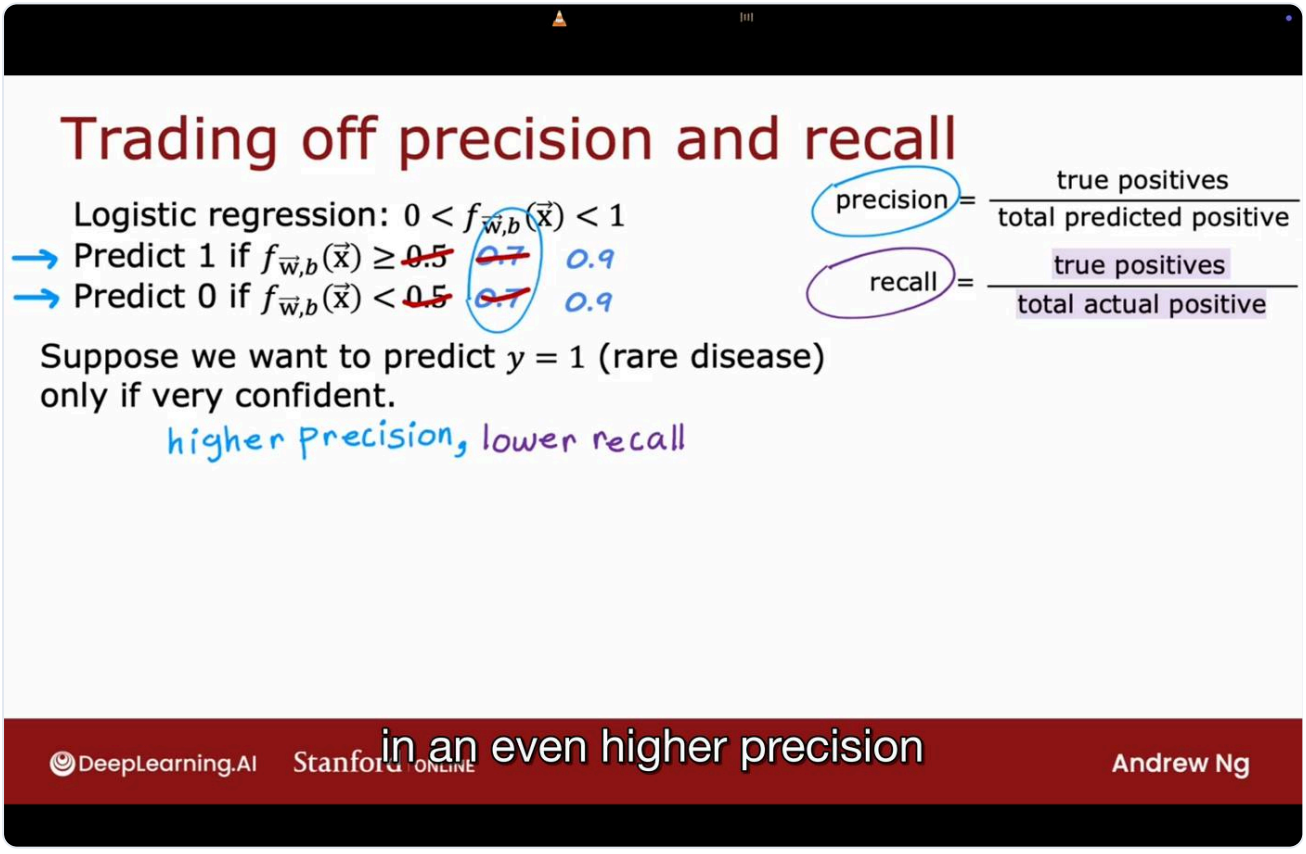


FIGURE 2.4 | Trading Off Precision and Recall

In fact, if you want to predict y equals 1 only if you are very confident, you can even raise this higher to 0.9 and that results in an even higher precision and so whenever you predict the patient has the disease, you're probably right and this will give you a very high precision. The recall will go even further down.

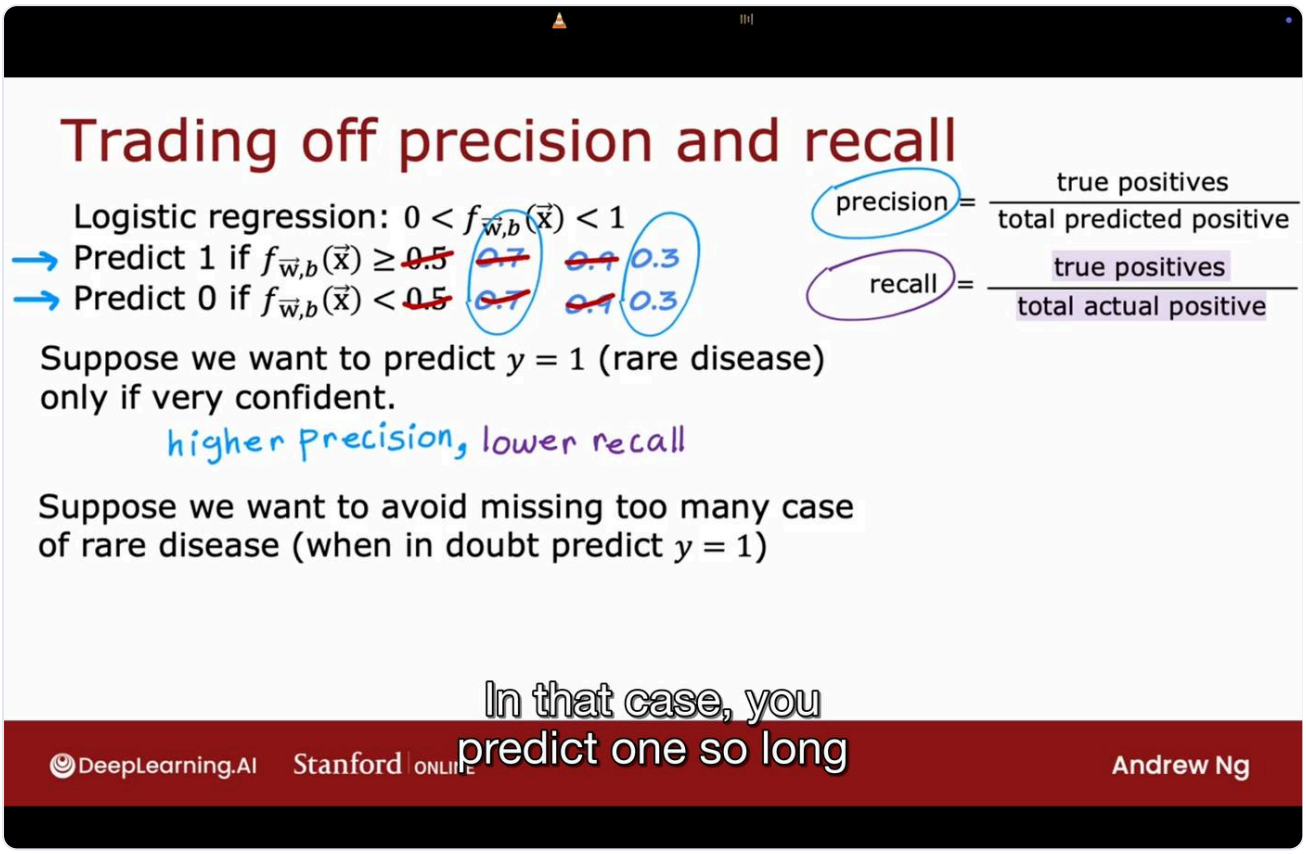


FIGURE 2.5

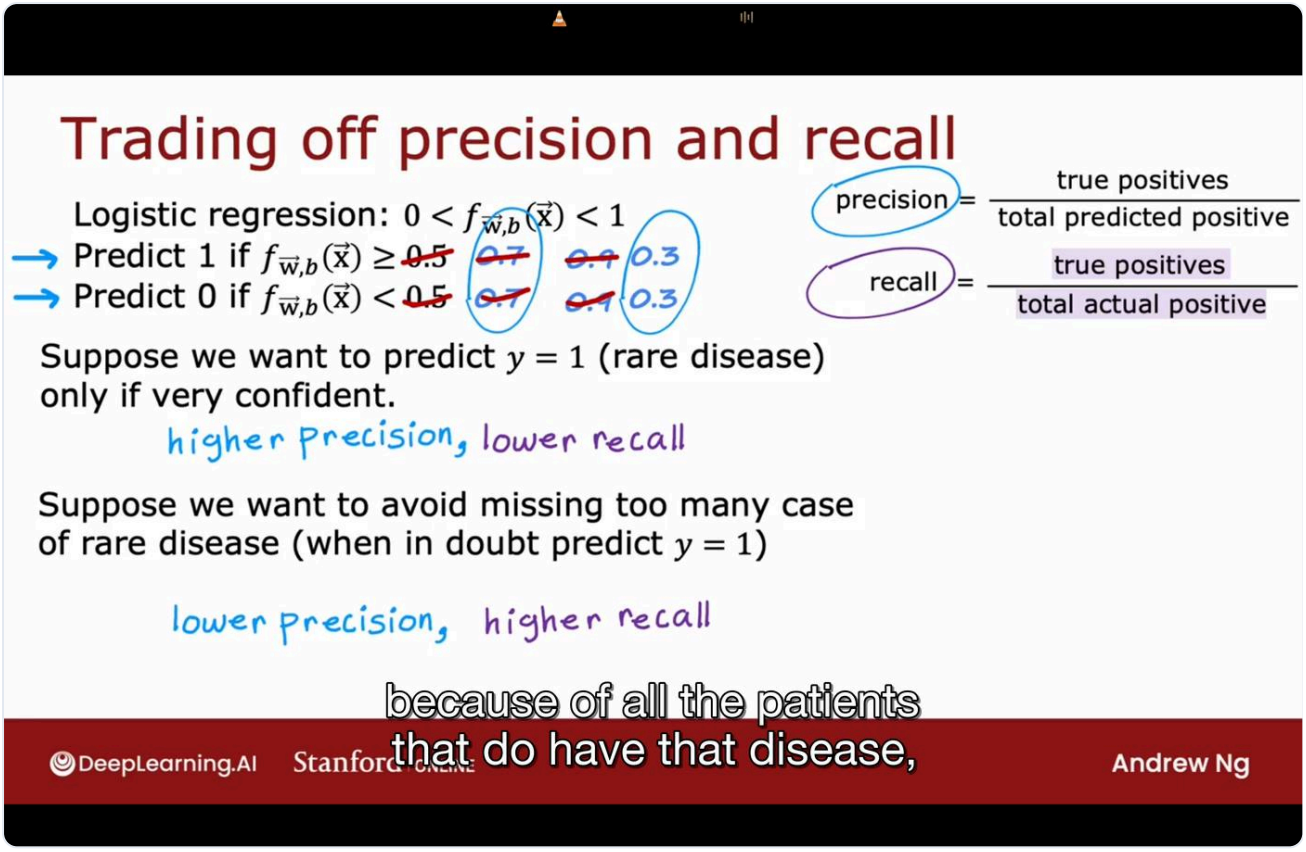


FIGURE 2.6

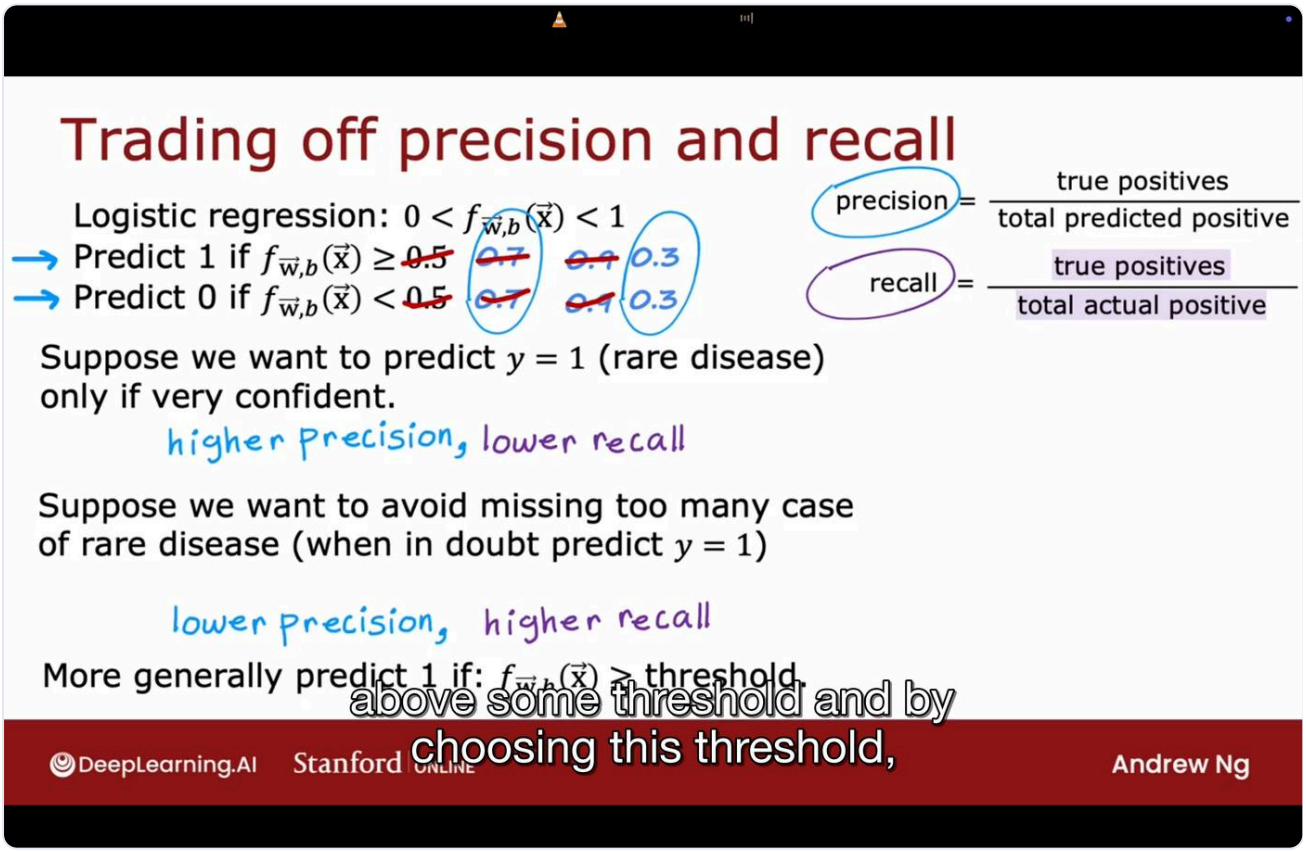


FIGURE 2.7 | Trading Off Precision and Recall

On the flip side, suppose we want to avoid missing too many cases of the rare disease, so if what we want is when in doubt, predict y equals 1, this might be the case where if treatment is not too invasive or painful or expensive but leaving a disease untreated has much worse consequences for the patient. In that case, you might say, when in doubt in the interests of safety let's just predict that they have it and consider them for treatment because untreated cases could be quite bad. If for your application, that is the better way to make decisions, then you would take this threshold instead lower it, say, set it to 0.3. In that case, you predict one so long as you think there's maybe a 30 percent chance or better of the disease being present and you predict zero only if you're pretty sure that the disease is absent.

As you can imagine, the impact on precision and recall will be opposite to what you saw up here, and lowering this threshold will result in lower precision because we're now looser, we're more willing to predict one even if we aren't sure but to result in higher recall, because of all the patients that do have that disease, we're probably going to correctly identify more of them.

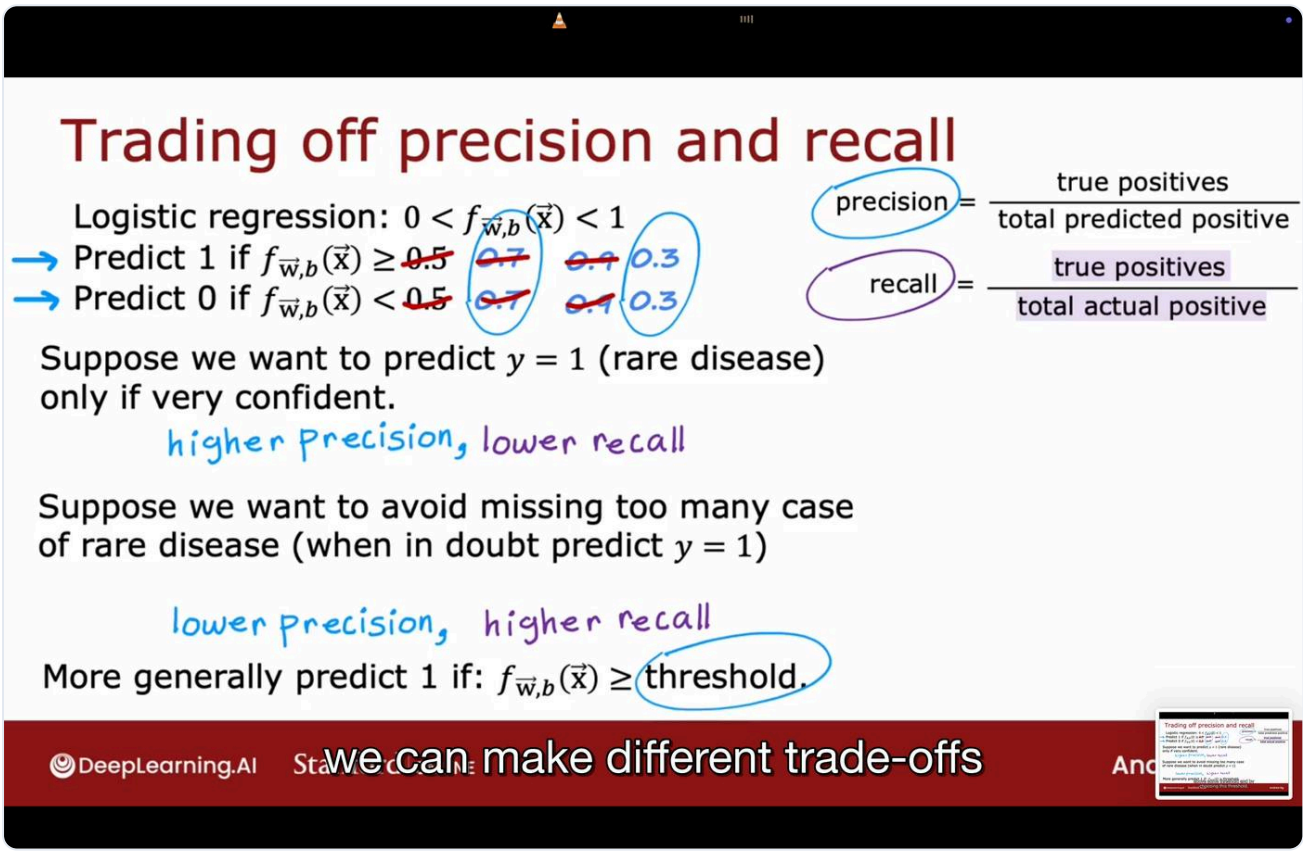


FIGURE 2.8

More generally, we have the flexibility to predict one only if f is above some threshold and by choosing this threshold, we can make different trade-offs between precision and recall. It turns out that for most learning algorithms, there is a trade-off between precision and recall.

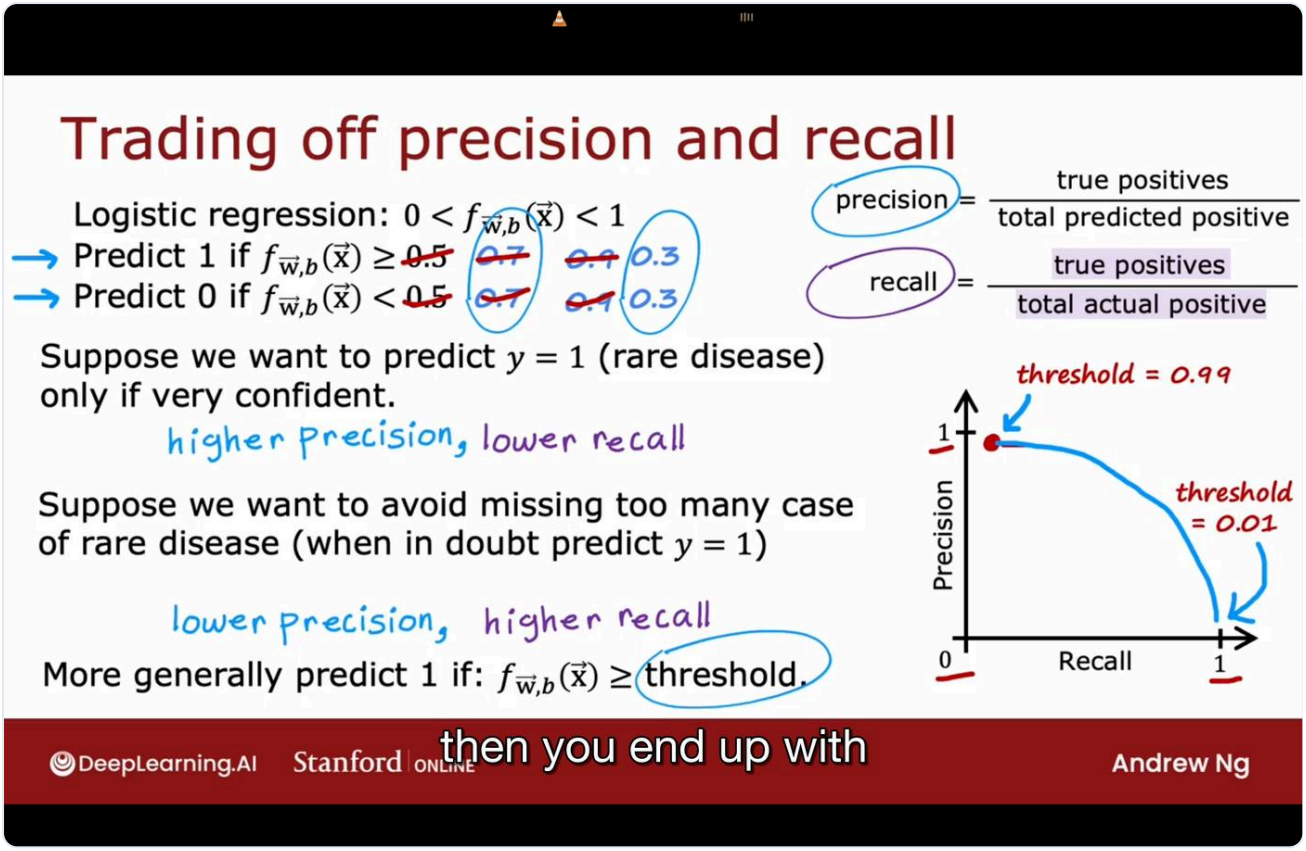


FIGURE 2.9

Precision and recall both go between zero and one and if you were to set a very high threshold, say a threshold of 0.99, then you enter with very high precision, but lower recall and as you reduce the value of this threshold, you then end up with a curve that trades off precision and recall until eventually, if you have a very low threshold, so the threshold equals 0.01, then you end up with very low precision but relatively high recall. Sometimes by plotting this curve, you can then try to pick a threshold which corresponds to picking a point on this curve.

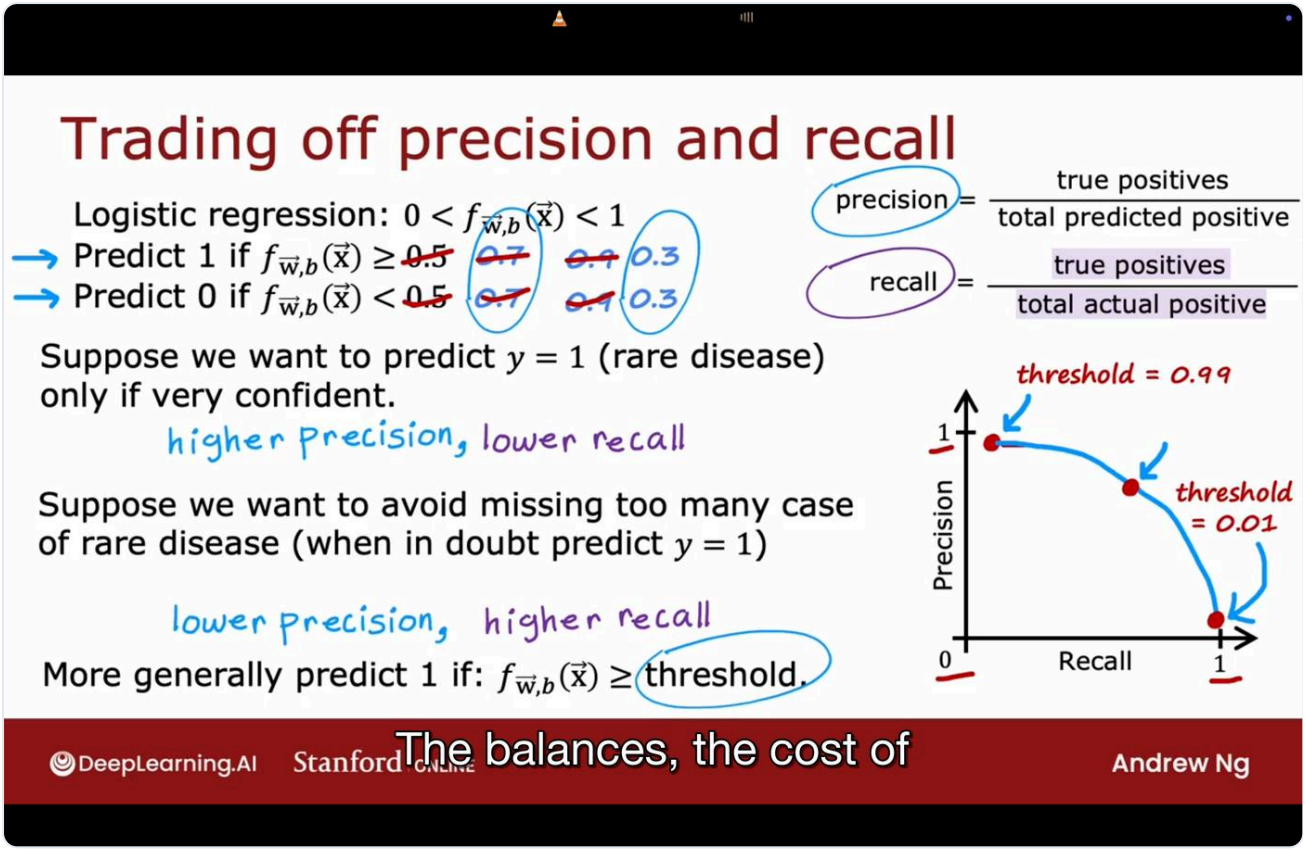


FIGURE 2.10 | Trading Off Precision and Recall

The balances, the cost of false positives and false negatives or the balances, the benefits of high precision and high recall. Plotting precision and recall for different values of the threshold allows you to pick a point that you want. Notice that picking the threshold is not something you can really do with cross-validation because it's up to you to specify the best points. For many applications, manually picking the threshold to trade-off precision and recall will be what you end up doing. It turns out that if you want to automatically trade-off precision and recall rather than have to do so yourself, there is another metric called the F1 score that is sometimes used to automatically combine precision recall to help you pick the best value or the best trade-off between the two.

One challenge with precision recall is you're now evaluating your algorithms using two different metrics, so if you've trained three different algorithms and the precision-recall numbers look like this, is not that obvious how to pick which algorithm to use.

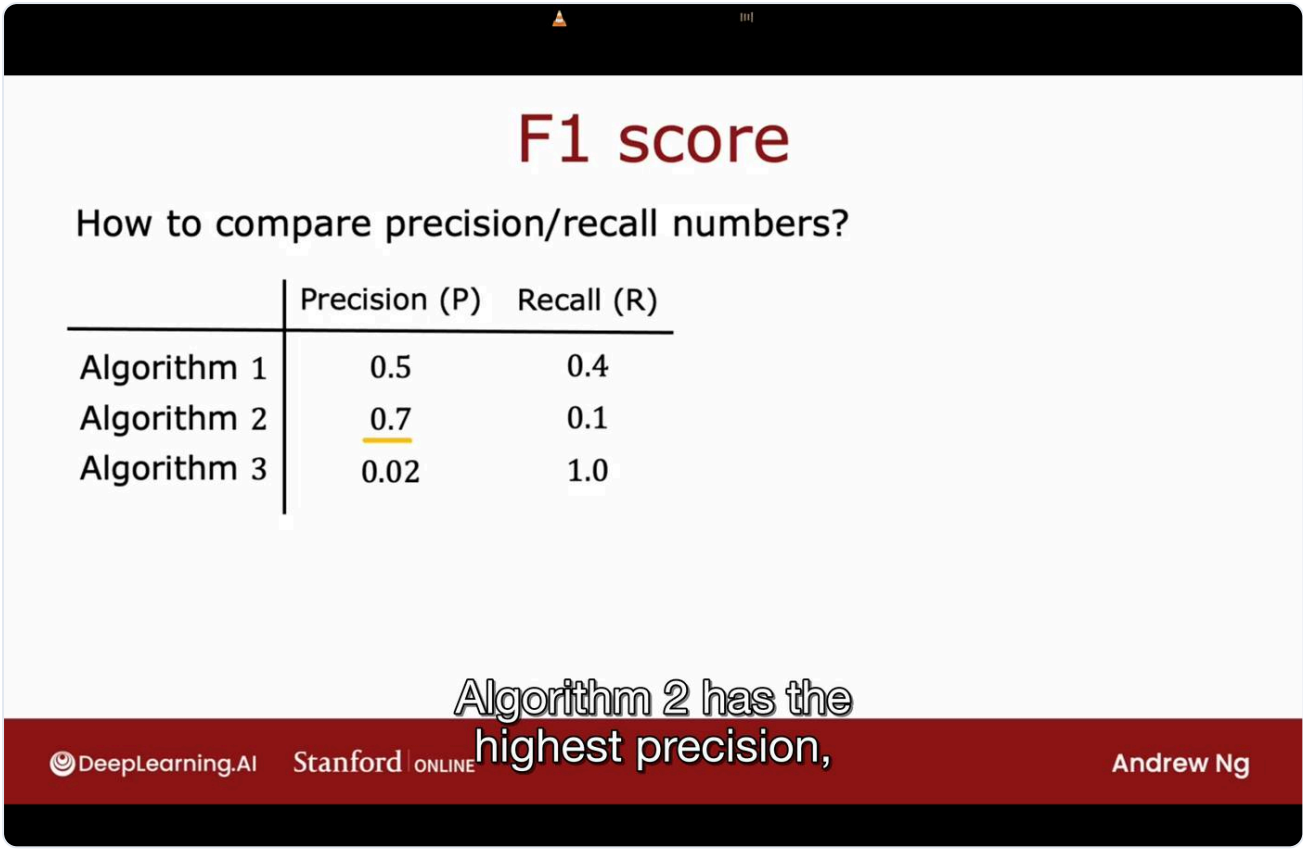


FIGURE 2.11 | How to Compare Precision/recall Numbers?

If there was an algorithm that's better on precision and better on recall, then you probably want to go with that one. But in this example, Algorithm 2 has the highest precision, but Algorithm 3 has the highest recall, and Algorithm 1 trades off the two in-between, and so no one algorithm is obviously the best choice. In order to help you decide which algorithm to pick, it may be useful to find a way to combine precision and recall into a single score, so you can just look at which algorithm has the highest score and maybe go with that one. One way you could combine precision and recall is to take the average, this turns out not to be a good way, so I don't really recommend this.

But if we were to take the average, you get 0.45, 0.4, and 0.5.

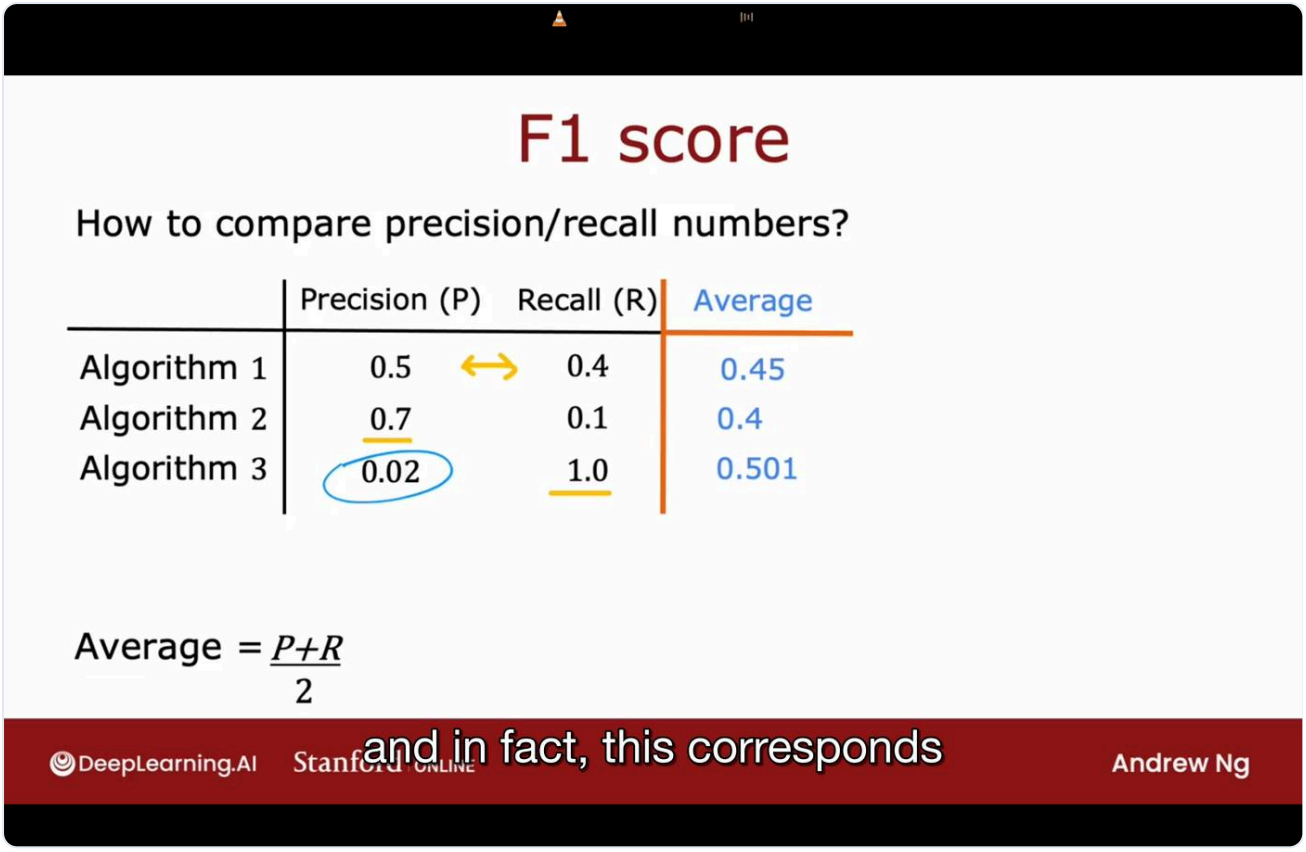


FIGURE 2.12 | How to Compare Precision/recall Numbers?

But it turns out that computing the average and picking the algorithm with the highest average between precision and recall doesn't work that well because this algorithm has very low precision, and in fact, this corresponds maybe to an algorithm that actually does print y equals 1 and diagnosis all patients as having the disease, that's why recall is perfect but the precision is really low. Algorithm 3 is actually not a particularly useful algorithm, even though the average between precision and recall is quite high. Let's not use the average between precision and recall. Instead, the most common way of combining precision recall is a compute something called the F1 score, and the F1 score is a way of combining P and R precision and recall but that gives more emphasis to whichever of these values is lower.

Because it turns out if an algorithm has very low precision or very low recall is pretty not that useful. The F1 score is a way of computing an average of sorts that pays more attention to whichever is lower.

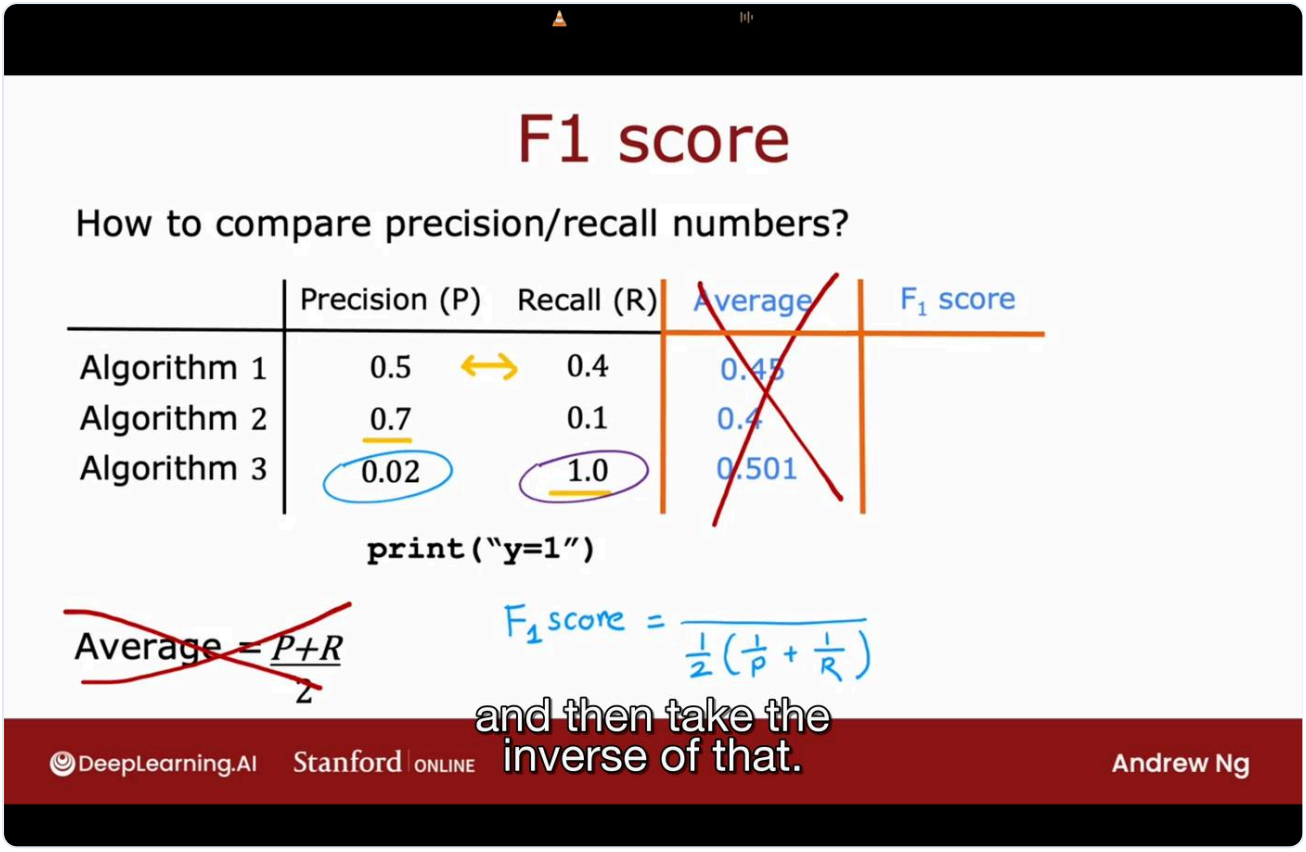


FIGURE 2.13 | How to Compare Precision/recall Numbers?

The formula for computing F1 score is this, you're going to compute one over P and one over R, and average them, and then take the inverse of that.

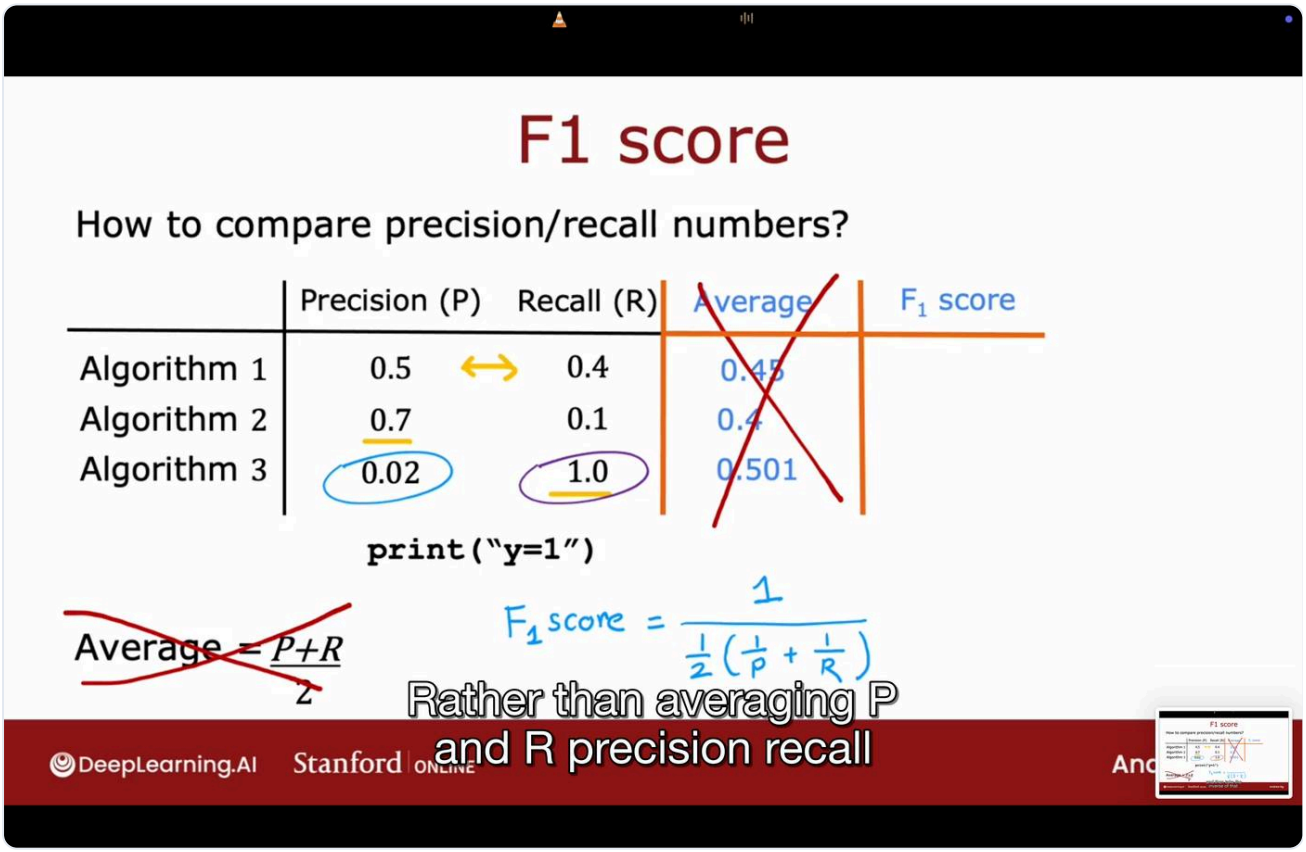


FIGURE 2.14 | How to Compare Precision/recall Numbers?

Rather than averaging P and R precision recall we're going to average one over P and one over R, and then take one over that. If you simplify this equation it can also be computed as follows.

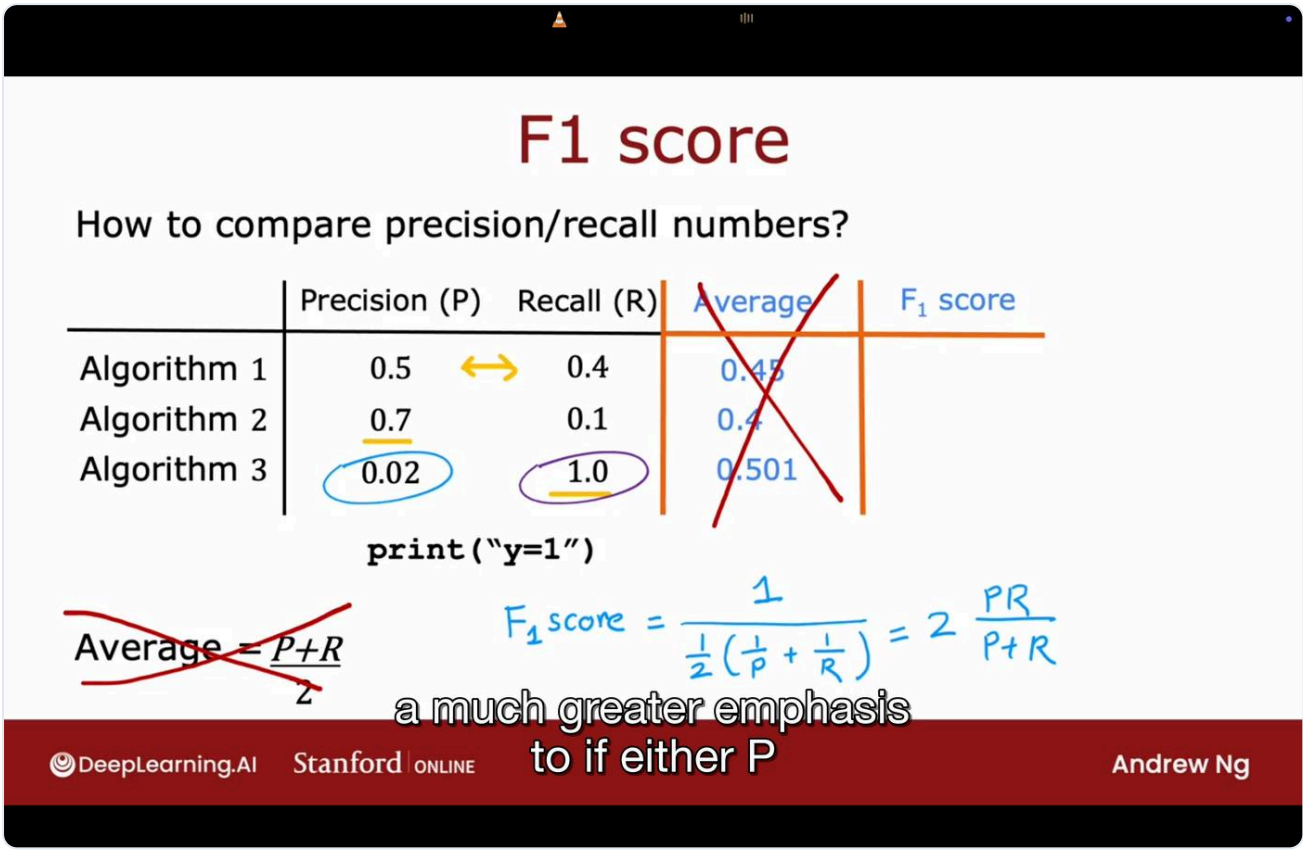


FIGURE 2.15 | How to Compare Precision/recall Numbers?

But by averaging one over P and one over R this gives a much greater emphasis to if either P or R turns out to be very small. If you were to compute the F1 score for these three algorithms, you'll find that the F1 score for Algorithm 1 is 0.444, and for the second algorithm is 0.175. You notice that 0.175 is much closer to the lower value than the higher value and for the third algorithm is 0.0392.

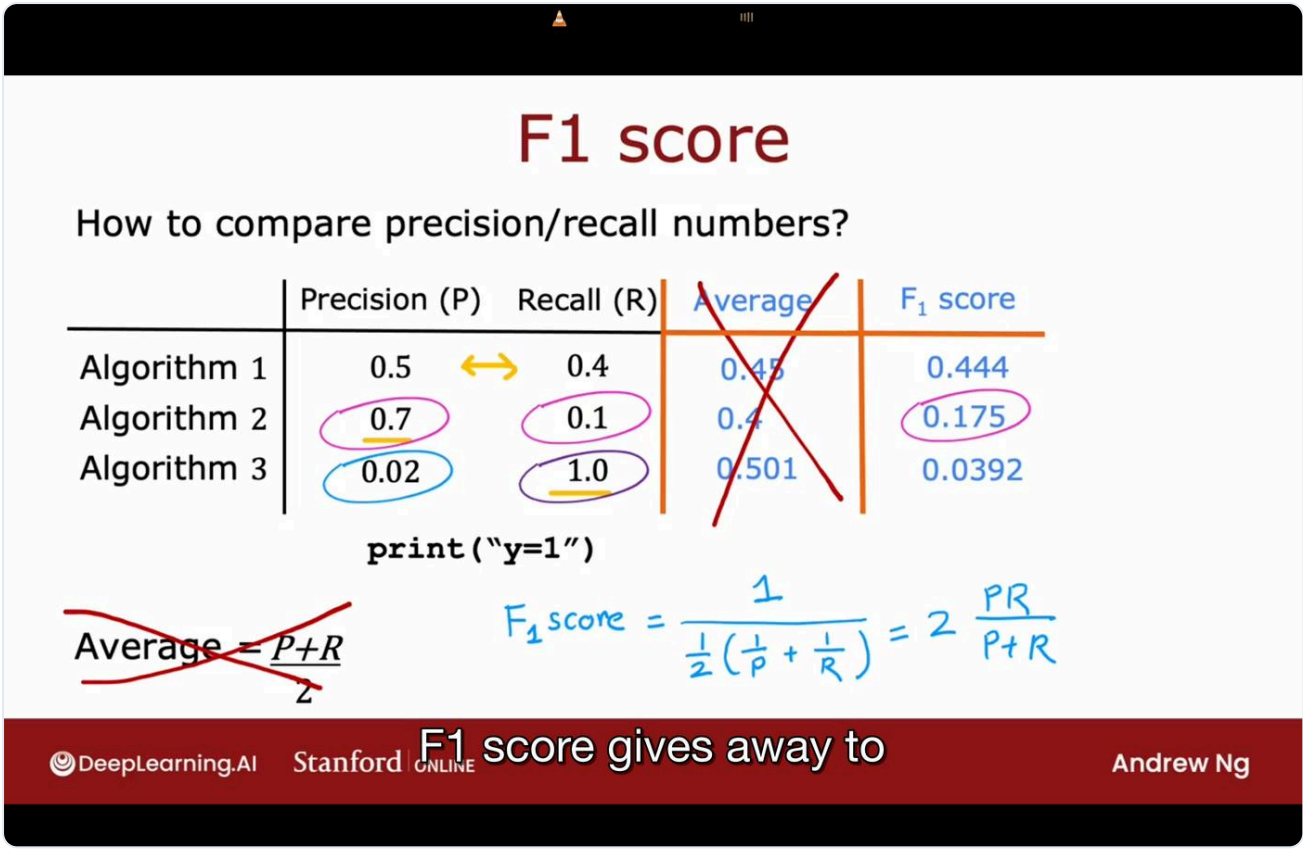


FIGURE 2.16 | How to Compare Precision/recall Numbers?

F1 score gives away to trade-off precision and recall, and in this case, it will tell us that maybe the first algorithm is better than the second or the third algorithms.

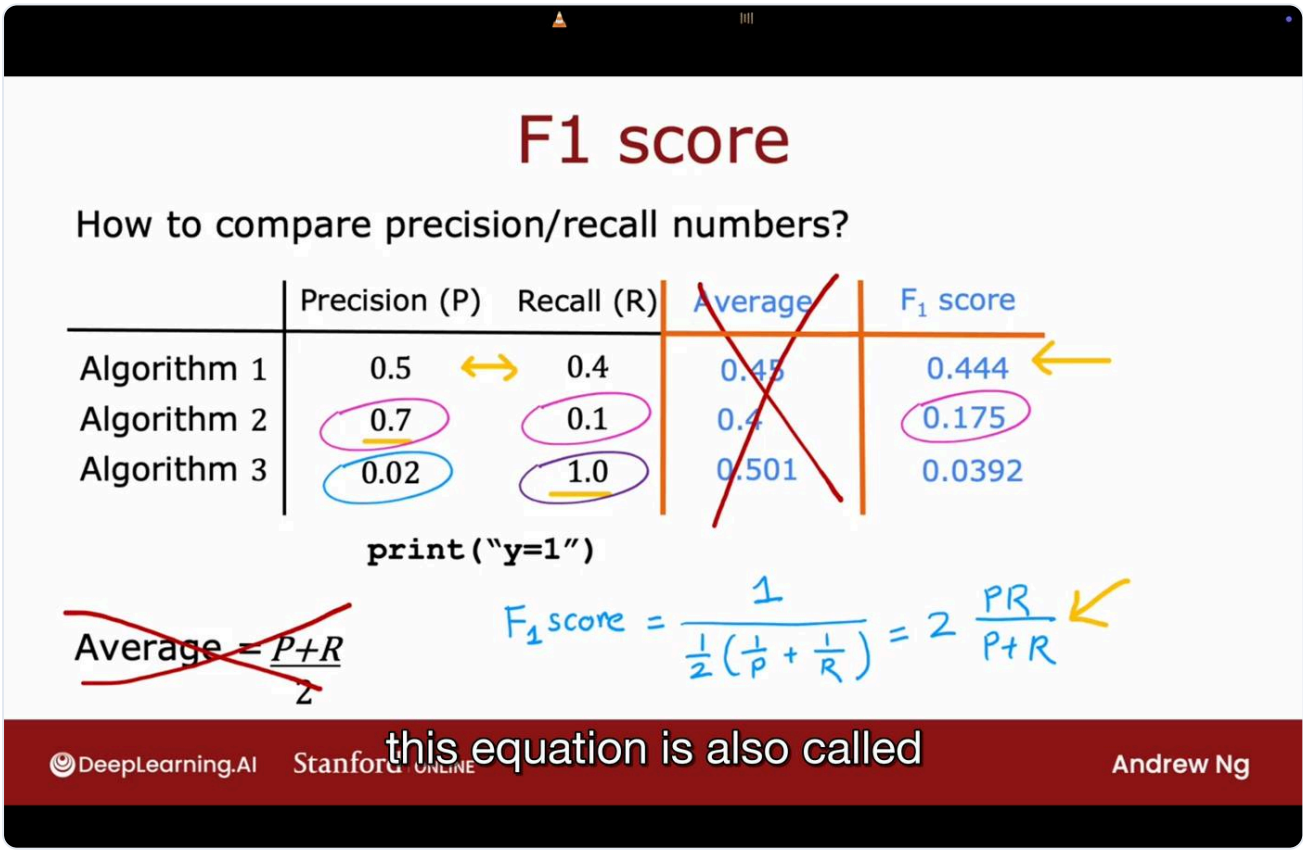


FIGURE 2.17 | How to Compare Precision/recall Numbers?

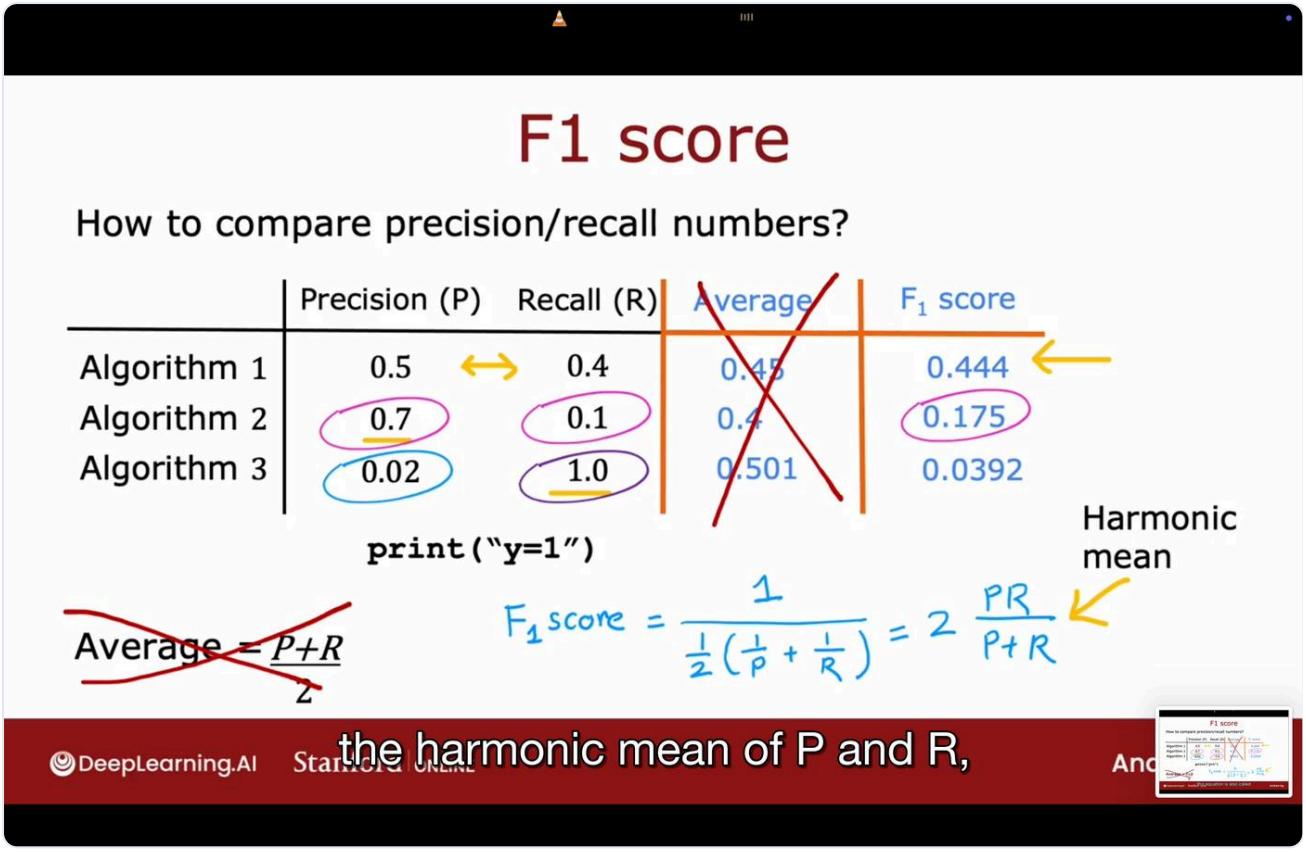


FIGURE 2.18 | How to Compare Precision/recall Numbers?

By the way, in math, this equation is also called the harmonic mean of P and R, and the harmonic mean is a way of taking an average that emphasizes the smaller values more. But for the purposes of this class, you don't need to worry about that terminology of the harmonic mean. Congratulations on getting to the last video of this week and thank you also for sticking with me through these two optional videos. In this week, you've learned a lot of practical tips, practical advice for how to build a machine learning system, and by applying these ideas, I think you'd be very effective at building machine learning algorithms. Next week, we'll come back to talk about another very powerful machine learning algorithm.

In fact, of the advanced techniques that why we use in many commercial production settings, I think at the top of the list would be neural networks and decision trees. Next week we'll talk about decision trees, which I think will be another very powerful technique that you're going to use to build many successful applications as well. I look forward to seeing you next week.

LECTURE 50

Decision Tree

Welcome to the final week of this course on Advanced Learning Algorithms. One of the learning algorithms that is very powerful, widely used in many applications, also used by many to win machine learning competitions is decision trees and tree ensembles. Despite all the successes of decision trees, they somehow haven't received that much attention in academia, and so you may not hear about decision trees nearly that much, but it is a tool well worth having in your toolbox. In this week, we'll learn about decision trees and you'll see how to get them to work for yourself. Let's dive in. To explain how decision trees work, I'm going to use as a running example this week a cat classification example.

You are running a cat adoption center and given a few features, you want to train a classifier to quickly tell you if an animal is a cat or not. I have here 10 training examples. Associated with each of these 10 examples, we're going to have features regarding the animal's ear shape, face shape, whether it has whiskers, and then the ground truth label that you want to predict this animal cat. The first example has pointy ears, round face, whiskers are present, and it is a cat. The second example has floppy ears, the face shape is not round, whiskers are present, and yes, that is a cat, and so on for the rest of the examples. This dataset has five cats and five dogs in it.

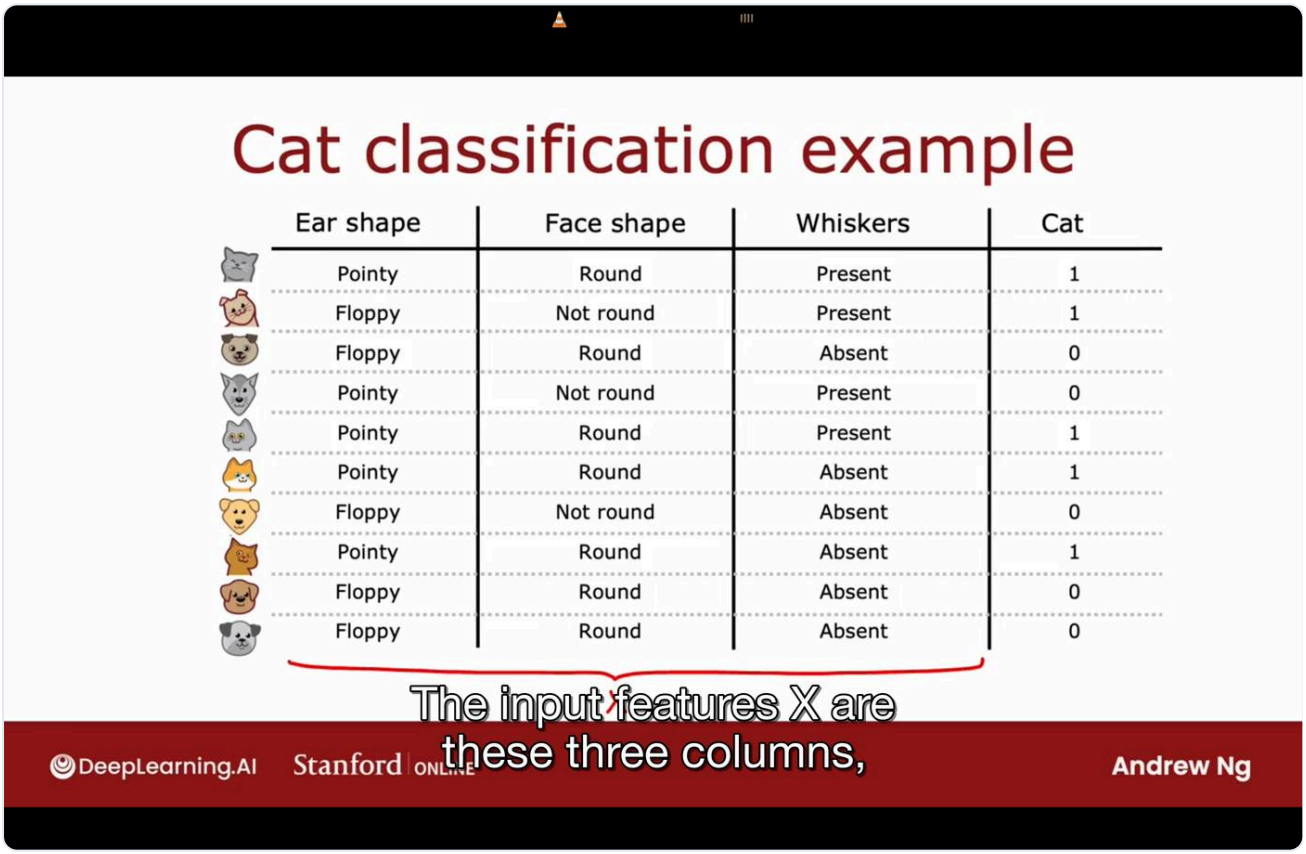


FIGURE 3.1 | Cat Classification Example

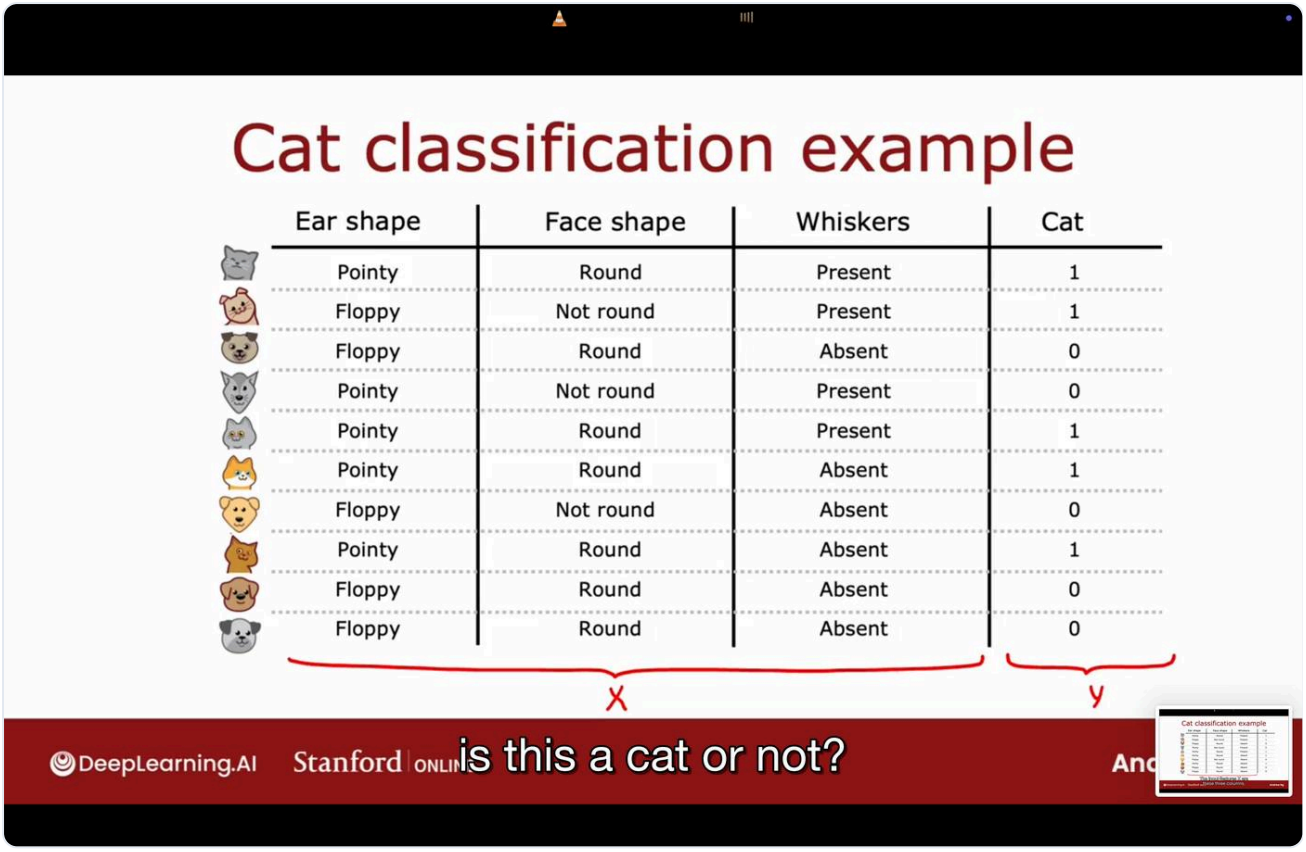


FIGURE 3.2 | Cat Classification Example

The input features X are these three columns, and the target output that you want to predict, Y, is this final column of, is this a cat or not? In this example, the features X take on categorical values.

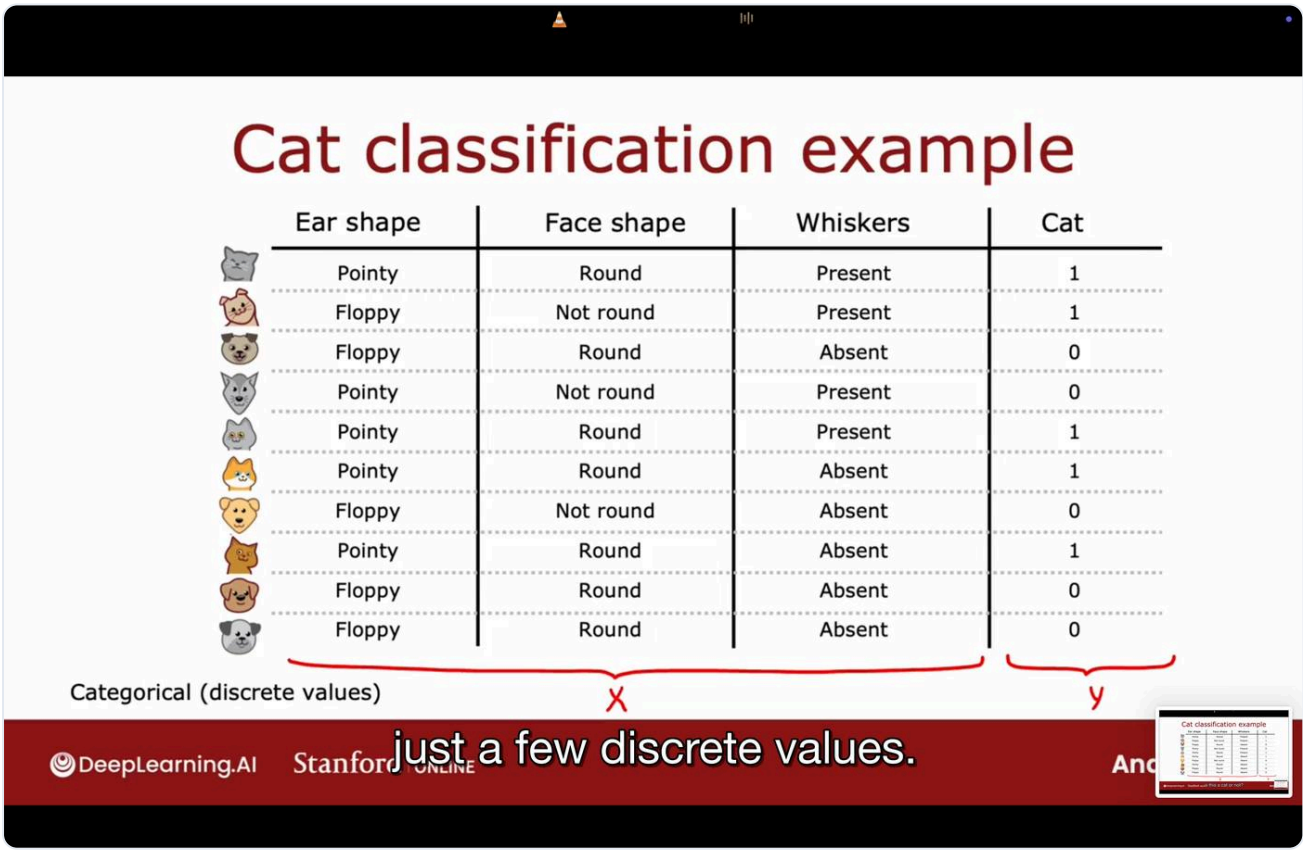


FIGURE 3.3 | Cat Classification Example

In other words, the features take on just a few discrete values. Your shapes are either pointy or floppy. The face shape is either round or not round and whiskers are either present or absent. This is a binary classification task because the labels are also one or zero. For now, each of the features X_1, X_2, and X_3 take on only two possible values. We'll talk about features that can take on more than two possible values, as well as continuous-valued features later in this week. What is a decision tree?

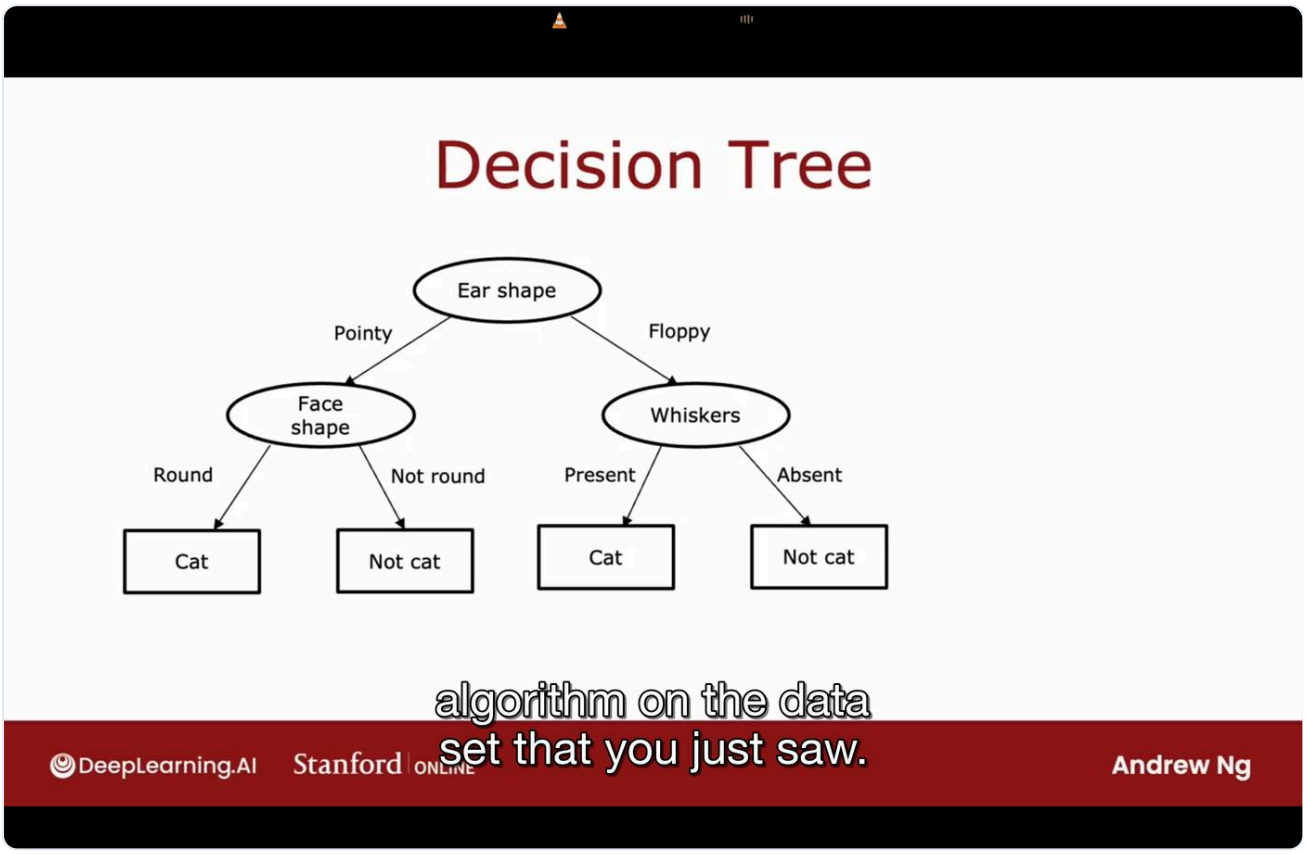


FIGURE 3.4 | Decision Tree

Here's an example of a model that you might get after training a decision tree learning algorithm on the data set that you just saw. The model that is output by the learning algorithm looks like a tree, and a picture like this is what computer scientists call a tree. If it looks nothing like the biological trees that you see out there to you, it's okay, don't worry about it. We'll go through an example to make sure that this computer science definition of a tree makes sense to you as well. Every one of these ovals or rectangles is called a node in the tree. The way this model works is if you have a new test example, she has a cat where the ear-shaped has pointy, face shape is round, and whiskers are present.

The way this model will look at this example and make a classification decision is will start with this example at this topmost node of the tree, this is called the root node of the tree, and we will look at the feature written inside, which is ear shape. Based on the value of the ear shape of this example we'll either go left or go right. The value of the ear-shape with this example is pointy, and so we'll go down the left branch of the tree, like so, and end up at this oval node over here. We then look at the face shape of this example, which turns out to be round, and so we will follow this arrow down over here. The algorithm will make a inference that it thinks this is a cat.

You get to this node and the algorithm will make a prediction that this is a cat.

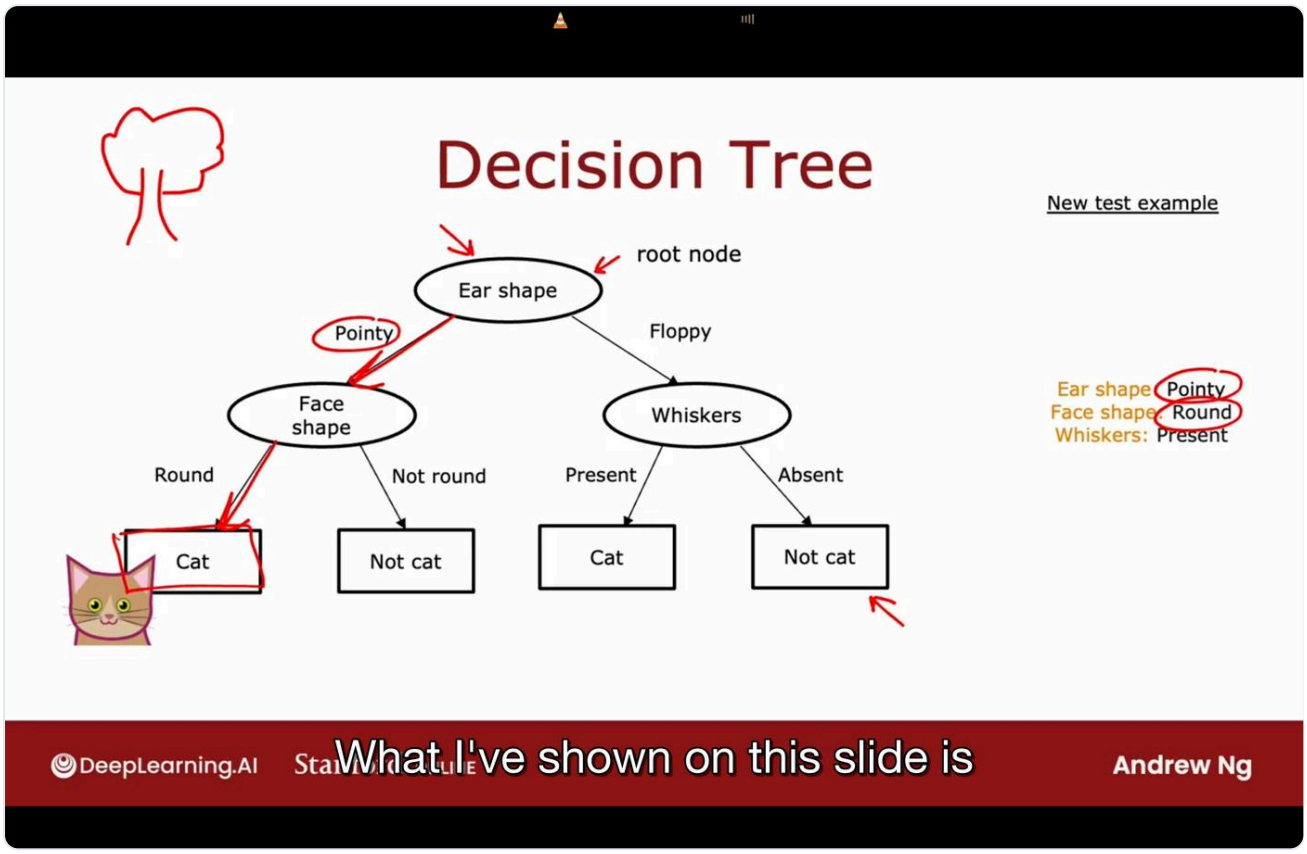


FIGURE 3.5 | Decision Tree

What I've shown on this slide is one specific decision tree model. To introduce a bit more terminology, this top-most node in the tree is called the root node. All of these nodes, that is, all of these oval shapes, but excluding the boxes at the bottom, all of these are called decision nodes. They're decision nodes because they look at a particular feature and then based on the value of the feature, cause you to decide whether to go left or right down the tree. Finally, these nodes at the bottom, these rectangular boxes are called leaf nodes.

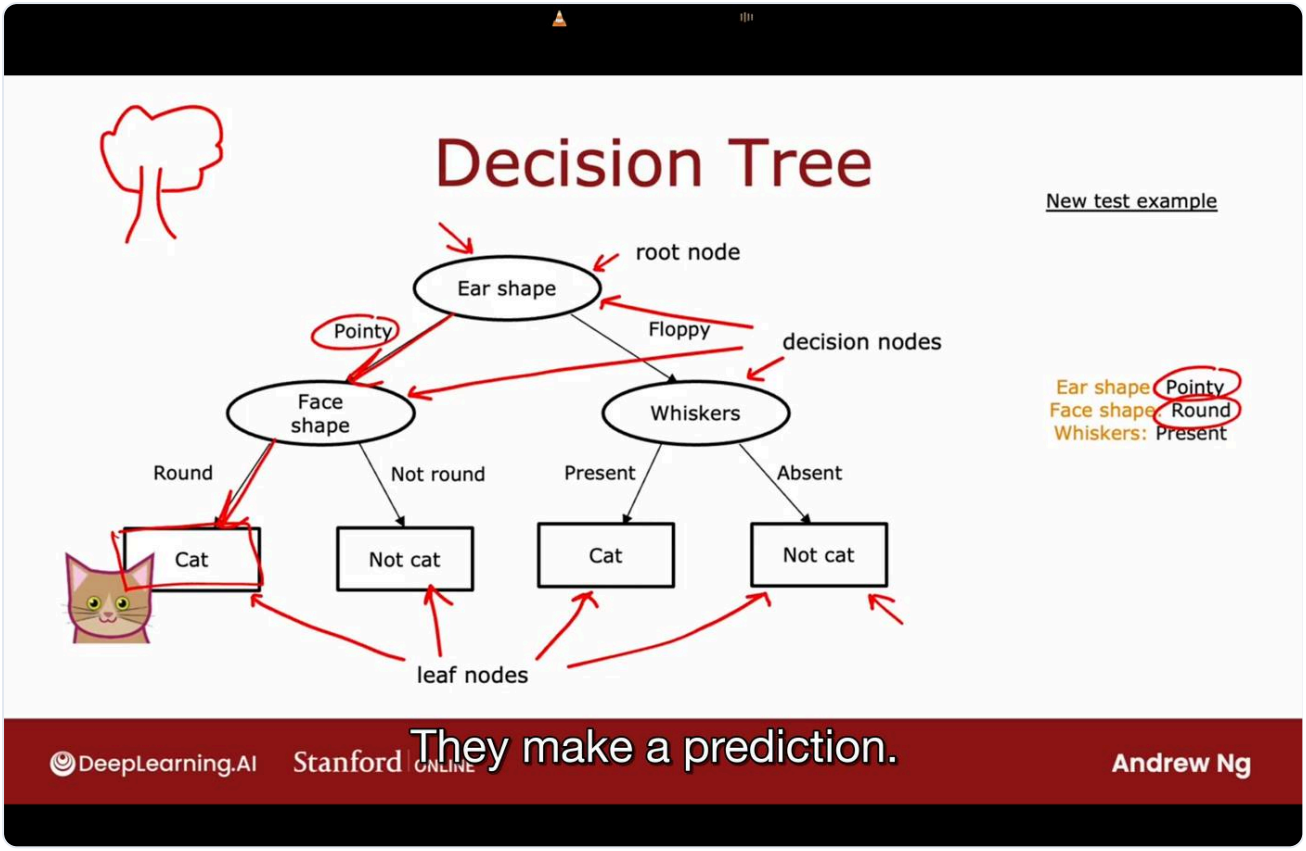


FIGURE 3.6 | Decision Tree

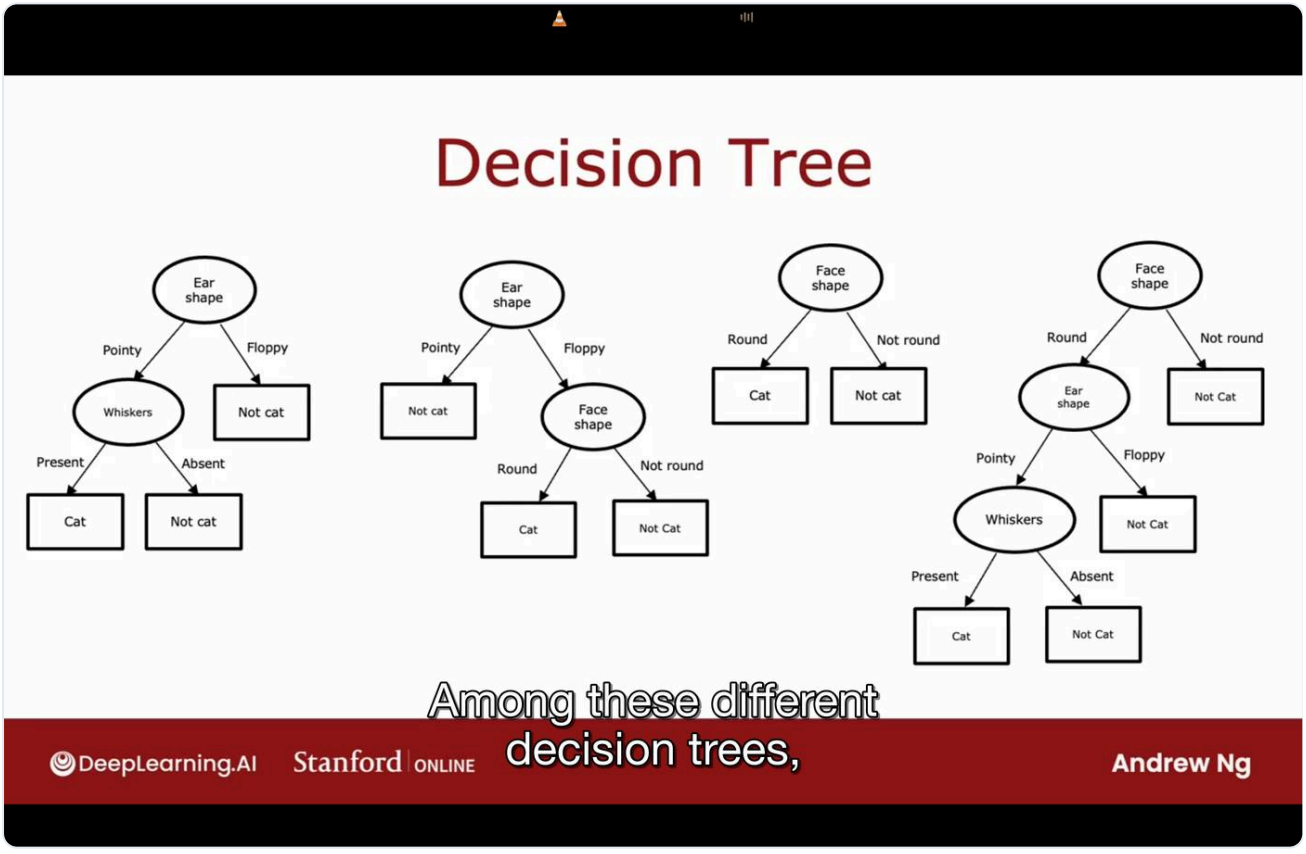


FIGURE 3.7 | Decision Tree

They make a prediction. If you haven't seen computer scientists' definitions of trees before, it may seem non-intuitive that the roots of the tree is at the top and the leaves of the tree are down at the bottom. Maybe one way to think about this is this is more akin to an indoor hanging plant, which is why the roots are up top, and then the leaves tend to fall down to the bottom of the tree. In this slide, I've shown just one example of a decision tree. Here are a few others. This is a different decision tree for trying to classify cat versus not cat. In this tree, to make a classification decision, you would again start at this topmost root node. Depending on their ear shape of an example, you'd go either left or right.

If the ear shape is pointy, then you look at the whiskers feature, and depending on whether whiskers are present or absent, you go left or right to gain and classify cat versus not cat. Just for fun, here's a second example of a decision tree, here's a third one, and here's a fourth one. Among these different decision trees, some will do better and some will do worse on the training sets or on the cross-validation and test sets. The job of the decision tree learning algorithm is, out of all possible decision trees, to try to pick one that hopefully does well on the training set, and then also ideally generalizes well to new data such as your cross-validation and test sets as well.

Seems like there are lots of different decision trees one could build for a given application. How do you get an algorithm to learn a specific decision tree based on a training set? Let's take a look at that in the next video.

LECTURE 51

Decision Tree Learning

The process of building a decision tree given a training set has a few steps. In this video, let's take a look at the overall process of what you need to do to build a decision tree. Given a training set of 10 examples of cats and dogs like you saw in the last video. The first step of decision tree learning is, we have to decide what feature to use at the root node. That is the first node at the very top of the decision tree. Via an algorithm that we'll talk about in the next few videos.

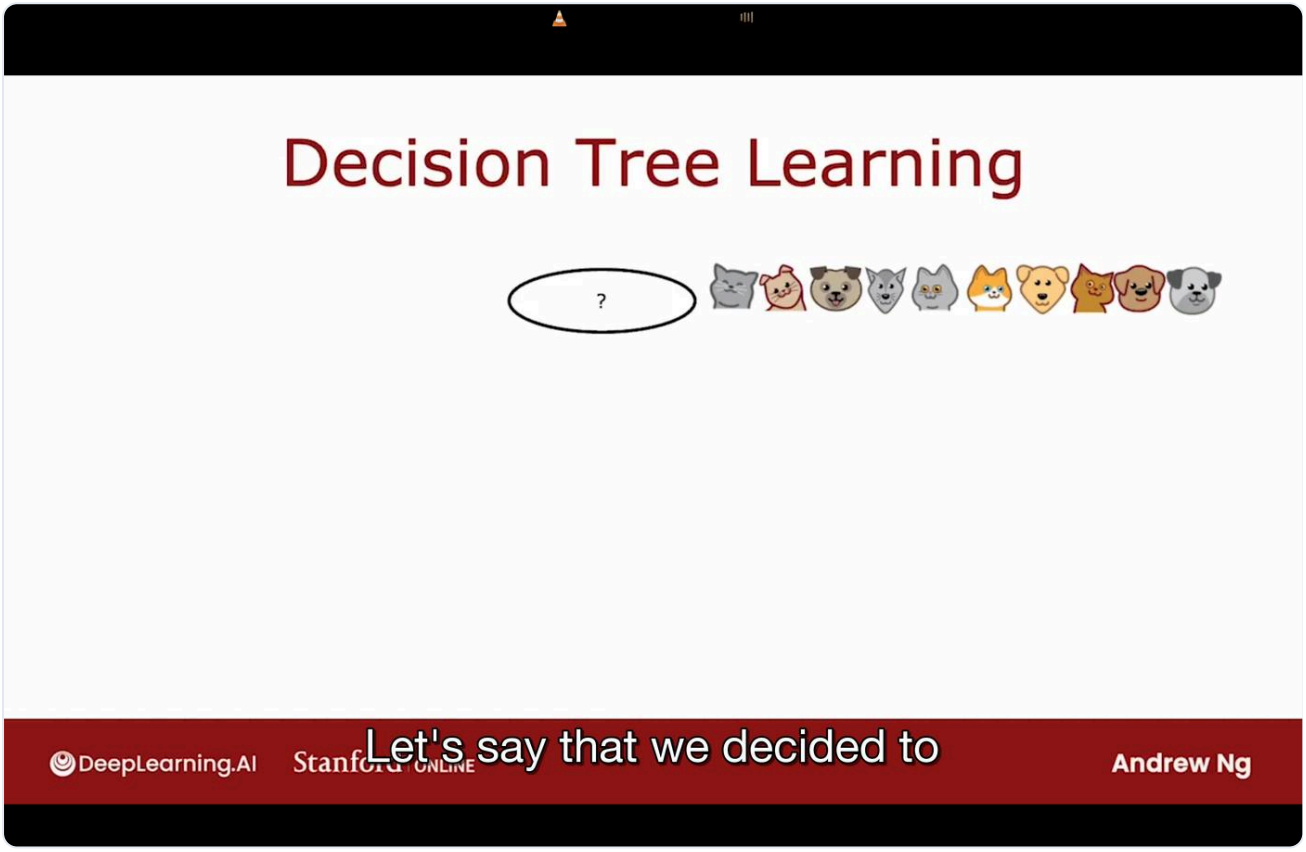


FIGURE 4.1 | Decision Tree Learning

Let's say that we decided to pick as the feature and the root node, the ear shape feature. What that means is we will decide to look at all of our training examples, all ten examples shown here, I split them according to the value of the ear shape feature. In particular, let's pick out the five examples with pointy ears and move them over down to the left. Let's pick the five examples with floppy ears and move them down to the right.



FIGURE 4.2 | Decision Tree Learning

The second step is focusing just on the left part or sometimes called the left branch of the decision tree to decide what nodes to put over there. In particular, what feature that we want to split on or what feature do we want to use next. Via an algorithm that again, we'll talk about later this week. Let's say you decide to use the face shape feature there. What we'll do now is take these five examples and split these five examples into two subsets based on their value of the face shape. We'll take the four examples out of these five with a round face shape and move them down to the left.

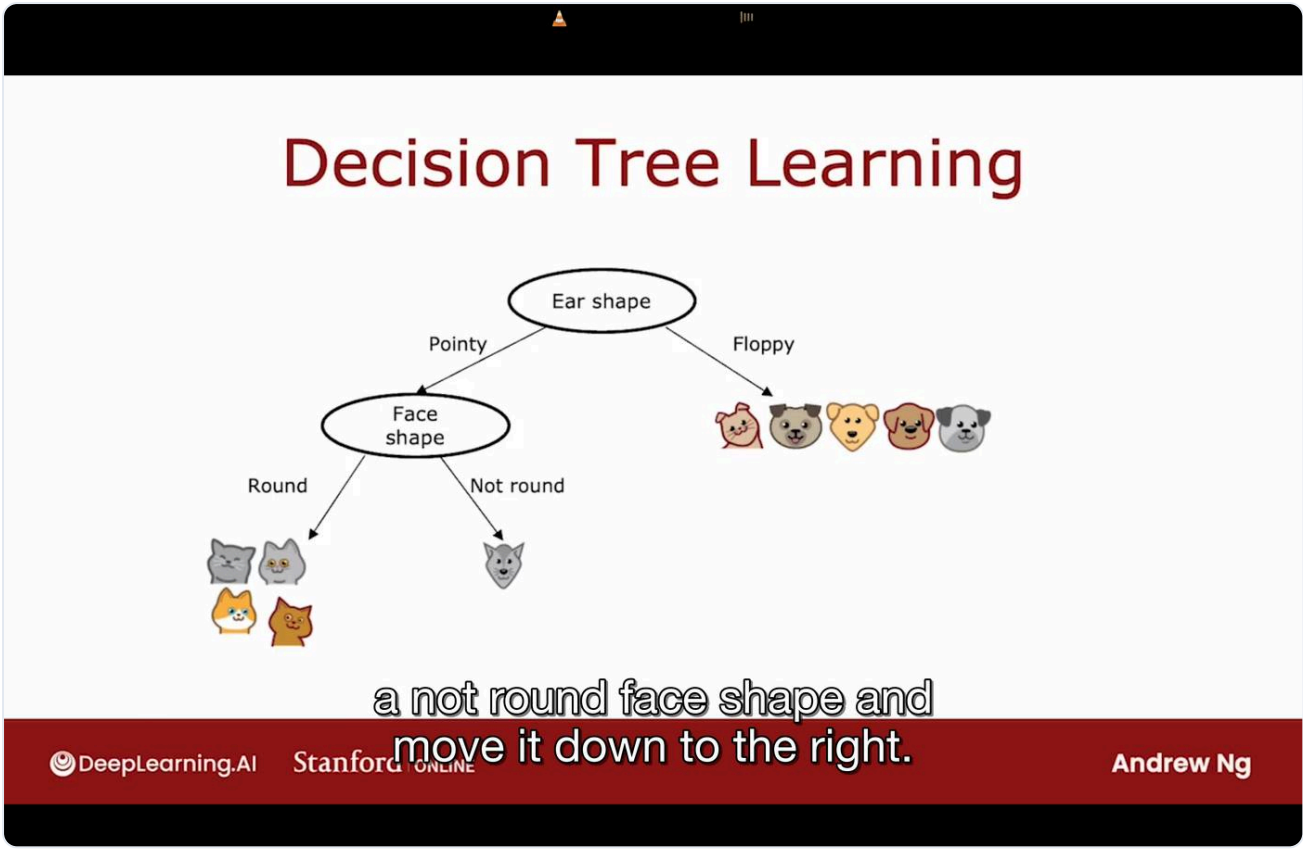


FIGURE 4.3 | Decision Tree Learning

The one example with a not round face shape and move it down to the right. Finally, we notice that these four examples are all cats four of them are cats. Rather than splitting further, were created a leaf node that makes a prediction that things that get down to that node of cats. Over here we notice that none of the examples zero of the one examples are cats or alternative 100 percent of the examples here are dogs. We can create a leaf node here that makes a prediction of not cat.

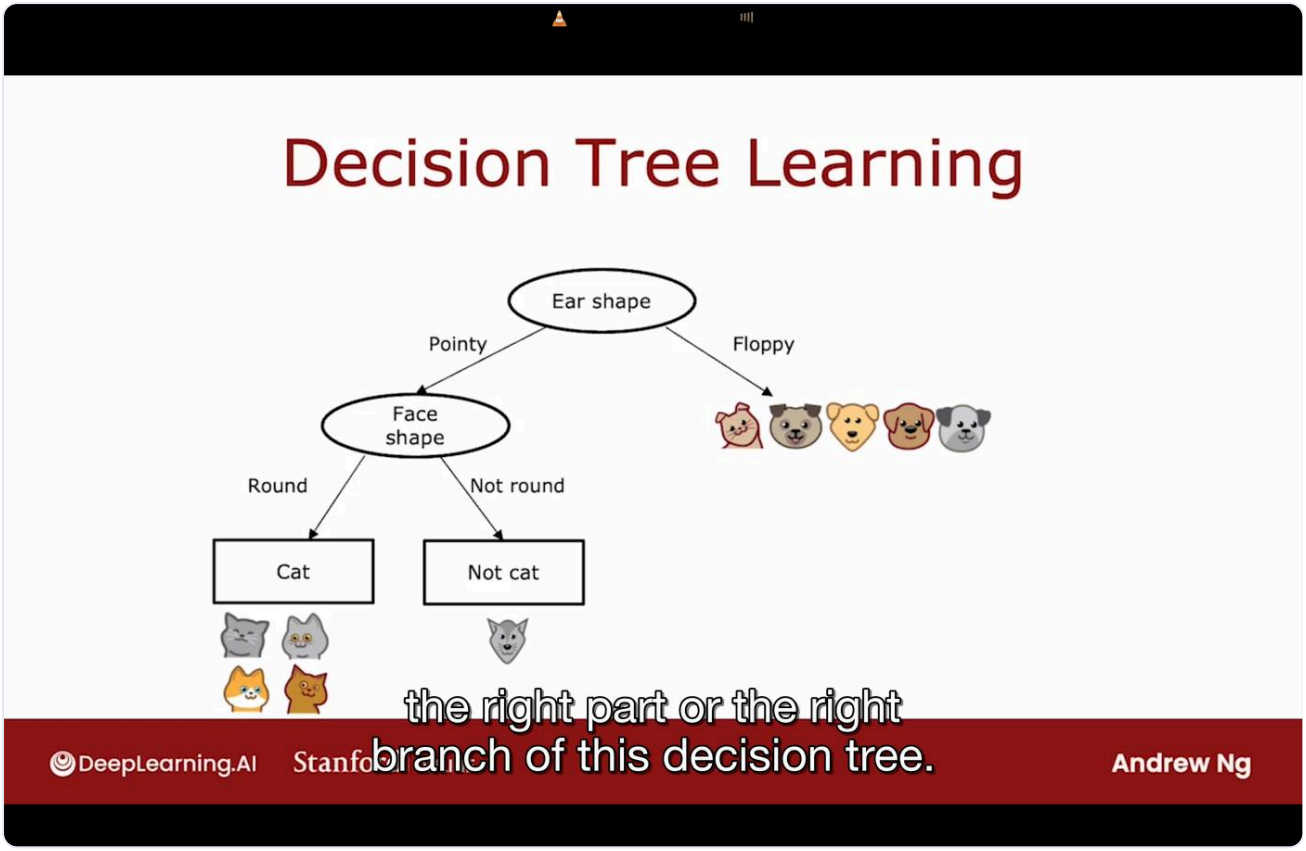


FIGURE 4.4

Having done this on the left part to the left branch of this decision tree, we now repeat a similar process on the right part or the right branch of this decision tree.

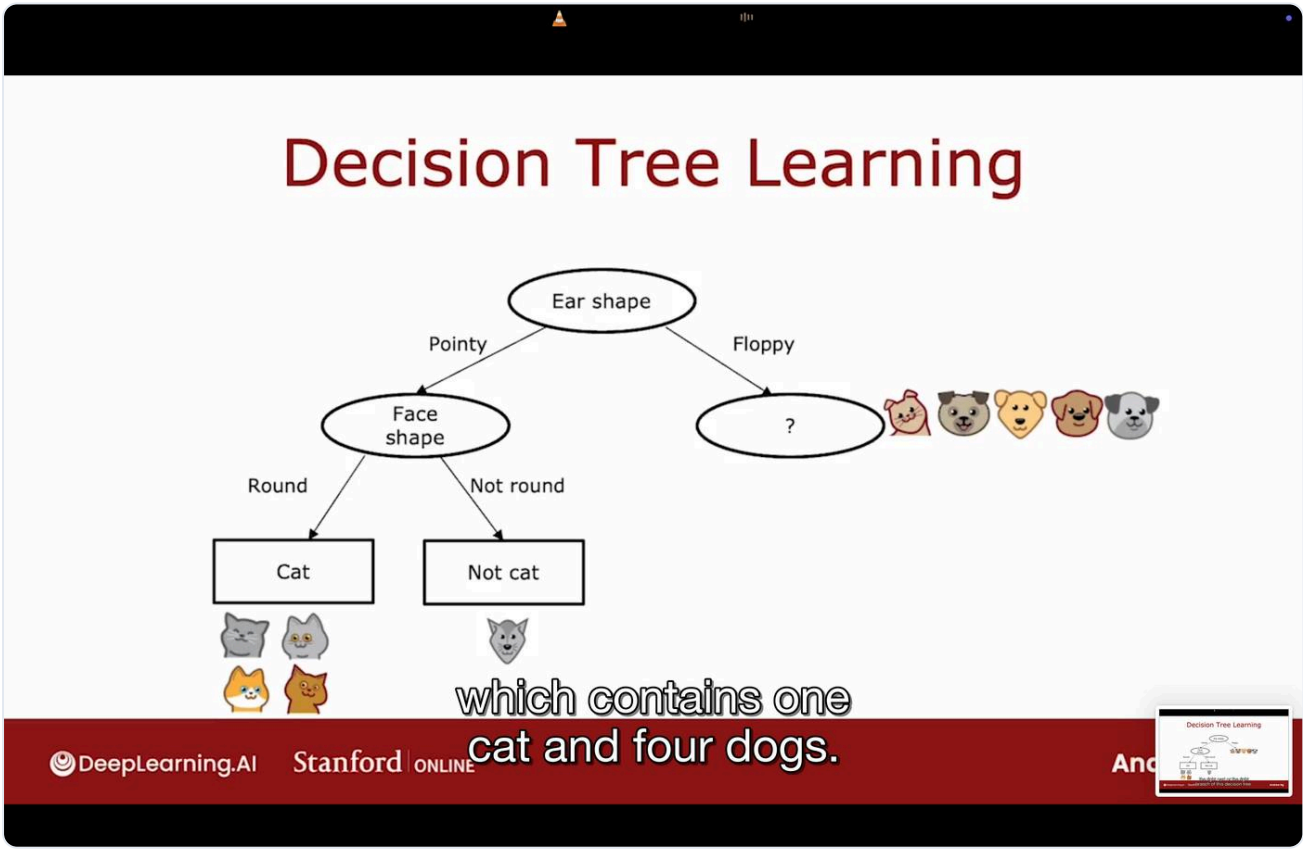


FIGURE 4.5 | Decision Tree Learning

Focus attention on just these five examples, which contains one cat and four dogs. We would have to pick some feature over here to use the split these five examples further, if we end up choosing the whiskers feature, we would then split these five examples based on where the whiskers are present or absent, like so.

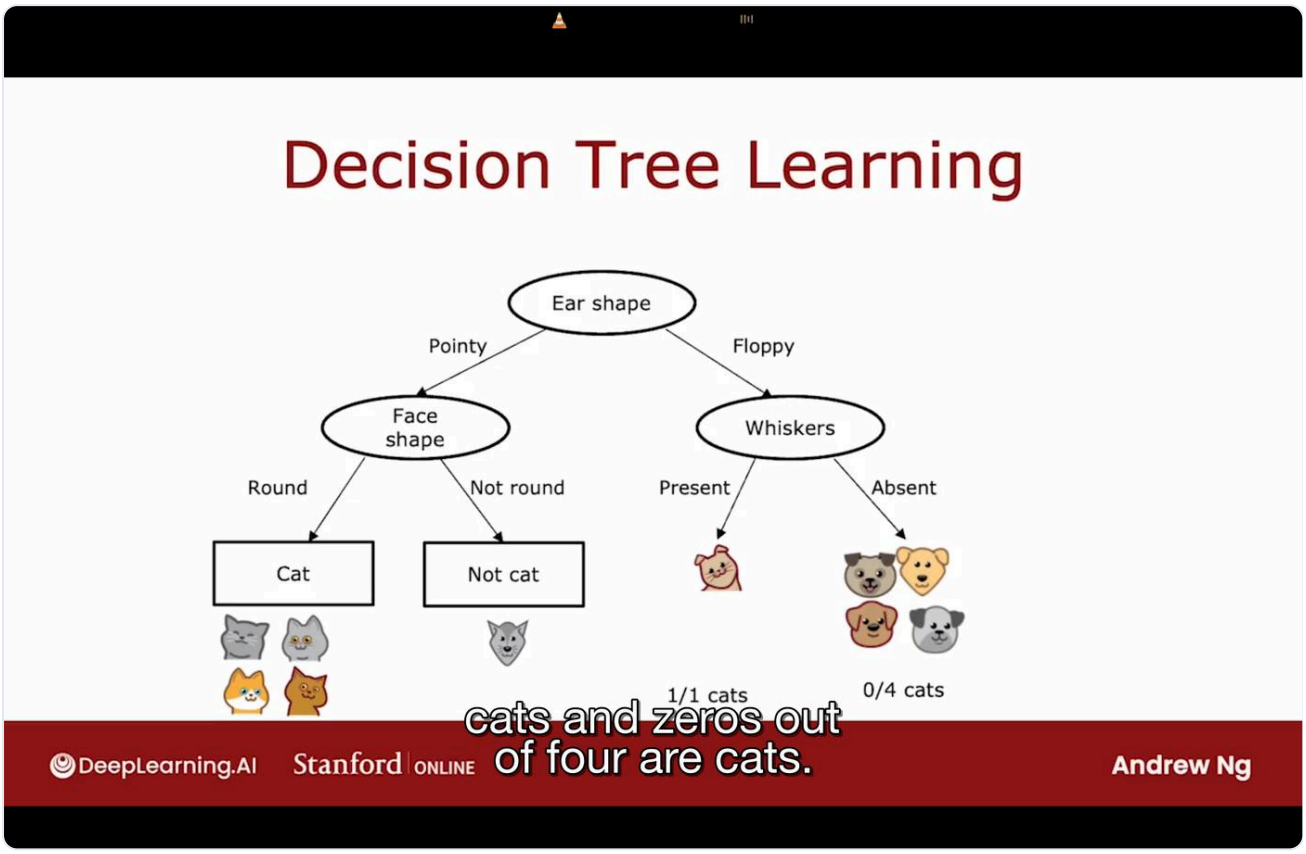


FIGURE 4.6 | Decision Tree Learning

You notice that one out of one examples on the left for cats and zeros out of four are cats. Each of these nodes is completely pure, meaning that is, all cats or not cats and there's no longer a mix of cats and dogs. We can create these leaf nodes, making a cat prediction on the left and a nightcap prediction here on the right. This is a process of building a decision tree. Through this process, there were a couple of key decisions that we had to make at various steps during the algorithm.

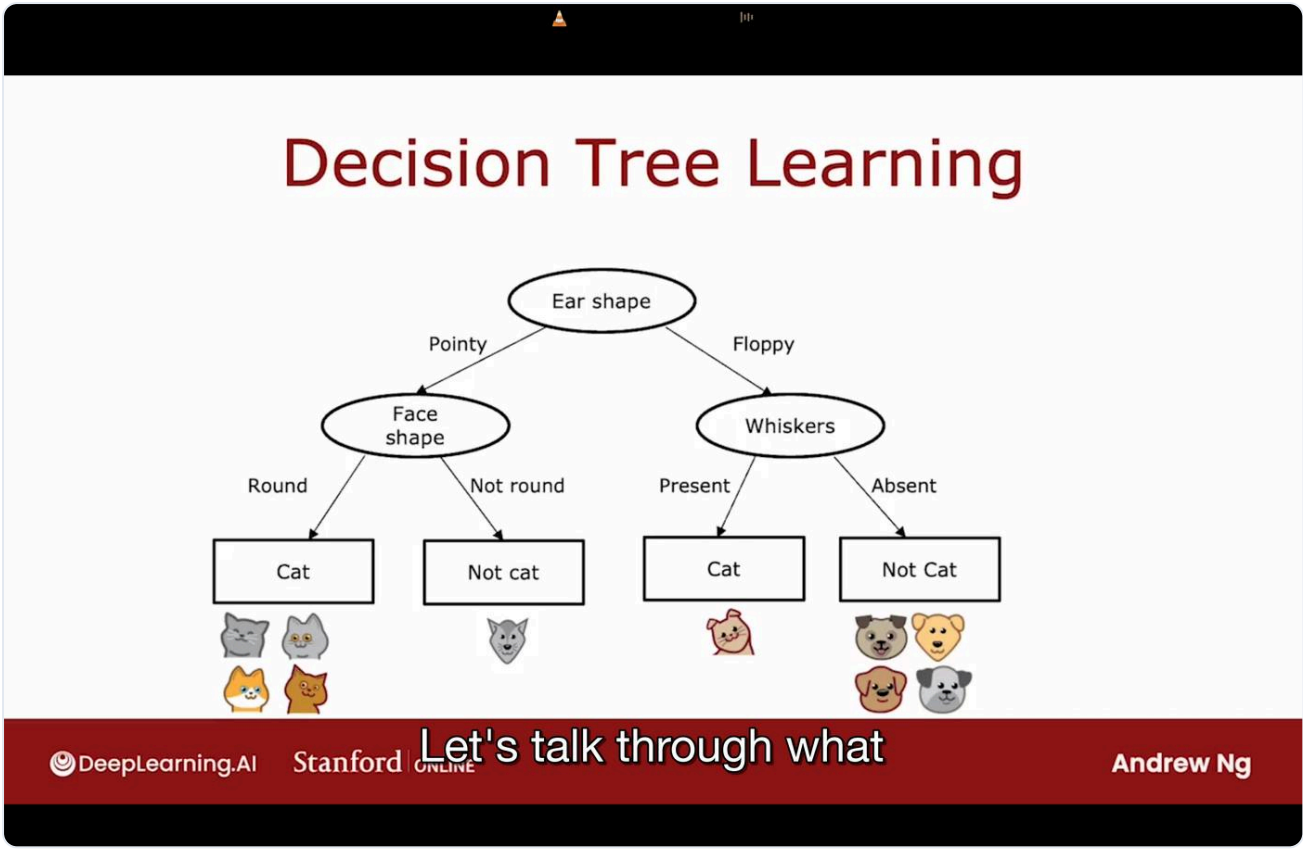


FIGURE 4.7 | Decision Tree Learning

Let's talk through what those key decisions were and we'll keep on session of the details of how to make these decisions in the next few videos. The first key decision was, how do you choose what features to use to split on at each node? At the root node, as well as on the left branch and the right branch of the decision tree, we had to decide if there were a few examples at that node comprising a mix of cats and dogs. Do you want to split on the ear-shaped feature or the facial feature or the whiskers feature? We'll see in the next video, that decision trees will choose what feature to split on in order to try to maximize purity. By purity, I mean, you want to get to what subsets, which are as close as possible to all cats or all dogs.

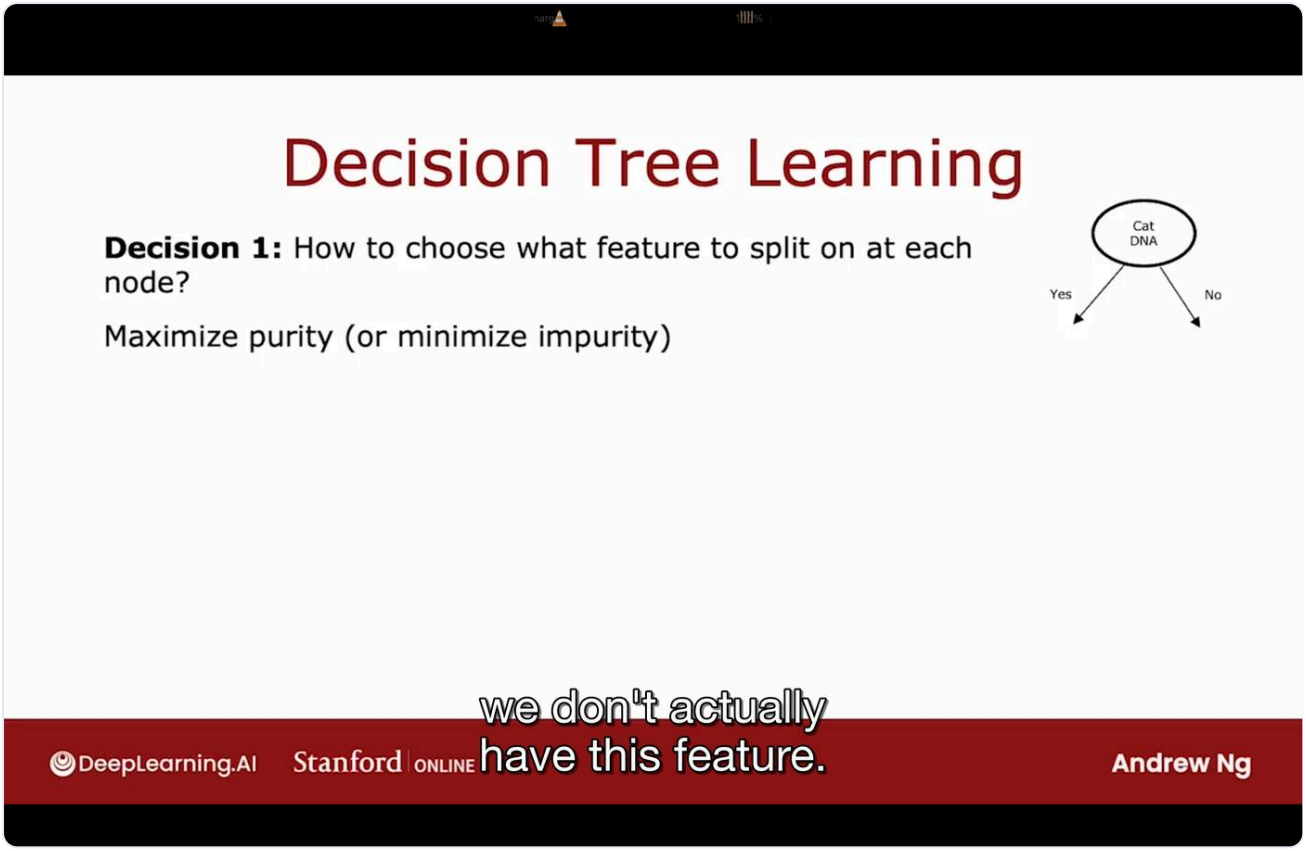


FIGURE 4.8 | Decision Tree Learning

For example, if we had a feature that said, does this animal have cat DNA, we don't actually have this feature. But if we did, we could have split on this feature at the root node, which would have resulted in five out of five cats in the left branch and zero of the five cats in the right branch.

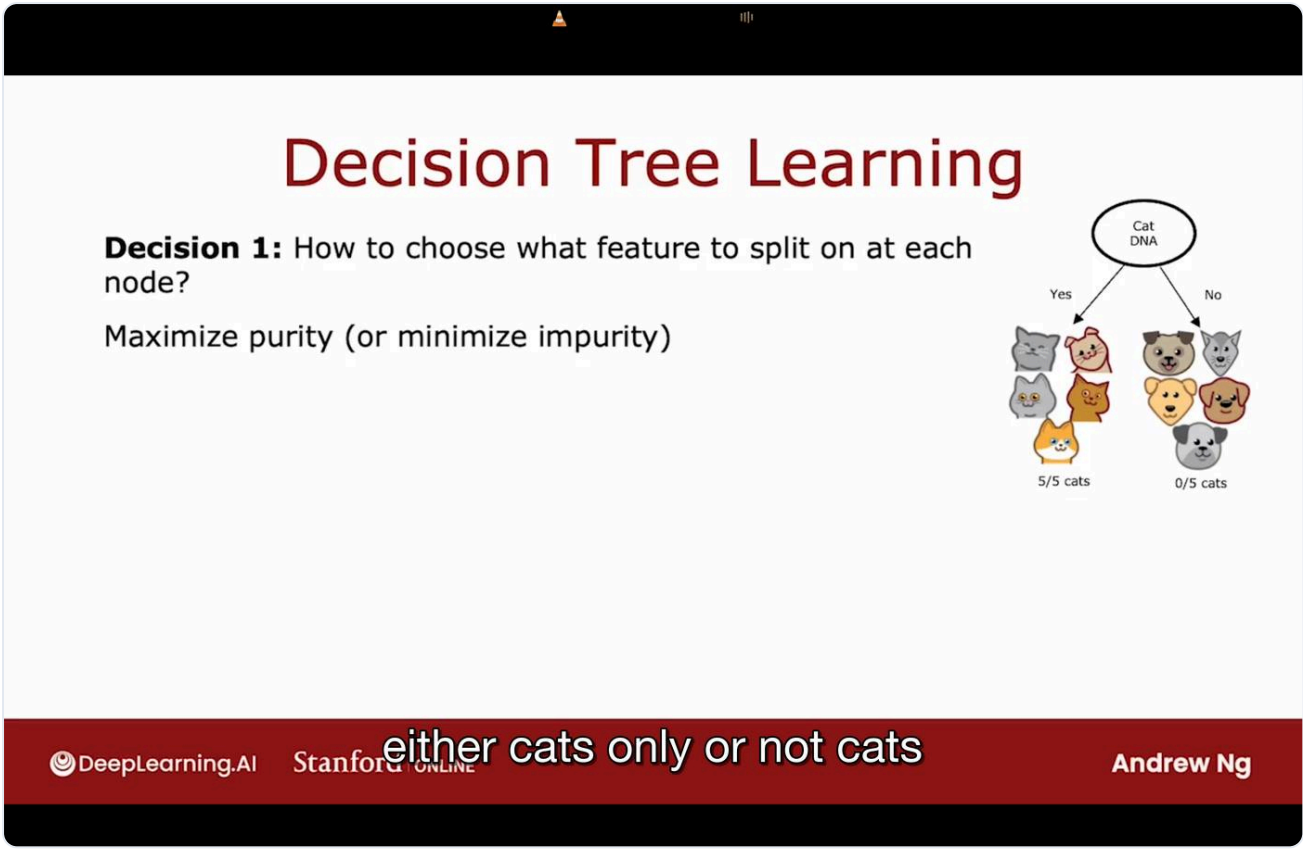


FIGURE 4.9 | Decision Tree Learning

Both these left and right subsets of the data are completely pure, meaning that there's only one class, either cats only or not cats only in both of these left and right sub-branches, which is why the cat DNA feature if we had this feature, would have been a great feature to use.

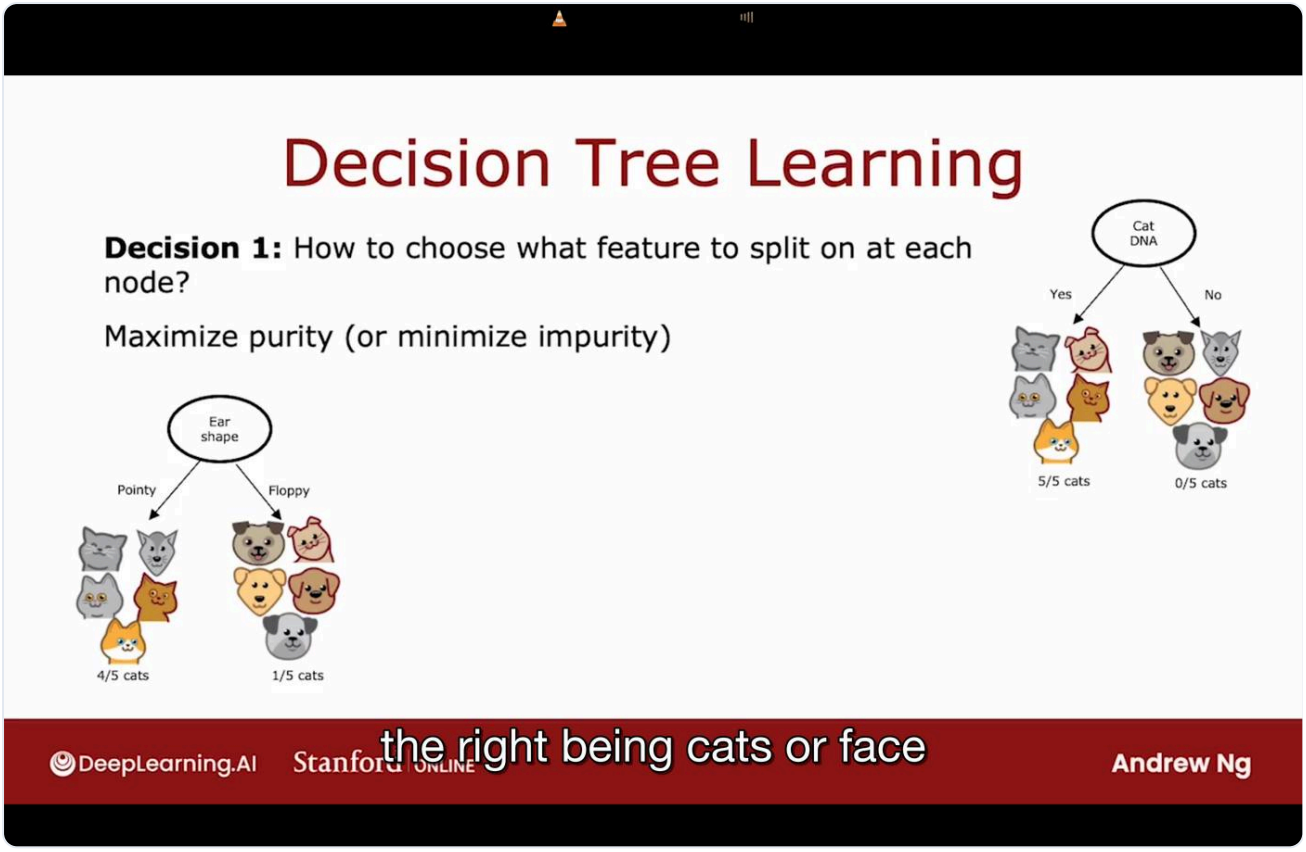


FIGURE 4.10 | Decision Tree Learning

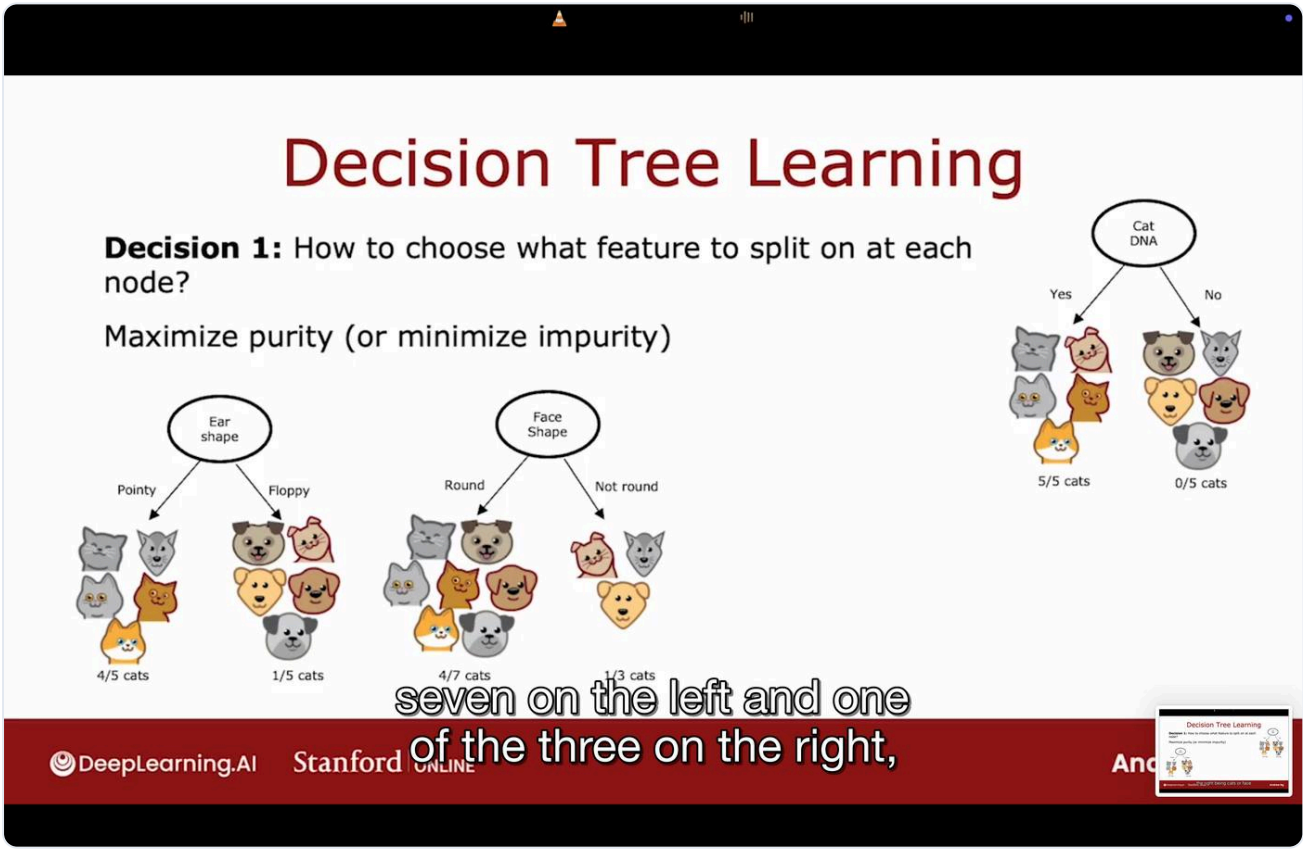


FIGURE 4.11 | Decision Tree Learning

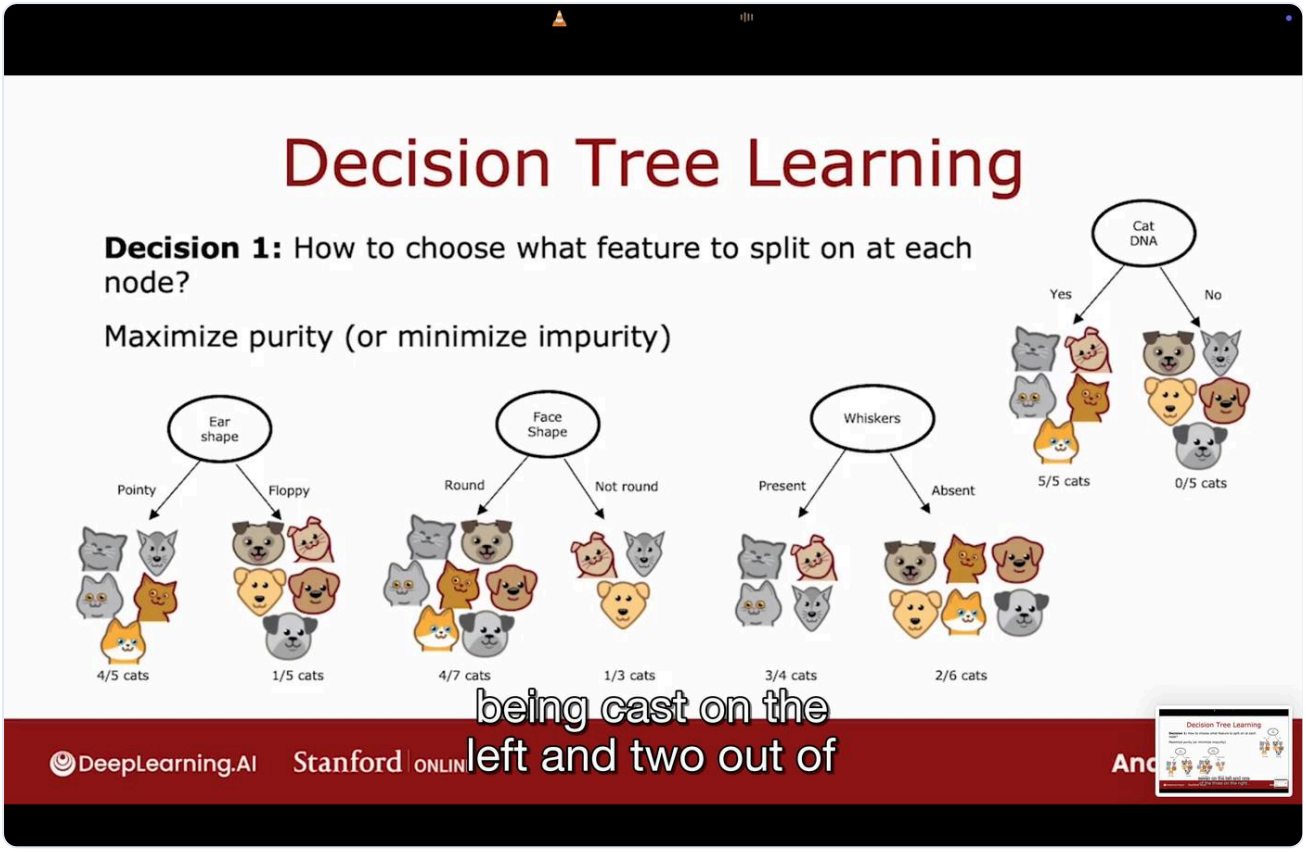


FIGURE 4.12 | Decision Tree Learning

But with the features that we actually have, we had to decide, what is the split on year shape, which result in four out of five examples on the left being cats, and one of the five examples on the right being cats or face shape where it resulted in the four of the seven on the left and one of the three on the right, or whiskers, which resulted in three out four examples being cast on the left and two out of six being not cats on the right. The decision tree learning algorithm has to choose between ear-shaped, face shape, and whiskers. Which of these features results in the greatest purity of the labels on the left and right sub branches? Because it is if you can get to a highly pure subsets of examples, then you can either predict cat or predict not cat and get it mostly right.

The next video on entropy, we'll talk about how to estimate impurity and how to minimize impurity. The first decision we have to make when learning a decision tree is how to choose which feature to split on on each node. The second key decision you need to make when building a decision tree is to decide when do you stop splitting. The criteria that we use just now was until I know there's either 100 percent, all cats or a 100 percent of dogs and not cats. Because at that point is seems natural to build a leaf node that just makes a classification prediction. Alternatively, you might also decide to stop splitting when splitting and no further results in the tree exceeding the maximum depth.

Where the maximum depth that you allow the tree to go to, is a parameter that you could just say. In decision tree, the depth of a node is defined as the number of hops that it takes to get from the root node that is denoted the very top to that particular node. So the root node takes zero hops, to get to itself and is at Depth 0.

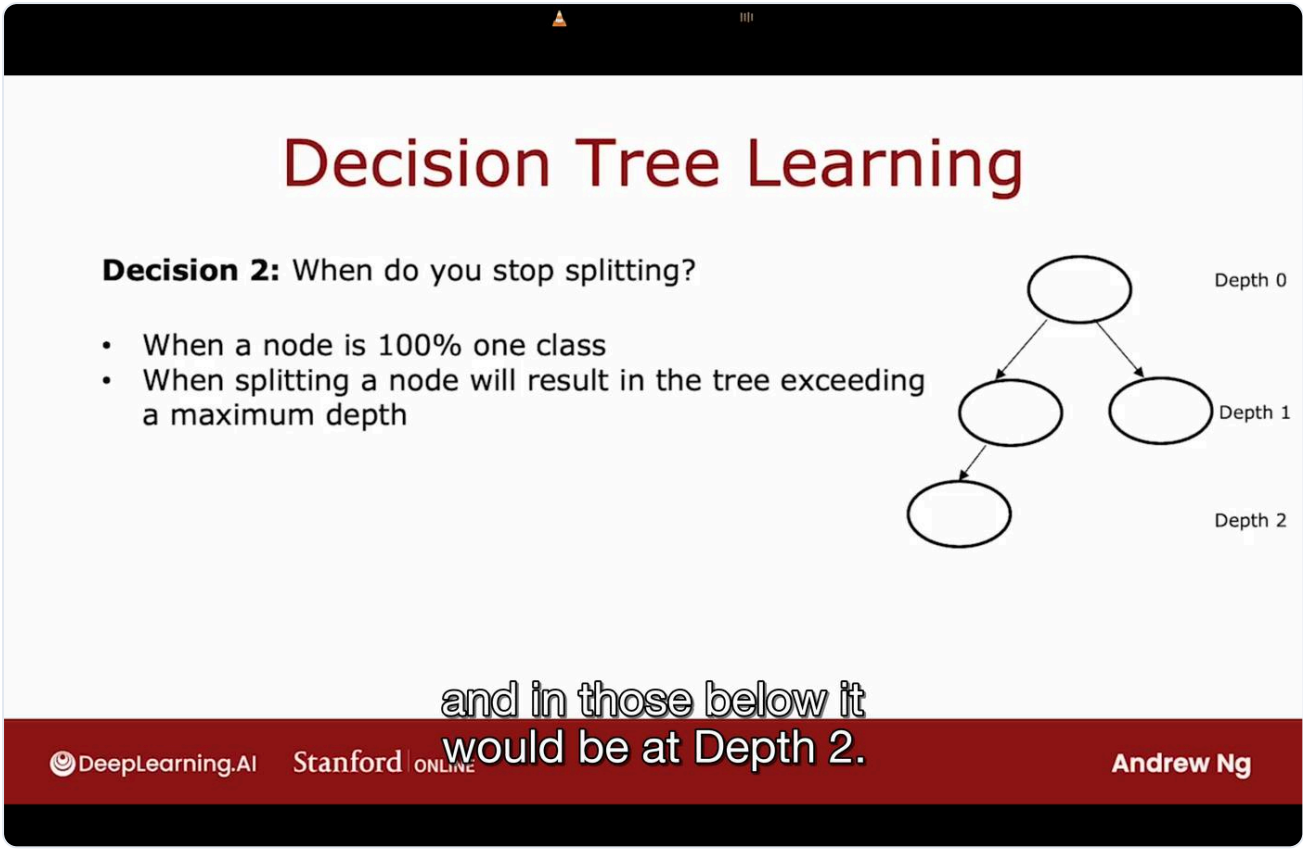


FIGURE 4.13 | Decision Tree Learning

The notes below it are at depth one and in those below it would be at Depth 2. If you had decided that the maximum depth of the decision tree is say two, then you would decide not to split any nodes below this level so that the tree never gets to Depth 3. One reason you might want to limit the depth of the decision tree is to make sure for us to tree doesn't get too big and unwieldy and second, by keeping the tree small, it makes it less prone to overfitting. Another criteria you might use to decide to stop splitting might be if the improvements in the purity score, which you see in a later video of below a certain threshold.

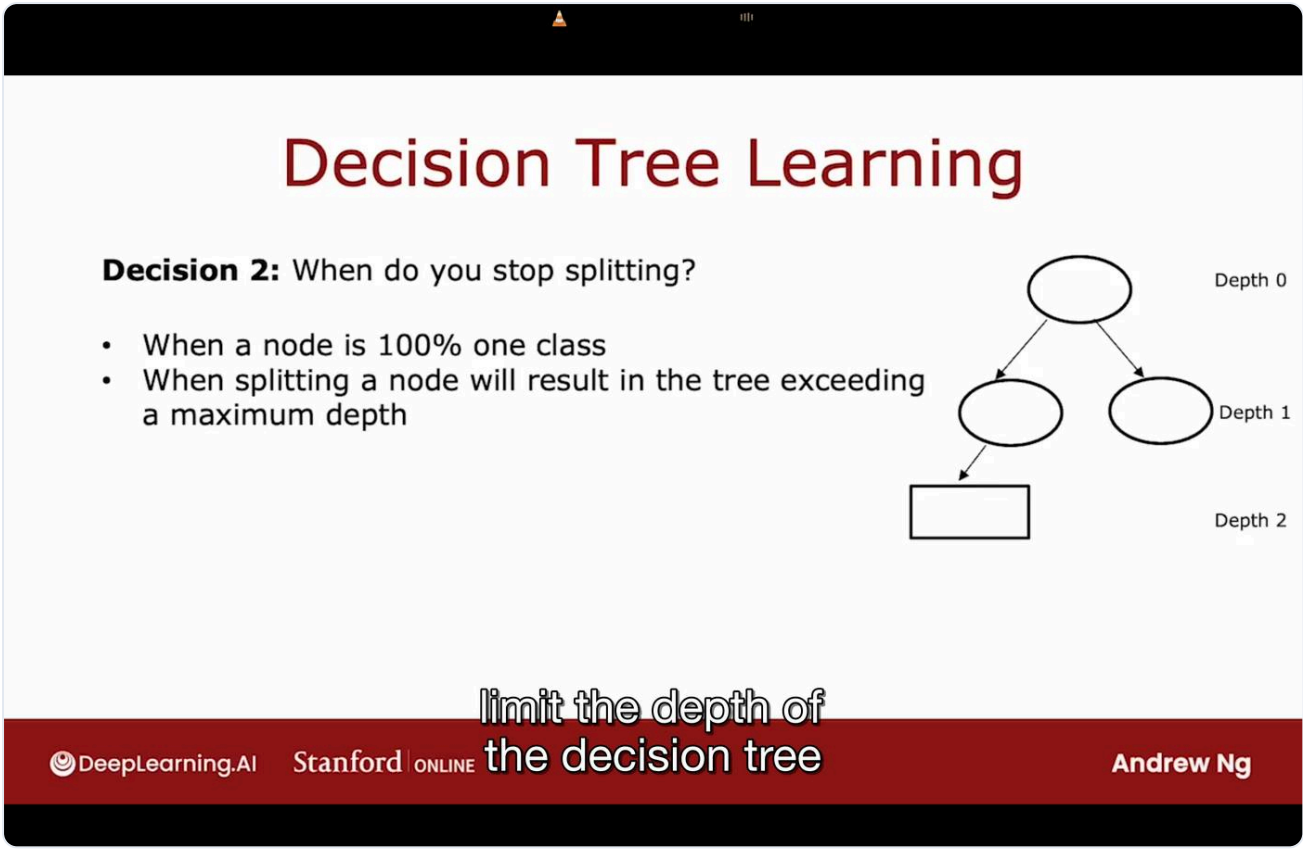


FIGURE 4.14 | Decision 2

If splitting a node results in minimum improvements to purity or you see later is actually decreases in impurity. But if the gains are too small, they might not bother. Again, both to keep the trees smaller and to reduce the risk of overfitting.

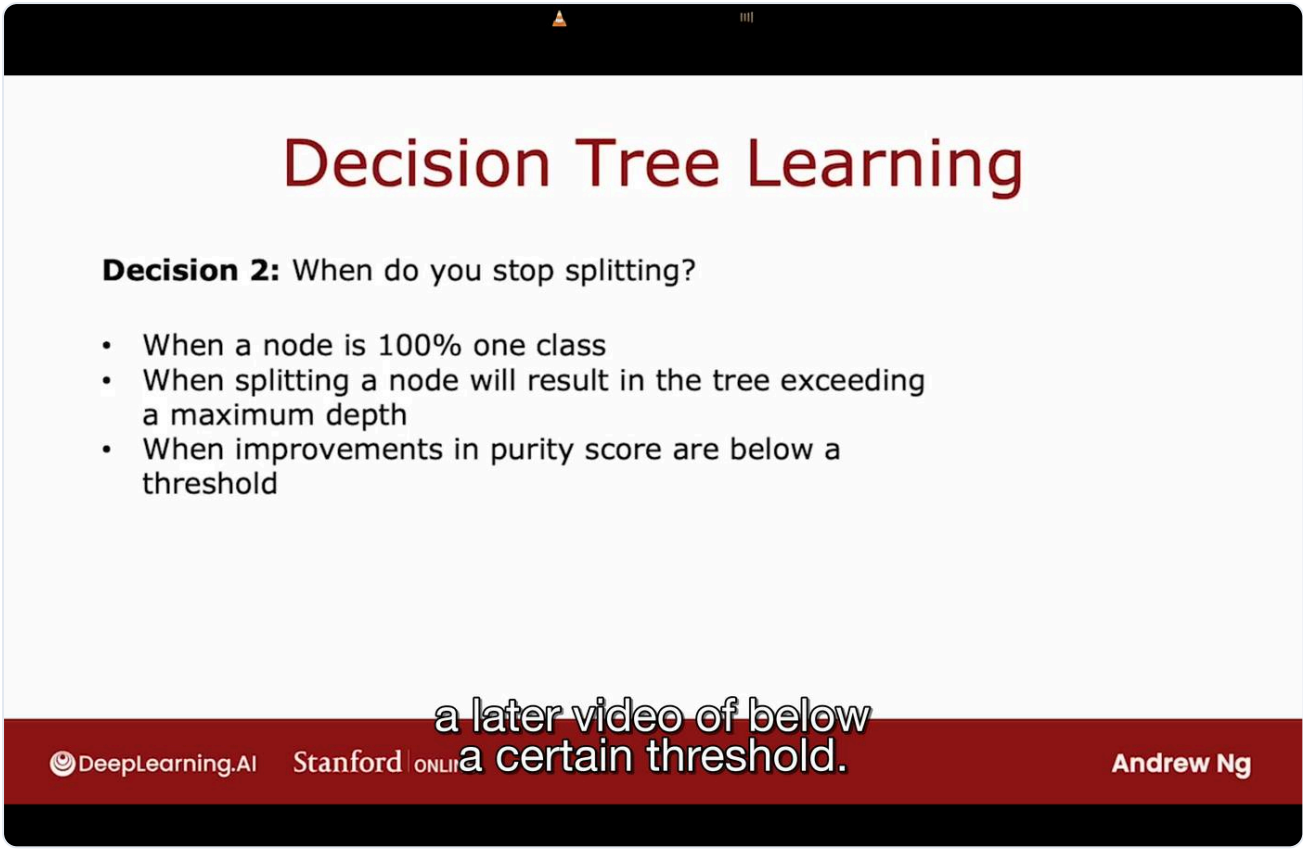


FIGURE 4.15 | Decision Tree Learning

Finally, if the number of examples that a node is below a certain threshold, then you might also decide to stop splitting.

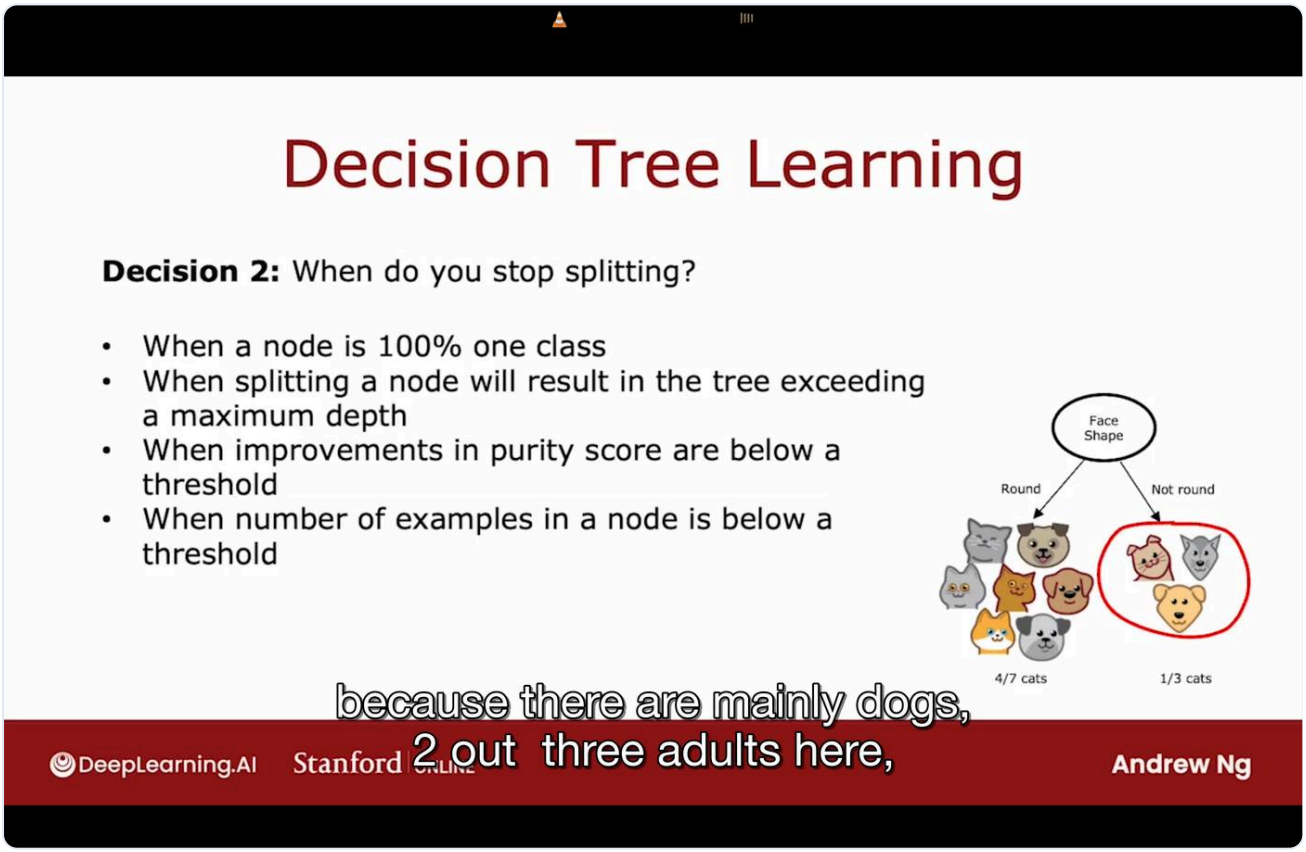


FIGURE 4.16 | Decision Tree Learning

For example, if at the root node we have split on the face shape feature, then the right branch will have had just three training examples with one cat and two dogs and rather than splitting this into even smaller subsets, if you decided not to split further set of examples with just three of your examples, then you will just create a decision node and because there are mainly dogs, 2 out three adults here, this would be a node and this makes a prediction of not cat.

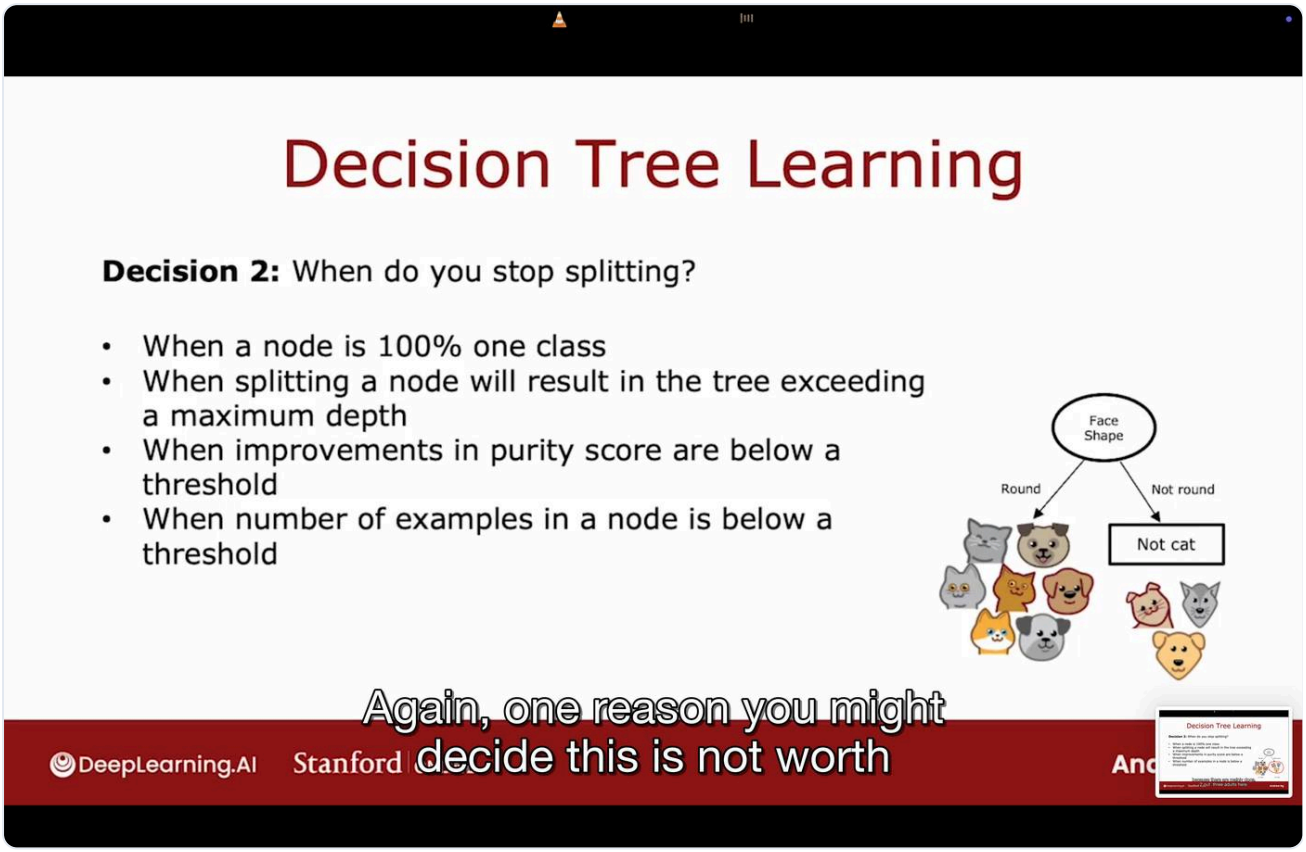


FIGURE 4.17 | Decision Tree Learning

Again, one reason you might decide this is not worth splitting on is to keep the tree smaller and to avoid overfitting. When I look at decision tree learning algorithms myself, sometimes I feel like, boy, there's a lot of different pieces and lots of different things going on in this algorithm. Part of the reason it might feel is in the evolution of decision trees. There was one researcher that proposed a basic version of decision trees and then a different researcher said, oh, we can modify this thing this way, such as his new criteria for splitting. Then a different researcher comes up with a different thing like, oh, maybe we should stop splitting when it reaches a certain maximum depth.

Over the years, different researchers came up with different refinements to the algorithm. As a result of that, it does work really well but we look at all the details of how to implement a decision tree. It feels a lot of different pieces such as why there's so many different ways to decide when to stop splitting. If it feels like a somewhat complicated, messy algorithm to you, it does to me too. But these different pieces, they do fit together into a very effective learning algorithm and what you learn in this course is the key, most important ideas for how to make it work well, and then at the end of this week, I'll also share with you some guidance, some suggestions for how to use open source packages so that you don't have to have too complicated the procedure for making all these decisions.

Like how do I decide to stop splitting? You really get these algorithms to work well for yourself. But I want to reassure you that if this algorithm seems complicated and messy, it frankly does to me too, but it does work well. Now, the next key decision that I want to dive more deeply into is how do you decide how to split a node. In the next video, let's take a look at this definition of entropy, which would be a way for us to measure purity, or more precisely, impurity in a node. Let's go on to the next video.

LECTURE 52

Entropy as a Measure of Impurity

In this video, we'll look at the way of measuring the purity of a set of examples. If the examples are all cats of a single class then that's very pure, if it's all not cats that's also very pure, but if it's somewhere in between how do you quantify how pure is the set of examples? Let's take a look at the definition of entropy, which is a measure of the impurity of a set of data. Given a set of six examples like this, we have three cats and three dogs, let's define p_1 to be the fraction of examples that are cats, that is, the fraction of examples with label one, that's what the subscript one indicates. p_1 in this example is equal to 3/6.

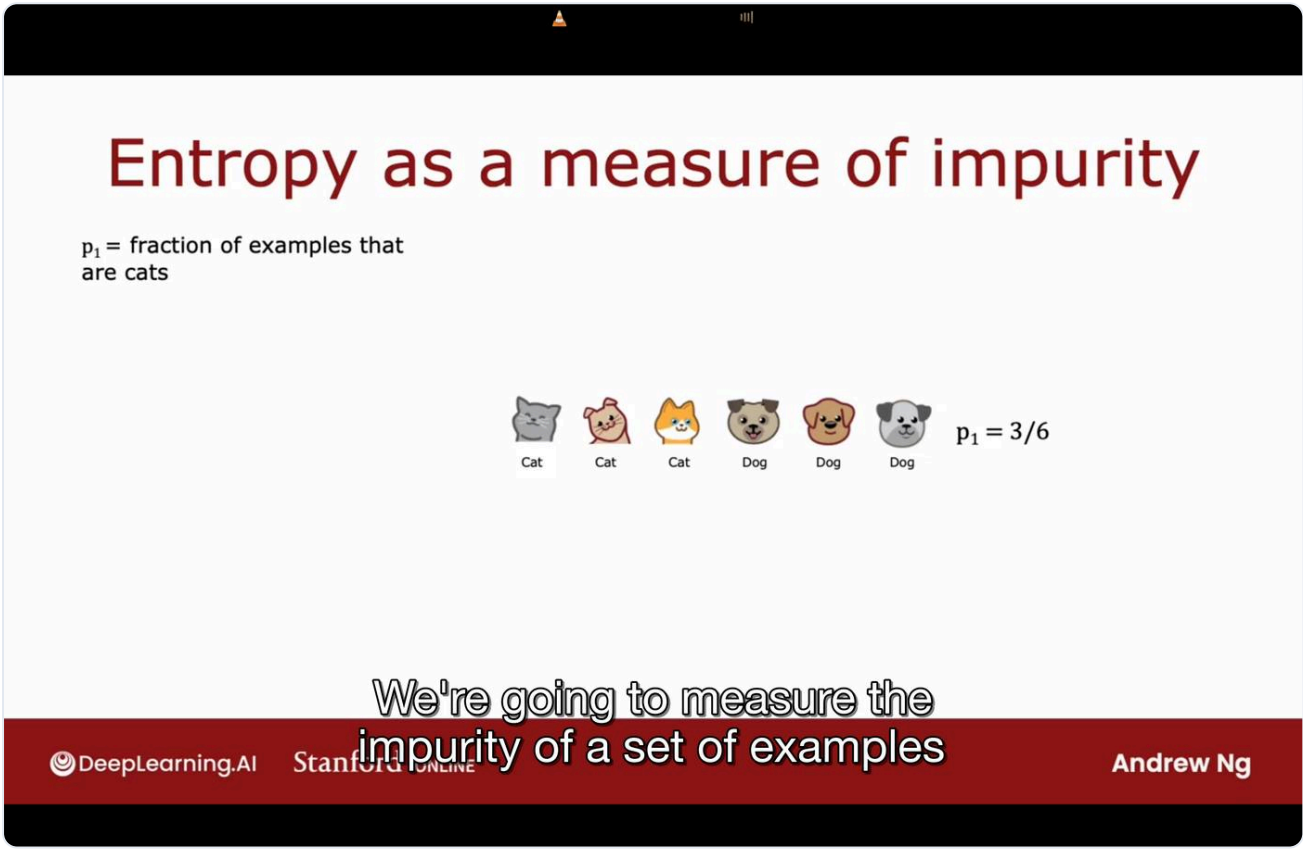


FIGURE 5.1 | Entropy as a Measure of Impurity

We're going to measure the impurity of a set of examples using a function called the entropy which looks like this.

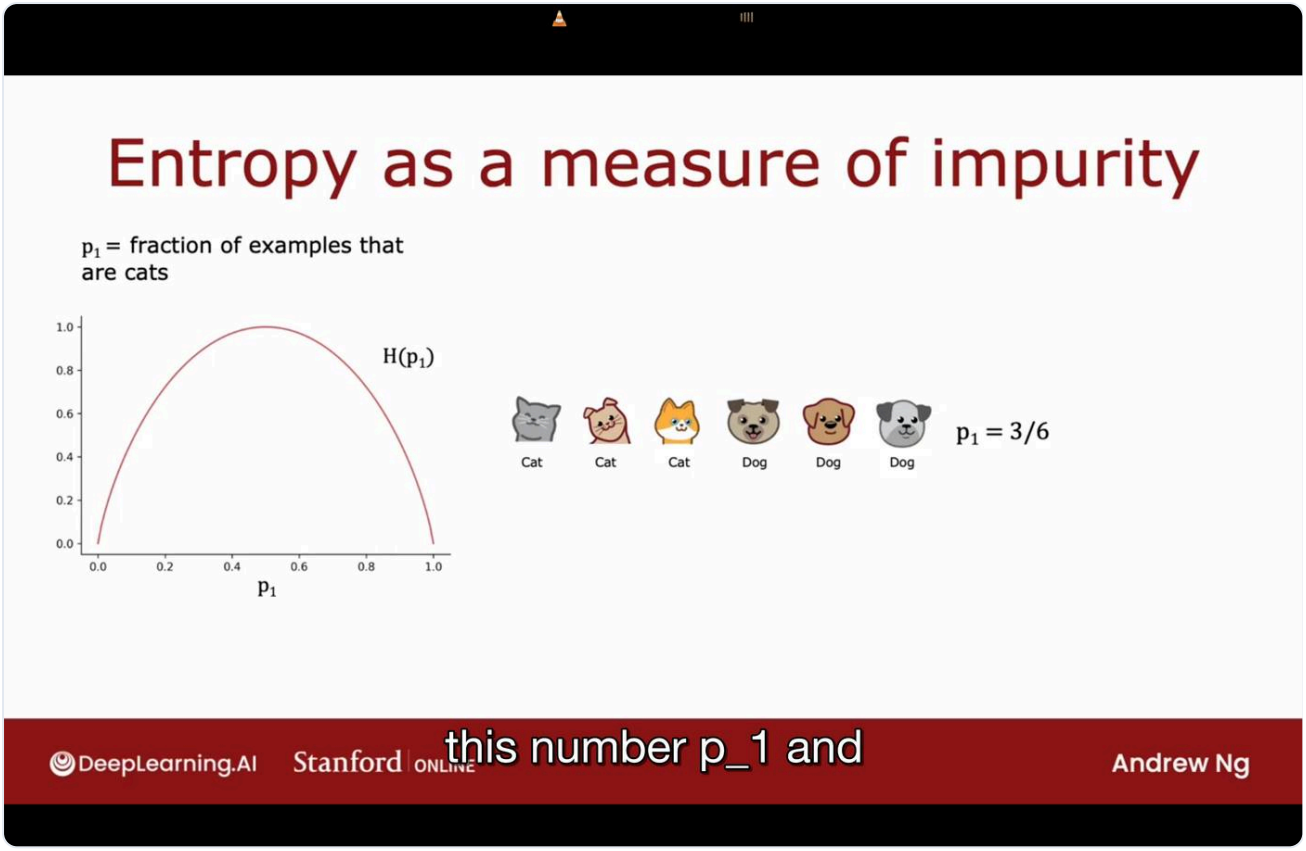


FIGURE 5.2 | Entropy as a Measure of Impurity

The entropy function is conventionally denoted as capital H of this number p_1 and the function looks like this curve over here where the horizontal axis is p_1, the fraction of cats in the sample, and the vertical axis is the value of the entropy. In this example where p_1 is 3/6 or 0.5, the value of the entropy of p_1 would be equal to one. You notice that this curve is highest when your set of examples is 50-50, so it's most impure as an impurity of one or with an entropy of one when your set of examples is 50-50, whereas in contrast if your set of examples was either all cats or not cats then the entropy is zero.

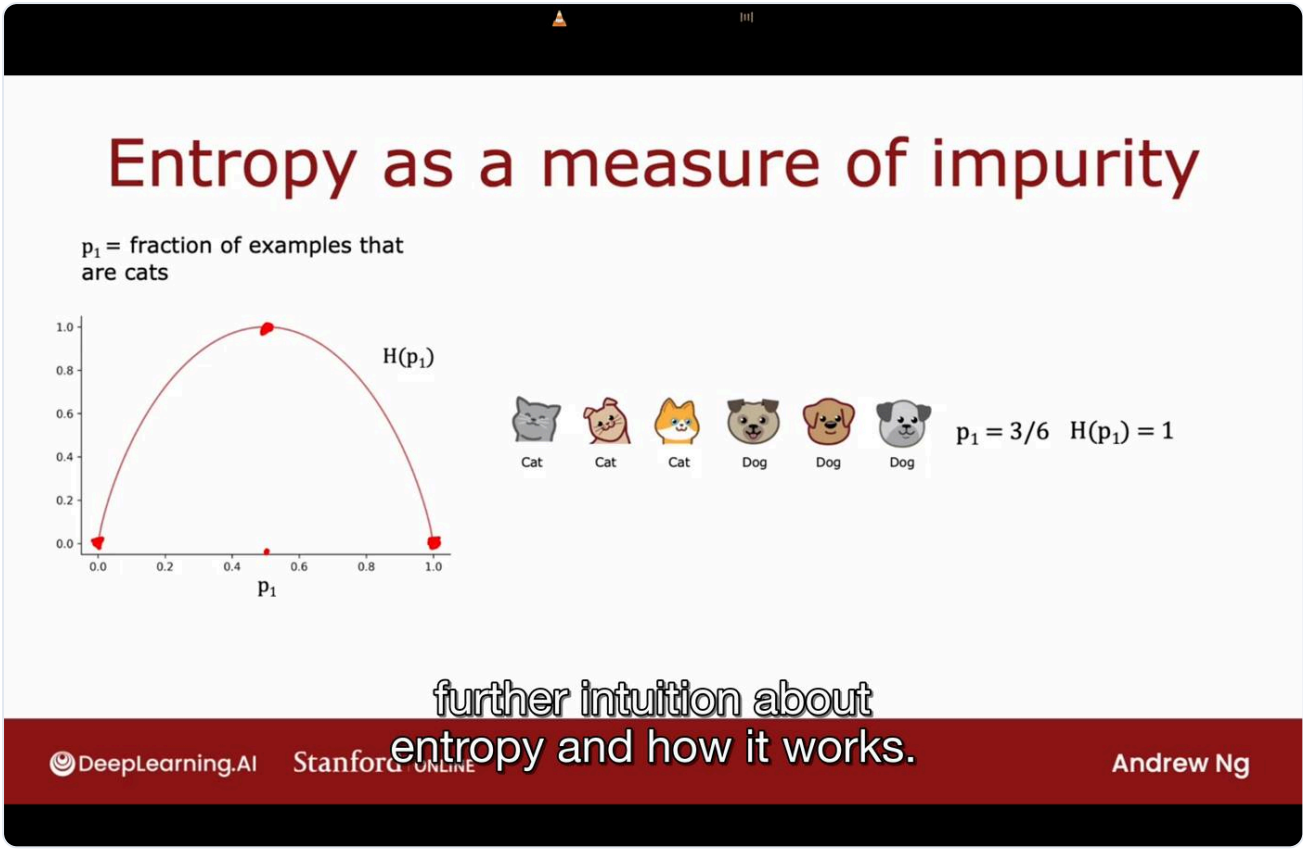


FIGURE 5.3 | Entropy as a Measure of Impurity

Let's just go through a few more examples to gain further intuition about entropy and how it works. Here's a different set of examples with five cats and one dog, so p_1 the fraction of positive examples, a fraction of examples labeled one is 5/6 and so p_1 is about 0.83. If you read off that value at about 0.83 we find that the entropy of p_1 is about 0.65.

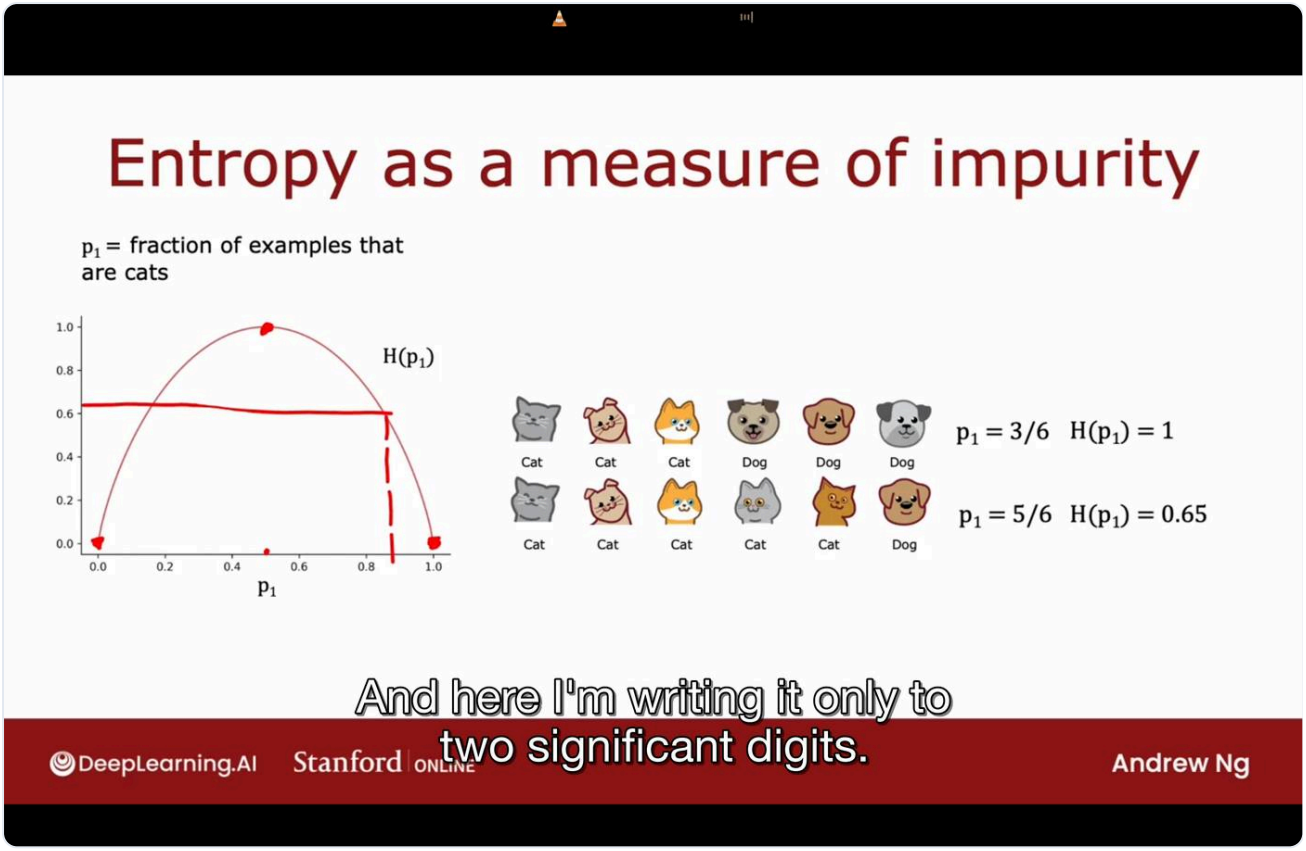


FIGURE 5.4 | Entropy as a Measure of Impurity

And here I'm writing it only to two significant digits. Here's one more example.

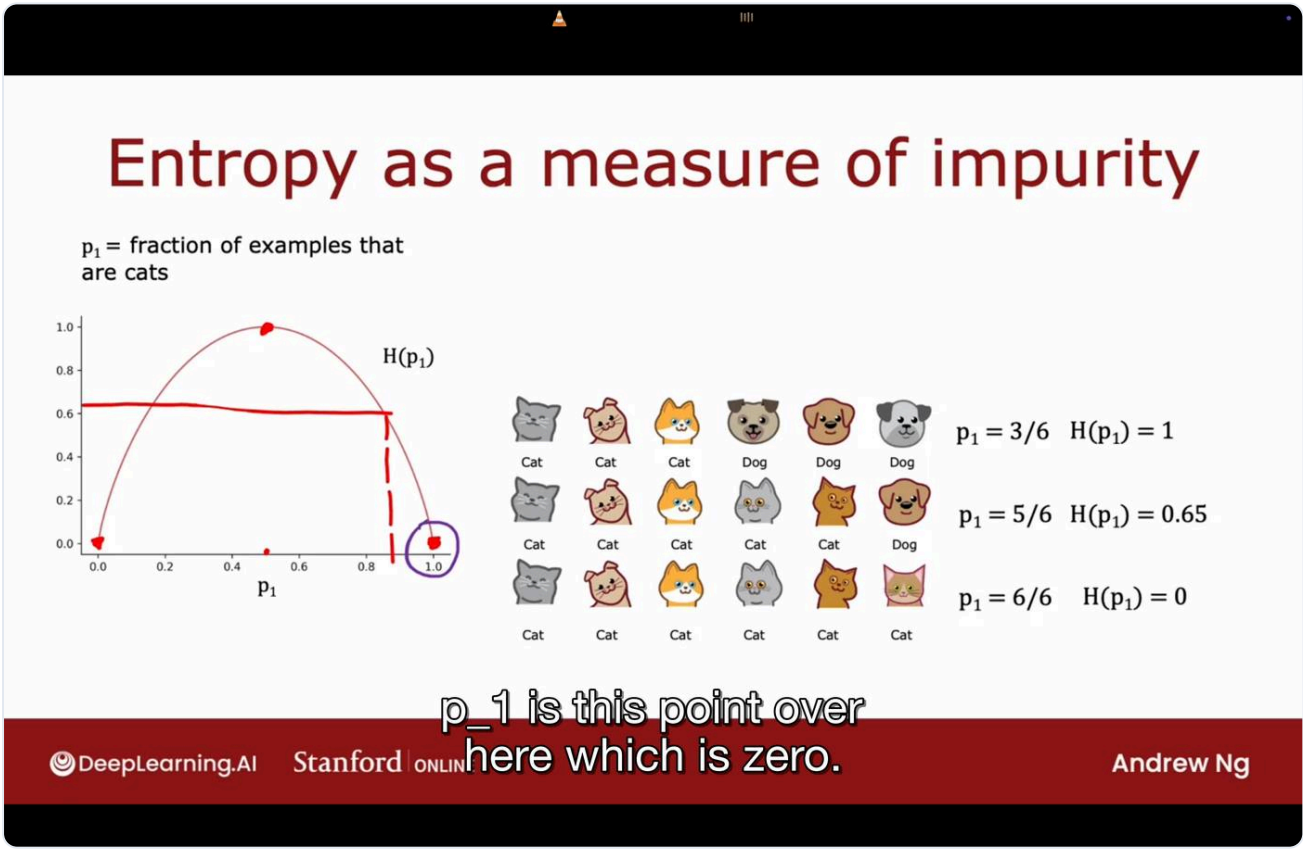


FIGURE 5.5 | Entropy as a Measure of Impurity

This sample of six images has all cats so p_1 is six out of six because all six are cats and the entropy of p_1 is this point over here which is zero. We see that as you go from 3/6 to six out of six cats, the impurity decreases from one to zero or in other words, the purity increases as you go from a 50-50 mix of cats and dogs to all cats. Let's look at a few more examples.

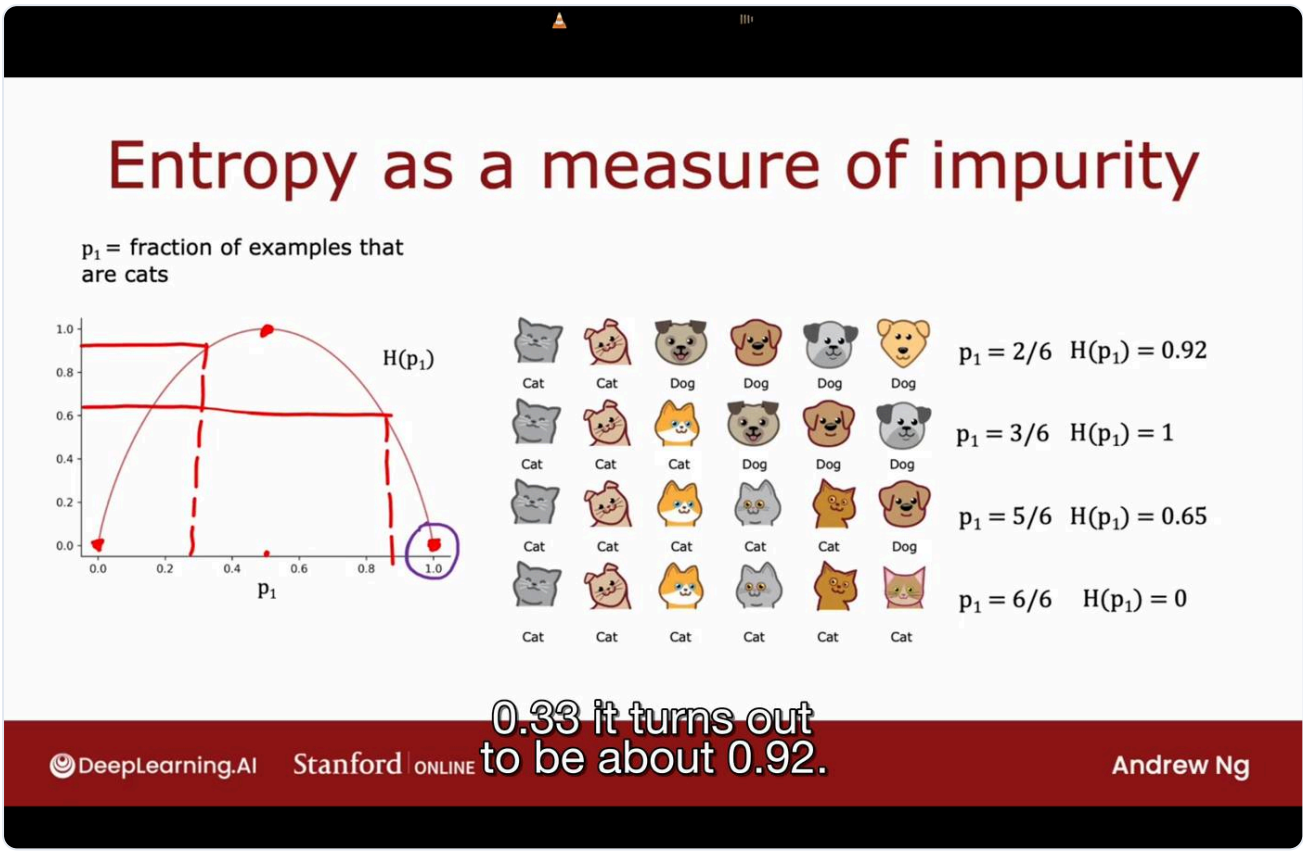


FIGURE 5.6 | Entropy as a Measure of Impurity

Here's another sample with two cats and four dogs, so p_1 here is 2/6 which is 1/3, and if you read off the entropy at 0.33 it turns out to be about 0.92. This is actually quite impure and in particular this set is more impure than this set because it's closer to a 50-50 mix, which is why the impurity here is 0.92 as opposed to 0.65.

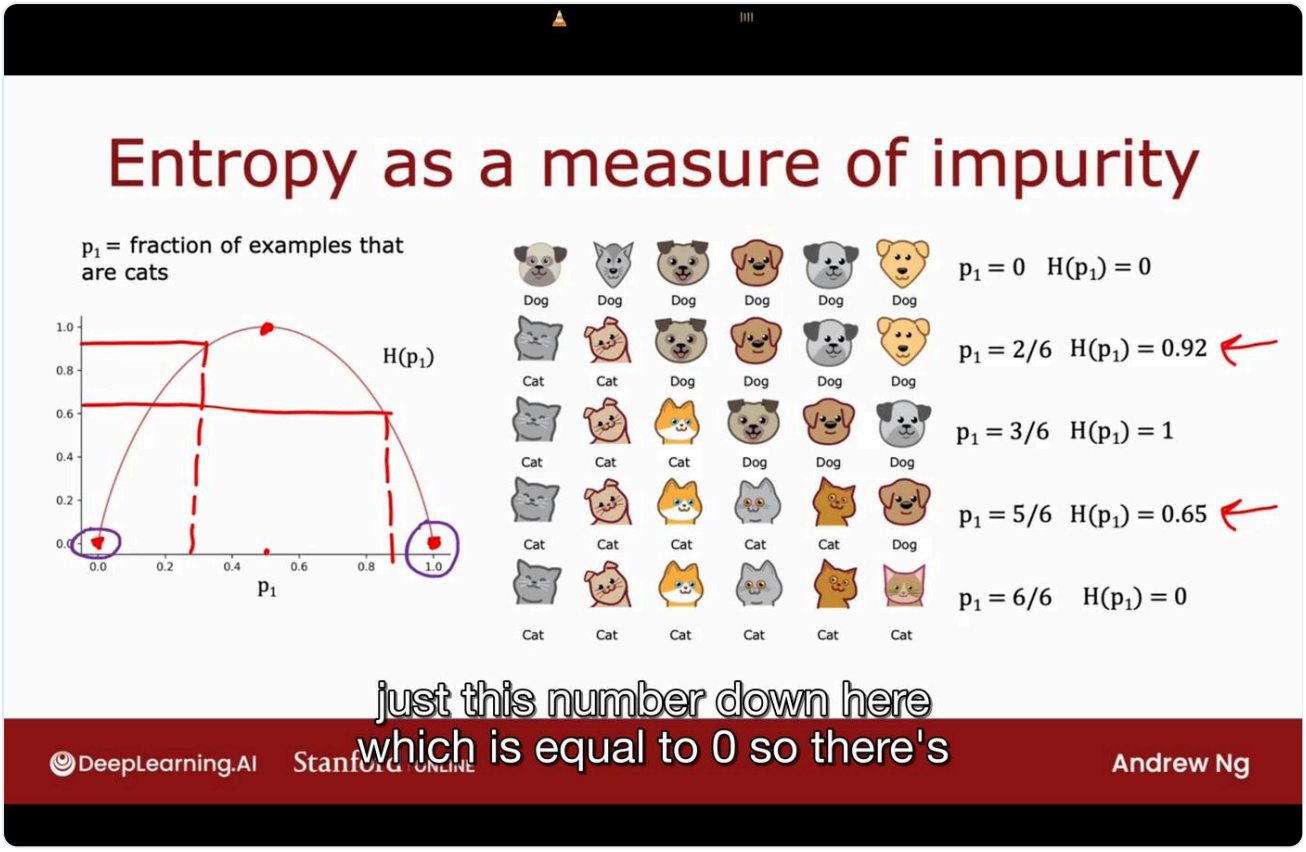


FIGURE 5.7 | Entropy as a Measure of Impurity

Finally, one last example, if we have a set of all six dogs then p_1 is equal to 0 and the entropy of p_1 is just this number down here which is equal to 0 so there's zero impurity or this would be a completely pure set of all not cats or all dogs. Now, let's look at the actual equation for the entropy function H(p_1). Recall that p_1 is the fraction of examples that are equal to cats so if you have a sample that is 2/3 cats then that sample must have 1/3 not cats. Let me define p_0 to be equal to the fraction of examples that are not cats to be just equal to 1 minus p_1.

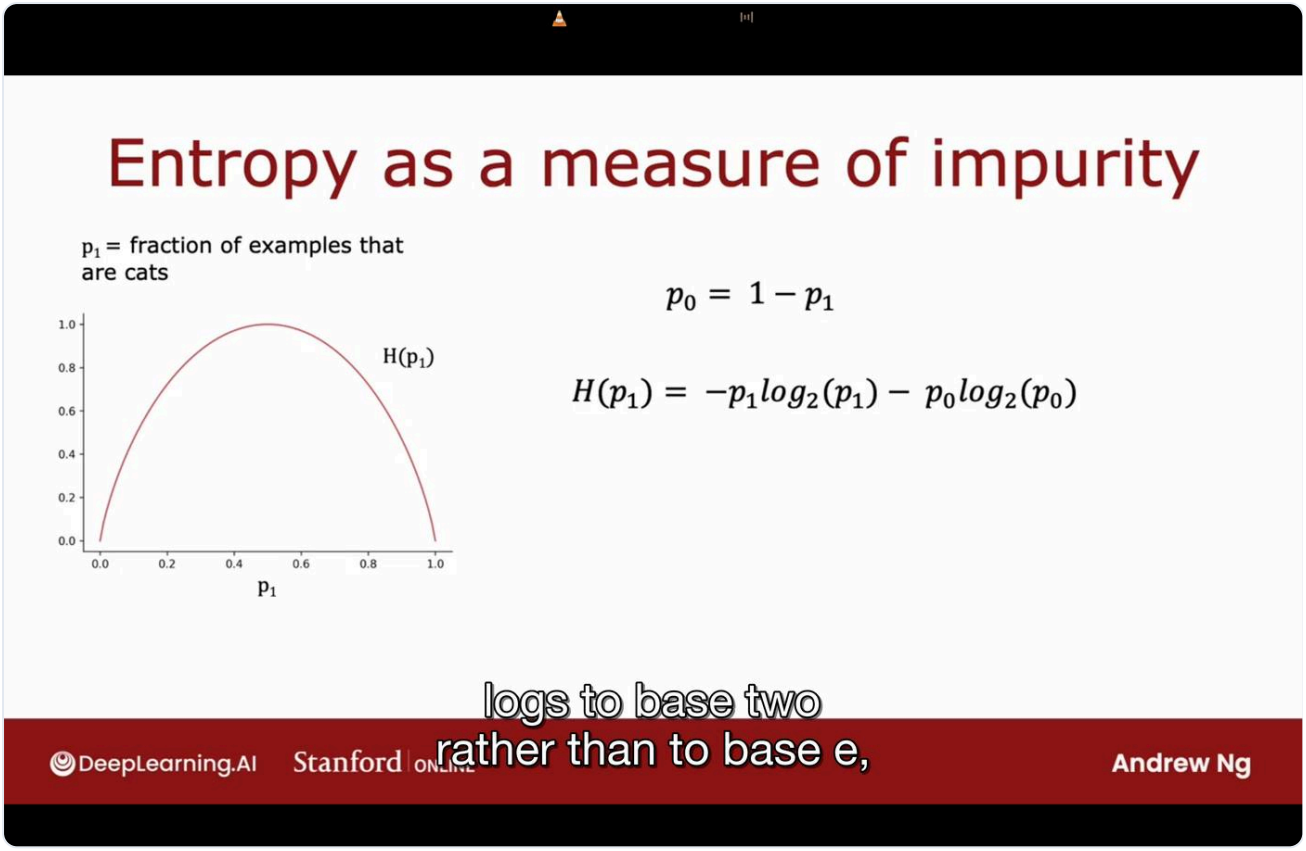


FIGURE 5.8 | Entropy as a Measure of Impurity

The entropy function is then defined as negative p_1log_2 (p_1), and by convention when computing entropy we take logs to base two rather than to base e, and then minus p_0log_2(p_0).

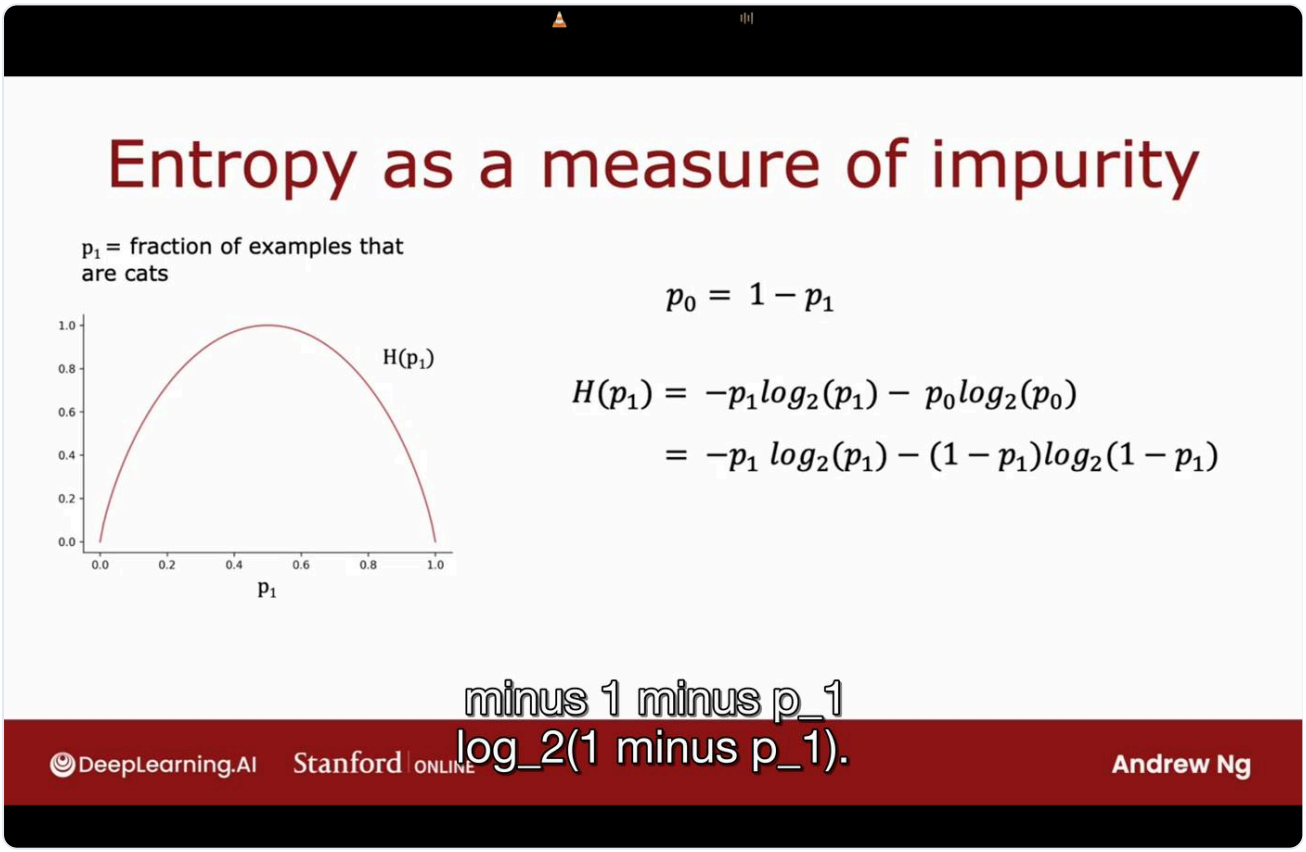


FIGURE 5.9 | Entropy as a Measure of Impurity

Alternatively, this is also equal to negative p_1log_2(p_1) minus 1 minus p_1 log_2(1 minus p_1). If you were to plot this function in a computer you will find that it will be exactly this function on the left.

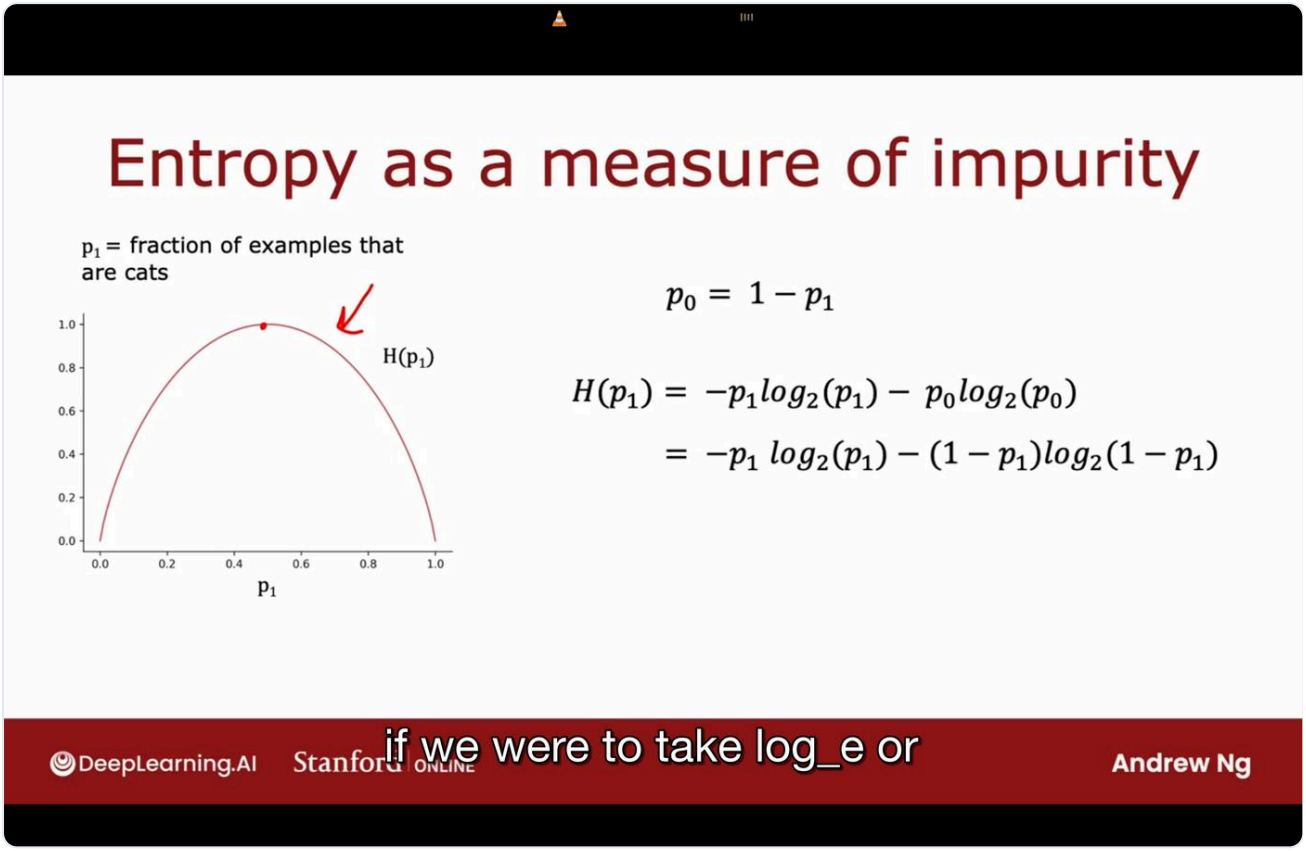


FIGURE 5.10 | Entropy as a Measure of Impurity

We take log_2 just to make the peak of this curve equal to one, if we were to take log_e or the base of natural logarithms, then that just vertically scales this function, and it will still work but the numbers become a bit hard to interpret because the peak of the function isn't a nice round number like one anymore. One note on computing this function, if p_1 or p_0 is equal to 0 then an expression like this will look like 0log(0), and log(0) is technically undefined, it's actually negative infinity.

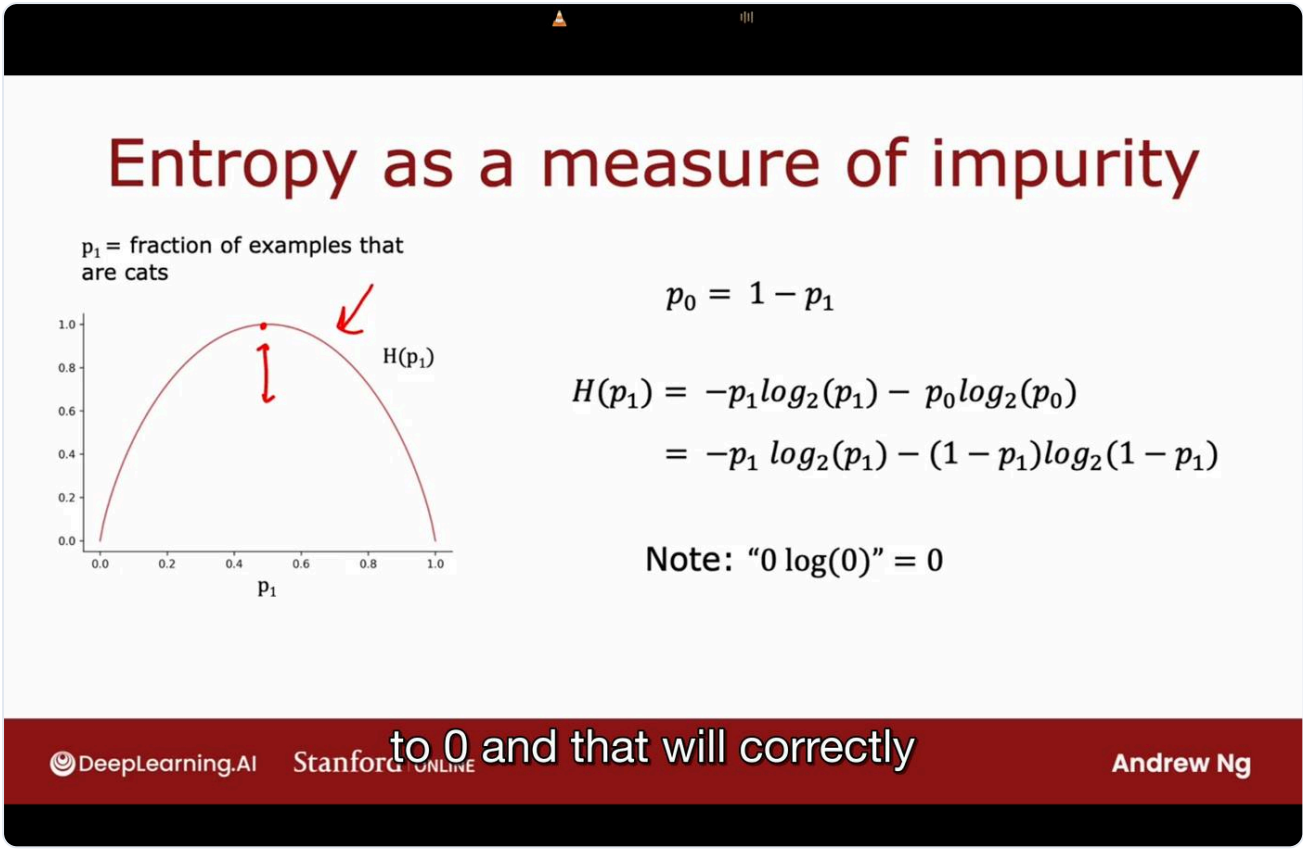


FIGURE 5.11 | Entropy as a Measure of Impurity

But by convention for the purposes of computing entropy, we'll take 0log(0) to be equal to 0 and that will correctly compute the entropy as zero or as one to be equal to zero. If you're thinking that this definition of entropy looks a little bit like the definition of the logistic loss that we learned about in the last course, there is actually a mathematical rationale for why these two formulas look so similar. But you don't have to worry about it and we won't get into it in this class. But applying this formula for entropy should work just fine when you're building a decision tree. To summarize, the entropy function is a measure of the impurity of a set of data.

It starts from zero, goes up to one, and then comes back down to zero as a function of the fraction of positive examples in your sample. There are other functions that look like this, they go from zero up to one and then back down. For example, if you look in open source packages you may also hear about something called the Gini criteria, which is another function that looks a lot like the entropy function, and that will work well as well for building decision trees. But for the sake of simplicity, in these videos I'm going to focus on using the entropy criteria which will usually work just fine for most applications. Now that we have this definition of entropy, in the next video let's take a look at how you can actually use it to make decisions as to what feature to split on in the nodes of a decision tree.

LECTURE 53

Choosing a Split

When building a decision tree, the way we'll decide what feature to split on at a node will be based on what choice of feature reduces entropy the most. Reduces entropy or reduces impurity, or maximizes purity. In decision tree learning, the reduction of entropy is called information gain. Let's take a look, in this video, at how to compute information gain and therefore choose what features to use to split on at each node in a decision tree. Let's use the example of deciding what feature to use at the root node of the decision tree we were building just now for recognizing cats versus not cats. If we had split using their ear shape feature at the root node, this is what we would have gotten, five examples on the left and five on the right.

On the left, we would have four out of five cats, so P1 would be equal to 4/5 or 0.8. On the right, one out of five are cats, so P1 is equal to 1/5 or 0.2. If you apply the entropy formula from the last video to this left subset of data and this right subset of data, we find that the degree of impurity on the left is entropy of 0.8, which is about 0.72, and on the right, the entropy of 0.2 turns out also to be 0.72. This would be the entropy at the left and right subbranches if we were to split on the ear shape feature. One other option would be to split on the face shape feature. If we'd done so then on the left, four of the seven examples would be cats, so P1 is 4/7 and on the right, 1/3 are cats, so P1 on the right is 1/3.

The entropy of 4/7 and the entropy of 1/3 are 0.99 and 0.92. So the degree of impurity in the left and right nodes seems much higher, 0.99 and 0.92 compared to 0.72 and 0.72. Finally, the third possible choice of feature to use at the root node would be the whiskers feature in which case you split based on whether whiskers are present or absent. In this case, P1 on the left is 3/4, P1 on the right is 2/6, and the entropy values are as follows.

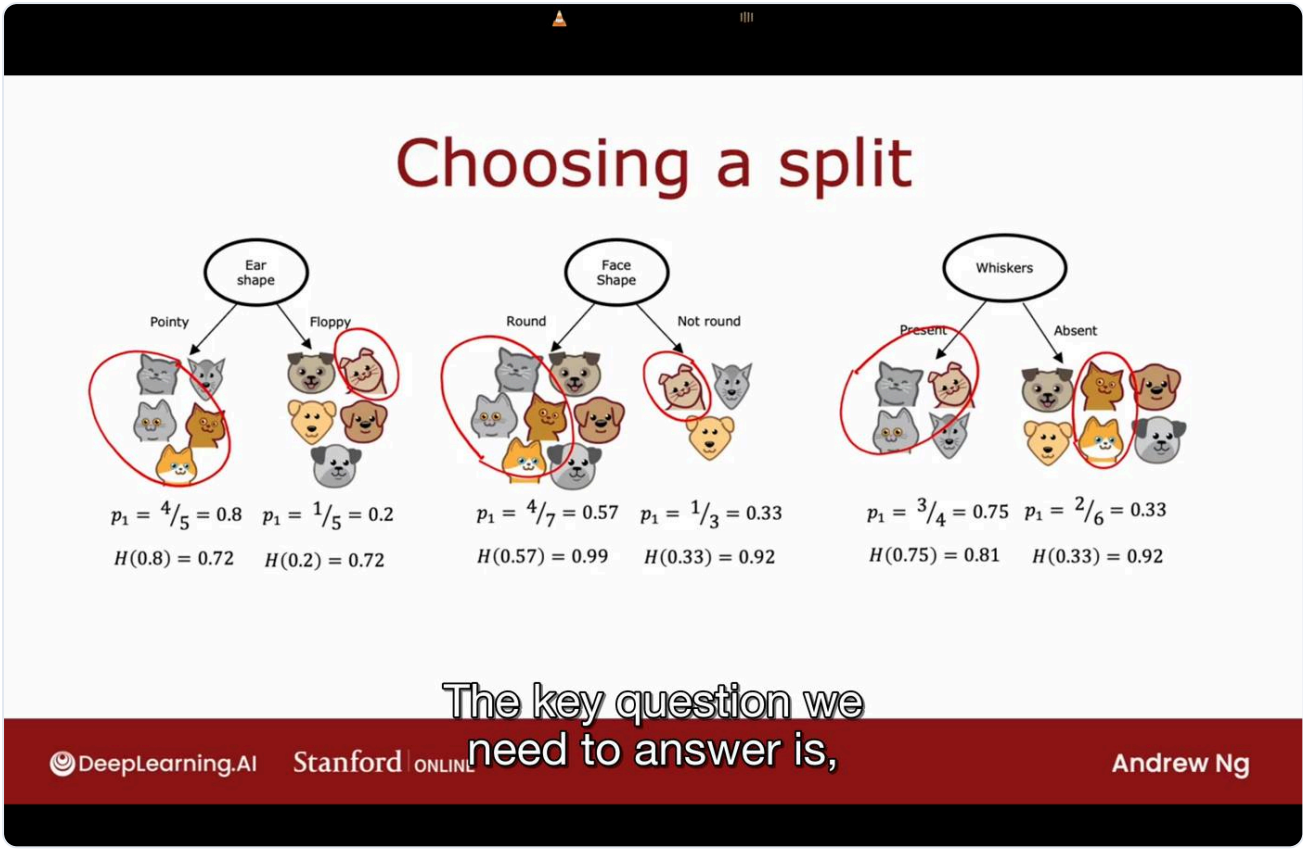


FIGURE 6.1 | Choosing a Split

The key question we need to answer is, given these three options of a feature to use at the root node, which one do we think works best? It turns out that rather than looking at these entropy numbers and comparing them, it would be useful to take a weighted average of them, and here's what I mean. If there's a node with a lot of examples in it with high entropy that seems worse than if there was a node with just a few examples in it with high entropy. Because entropy, as a measure of impurity, is worse if you have a very large and impure dataset compared to just a few examples and a branch of the tree that is very impure. The key decision is, of these three possible choices of features to use at the root node, which one do we want to use?

Associated with each of these splits is two numbers, the entropy on the left sub-branch and the entropy on the right sub-branch. In order to pick from these, we like to actually combine these two numbers into a single number. So you can just pick of these three choices, which one does best? The way we're going to combine these two numbers is by taking a weighted average. Because how important it is to have low entropy in, say, the left or right sub-branch also depends on how many examples went into the left or right sub-branch. Because if there are lots of examples in, say, the left sub-branch then it seems more important to make sure that that left sub-branch's entropy value is low.

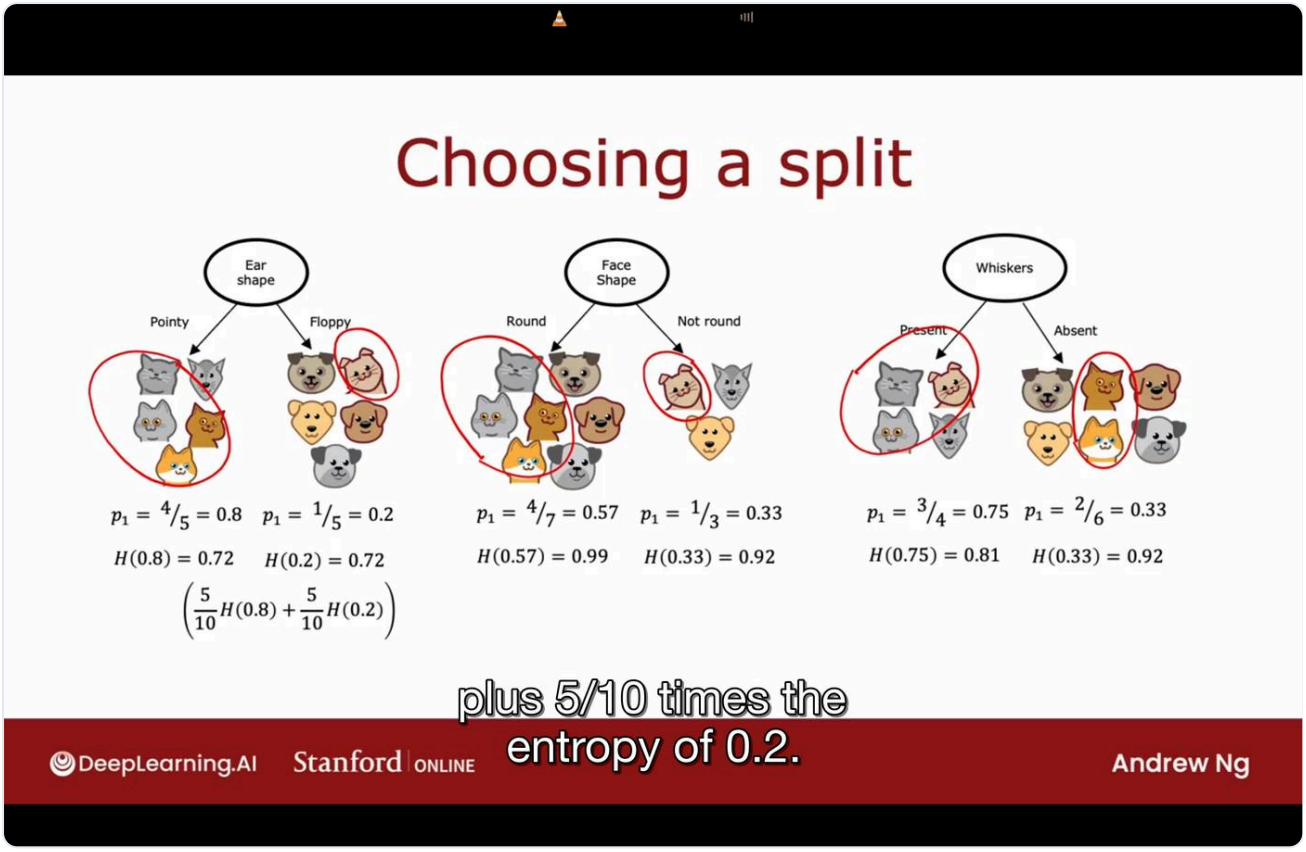


FIGURE 6.2 | Choosing a Split

In this example we have, five of the 10 examples went to the left sub-branch, so we can compute the weighted average as 5/10 times the entropy of 0.8, and then add to that 5/10 examples also went to the right sub-branch, plus 5/10 times the entropy of 0.2.

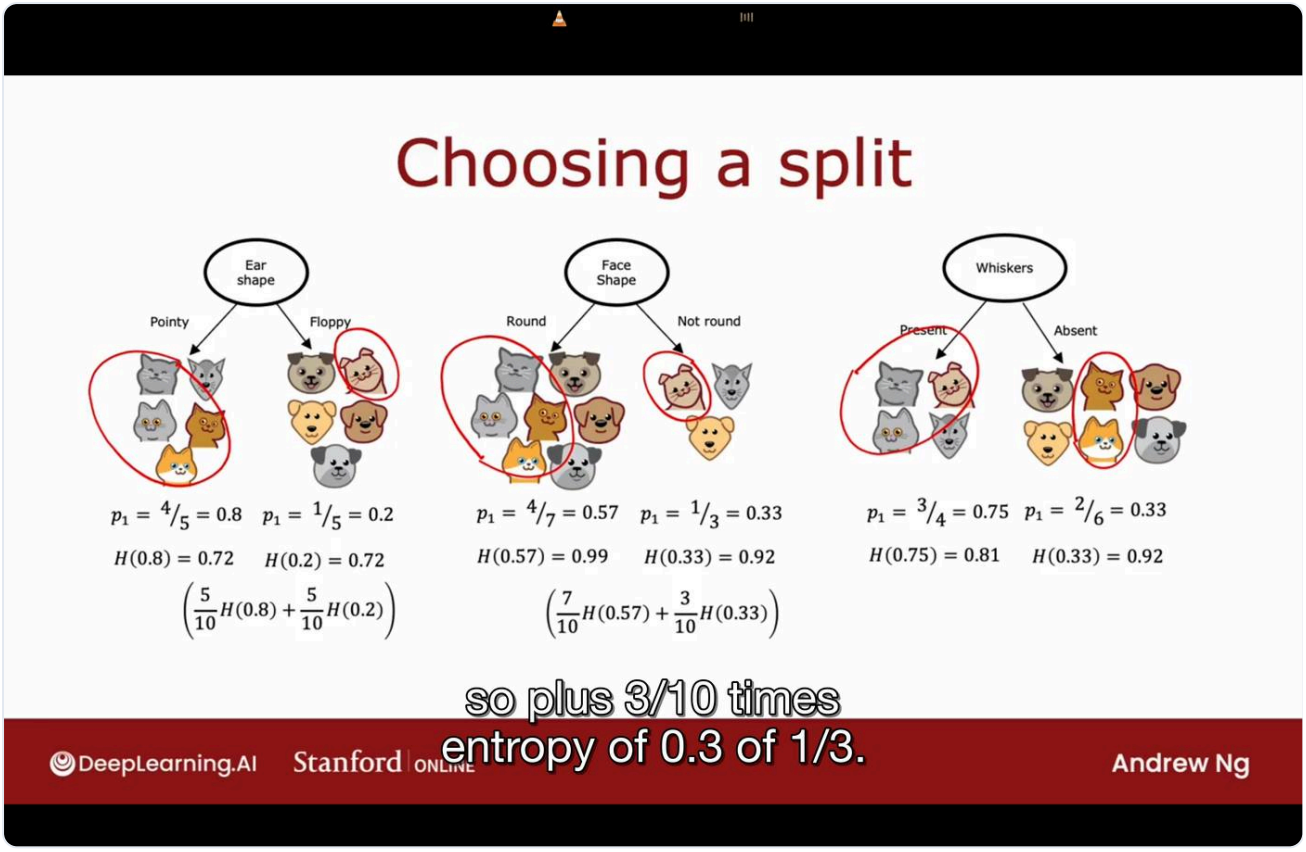


FIGURE 6.3 | Choosing a Split

Now, for this example in the middle, the left sub-branch had received seven out of 10 examples. and so we're going to compute 7/10 times the entropy of 0.57 plus, the right sub-branch had three out of 10 examples, so plus 3/10 times entropy of 0.3 of 1/3.

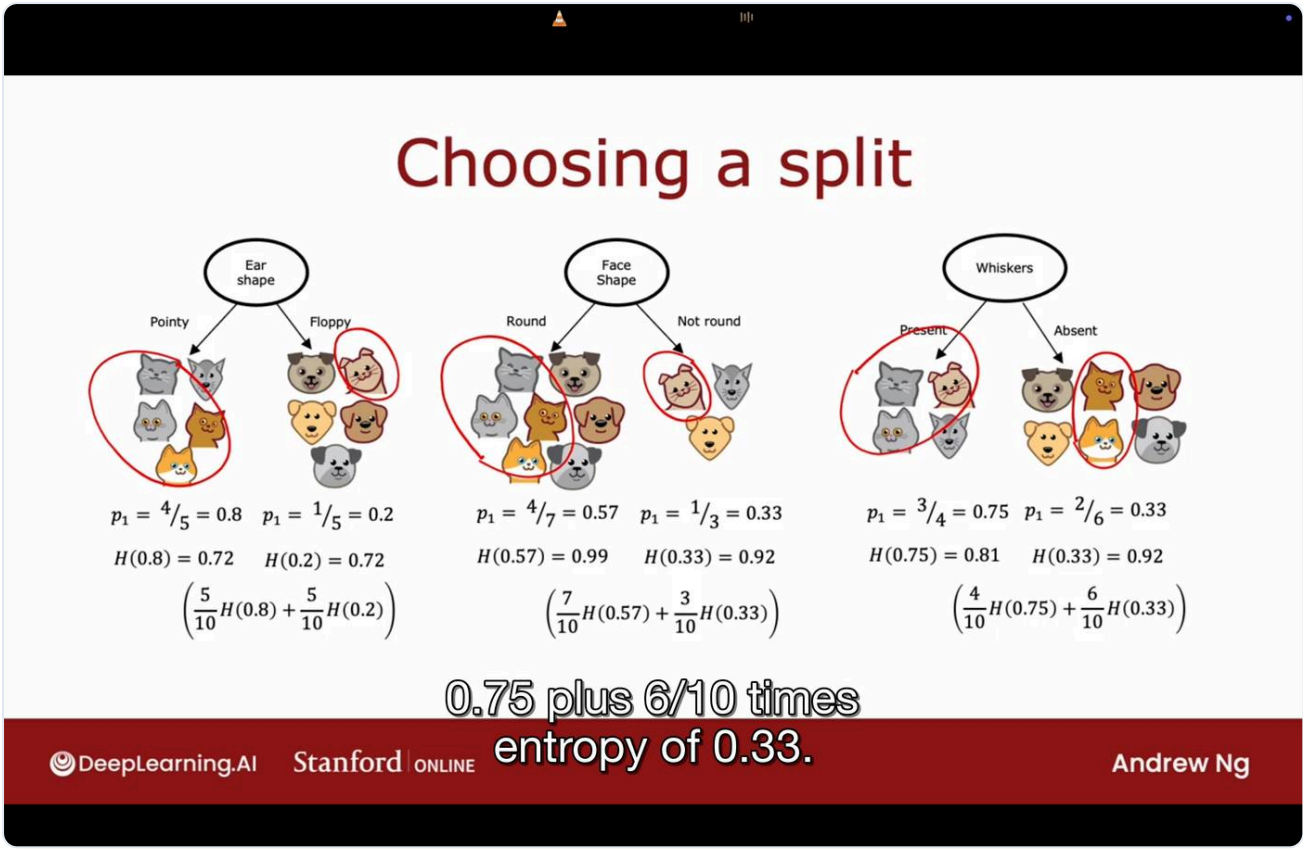


FIGURE 6.4 | Choosing a Split

Finally, on the right, we'll compute 4/10 times entropy of 0.75 plus 6/10 times entropy of 0.33. The way we will choose a split is by computing these three numbers and picking whichever one is lowest because that gives us the left and right sub-branches with the lowest average weighted entropy. In the way that decision trees are built, we're actually going to make one more change to these formulas to stick to the convention in decision tree building, but it won't actually change the outcome. Which is rather than computing this weighted average entropy, we're going to compute the reduction in entropy compared to if we hadn't split at all. If we go to the root node, remember that the root node we have started off with all 10 examples in the root node with five cats and dogs, and so at the root node, we had p_1 equals 5/10 or 0.5.

The entropy of the root nodes, entropy of 0.5 was actually equal to 1. This was maximum impurity because it was five cats and five dogs. The formula that we're actually going to use for choosing a split is not this weighted entropy at the left and right sub-branches, instead is going to be the entropy at the root node, which is entropy of 0.5, then minus this formula. In this example, if you work out the math, it turns out to be 0.28. For the face shape example, we can compute entropy of the root node, entropy of 0.5 minus this, which turns out to be 0.03, and for whiskers, compute that, which turns out to be 0.12.

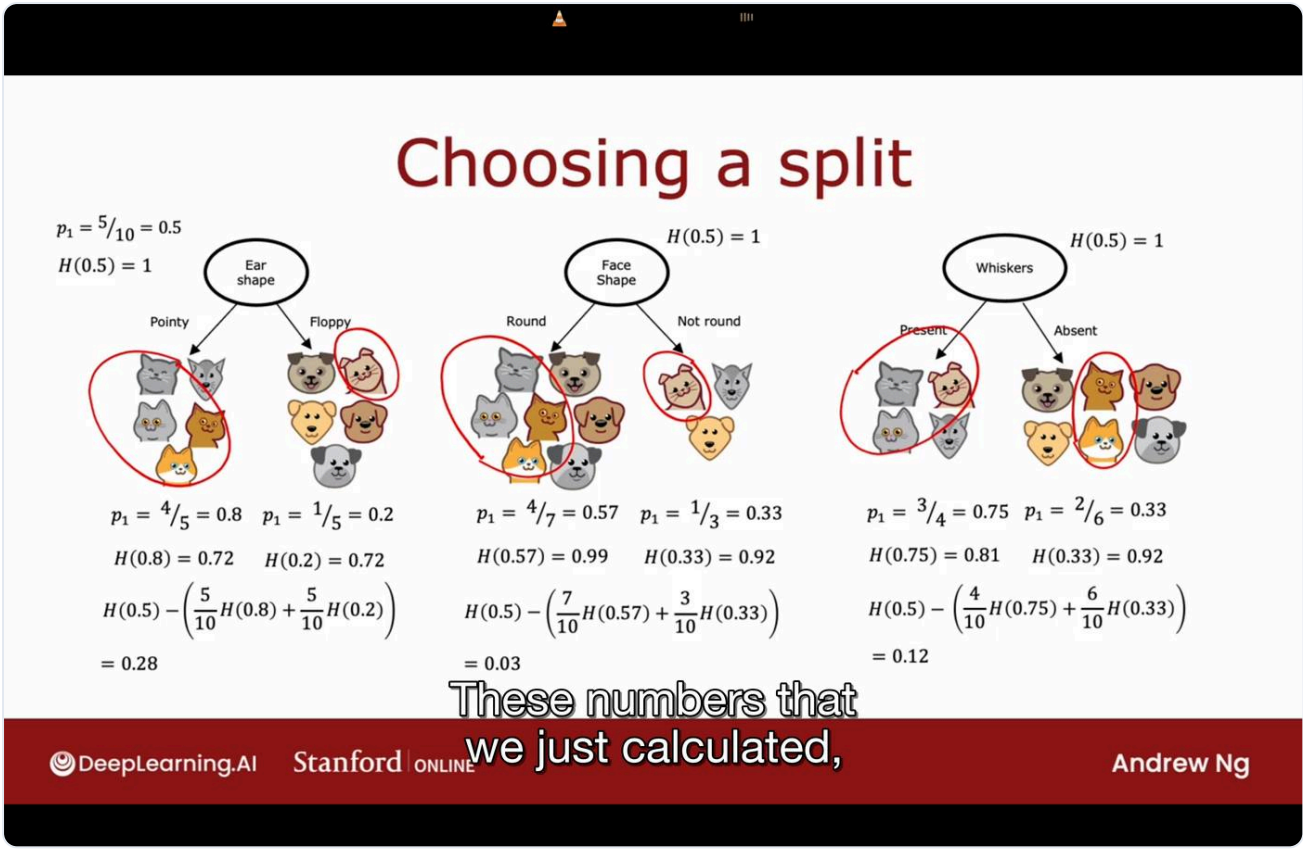


FIGURE 6.5 | Choosing a Split

These numbers that we just calculated, 0.28, 0.03, and 0.12, these are called the information gain, and what it measures is the reduction in entropy that you get in your tree resulting from making a split. Because the entropy was originally one at the root node and by making the split, you end up with a lower value of entropy and the difference between those two values is a reduction in entropy, and that's 0.28 in the case of splitting on the ear shape. Why do we bother to compute reduction in entropy rather than just entropy at the left and right sub-branches? It turns out that one of the stopping criteria for deciding when to not bother to split any further is if the reduction in entropy is too small.

In which case you could decide, you're just increasing the size of the tree unnecessarily and risking overfitting by splitting and just decide to not bother if the reduction in entropy is too small or below a threshold. In this other example, spitting on ear shape results in the biggest reduction in entropy, 0.28 is bigger than 0.03 or 0.12 and so we would choose to split onto ear shape feature at the root node. On the next slide, let's give a more formal definition of information gain. By the way, one additional piece of notation that we'll introduce also in the next slide is these numbers, 5/10 and 5/10.

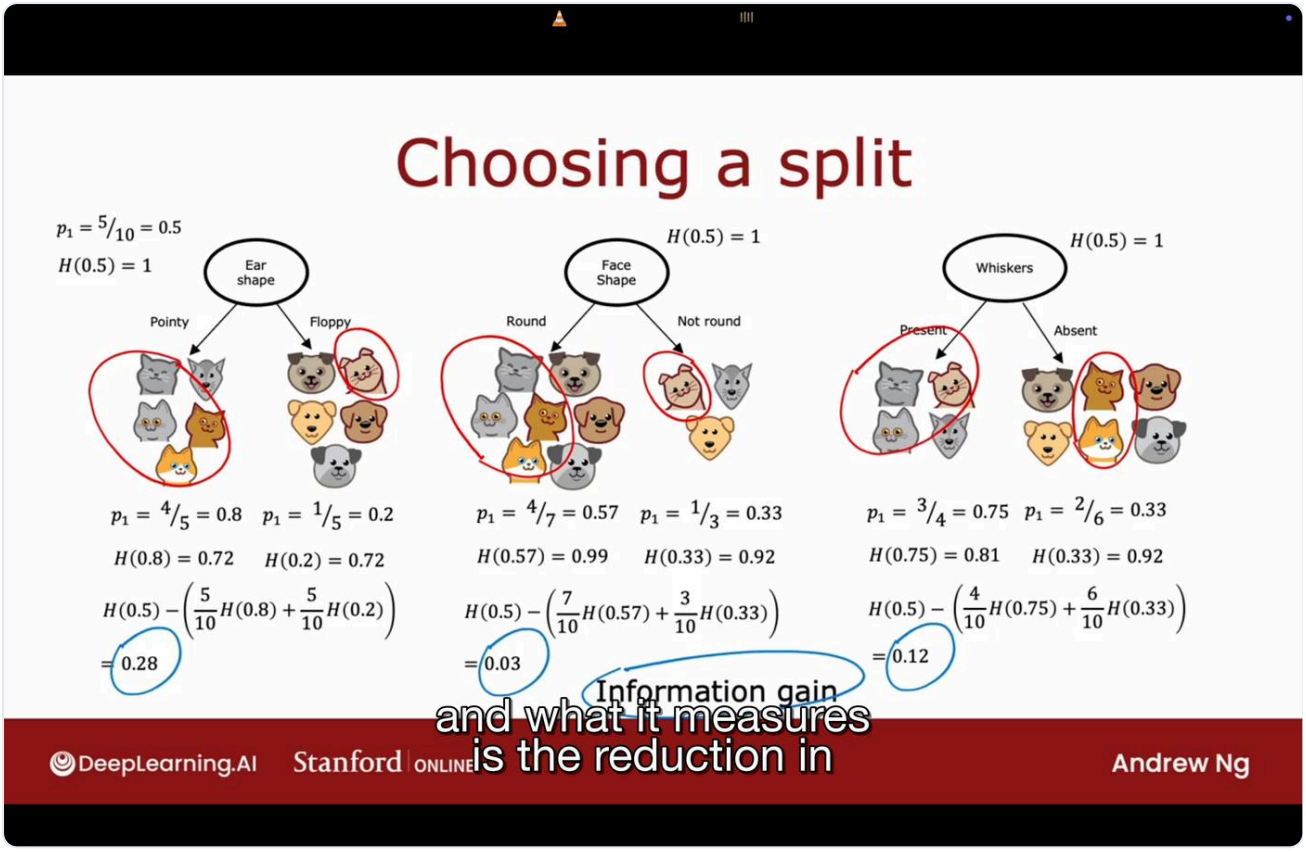


FIGURE 6.6 | Choosing a Split

I'm going to call this w^left because that's the fraction of examples that went to the left branch, and I'm going to call this w^right because that's the fraction of examples that went to the right branch. Whereas for this another example, w^left would be 7/10, and w^right will be 3/10. Let's now write down the general formula for how to compute information gain. Using the example of splitting on the ear shape feature, let me define p_1^left to be equal to the fraction of examples in the left subtree that have a positive label, that are cats. In this example, p_1^left will be equal to 4/5. Also, let me define w^left to be the fraction of examples of all of the examples of the root node that went to the left sub-branch, and so in this example, w^left would be 5/10.

Similarly, let's define p_1^right to be of all the examples in the right branch. The fraction that are positive examples and so one of the five of these examples being cats, there'll be 1/5, and similarly, w^right is 5/10 the fraction of examples that went to the right sub-branch. Let's also define p_1^root to be the fraction of examples that are positive in the root node.

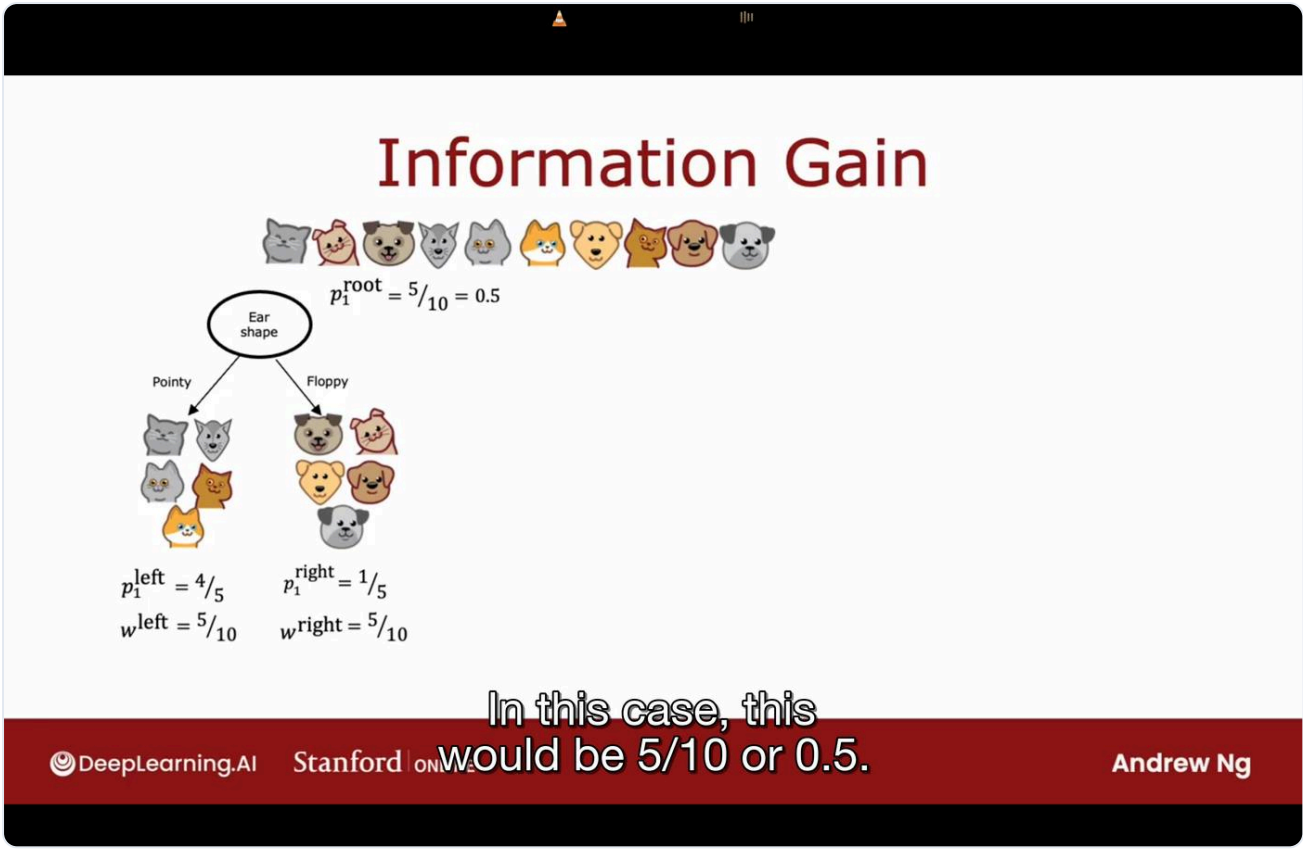


FIGURE 6.7 | Information Gain

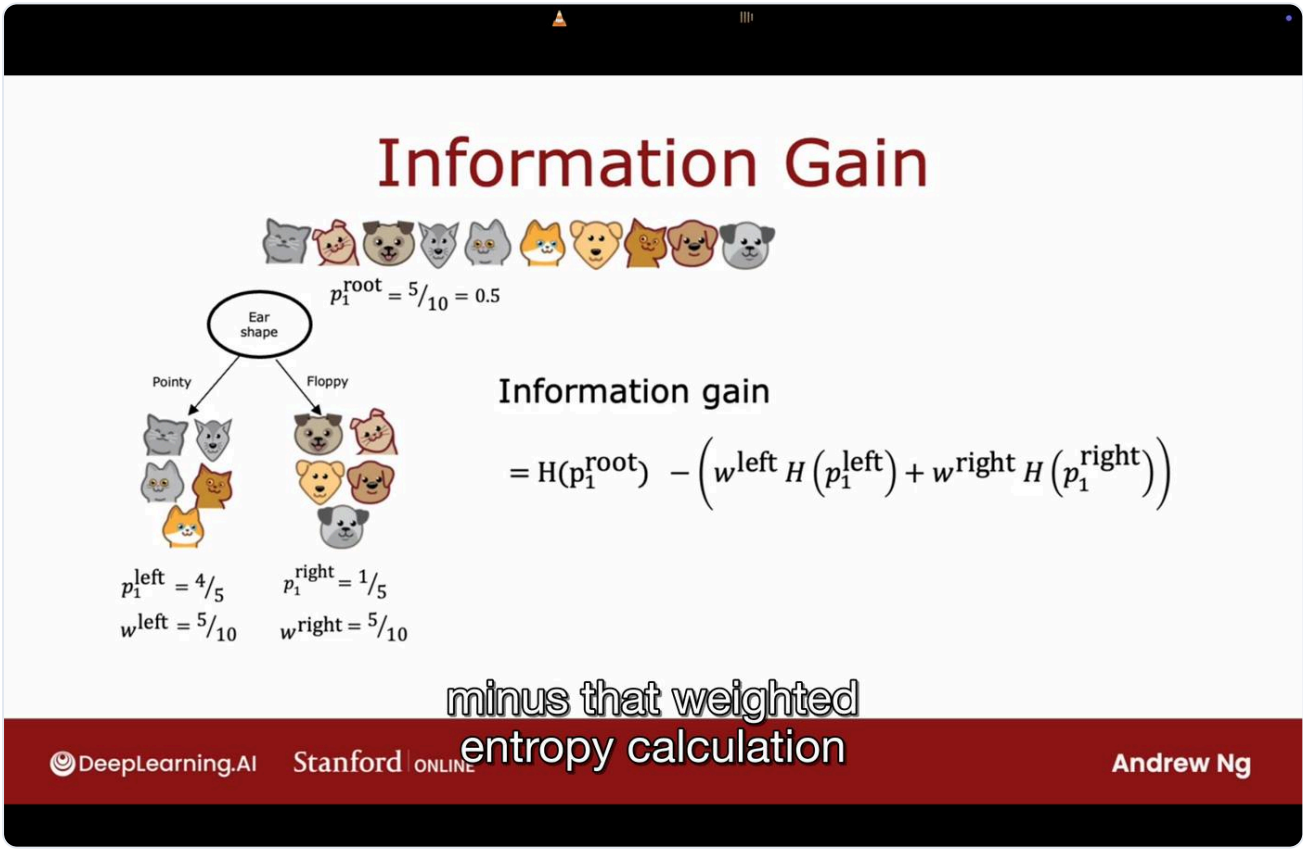


FIGURE 6.8 | Information Gain

In this case, this would be 5/10 or 0.5. Information gain is then defined as the entropy of p_1^root, so what's the entropy at the root node, minus that weighted entropy calculation that we had on the previous slide, minus w^left those were 5/10 in the example, times the entropy applied to p_1^left, that's entropy on the left sub-branch, plus w^right the fraction of examples that went to the right branch, times entropy of p_1^right. With this definition of entropy, and you can calculate the information gain associated with choosing any particular feature to split on in the node. Then out of all the possible futures, you could choose to split on, you can then pick the one that gives you the highest information gain.

That will result in, hopefully, increasing the purity of your subsets of data that you get on the left and right sub-branches of your decision tree and that will result in choosing a feature to split on that increases the purity of your subsets of data in both the left and right sub-branches of your decision tree. Now that you know how to calculate information gain or reduction in entropy, you know how to pick a feature to split on another node. Let's put all the things we've talked about together into the overall algorithm for building a decision tree given a training set. Let's go see that in the next video.

LECTURE 54

Recursive Splitting

The information gain criteria lets you decide how to choose one feature to split a one-node. Let's take that and use that in multiple places through a decision tree in order to figure out how to build a large decision tree with multiple nodes. Here is the overall process of building a decision tree. Starts with all training examples at the root node of the tree and calculate the information gain for all possible features and pick the feature to split on, that gives the highest information gain. Having chosen this feature, you would then split the dataset into two subsets according to the selected feature, and create left and right branches of the tree and send the training examples to either the left or the right branch, depending on the value of that feature for that example.

This allows you to have made a split at the root node. After that, you will then keep on repeating the splitting process on the left branch of the tree, on the right branch of the tree and so on. Keep on doing that until the stopping criteria is met.

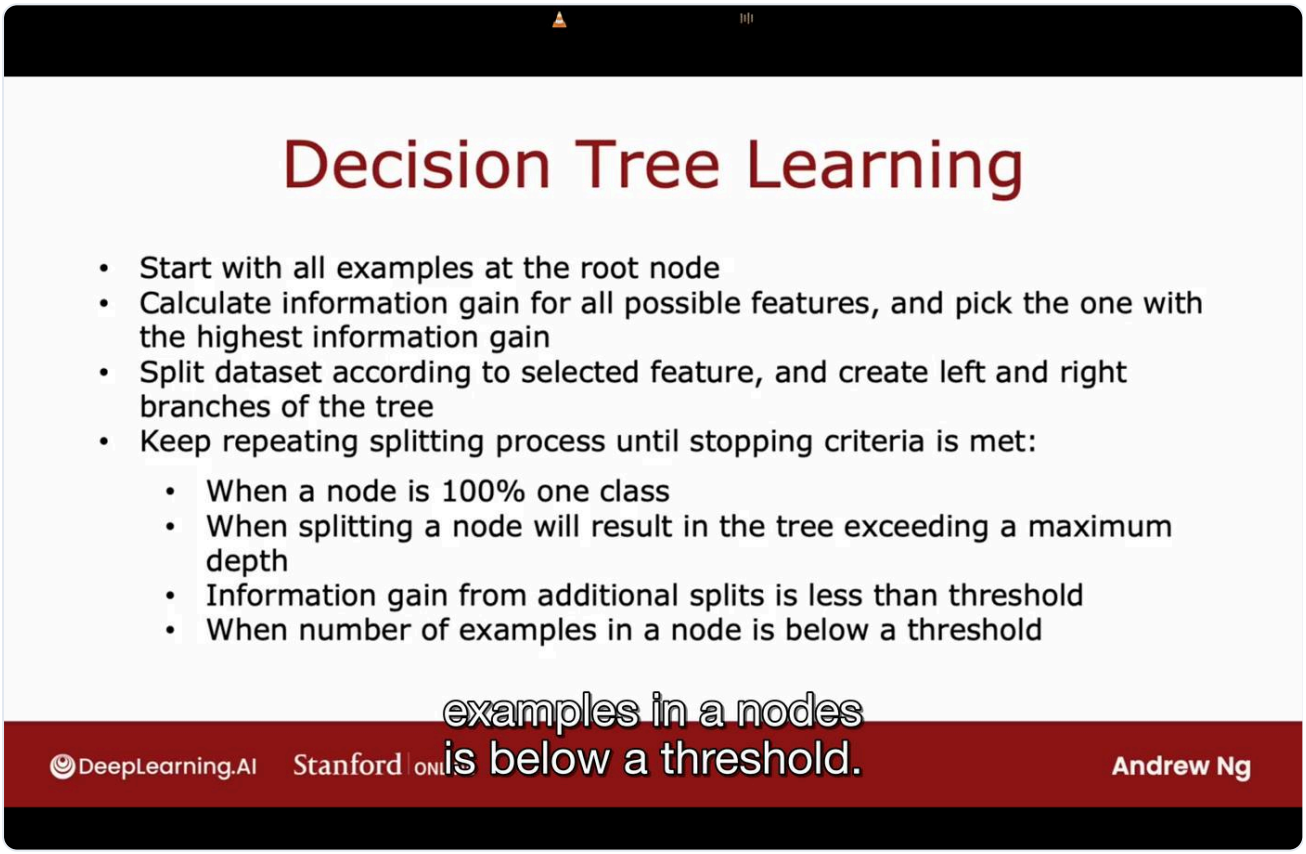


FIGURE 7.1 | Decision Tree Learning

Where the stopping criteria can be, when a node is 100 percent a single clause, someone has reached entropy of zero, or when further splitting a node will cause the tree to exceed the maximum depth that you had set, or if the information gain from an additional splits is less than the threshold, or if the number of examples in a nodes is below a threshold. You will keep on repeating the splitting process until the stopping criteria that you've chosen, which could be one or

more of these criteria is met. Let's look at an illustration of how this process will work. We started all of the examples at the root nodes and based on computing information gain for all three features, decide that ear-shaped is the best feature to split on.

Based on that, we create a left and right sub-branches and send the subsets of the data with pointy versus floppy ear to left and right sub-branches. Let me cover the root node and the right sub-branch and just focus on the left sub-branch where we have these five examples. Let's see off splitting criteria is to keep splitting until everything in the node belongs to a single class, so either all cancel all nodes. We will look at this node and see if it meets the splitting criteria, and it does not because there is a mix of cats and dogs here. The next step is to then pick a feature to split on. We then go through the features one at a time and compute the information gain of each of those features as if this node, were the new root node of a decision tree that was trained using just five training examples shown here.

We would compute the information gain for splitting on the whiskers feature, the information gain on splitting on the face shape feature. It turns out that the information gain for splitting on ear shape will be zero because all of these have the same point ear shape. Between whiskers and face shape, face shape turns out to have a highest information gain. We're going to split on face shape and that allows us to build left and right sub branches as follows.

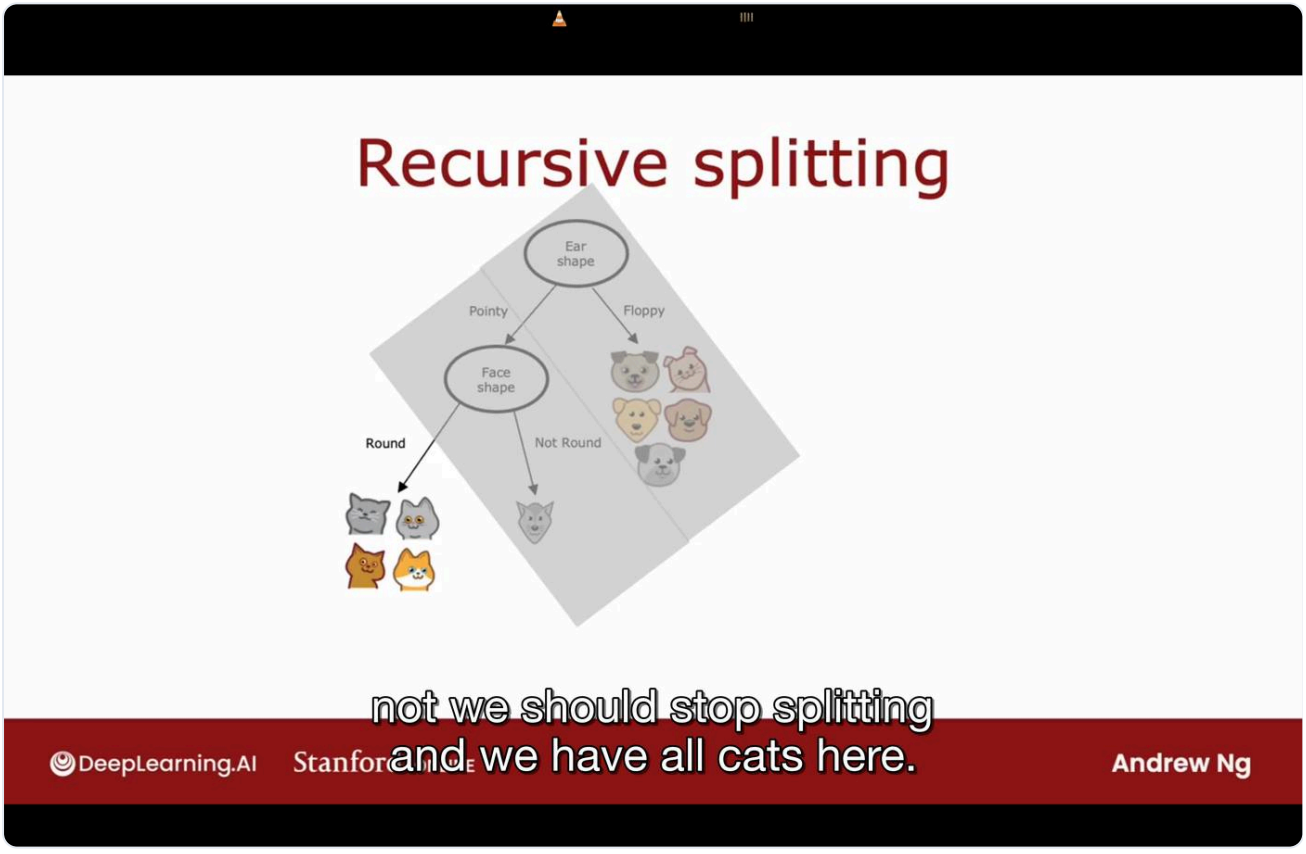


FIGURE 7.2 | Recursive Splitting

For the left sub-branch, we check for the criteria for whether or not we should stop splitting and we have all cats here. The stopping criteria is met and we create a leaf node that makes a prediction of cat. For the right sub-branch, we find that it is all dogs. We will also stop splitting since we've met the splitting criteria and put a leaf node there, that predicts not cat.

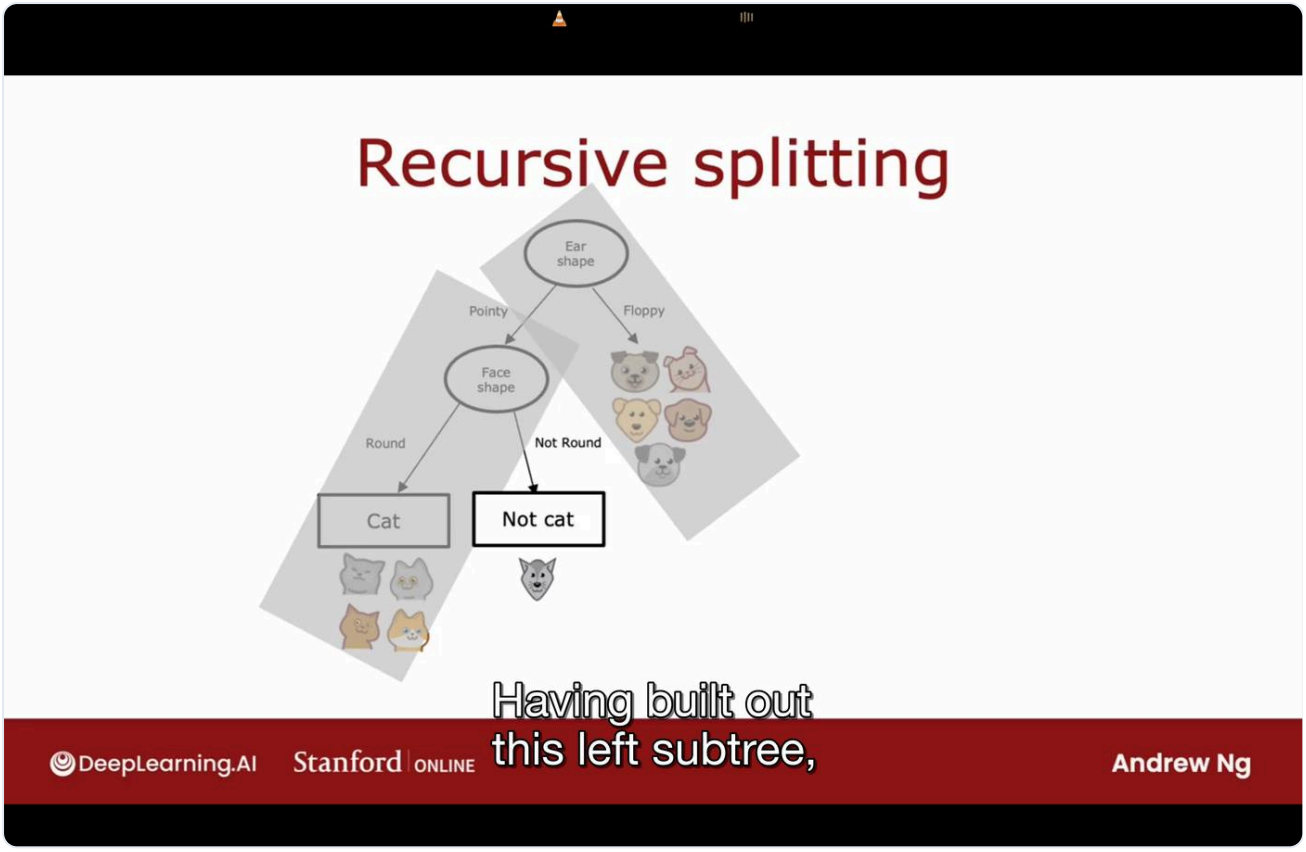


FIGURE 7.3 | Recursive Splitting

Having built out this left subtree, we can now turn our attention to building the right subtree. Let me now again cover up the root node and the entire left subtree. To build out the right subtree, we have these five examples here. Again, the first thing we do is check if the criteria to stop splitting has been met, their criteria being met or not. All the examples are a single class, we've not met that criteria. We'll decide to keep splitting in this right sub-branch as well.

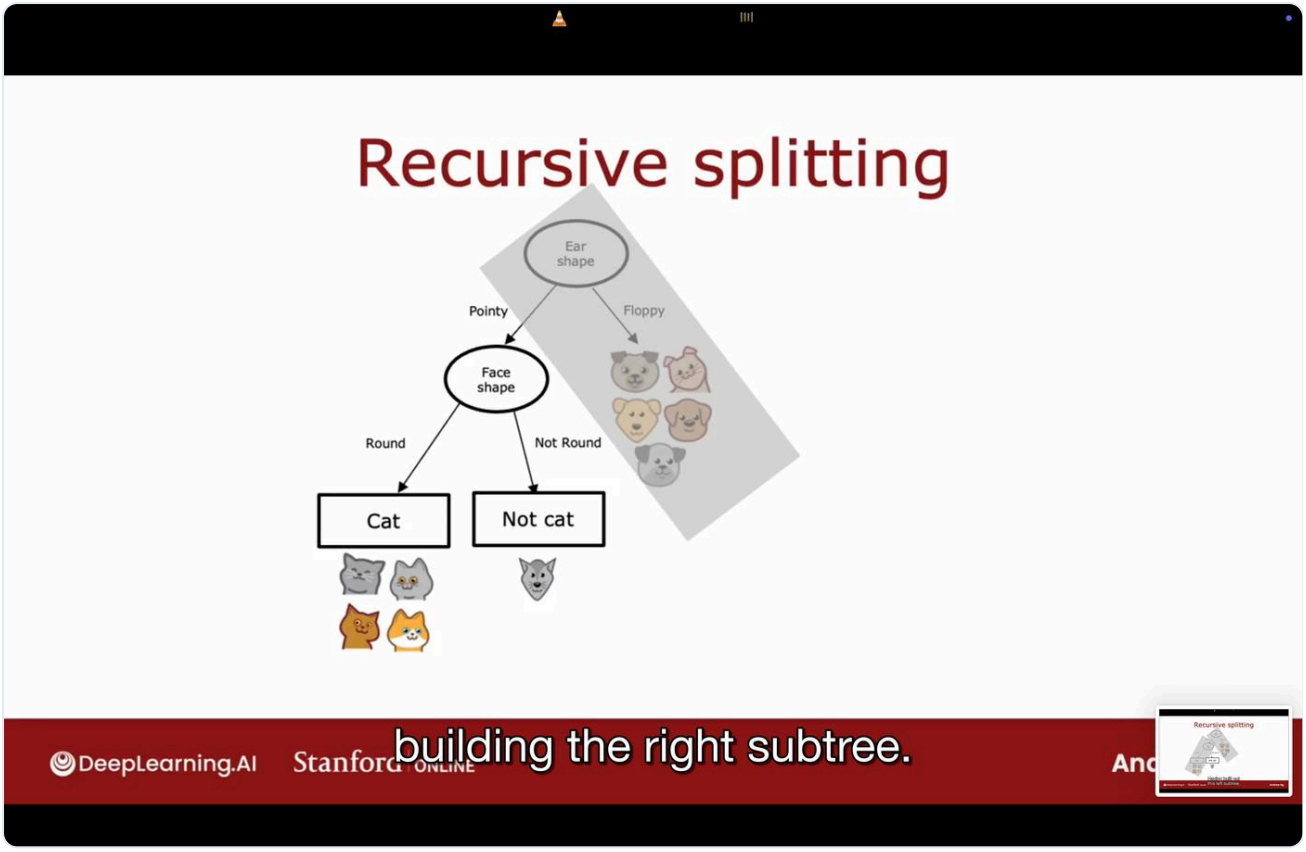


FIGURE 7.4 | Recursive Splitting

In fact, the procedure for building the right sub-branch will be a lot as if you were training a decision tree learning algorithm from scratch, where the dataset you have comprises just these five training examples. Again, computing information gain for all of the possible features to split on, you find that the whiskers feature use the highest information gain. Split this set of five examples according to whether whiskers are present or absent. Check if the criteria to stop splitting are met in the left and right sub-branches here and decide that they are. You end up with leaf nodes that predict cat and dog cat.

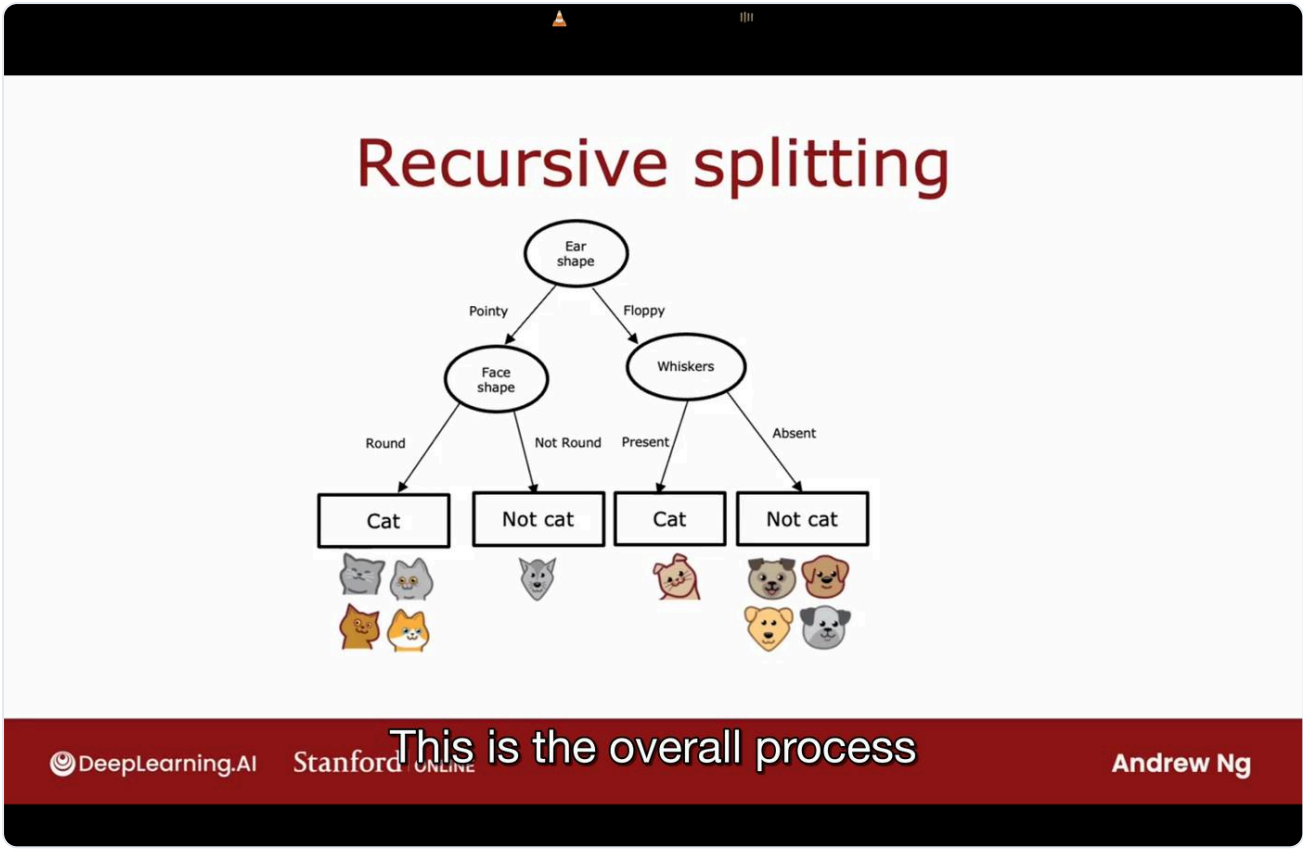


FIGURE 7.5 | Recursive Splitting

This is the overall process for building the decision tree. Notice that there's interesting aspects of what we've done, which is after we decided what to split on at the root node, the way we built the left subtree was by building a decision tree on a subset of five examples. The way we built the right subtree was by, again, building a decision tree on a subset of five examples.

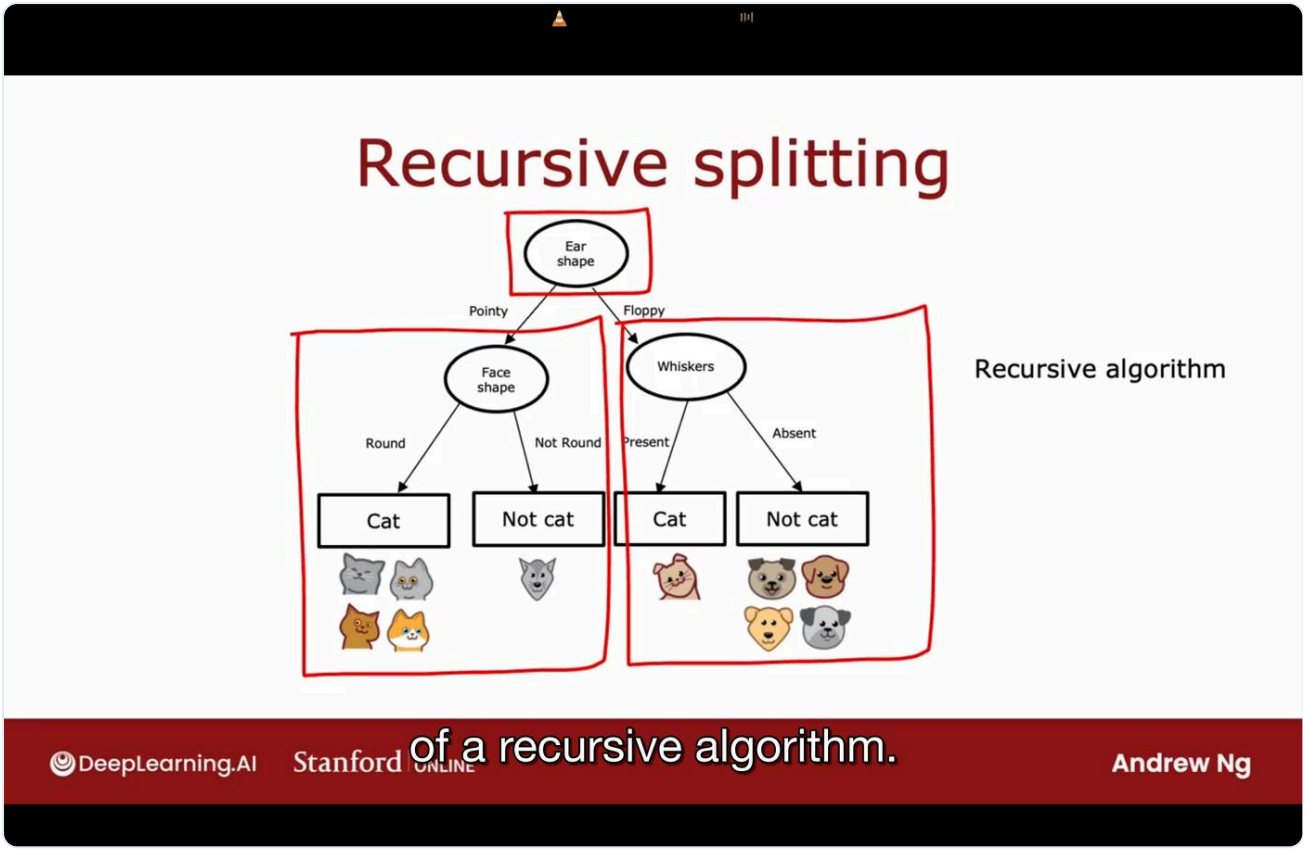


FIGURE 7.6 | Recursive Splitting

In computer science, this is an example of a recursive algorithm. All that means is the way you build a decision tree at the root is by building other smaller decision trees in the left and the right sub-branches. Recursion in computer science refers to writing code that calls itself. The way this comes up in building a decision tree is you build the overall decision tree by building smaller sub-decision trees and then putting them all together. That's why if you look at software implementations of decision trees, you'll see sometimes references to a recursive algorithm. But if you don't feel like you've fully understood this concept of recursive algorithms, don't worry about it.

You still be able to fully complete this week's assignments, as well as use libraries to get decision trees to work for yourself. But if you're implementing a decision tree algorithm from scratch, then a recursive algorithm turns out to be one of the steps you'd have to implement. By the way, you may be wondering how to choose the maximum depth parameter. There are many different possible choices, but some of the open-source libraries will have good default choices that you can use. One intuition is, the larger the maximum depth, the bigger the decision tree you're willing to build. This is a bit like fitting a higher degree polynomial or training a larger neural network.

It lets the decision tree learn a more complex model, but it also increases the risk of overfitting if this fitting a very complex function to your data. In theory, you could use cross-validation to pick parameters like the maximum depth, where you try out different values of the maximum depth and pick what works best on the cross-validation set. Although in practice, the open-source libraries have even somewhat better ways to choose this parameter for you. Or another criteria

that you can use to decide when to stop splitting is if the information gained from an additional split is less than a certain threshold. If any feature is splint on, achieves only a small reduction in entropy or a very small information gain, then you might also decide to not bother.

Finally, you can also decide to stop splitting when the number of examples in the node is below a certain threshold. That's the process of building a decision tree. Now that you've learned the decision tree, if you want to make a prediction, you can then follow the procedure that you saw in the very first video of this week, where you take a new example, say a test example, and started a route and keep on following the decisions down until you get to the leaf node, which then makes the prediction. Now that you know the basic decision tree learning algorithm, in the next few videos, I'd like to go into some further refinements of this algorithm. So far we've only used features to take on two possible values.

But sometimes you have a feature that takes on categorical or discrete values, but maybe more than two values. Let's take a look in the next video at how to handle that case.

LECTURE 55

One Hot Encoding

In the example we've seen so far each of the features could take on only one of two possible values. The ear shape was either pointy or floppy, the face shape was either round or not round and whiskers were either present or absent. But whether if you have features that can take on more than two discrete values, in this video we'll look at how you can use one-hot encoding to address features like that. Here's a new training set for our pet adoption center application where all the data is the same except for the ear shaped feature.

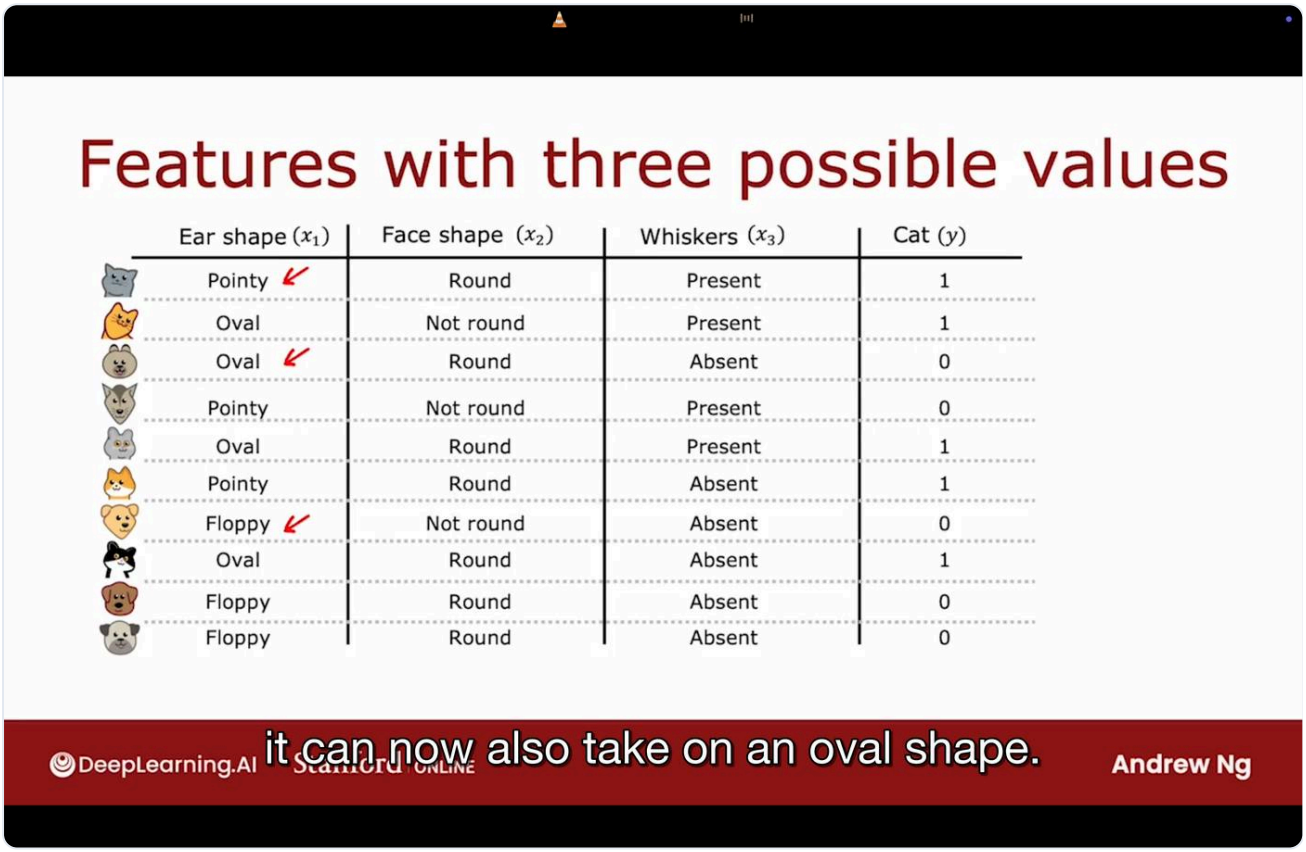


FIGURE 8.1 | Features with Three Possible Values

Rather than ear shape only being pointy and floppy, it can now also take on an oval shape. And so the initial feature is still a categorical value feature but it can take on three possible values instead of just two possible values. And this means that when you split on this feature you end up creating three subsets of the data and end up building three sub branches for this tree. But in this video I'd like to describe a different way of addressing features that can take on more than two values, which is to use the one-hot encoding. In particular rather than using an ear shaped feature, they can take on any of three possible values. We're instead going to create three new features where one feature is, does this animal have pointy ears, a second is does their floppy ears and the third is does it have oval ears.

And so for the first example whereas we previously had ear shape as pointy, we are now instead say that this animal has a value for the pointy ear feature of 1 and 0 for floppy and oval. Whereas previously for the second example, we previously said it had oval ears now we'll say that it has a value of 0 for pointy ears because it doesn't have pointy ears.

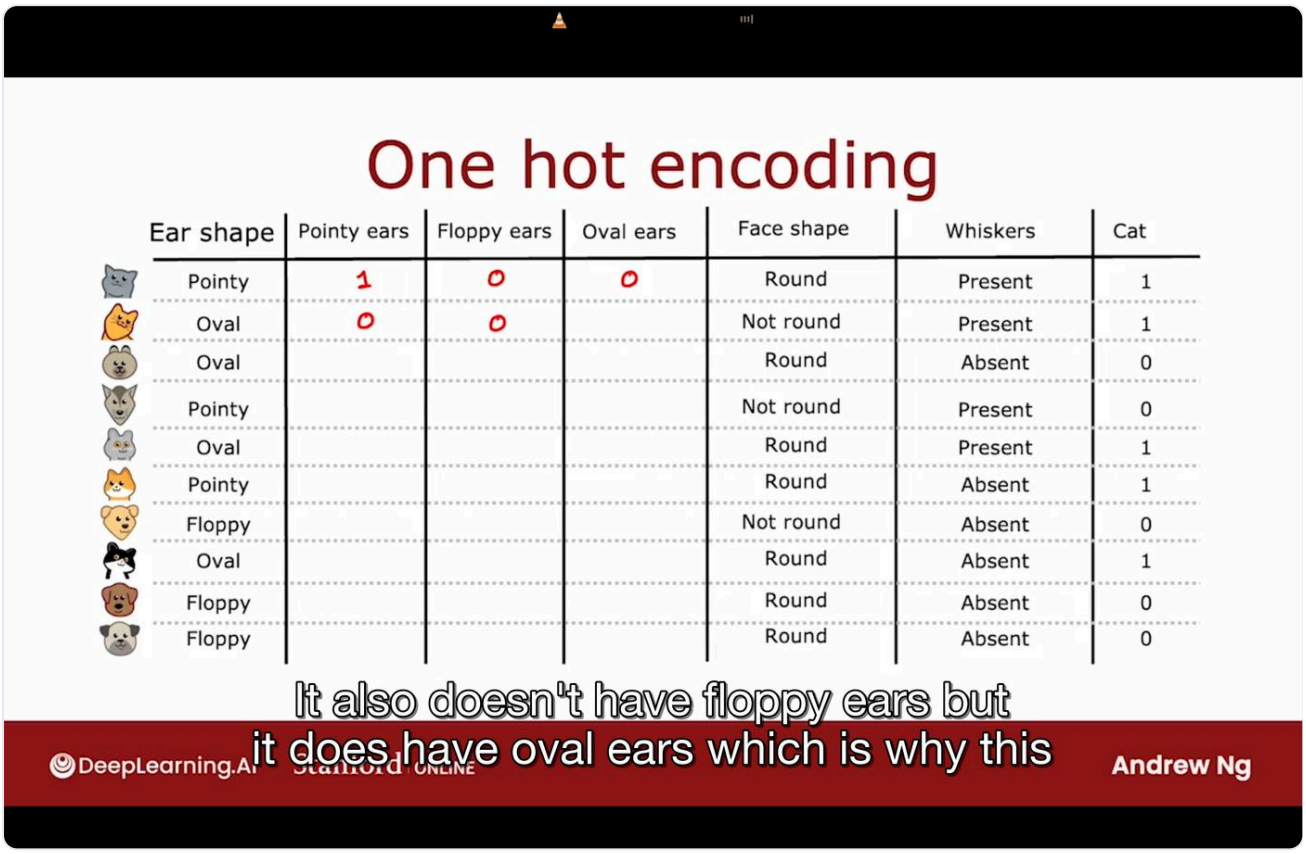


FIGURE 8.2 | One Hot Encoding

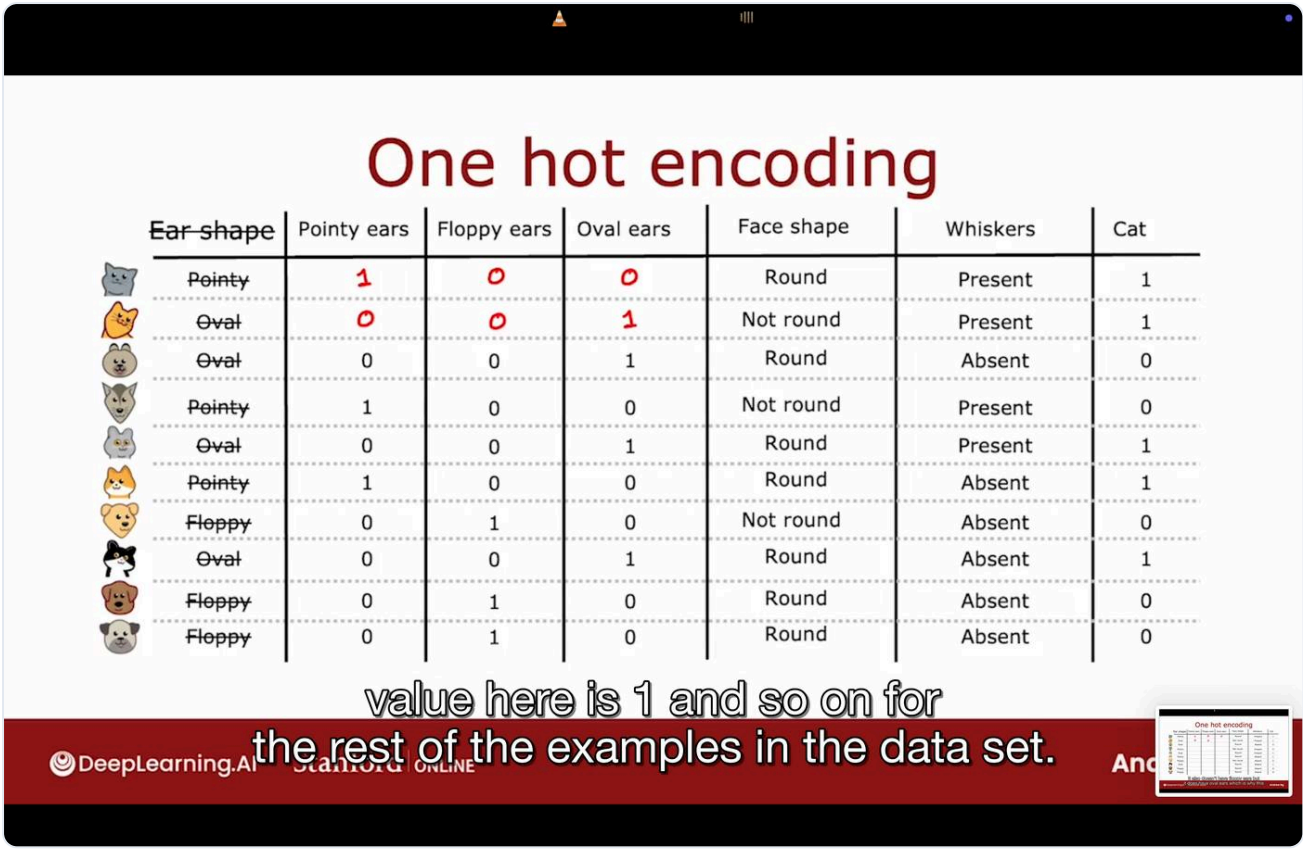


FIGURE 8.3 | One Hot Encoding

It also doesn't have floppy ears but it does have oval ears which is why this value here is 1 and so on for the rest of the examples in the data set. And so instead of one feature taking on three possible values, we've now constructed three new features each of which can take on only one of two possible values, either 0 or 1.

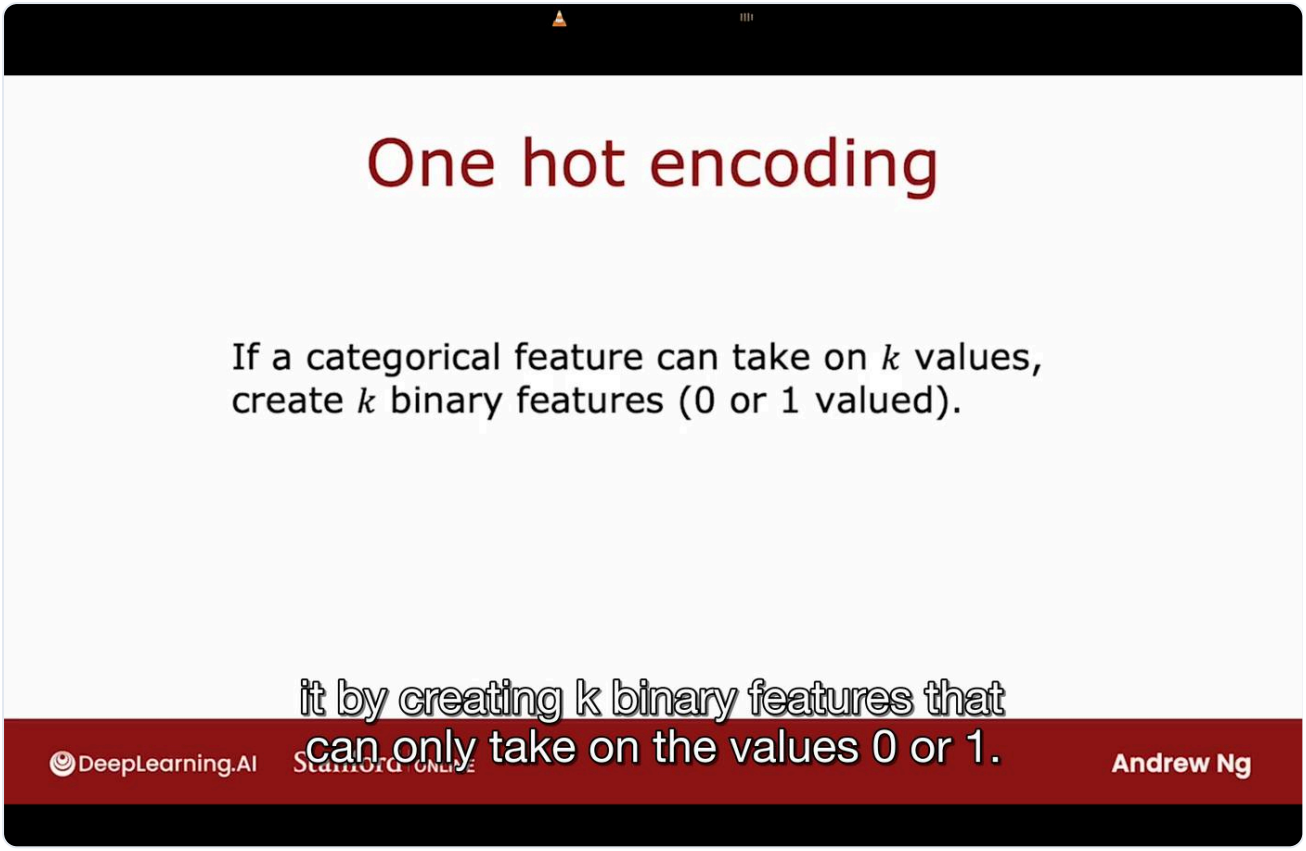


FIGURE 8.4 | One Hot Encoding

In a little bit more detail, if a categorical feature can take on k possible values, k was three in our example, then we will replace it by creating k binary features that can only take on the values 0 or 1. And you notice that among all of these three features, if you look at any role here, exactly 1 of the values is equal to 1. And that's what gives this method of future construction the name one-hot encoding. And because one of these features will always take on the value 1 that's the hot feature and hence the name one-hot encoding. And with this choice of features we're now back to the original setting of where each feature only takes on one of two possible values, and so the decision tree learning algorithm that we've seen previously will apply to this data with no further modifications.

Just an aside, even though this week's material has been focused on training decision tree models the idea of using one-hot encodings to encode categorical features also works for training neural networks. In particular if you were to take the face shape feature and replace round and not round with 1 and 0 where round gets matter 1, not round gets matter 0 and so on. And for whiskers similarly replace presence with 1 and absence with 0.

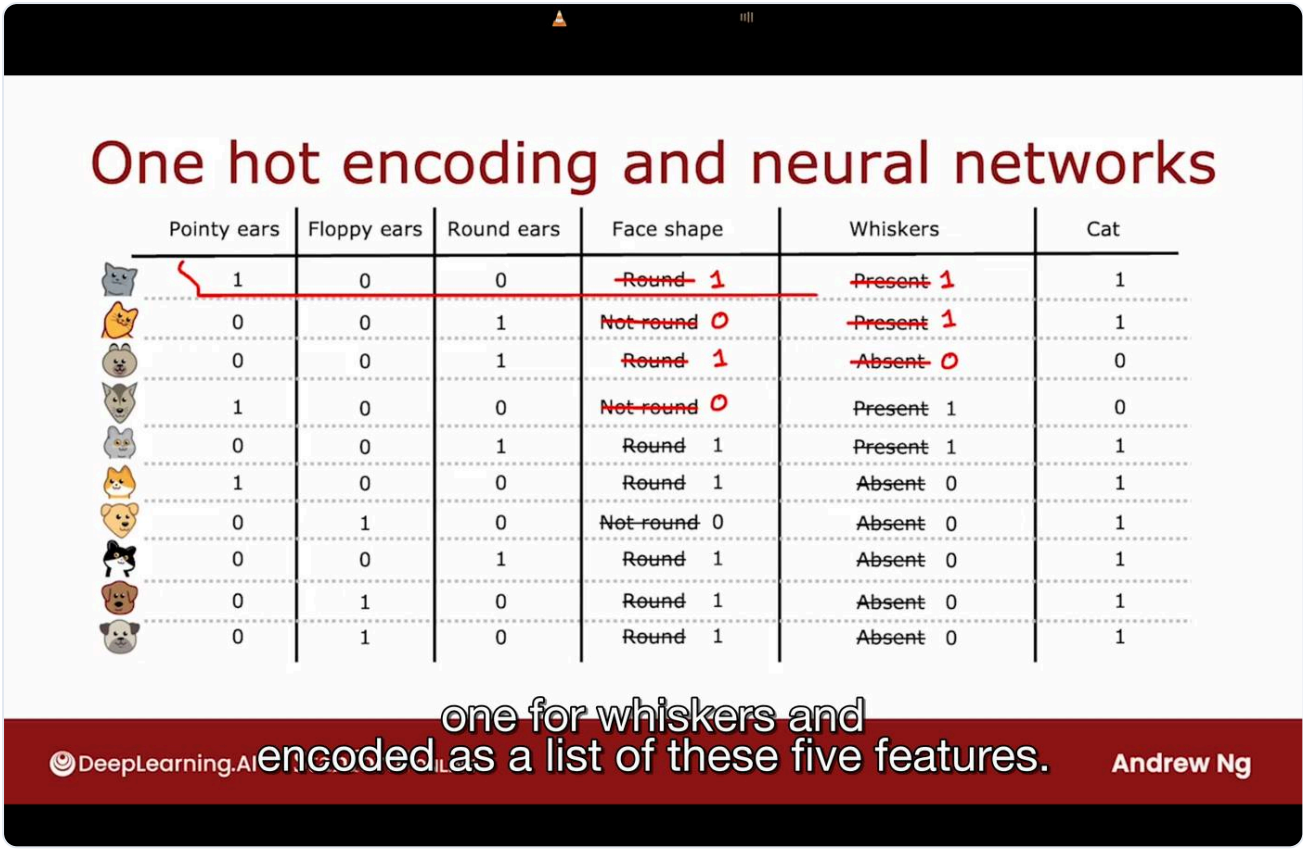


FIGURE 8.5 | One Hot Encoding and Neural Networks

They noticed that we have taken all the categorical features we had where we had three possible values for ear shape, two for face shape and one for whiskers and encoded as a list of these five features. Three from the one-hot encoding of ear shape, one from face shape and from whiskers and now this list of five features can also be fed to a new network or to logistic regression to try to train a cat classifier. So one-hot encoding is a technique that works not just for decision tree learning but also lets you encode categorical features using ones and zeros, so that it can be fed as inputs to a neural network as well which expects numbers as inputs.

So that's it, with a one-hot encoding you can get your decision tree to work on features that can take on more than two discrete values and you can also apply this to neural networks or linear regression or logistic regression training. But how about features that are numbers that can take on any value, not just a small number of discrete values. In the next video let's look at how you can get the decision tree to handle continuous value features that can be any number.

LECTURE 56

Not Cat

Let's look at how you can modify decision tree to work with features that aren't just discrete values but continuous values. That is features that can be any number.

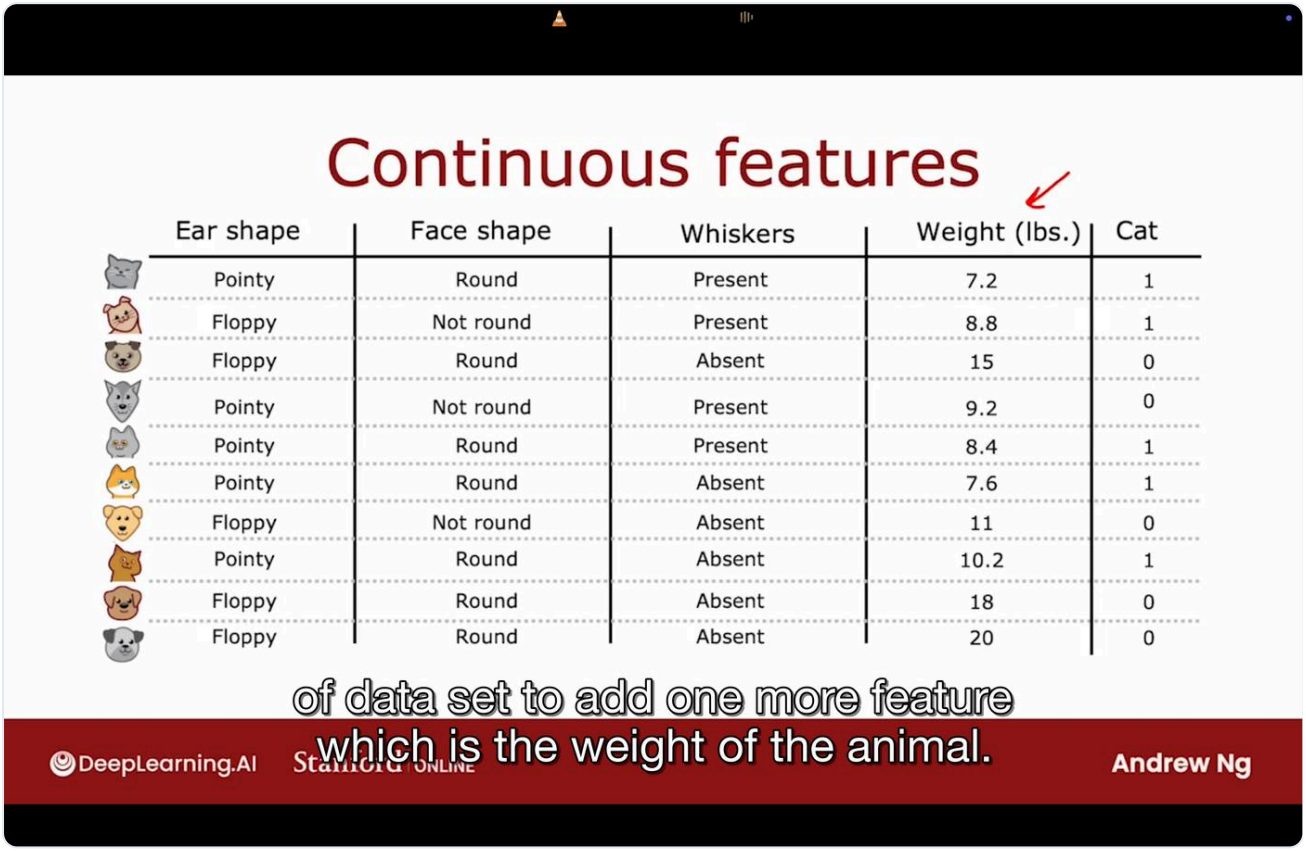


FIGURE 9.1 | Continuous Features

Let's start with an example, I have modified the cat adoption center of data set to add one more feature which is the weight of the animal. In pounds on average between cats and dogs, cats are a little bit lighter than dogs, although there are some cats are heavier than some dogs. But so the weight of an animal is a useful feature for deciding if it is a cat or not. So how do you get a decision tree to use a feature like this? The decision tree learning algorithm will proceed similarly as before except that rather than constraint splitting just on ear shape, face shape and whiskers. You have to consist splitting on ear shape, face shape whisker or weight.

And if splitting on the weight feature gives better information gain than the other options. Then you will split on the weight feature. But how do you decide how to split on the weight feature? Let's take a look. Here's a plot of the data at the root. Not a plotted on the horizontal axis. The way to the animal and the vertical axis is cat on top and not cat below. So the vertical axis indicates the label, y being 1 or 0. The way we were split on the weight feature would be if we were to split the data based on whether or not the weight is less than or equal to some value. Let's say 8 or some of the number. That will be the job of the learning algorithm to choose.

And what we should do when constraints splitting on the weight feature is to consider many different values of this threshold and then to pick the one that is the best. And by the best I mean the one that results in the best information gain. So in particular, if you were considering splitting the examples based on whether the weight is less than or equal to 8, then you will be splitting this data set into two subsets. Where the subset on the left has two cats and the subset on the right has three cats and five dogs.

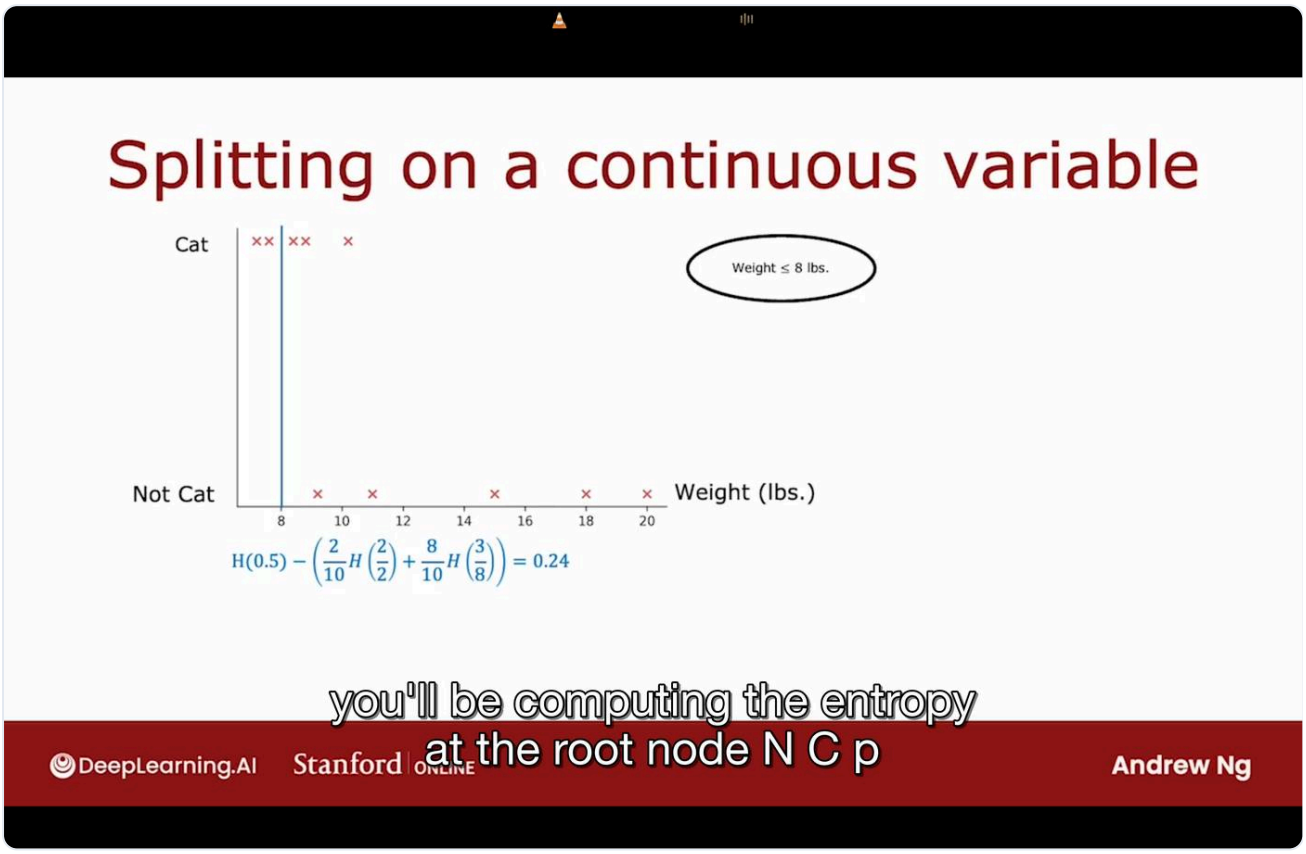


FIGURE 9.2 | Not Cat

So if you were to calculate our usual information gain calculation, you'll be computing the entropy at the root node N C p f 0.5 minus now 2/10 times entropy of the left split has two other two cats. So it should be 2/2 plus the right split has eight out of 10 examples and an entropy F. That's of the eight examples on the right three cats. To entry of 3/8 and this turns out to be 0.24. So this would be information gain if you were to split on whether the weight is less than equal to 8 but we should try other values as well. So what if you were to split on whether or not the weight is less than equal to 9 and that corresponds to this new line over here.

And the information gain calculation becomes H (0.5) minus.

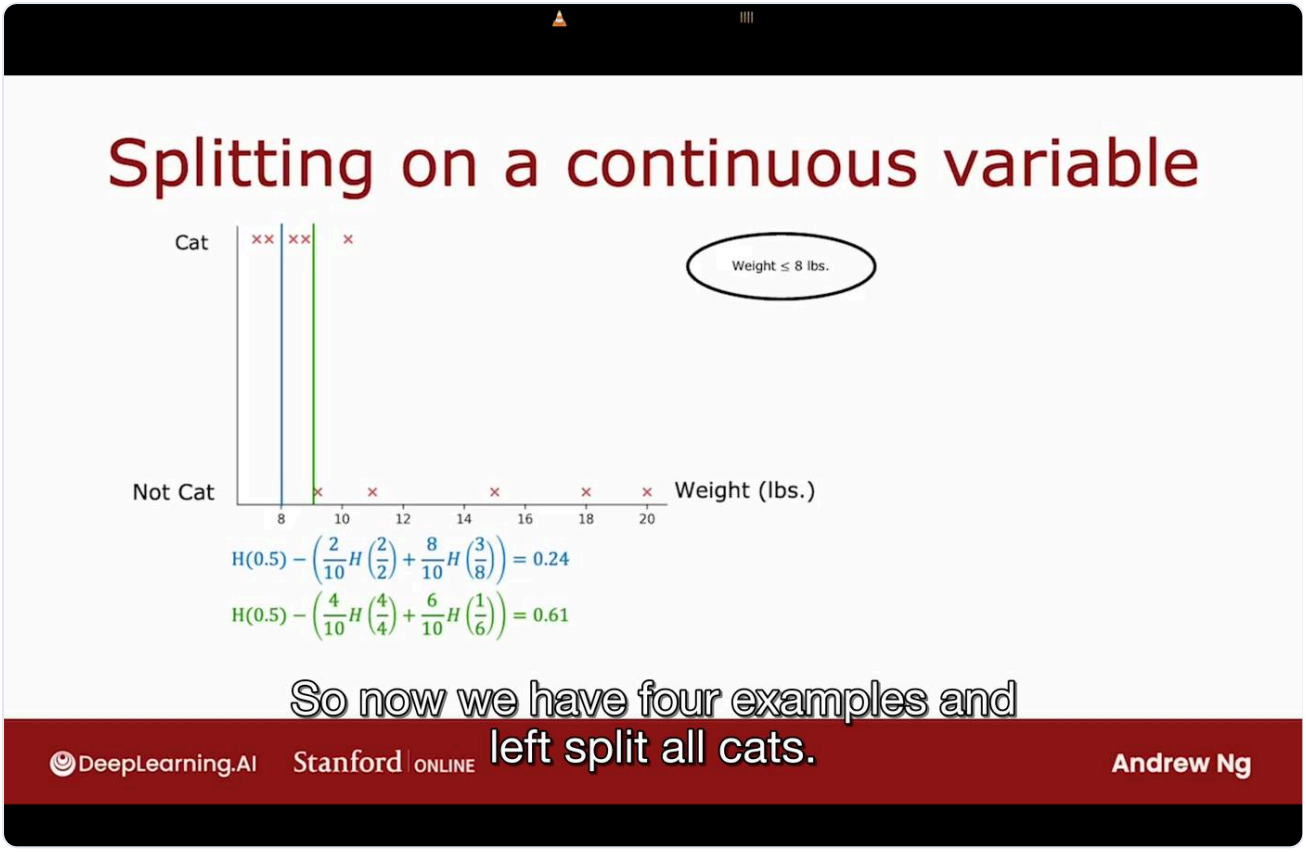


FIGURE 9.3 | Not Cat

So now we have four examples and left split all cats. So that's 4/10 times entropy of 4/4 plus six examples on the right of which you have one cat. So that's 6/10 times each of 1/6, which is equal to turns out 0.61. So the information gain here looks much better is 0.61 information gain which is much higher than 0.24. Or we could try another value say 13. And the calculation turns out to look like this, which is 0.40.

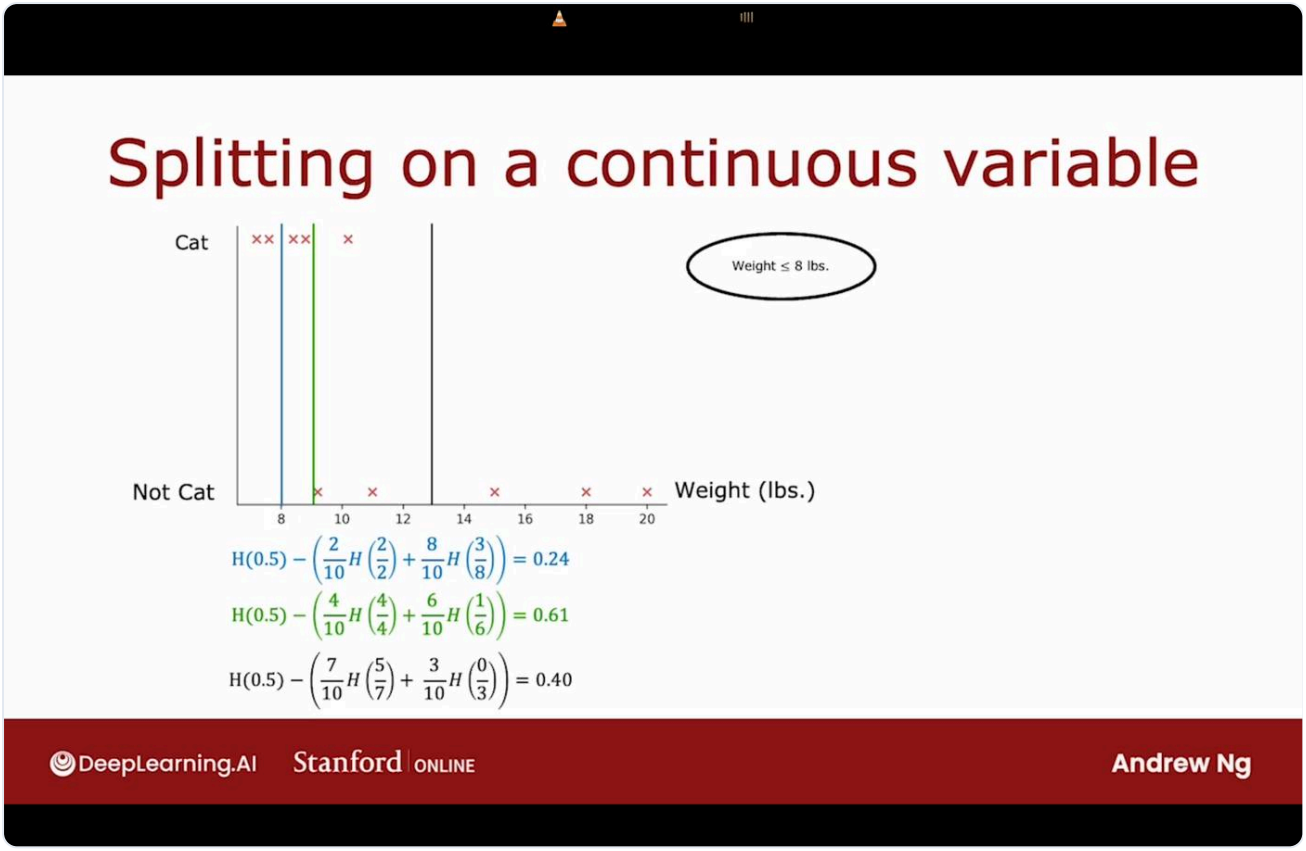


FIGURE 9.4 | Splitting on

In the more general case, we'll actually try not just three values, but multiple values along the X axis. And one convention would be to sort all of the examples according to the weight or according to the value of this feature and take all the values that are mid points between the sorted list of training. Examples as the values for consideration for this threshold over here. This way, if you have 10 training examples, you will test nine different possible values for this threshold and then try to pick the one that gives you the highest information gain. And finally, if the information gained from splitting on a given value of this threshold is better than the information gain from splitting on any other feature, then you will decide to split that node at that feature.

And in this example an information gain of 0.61 turns out to be higher than that of any other feature. It turns out they're actually two thresholds.

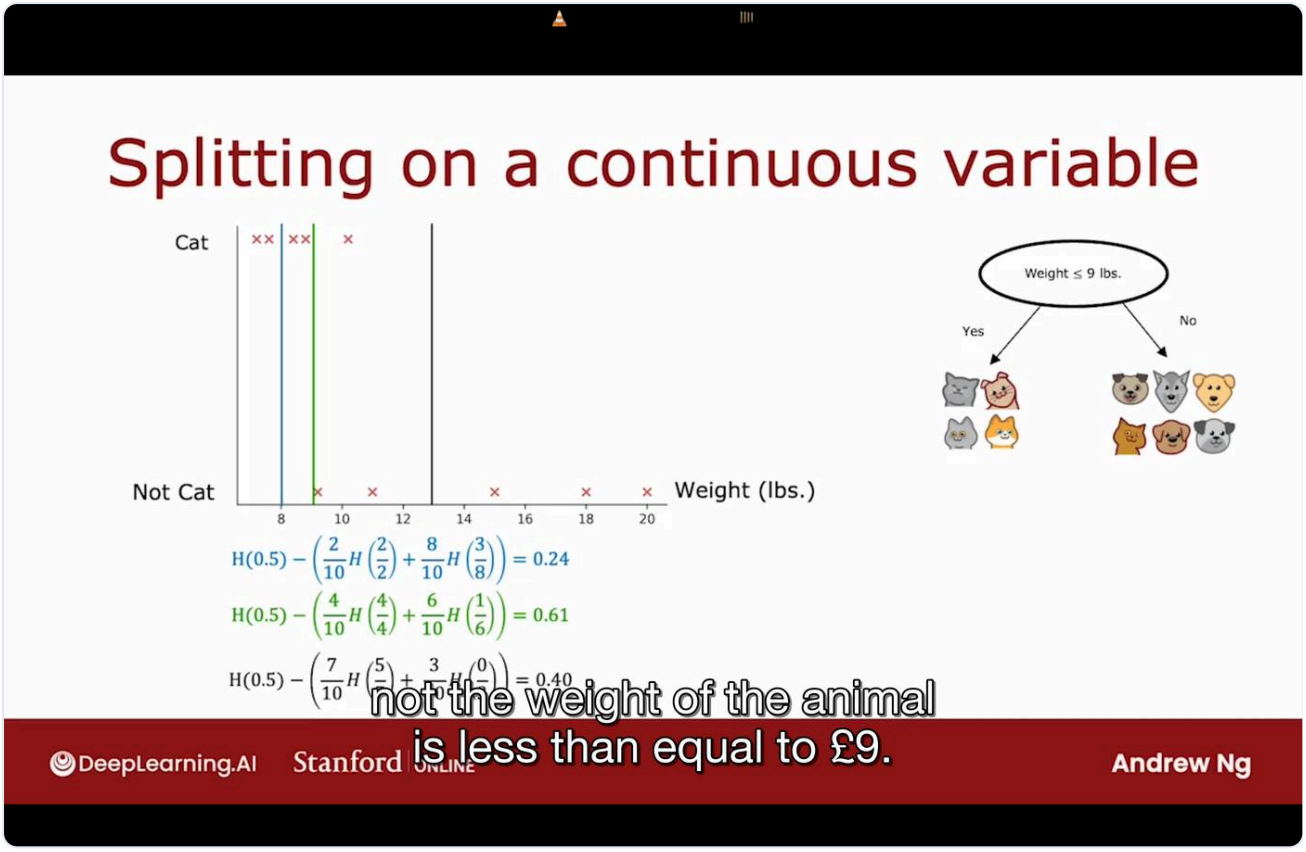


FIGURE 9.5 | Splitting on a Continuous Variable

And so assuming the algorithm chooses this feature to split on, you will end up splitting the data set according to whether or not the weight of the animal is less than equal to £9. And so you end up with two subsets of the data like this and you can then build recursively, additional decision trees using these two subsets of the data to build out the rest of the tree. So to summarize to get the decision tree to work on continuous value features at every note. When consuming splits, you would just consider different values to split on, carry out the usual information gain calculation and decide to split on that continuous value feature if it gives the highest possible information gain.

So that's how you get the decision tree to work with continuous value features. Try different thresholds, do the usual information gain calculation and split on the continuous value feature with the selected threshold if it gives you the best possible information gain out of all possible features to split on. And that's it for the required videos on the core decision tree algorithm After there's there is an optional video you can watch or not that generalizes the decision tree learning algorithm to regression trees. So far, we've only talked about using decision trees to make predictions that are classifications predicting a discrete category, such as cat or not cat.

But what if you have a regression problem where you want to predict a number in the next video. I'll talk about a generalization of decision trees to handle that.

LECTURE 57

Regression with Decision Trees

So far we've only been talking about decision trees as classification algorithms. In this optional video, we'll generalize decision trees to be regression algorithms so that we can predict a number. Let's take a look.

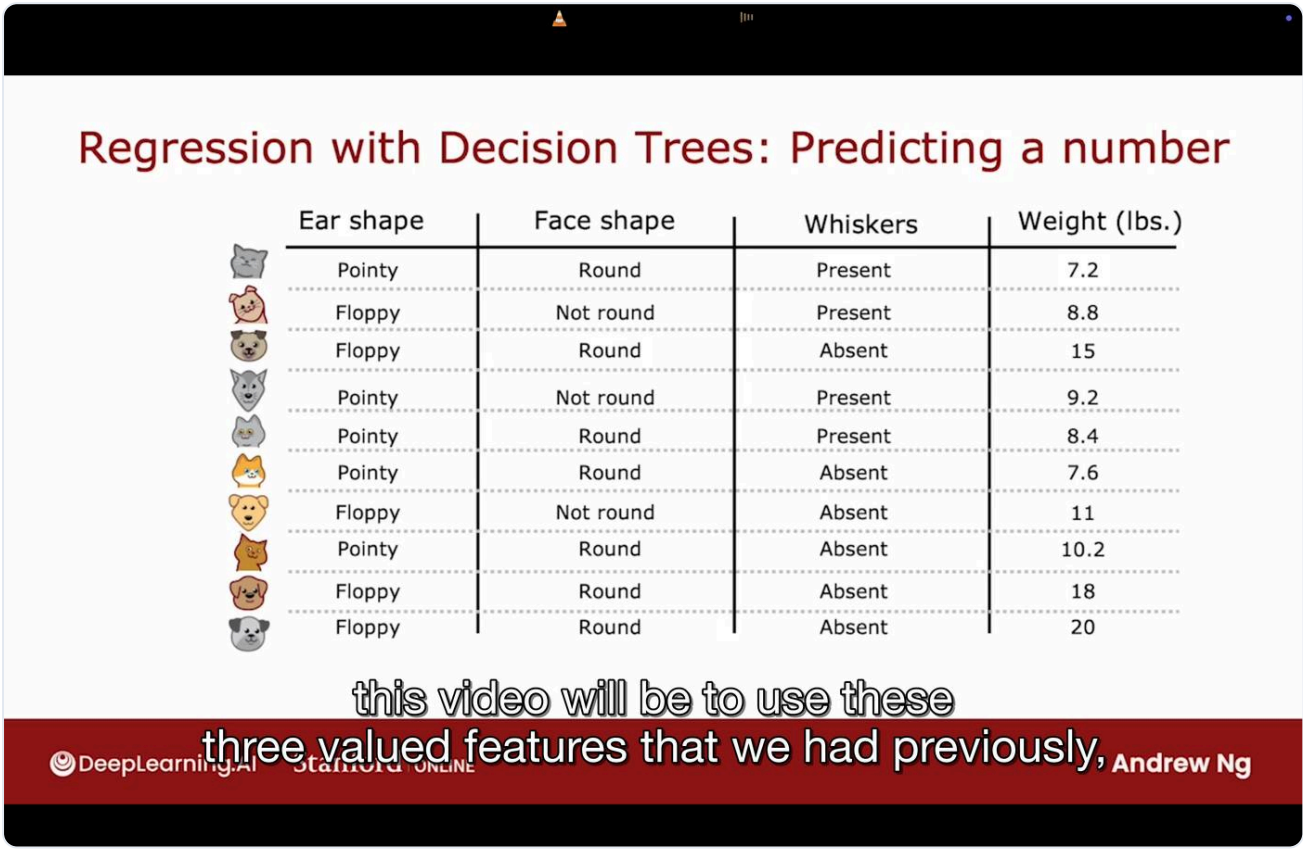


FIGURE 10.1 | Regression with Decision Trees: Predicting a Number

The example I'm going to use for this video will be to use these three valued features that we had previously, that is, these features X, In order to predict the weight of the animal, Y. So just to be clear, the weight here, unlike the previous video is no longer an input feature. Instead, this is the target output, Y, that we want to predict rather than trying to predict whether or not an animal is or is not a cat.

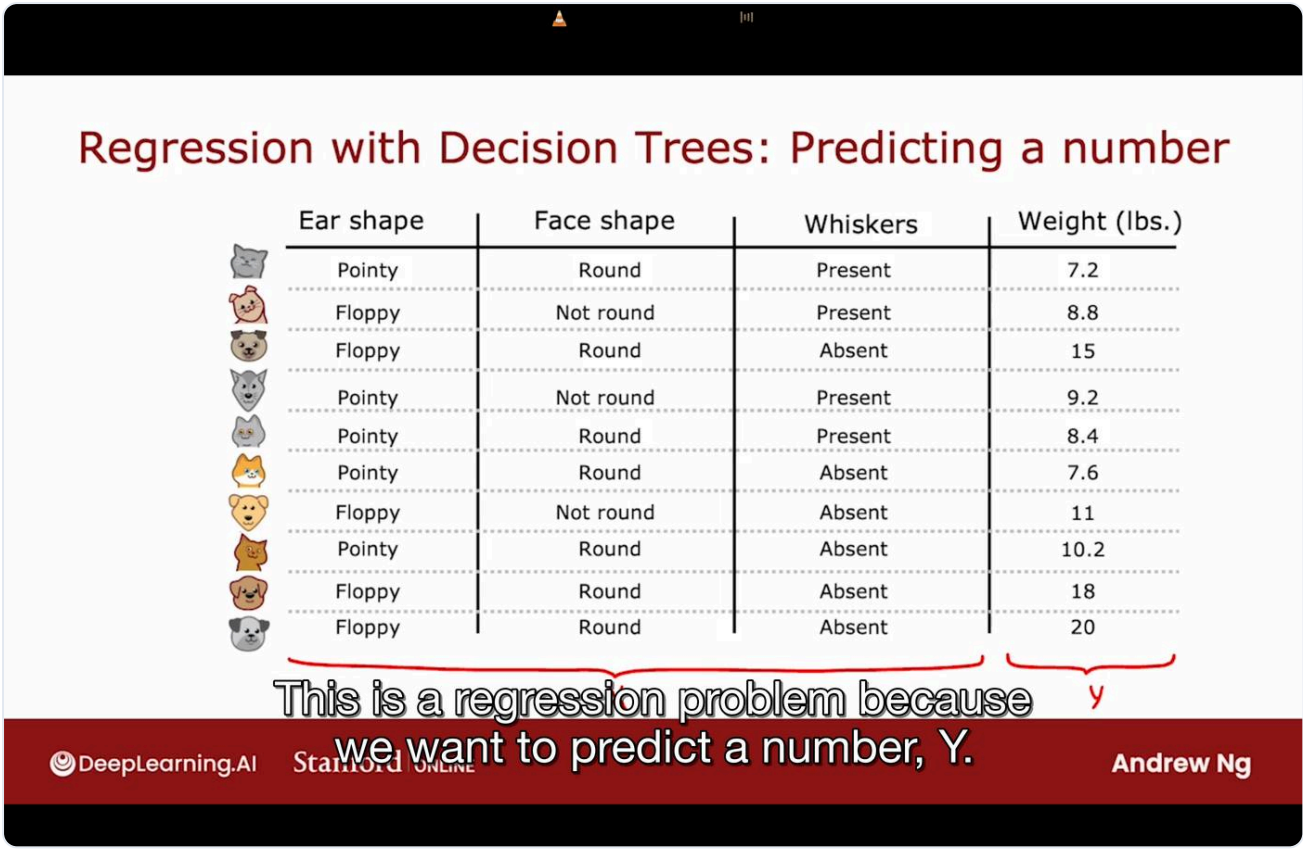


FIGURE 10.2 | Regression with Decision Trees: Predicting a Number

This is a regression problem because we want to predict a number, Y. Let's look at what a regression tree will look like. Here I've already constructed a tree for this regression problem where the root node splits on ear shape and then the left and right sub tree split on face shape and also face shape here on the right.

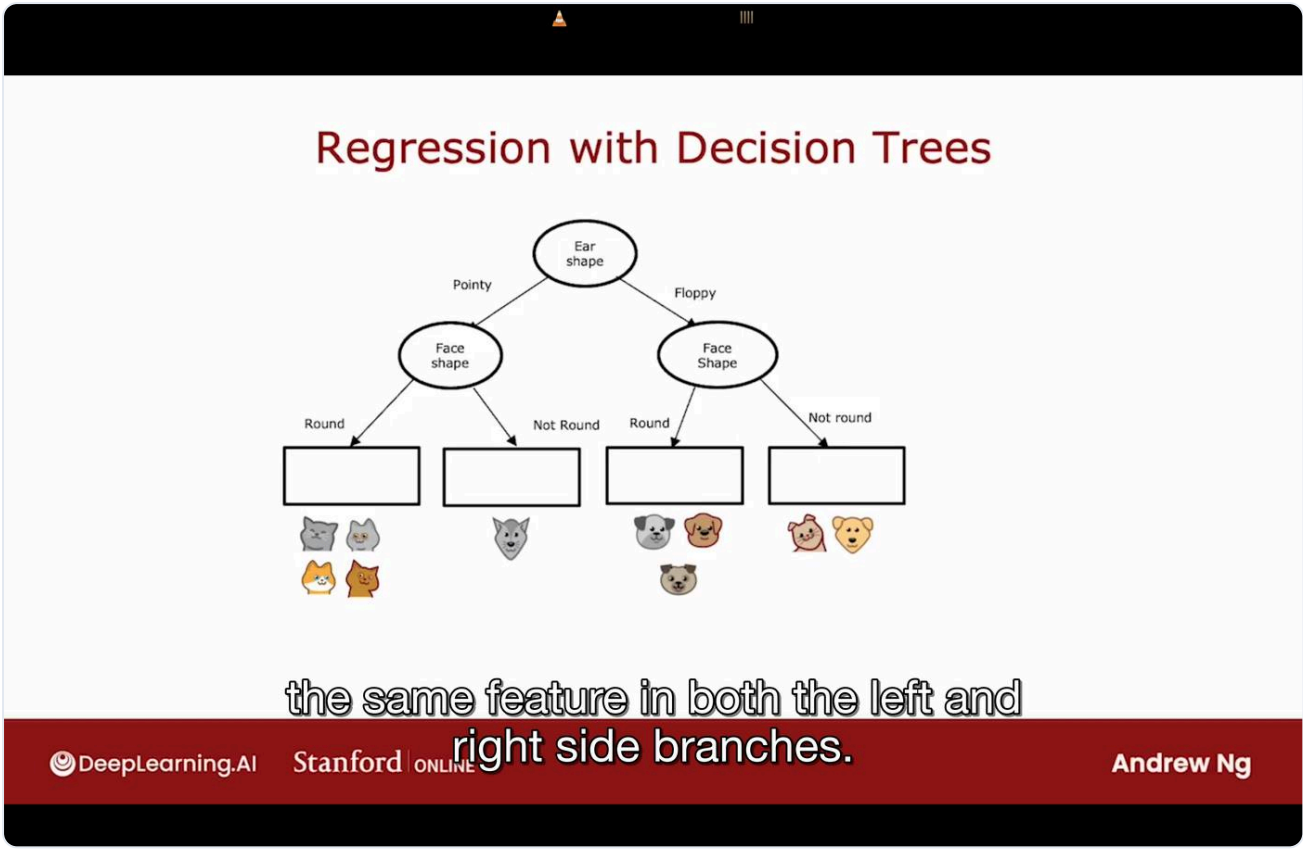


FIGURE 10.3 | Regression with Decision Trees

And there's nothing wrong with a decision tree that chooses to split on the same feature in both the left and right side branches. It's perfectly fine if the splitting algorithm chooses to do that. If during training, you had decided on these splits, then this node down here would have these four animals with weights 7.2, 7.6 and 10.2.

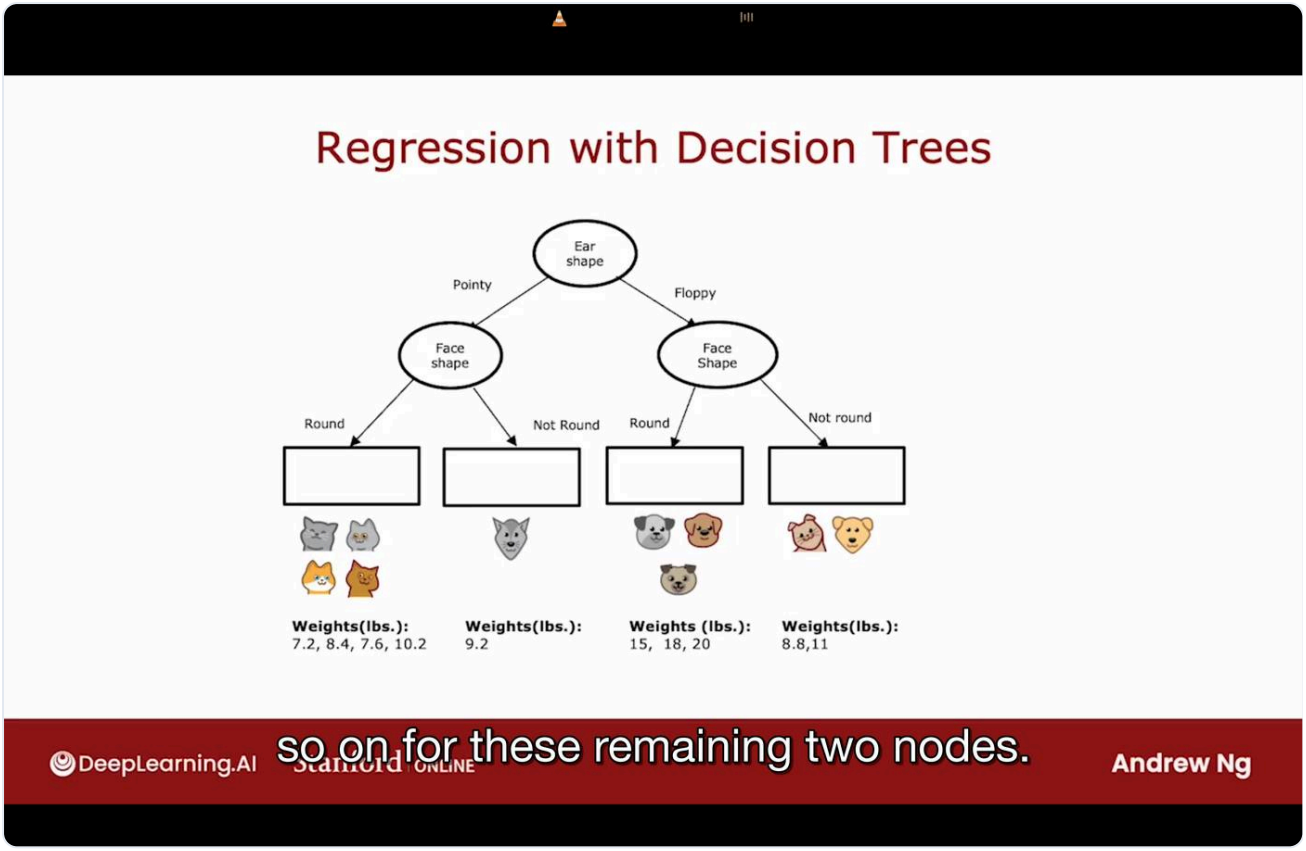


FIGURE 10.4 | Regression with Decision Trees

This node would have this one animal with weight 9.2 and so on for these remaining two nodes. So, the last thing we need to fill in for this decision tree is if there's a test example that comes down to this node, what is there weights that we should predict for an animal with pointy ears and a round face shape? The decision tree is going to make a prediction based on taking the average of the weights in the training examples down here. And by averaging these four numbers, it turns out you get 8.35. If on the other hand, an animal has pointy ears and a not round face shape, then it will predict 9.2 or 9.2 pounds because that's the weight of this one animal down here.

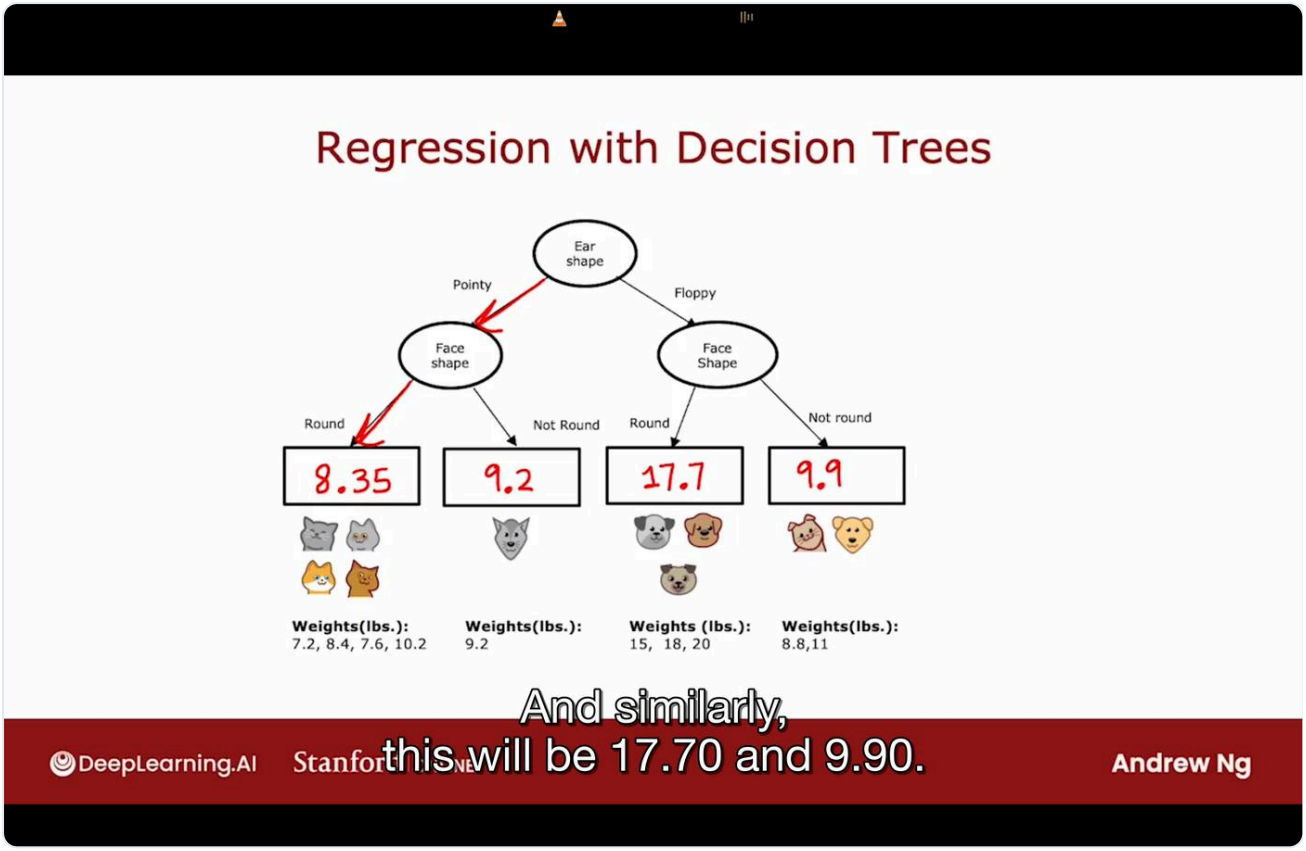


FIGURE 10.5 | Regression with Decision Trees

And similarly, this will be 17.70 and 9.90. So, what this model will do is given a new test example, follow the decision nodes down as usual until it gets to a leaf node and then predict that value at the leaf node which I had just computed by taking an average of the weights of the animals that during training had gotten down to that same leaf node. So, if you were constructing a decision tree from scratch using this data set in order to predict the weight. The key decision as you've seen earlier this week will be, how do you choose which feature to split on? Let me illustrate how to make that decision with an example. At the root node, one thing you could do is split on the ear shape and if you do that, you end up with left and right branches of the tree with five animals on the left and right with the following weights.

If you were to choose the split on the face shape, you end up with these animals on the left and right with the corresponding weights that are written below.

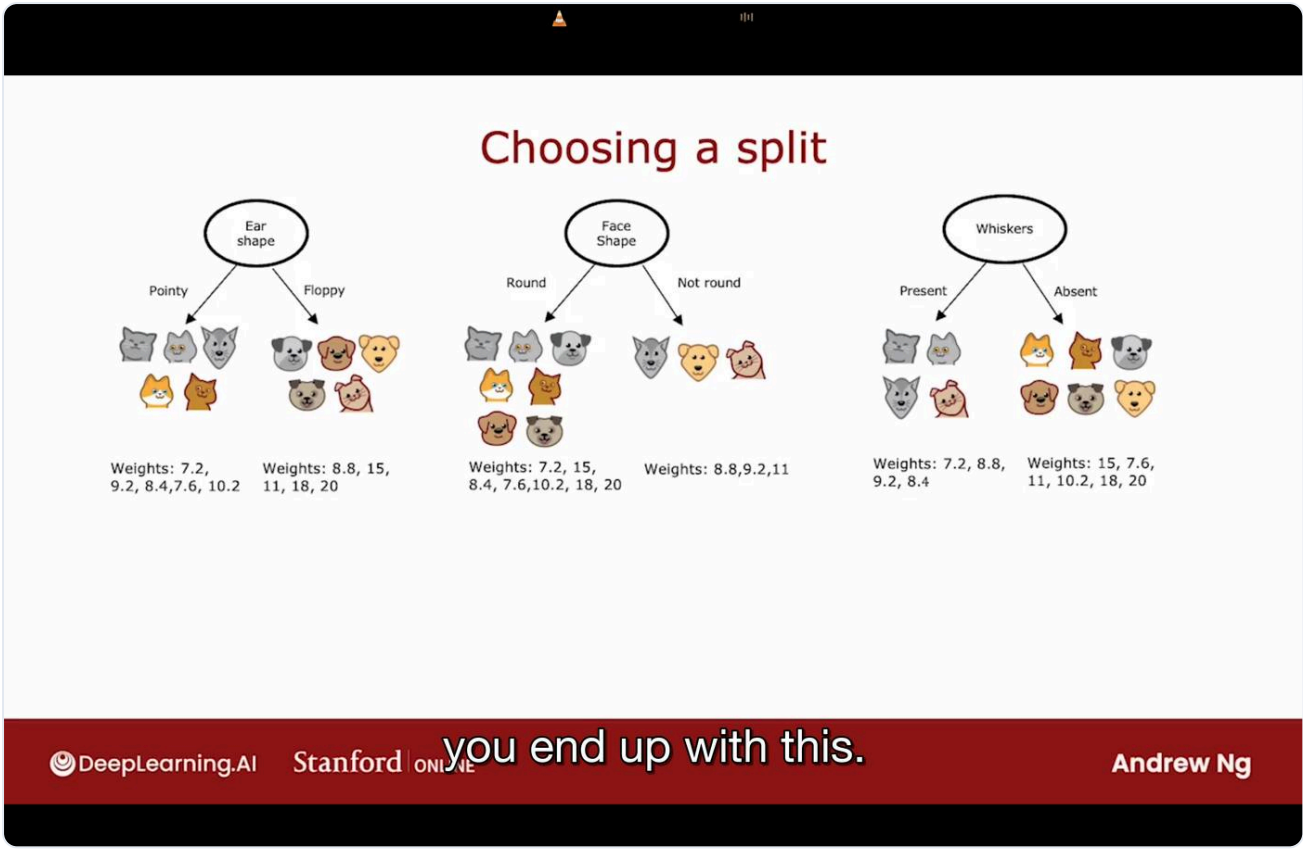


FIGURE 10.6

And if you were to choose to split on whiskers being present or absent, you end up with this. So, the question is, given these three possible features to split on at the root node, which one do you want to pick that gives the best predictions for the weight of the animal? When building a regression tree, rather than trying to reduce entropy, which was that measure of impurity that we had for a classification problem, we instead try to reduce the variance of the weight of the values Y at each of these subsets of the data. So, if you've seen the notion of variants in other contexts, that's great. This is the statistical mathematical notion of variants that we'll used in a minute.

But if you've not seen how to compute the variance of a set of numbers before, don't worry about it. All you need to know for this slide is that variants informally computes how widely a set of numbers varies. So for this set of numbers 7.2, 9.2 and so on, up to 10.2, it turns out the variance is 1.47, so it doesn't vary that much. Whereas, here 8.8, 15, 11, 18 and 20, these numbers go all the way from 8.8 all the way up to 20. And so the variance is much larger, turns out to the variance of 21.87. And so the way we'll evaluate the quality of the split is, we'll compute same as before, W left and W right as the fraction of examples that went to the left and right branches.

And the average variance after the split is going to be 5/10, which is W left times 1.47, which is the variance on the left and then plus 5/10 times the variance on the right, which is 21.87. So, this weighted average variance plays a very similar role to the weighted average entropy that we had used when deciding what split to use for a classification problem. And we can then repeat this calculation for the other possible choices of features to split on. Here in the tree in the middle, the variance of these numbers here turns out to be 27.80. The variance here is 1.37. And so with W left equals seven-tenths and W right as three-tenths, and so with these values, you can compute the weighted variance as follows.

Finally, for the last example, if you were to split on the whiskers feature, this is the variance on the left and right, there's W left and W right. And so the weight of variance is this.

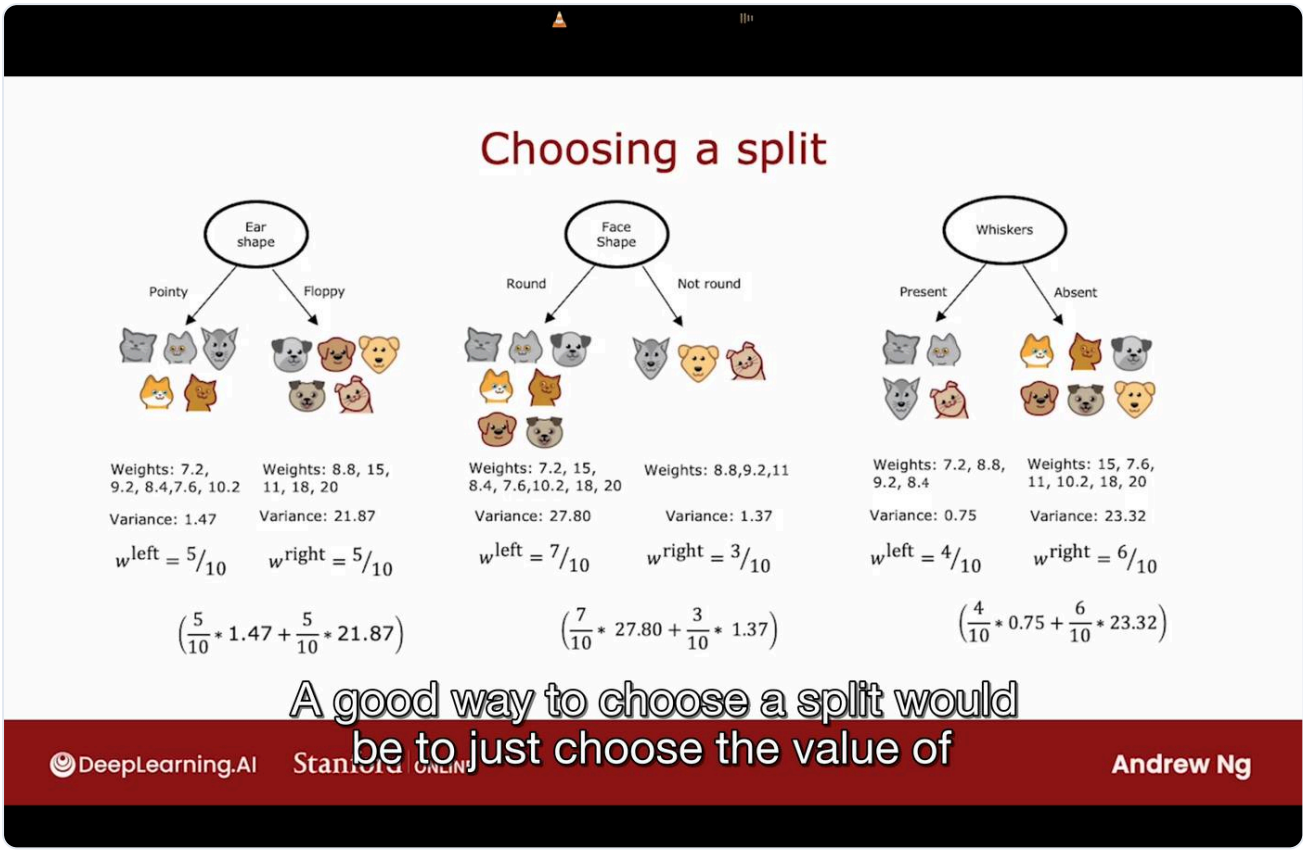


FIGURE 10.7

A good way to choose a split would be to just choose the value of the weighted variance that is lowest. Similar to when we're computing information gain, I'm going to make just one more modification to this equation. Just as for the classification problem, we didn't just measure the average weighted entropy, we measured the reduction in entropy and that was information gain. For a regression tree we'll also similarly measure the reduction in variance. Turns out, if you look at all of the examples in the training set, all ten examples and compute the variance of all of them, the variance of all the examples turns out to be 20.51.

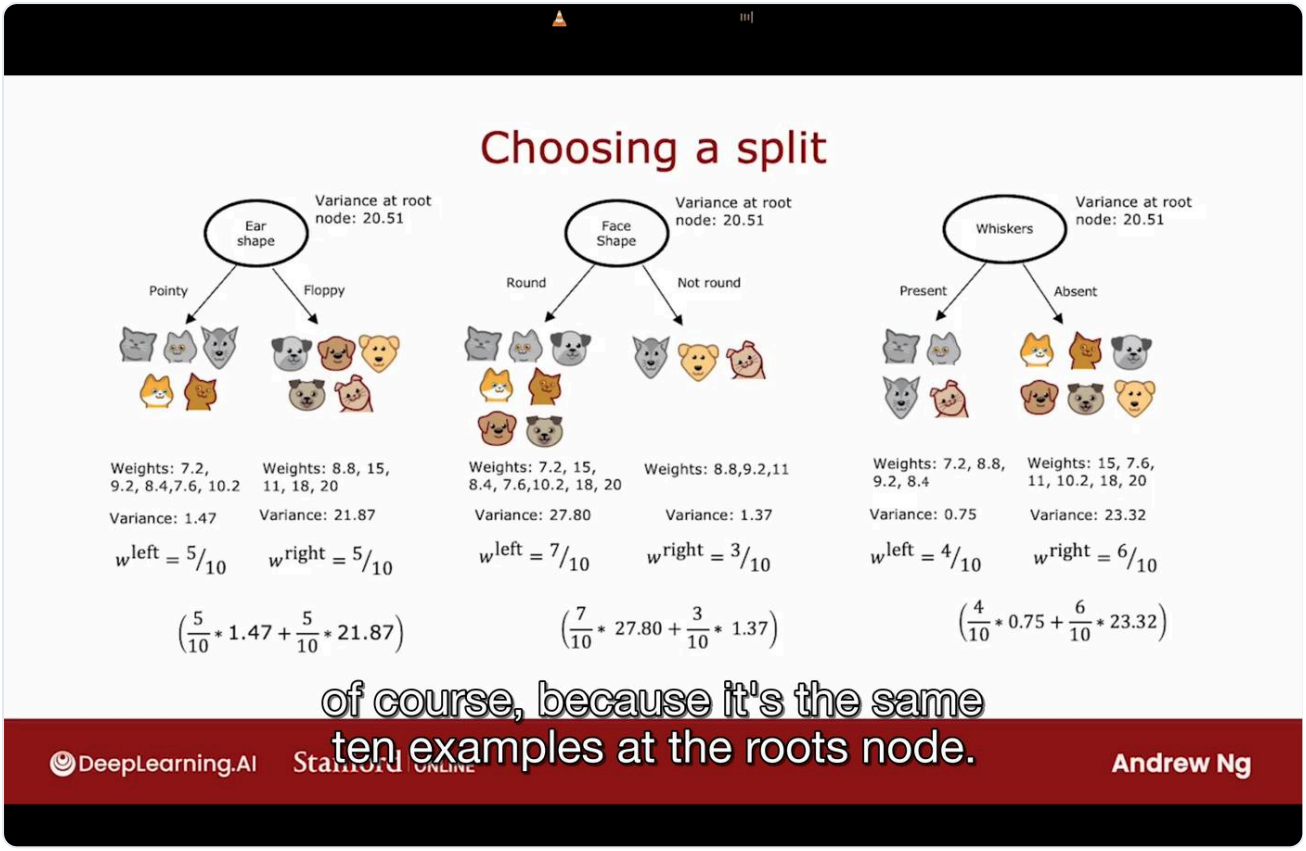


FIGURE 10.8 | Variance at Root

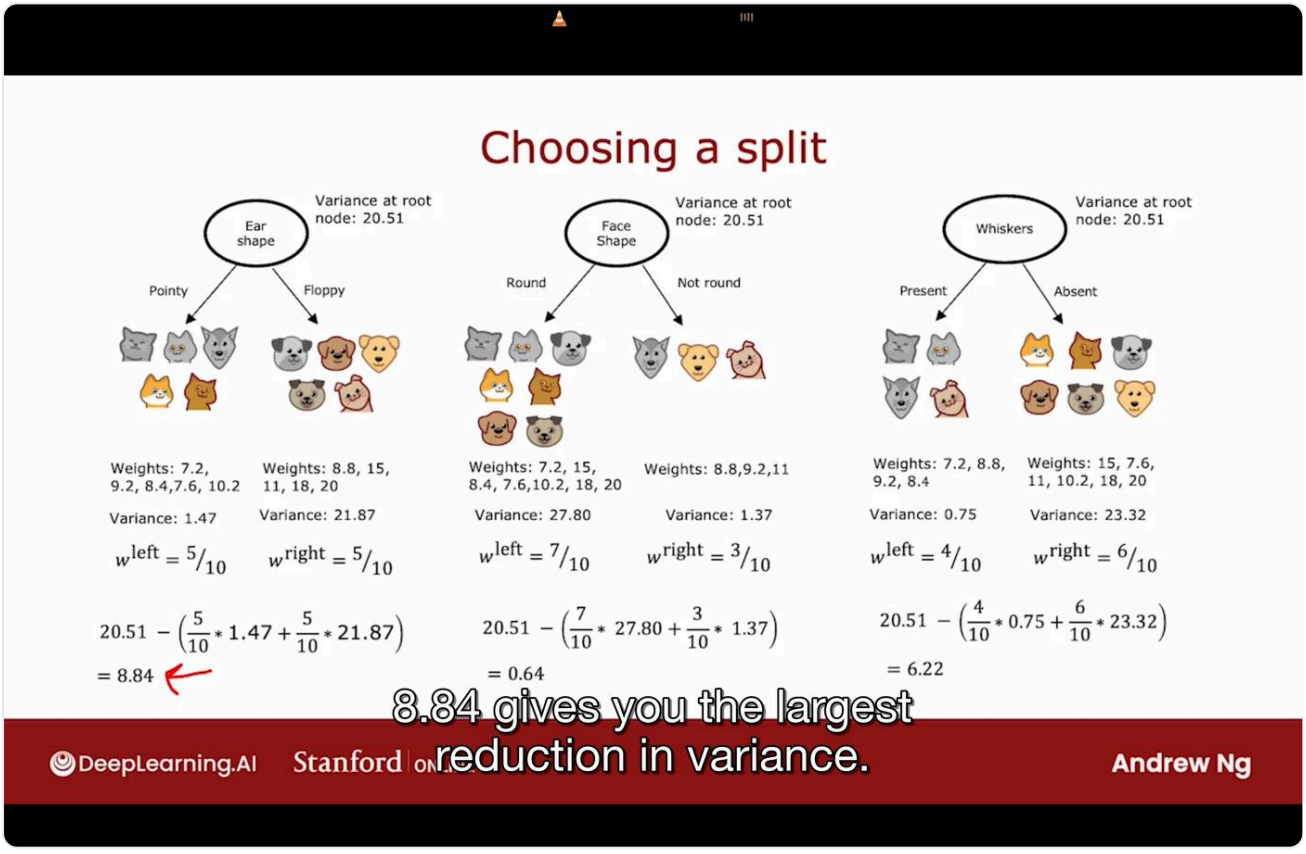


FIGURE 10.9 | Variance at Root

And that's the same value for the roots node in all of these, of course, because it's the same ten examples at the roots node. And so what we'll actually compute is the variance of the roots node, which is 20.51 minus this expression down here, which turns out to be equal to 8.84. And so at the roots node, the variance was 20.51 and after splitting on ear shape, the average weighted variance at these two nodes is 8.84 lower. So, the reduction in variance is 8.84. And similarly, if you compute the expression for reduction in variance for this example in the middle, it's 20.51 minus this expression that we had before, which turns out to be equal to 0.64.

So, this is a very small reduction in variance. And for the whiskers feature you end up with this which is 6.22. So, between all three of these examples, 8.84 gives you the largest reduction in variance. So, just as previously we would choose the feature that gives you the largest information gain for a regression tree, you will choose the feature that gives you the largest reduction in variance, which is why you choose ear shape as the feature to split on. Having chosen the ear shape feature to split on, you now have two subsets of five examples in the left and right side branches and you would then, again, we say recursively, where you take these five examples and do a new decision tree focusing on just these five examples, again, evaluating different options of features to split on and picking the one that gives you the biggest variance reduction.

And similarly on the right. And you keep on splitting until you meet the criteria for not splitting any further. And so that's it. With this technique, you can get your decision treat to not just carry out classification problems, but also regression problems. So far, we've talked about how to train a single decision tree. It turns out if you train a lot of decision trees, we call this an ensemble of decision trees, you can get a much better result. Let's take a look at why and how to do so in the next video.

LECTURE 58

Tree Ensemble

One of the weaknesses of using a single decision tree is that that decision tree can be highly sensitive to small changes in the data. One solution to make the algorithm less sensitive or more robust is to build not one decision tree, but to build a lot of decision trees, and we call that a tree ensemble. Let's take a look. With the example that we've been using, the best feature to split on at the root node turned out to be the ear shape resulting in these two subsets of the data and then building further sub trees on these two subsets of the data. But it turns out that if you were to take just one of the ten examples and change it to a different cat so that instead of having pointy ears, round face, whiskers absent, this new cat has floppy ears, round face, whiskers present, with just changing a single training example, the highest information gain feature to split on becomes the whiskers feature instead of the ear shape feature.

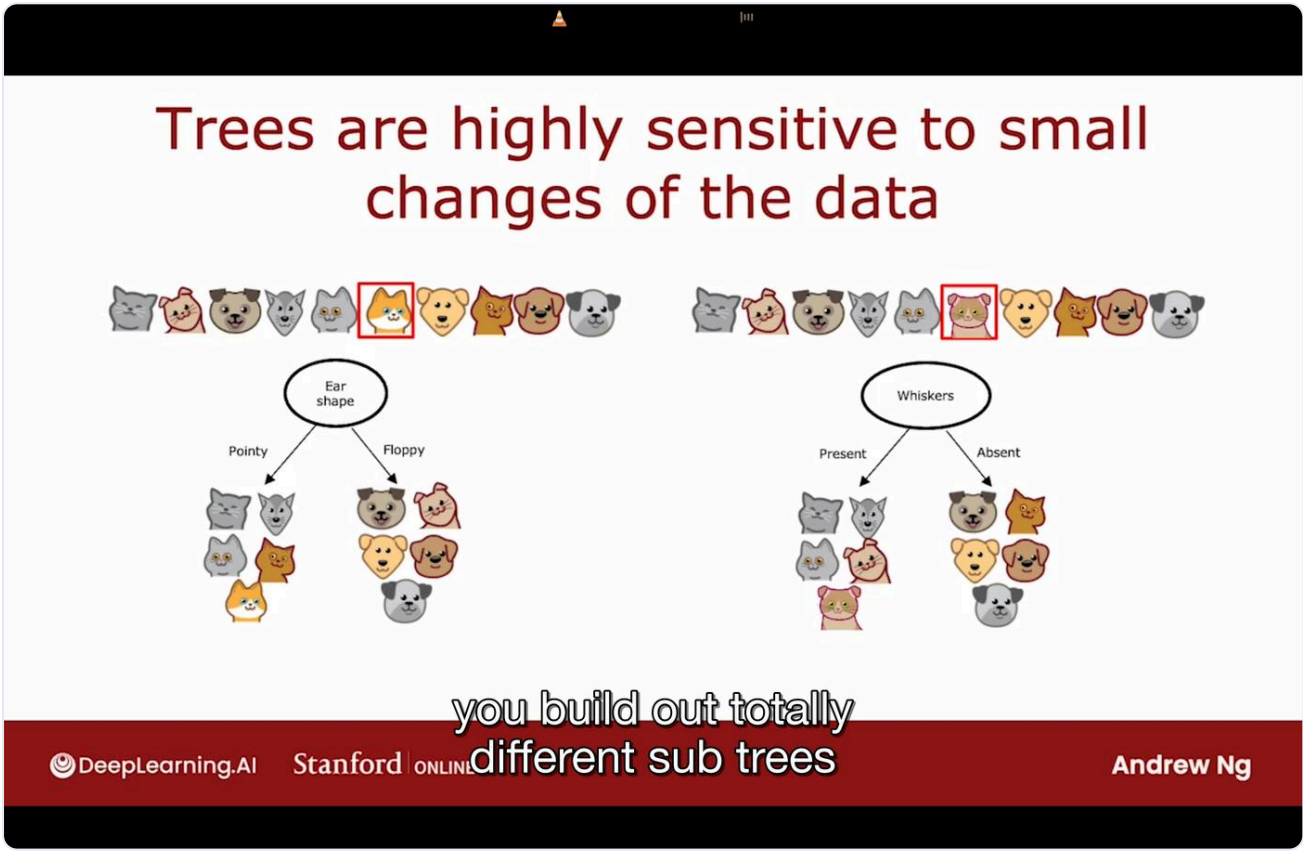


FIGURE 11.1 | Trees Are Highly Sensitive to Small

As a result of that, the subsets of data you get in the left and right sub-trees become totally different and as you continue to run the decision tree learning algorithm recursively, you build out totally different sub trees on the left and right. The fact that changing just one training example causes the algorithm to come up with a different split at the root and therefore a totally different tree, that makes this algorithm just not that robust. That's why when you're using decision trees, you often get a much better result, that is, you get more accurate predictions if you train not just a single decision tree but a whole bunch of different decision trees.

This is what we call a tree ensemble, which just means a collection of multiple trees. We'll see, in the next few videos, how to construct this ensemble of trees. But if you had this ensemble three trees, each one of these is maybe a plausible way to classify cat versus not cat. If you had a new test example that you wanted to classify, then what you would do is run all three of these trees on your new example and get them to vote on whether it's the final prediction.

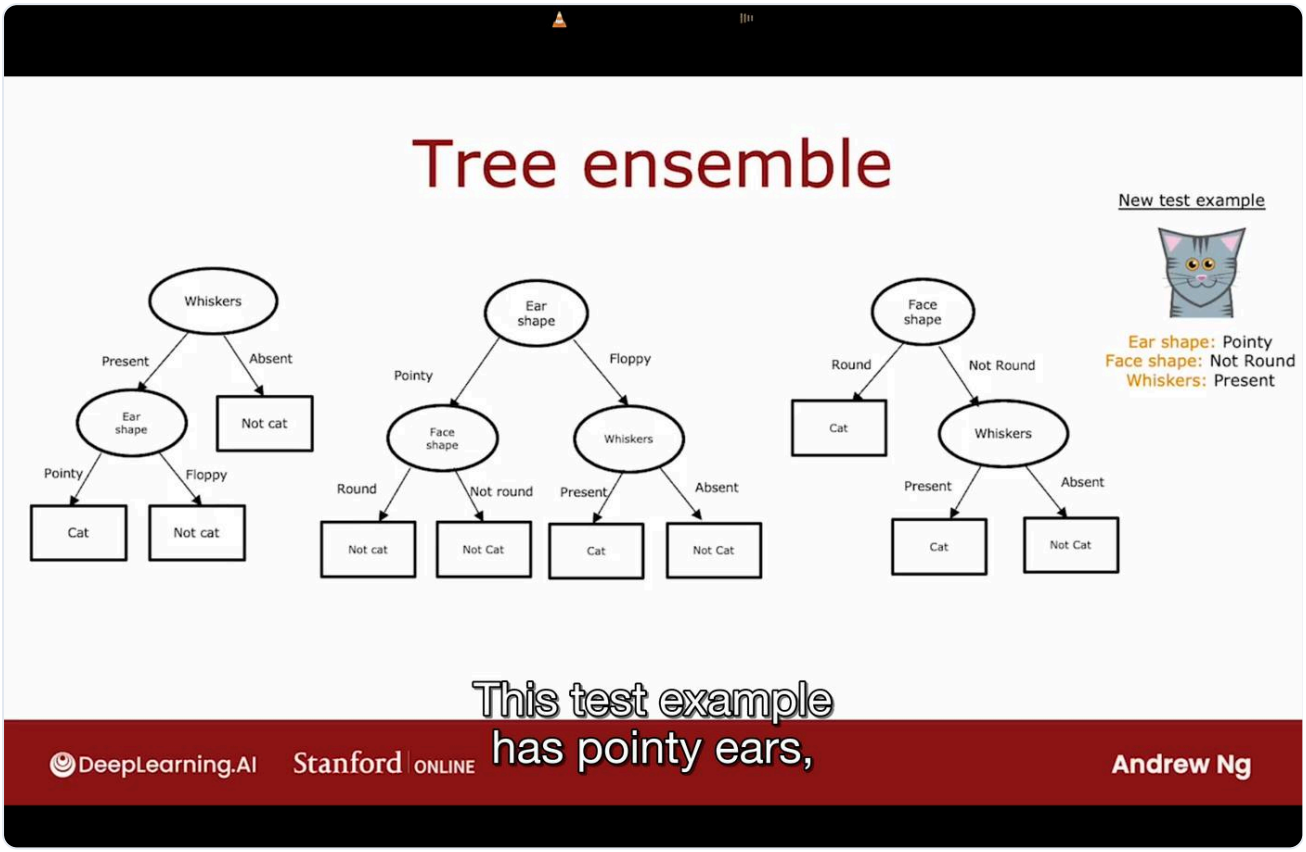


FIGURE 11.2 | Tree Ensemble

This test example has pointy ears, a not round face shape and whiskers are present and so the first tree would carry out inferences like this and predict that it is a cat. The second tree's inference would follow this path through the tree and therefore predict that is not cat. The third tree would follow this path and therefore predict that it is a cat.

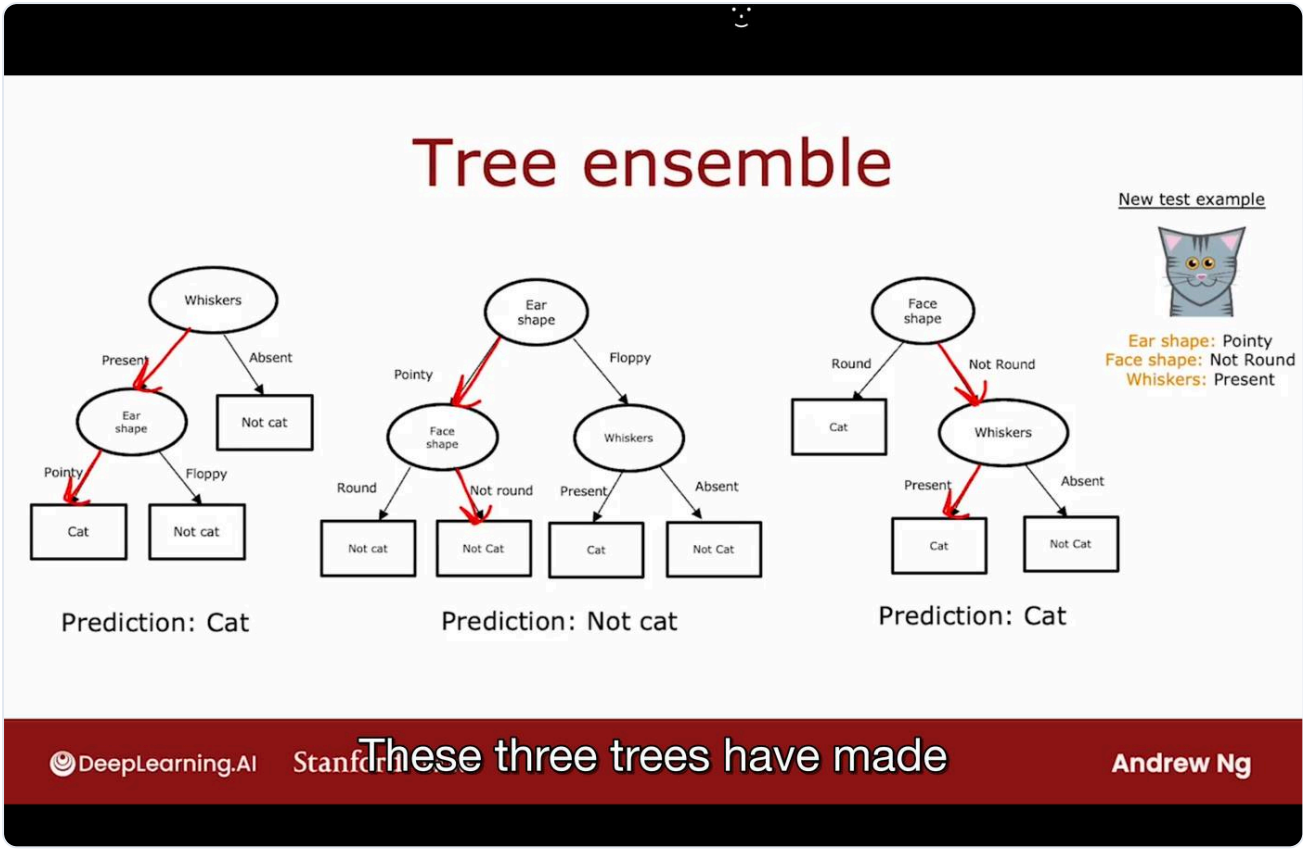


FIGURE 11.3 | Tree Ensemble

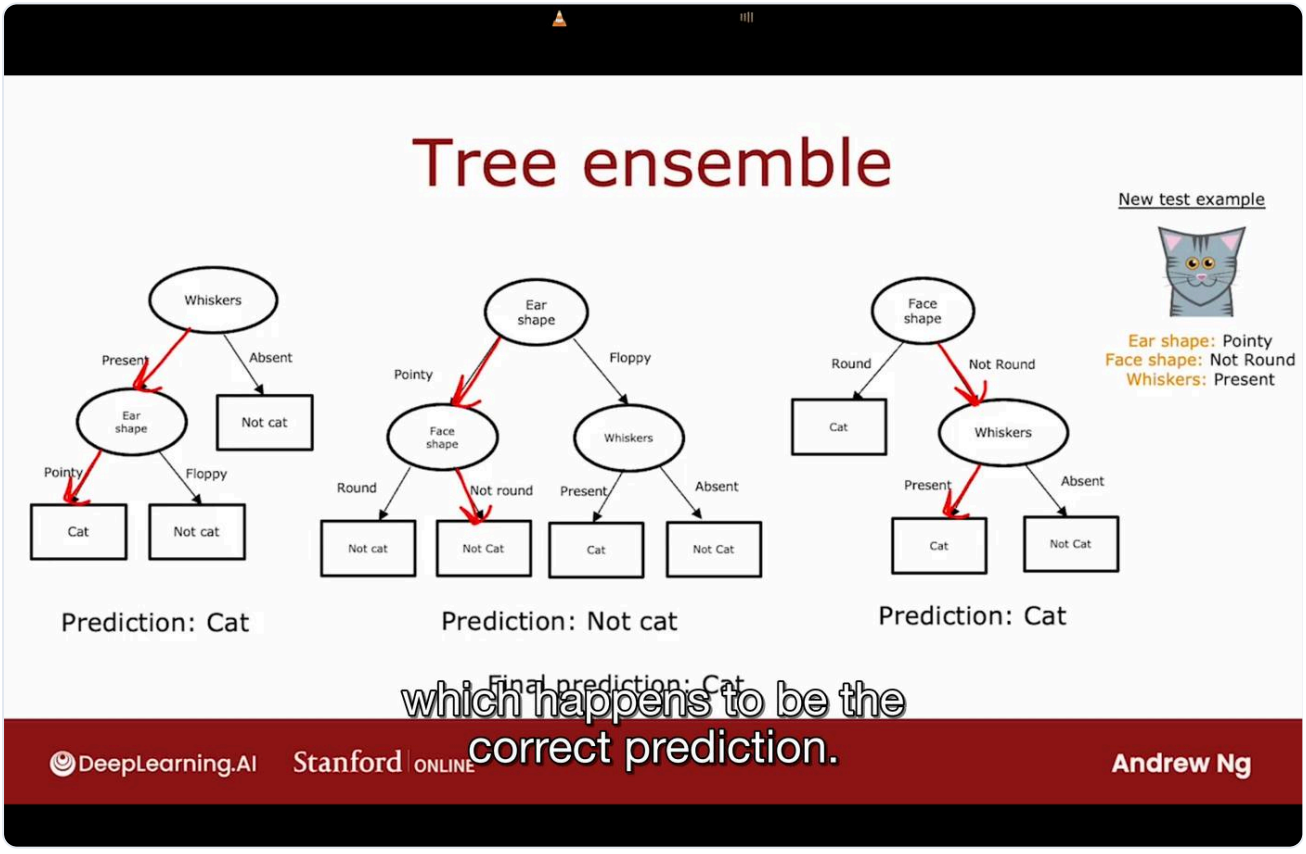


FIGURE 11.4 | Tree Ensemble

These three trees have made different predictions and so what we'll do is actually get them to vote. The majority votes of the predictions among these three trees is, cat. The final prediction of this ensemble of trees is that this is a cat which happens to be the correct prediction. The reason we use an ensemble of trees is by having lots of decision trees and having them vote, it makes your overall algorithm less sensitive to what any single tree may be doing because it gets only one vote out of three or one vote out of many, many different votes and it makes your overall algorithm more robust. But how do you come up with all of these different plausible but maybe slightly different decision trees in order to get them to vote?

In the next video, we'll talk about a technique from statistics called sampling with replacement and this will turn out to be a key technique that we'll use in the video after that in order to build this ensemble of trees. Let's go on to the next video to talk about sampling with replacement.

LECTURE 59

Sampling with Replacement

In order to build a tree ensemble, we're going to need a technique called sampling with replacement. Let's take a look at what that means. In order to illustrate how sampling with replacement works, I'm going to show you a demonstration of sampling with replacement using four tokens that are colored red, yellow, green, and blue. I actually have here with me four tokens of colors, red, yellow, green, and blue. I'm going to demonstrate what sampling with replacement using them look like. Here's a black velvet bag, empty. I'm going to take this example of four tokens, and drop them in. I'm going to sample four times with replacement out of this bag.

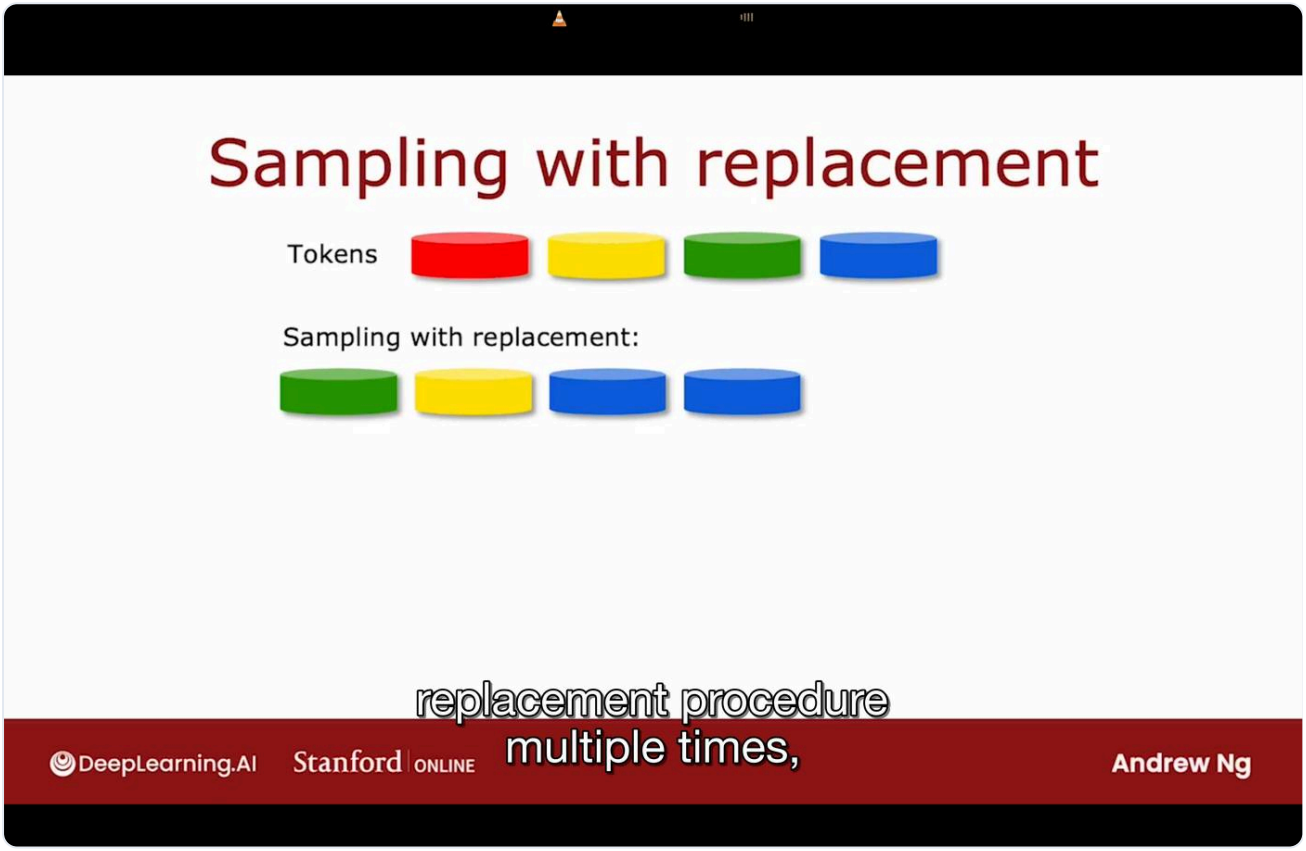


FIGURE 12.1 | Sampling with Replacement

What that means, I'm going to shake it up, and can't see when I'm picking, pick out one token, turns out to be green. The term with replacement means that if I take out the next token, I'm going to take this, and put it back in, and shake it up again, and then take on another one, yellow. Replace it. That's a little replacement part. Then go again, blue replace it again, and then pick on one more, which is blue again. That sequence of tokens I got was green, yellow, blue, blue. Notice that I got blue twice, and didn't get red even a single time.

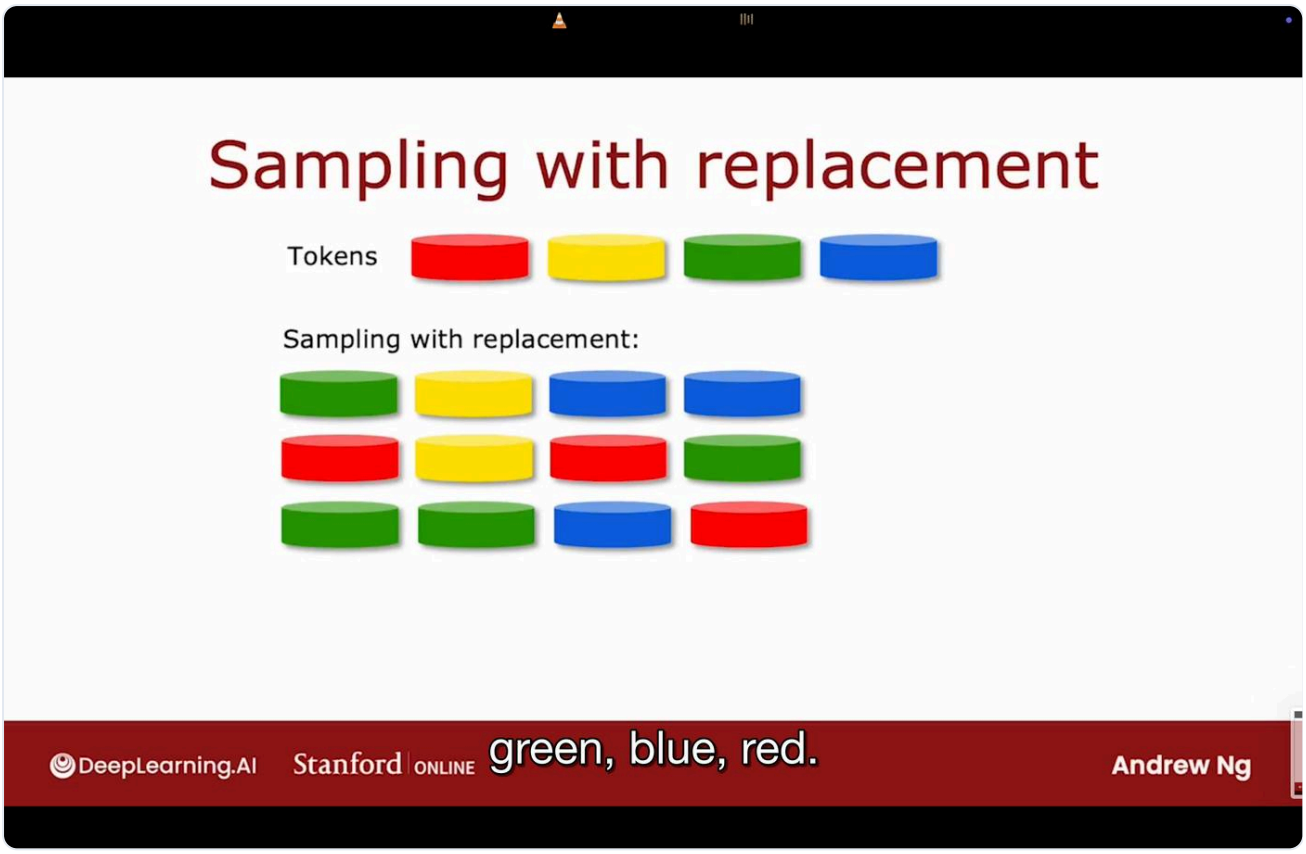


FIGURE 12.2 | Sampling with Replacement

If you were to repeat this sampling with replacement procedure multiple times, if you were to do it again, you might get red, yellow, red, green, or green, green, blue, red. Or you might also get red, blue, yellow, green.

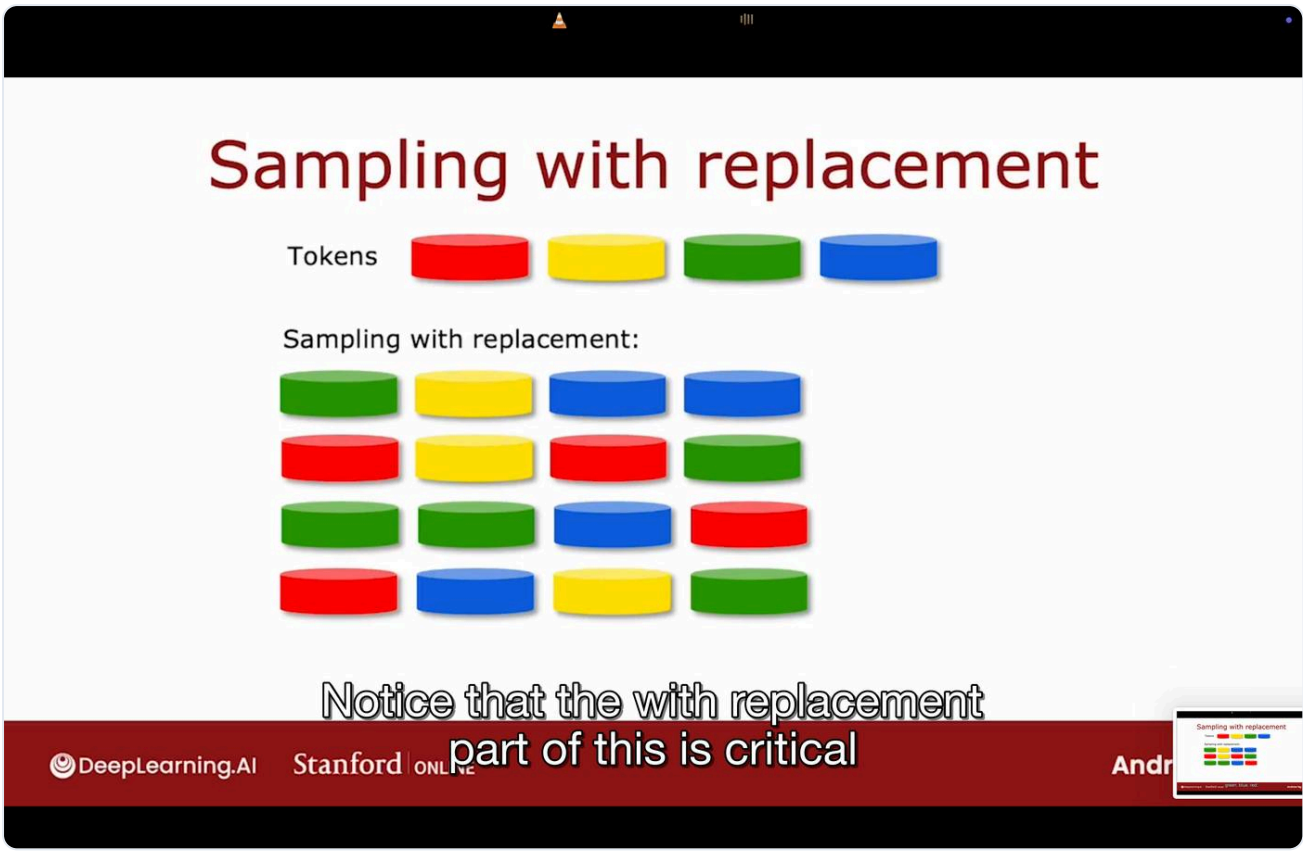


FIGURE 12.3 | Sampling with Replacement

Notice that the with replacement part of this is critical because if I were not replacing a token every time I sample, then if I were to pour four tokens from my bag of four, I will always just get the same four tokens. That's why replacing a token after I pull it out each time, is important to make sure I don't just get the same four tokens every single time. The way that sampling with replacement applies to building an ensemble of trees is as follows. We are going to construct multiple random training sets that are all slightly different from our original training set. In particular, we're going to take our 10 examples of cats and dogs. We're going to put the 10 training examples in a theoretical bag.

Please don't actually put a real cat or dog in a bag. That sounds inhumane, but you can take a training example and put it in a theoretical bag if you want. I'm using this theoretical bag, we're going to create a new random training set of 10 examples of the exact same size as the original data set. The way we'll do so is we're reaching and pick out one random training example. Let's say we get this training example. Then we put it back into the bag, and then again randomly pick out one training example and so you get that. You pick again and again and again. Notice now this fifth training example is identical to the second one that we had out there.

But that's fine. You keep going and keep going, and we get another repeats the example, and so on and so forth.

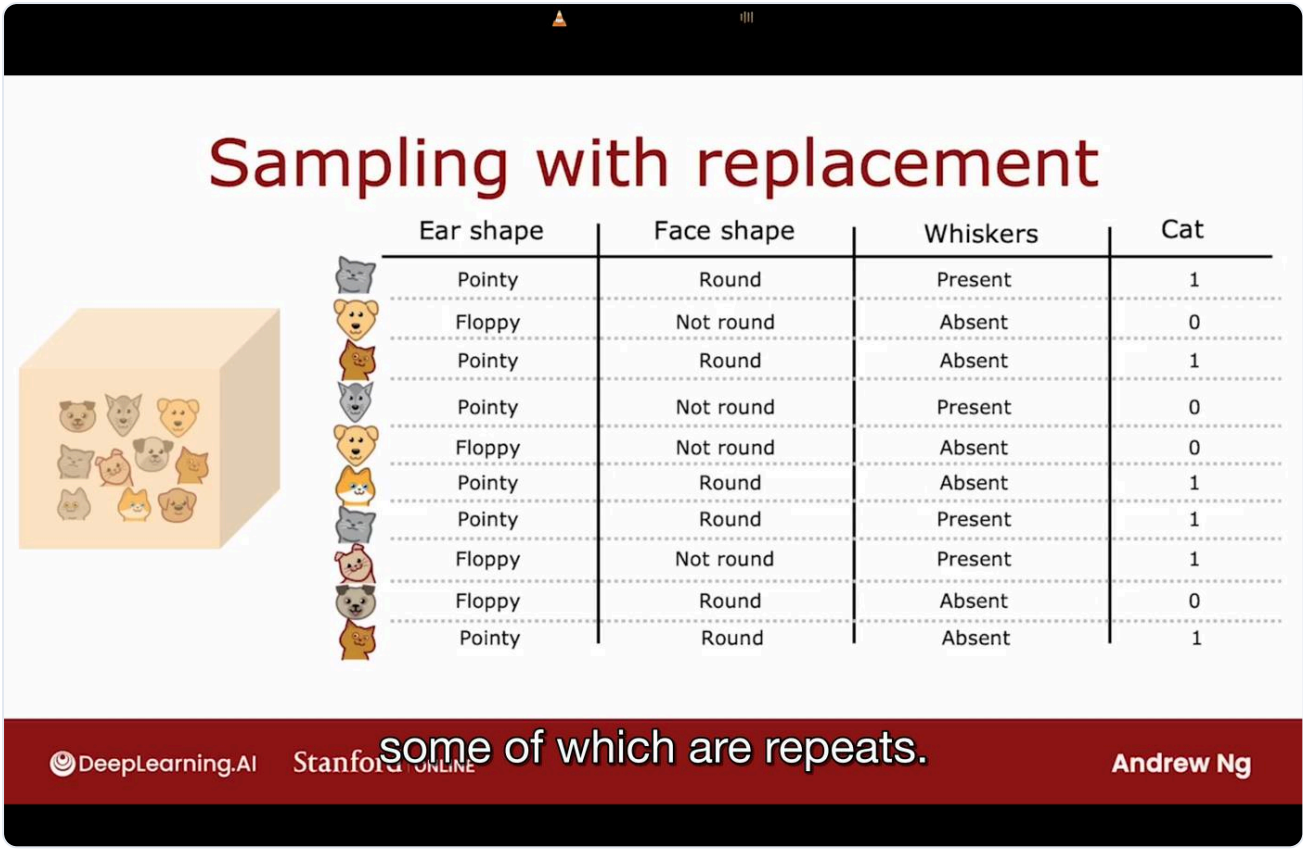


FIGURE 12.4 | Sampling with Replacement

Until eventually you end up with 10 training examples, some of which are repeats. You notice also that this training set does not contain all 10 of the original training examples, but that's okay. That is part of the sampling with replacement procedure. The process of sampling with replacement, lets you construct a new training set that's a little bit similar to, but also pretty different from your original training set. It turns out that this would be the key building block for building an ensemble of trees. Let's take a look in the next video and how you could do that.

LECTURE 60

Randomizing the Feature Choice

Now that we have a way to use sampling with replacement to create new training sets that are a bit similar to but also quite different from the original training set. We're ready to build our first tree ensemble algorithm. In particular in this video, we'll talk about the random forest algorithm which is one powerful tree ensemble algorithm that works much better than using a single decision tree. Here's how we can generate an ensemble of trees. If you are given a training set of size M, then for B equals 1 to capital b so we do this capital B times.

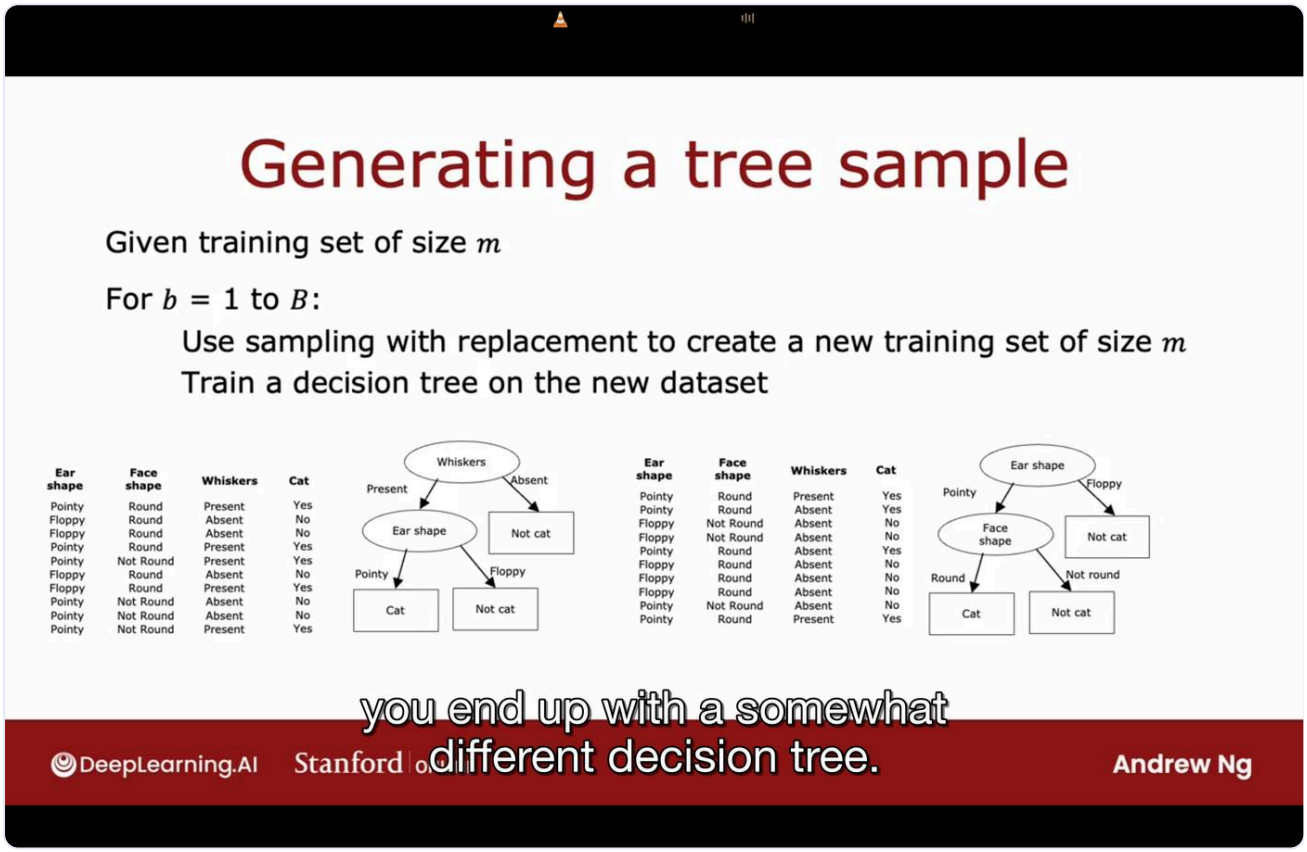


FIGURE 13.1 | Generating a Tree Sample

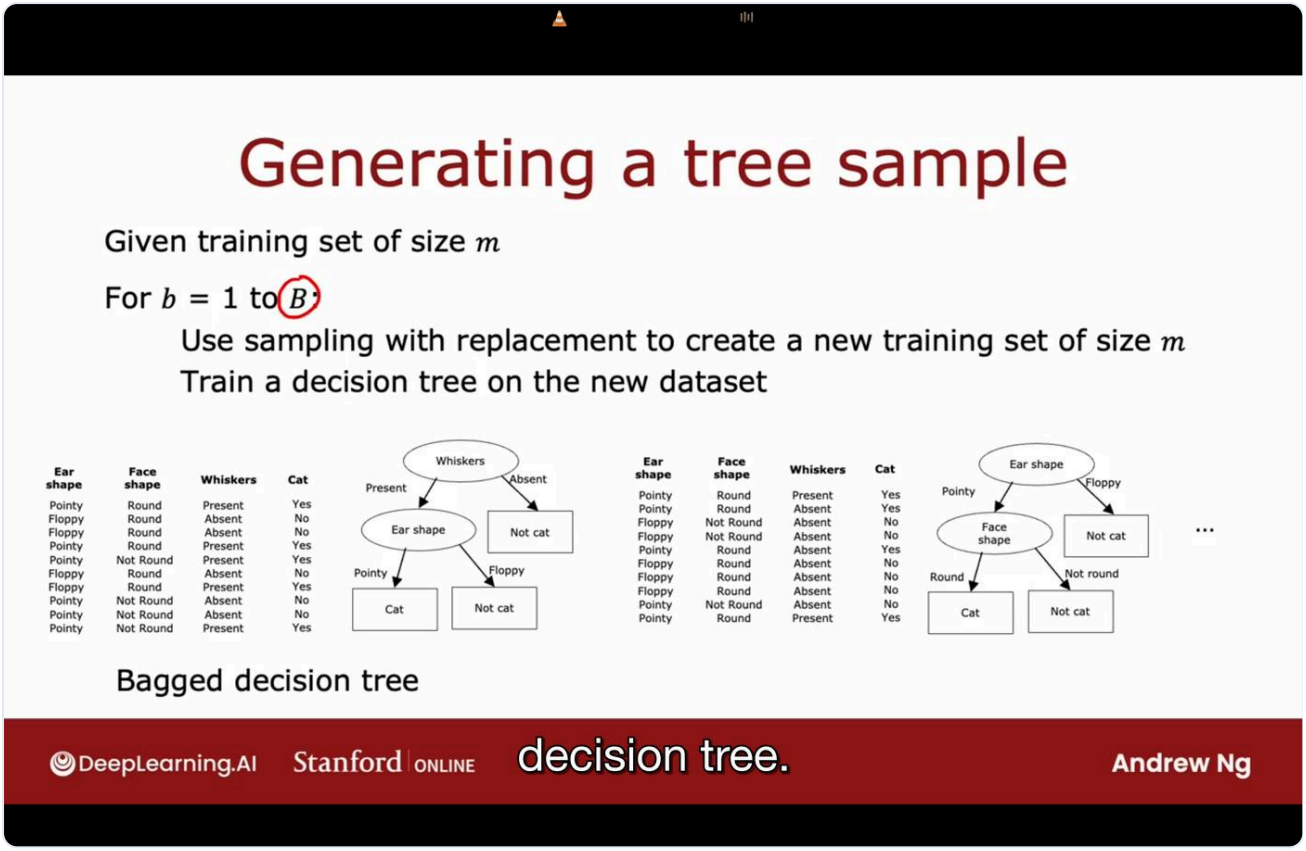


FIGURE 13.2 | Generating a Tree Sample

You can use something with replacement to create a new training set of size M. So if you have 10 training examples, you will put the 10 training examples in that virtual bag and sample of replacement 10 times to generate a new training set with also 10 examples, and then you would train a decision tree on this data set. So here's the data set I've generated using something with replacement. If you look carefully, you may notice that some of the training examples are repeated and that's okay. And if you train the decision on this data said you end up with this decision tree. And having done this once, we would then go and repeat this a second time. Use something with replacement to generate another training set of M or 10 training examples.

This again looks a bit like the original training set but it's also a little bit different. You then train the decision tree on this new data set and you end up with a somewhat different decision tree. And so on. And you may do this a total of capital B times. Typical choice of capital B the number of such trees you built might be around a 100 people recommend any value from Say 64, 128. And having built an ensemble of say 100 different trees, you would then when you're trying to make a prediction, get these trees all votes on the correct final prediction. It turns out that setting capital B to be larger, never hurts performance, but beyond a certain point, you end up with diminishing returns and it doesn't actually get that much better when B is much larger than say 100 or so.

And that's why I never use say 1000 trees that just slows down the computation significantly without meaningfully increasing the performance of the overall algorithm. Just to give this particular algorithm a name. This specific instance creation of tree ensemble is sometimes also called a bagged decision tree. And that refers to putting your training examples in that virtual

bag. And that's why also we use the let us lower case B an uppercase B here because that stands for bag. There's one modification to this album that will actually make it work even much better and that changes this algorithm the bagged decision tree into the random forest algorithm.

The key idea is that even with this sampling with replacement procedure sometimes you end up with always using the same split at the root node and very similar splits near the root note. That didn't happen in this particular example where a small change the trainings that resulted in a different split at the root note. But for other training sets it's not uncommon that for many or even all capital B training sets, you end up with the same choice of feature at the root node and at a few of the nodes near the root node.

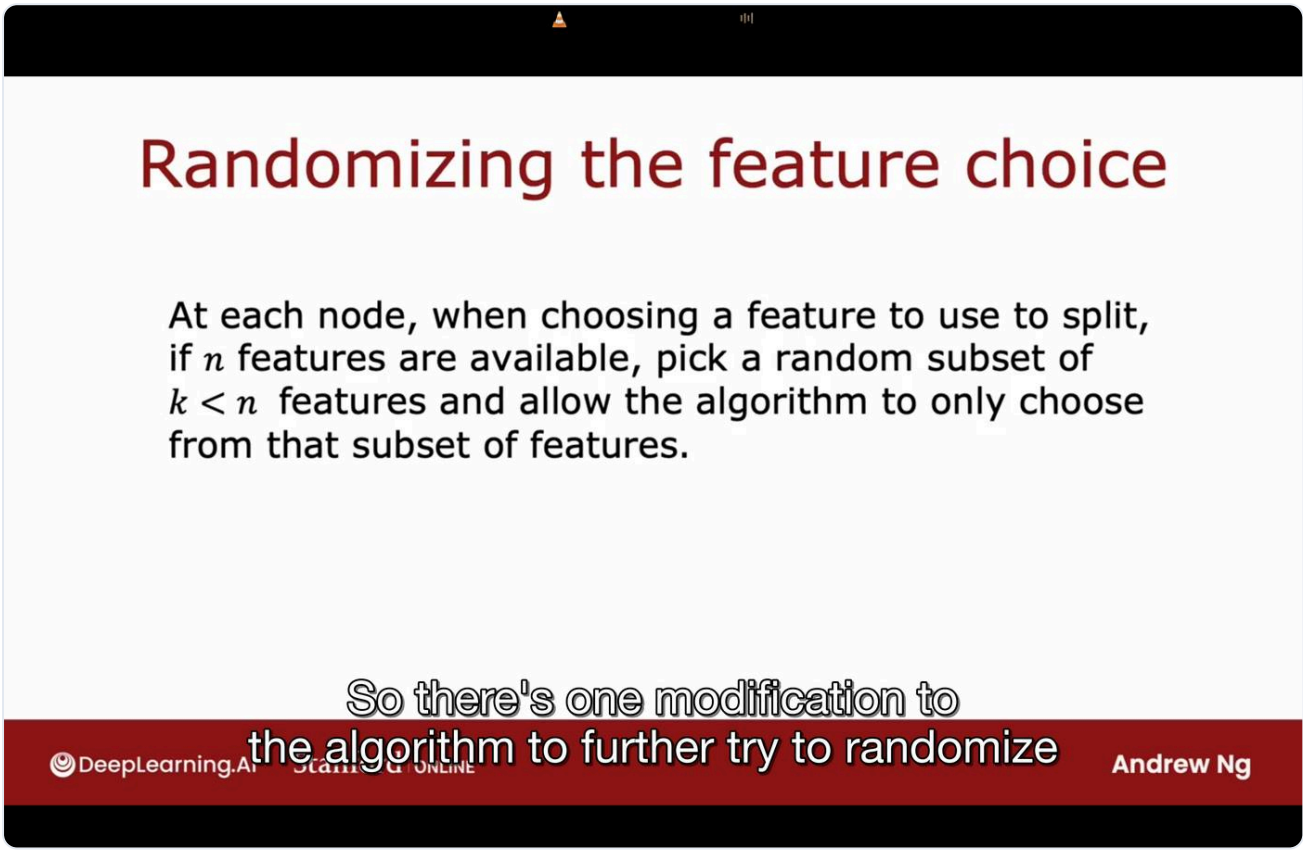


FIGURE 13.3 | Randomizing the Feature Choice

So there's one modification to the algorithm to further try to randomize the feature choice at each node that can cause the set of trees and you learn to become more different from each other. So when you vote them, you end up with an even more accurate prediction. The way this is typically done is at every note when choosing a feature to use to split if end features are available. So in our example we had three features available rather than picking from all end features, we will instead pick a random subset of K less than N features. And allow the algorithm to choose only from that subset of K features. So in other words, you would pick K features as the allowed features and then out of those K features choose the one with the highest information gain as the choice of feature to use the split.

When N is large, say n is Dozens or 10's or even hundreds.

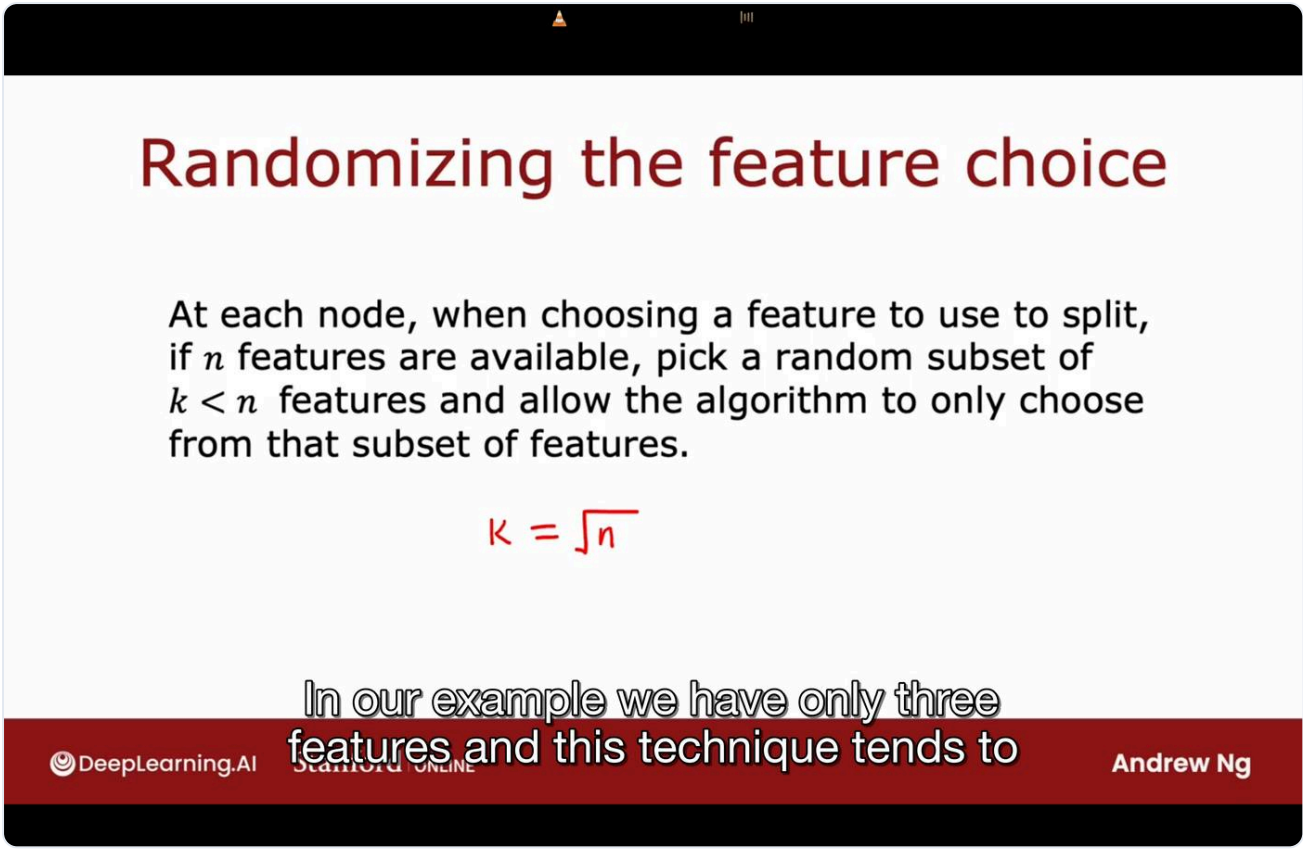


FIGURE 13.4 | Randomizing the Feature Choice

A typical choice for the value of K would be to choose it to be square root of N, In our example we have only three features and this technique tends to be used more for larger problems with a larger number of features.

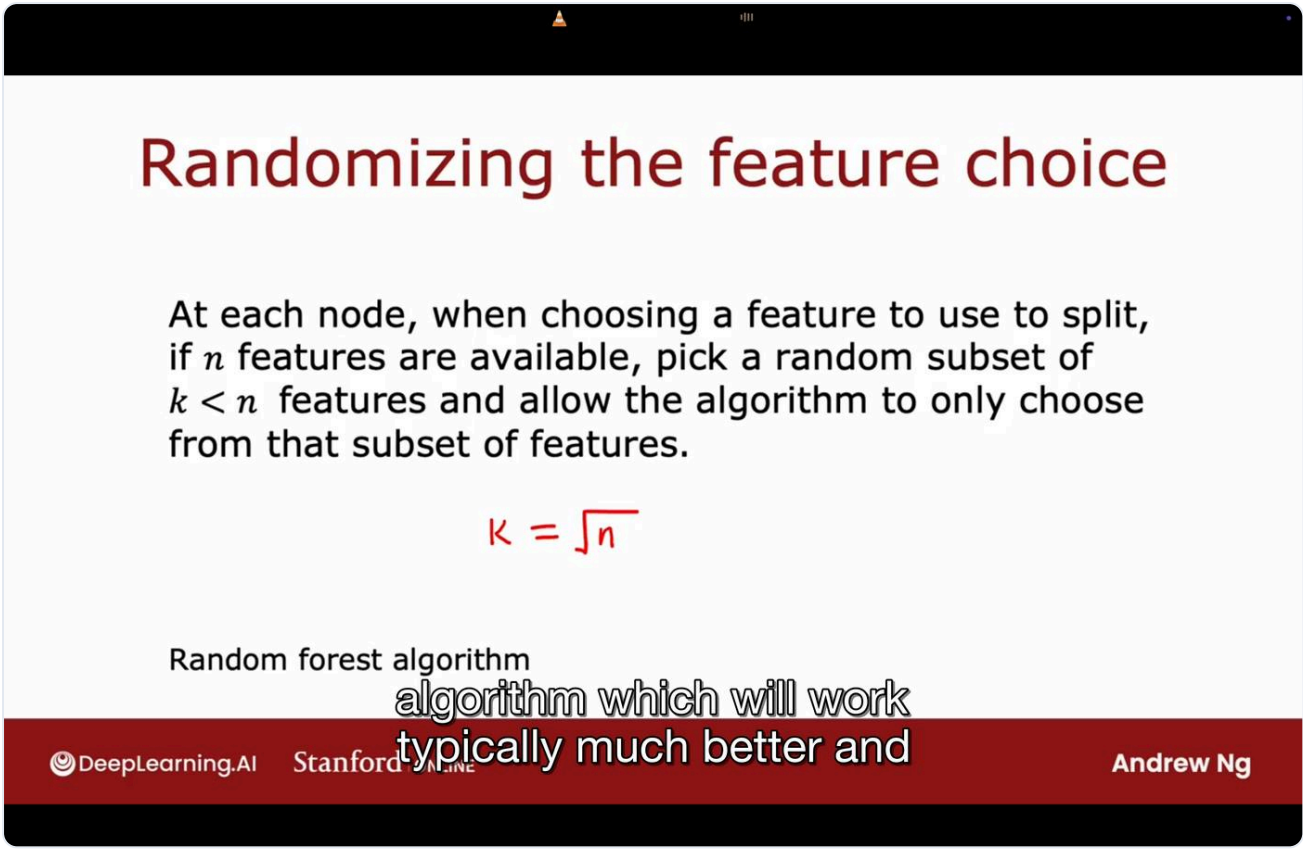


FIGURE 13.5 | Randomizing the Feature Choice

And will just further change the algorithm you end up with the random Forest algorithm which will work typically much better and becomes much more robust than just a single decision tree. One way to think about why this is more robust to than a single decision tree is the sampling with replacement procedure causes the algorithm to explore a lot of small changes to the data already and it's training different decision trees and is averaging over all of those changes to the data that the sampling with replacement procedure causes. And so this means that any little change further to the training set makes it less likely to have a huge impact on the overall output of the overall random forest algorithm.

Because it's already explored and it's averaging over a lot of small changes to the training set. Before wrapping up this video there's just one more thought I want to share with you Which is, where does a machine learning engineer go camping? In a random forest. All right. Go and tell that joke to your friends. I hope you enjoy it. The random forest is an effective algorithm and I hope you better use it in your work. Beyond the random forest It turns out there's one other algorithm that works even better. Which is a boosted decision tree. In the next video, let's talk about a boosted decision tree algorithm called X G boost.

LECTURE 61

Boosted Trees Intuition

Over the years, machine learning researchers have come up with a lot of different ways to build decision trees and decision tree ensembles. Today by far the most commonly used way or implementation of decision tree ensembles or decision trees there's an algorithm called XGBoost. It runs quickly, the open source implementations are easily used, has also been used very successfully to win many machine learning competitions as well as in many commercial applications. Let's take a look at how XGBoost works. There's a modification to the back decision tree algorithm that we saw in the last video that can make it work much better. Here again, is the algorithm that we had written down previously.

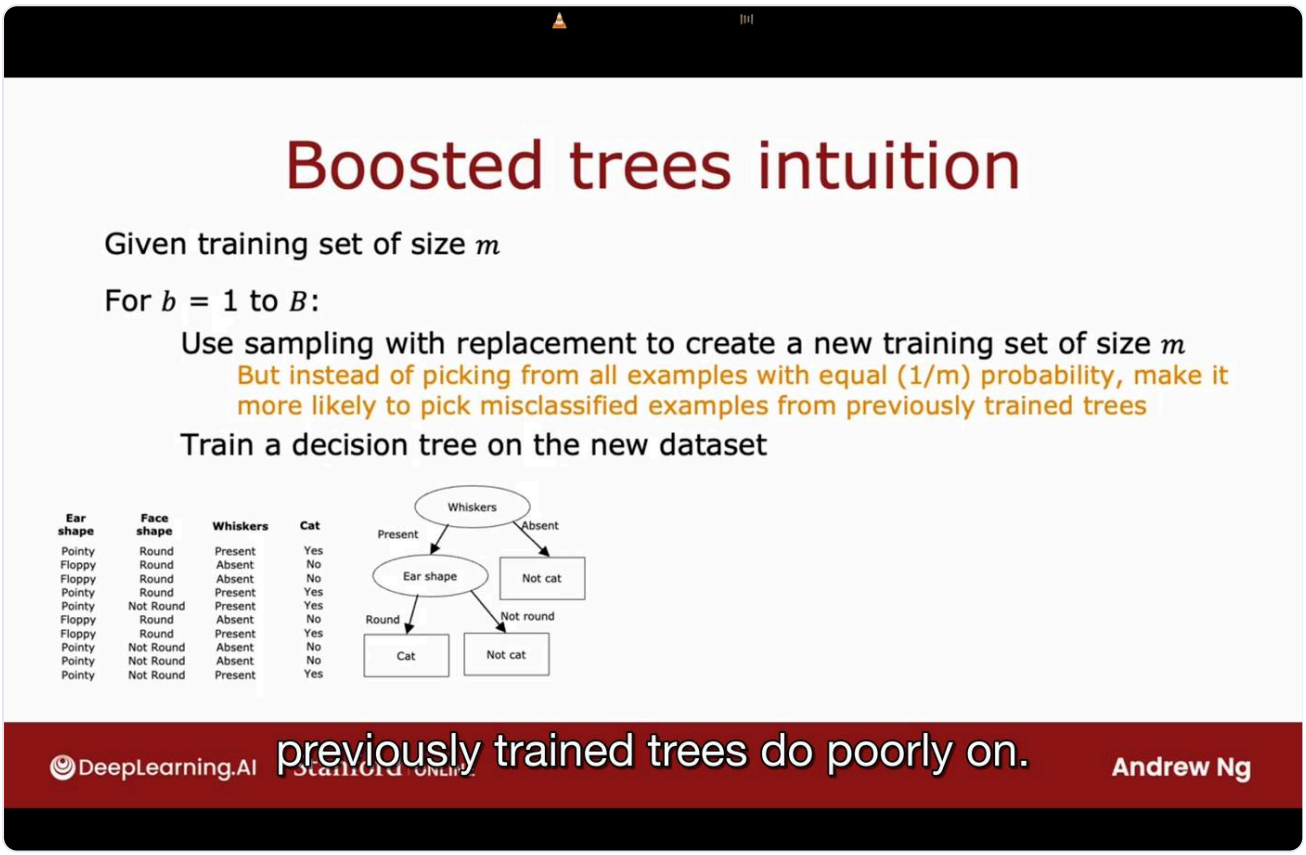


FIGURE 14.1 | Boosted Trees Intuition

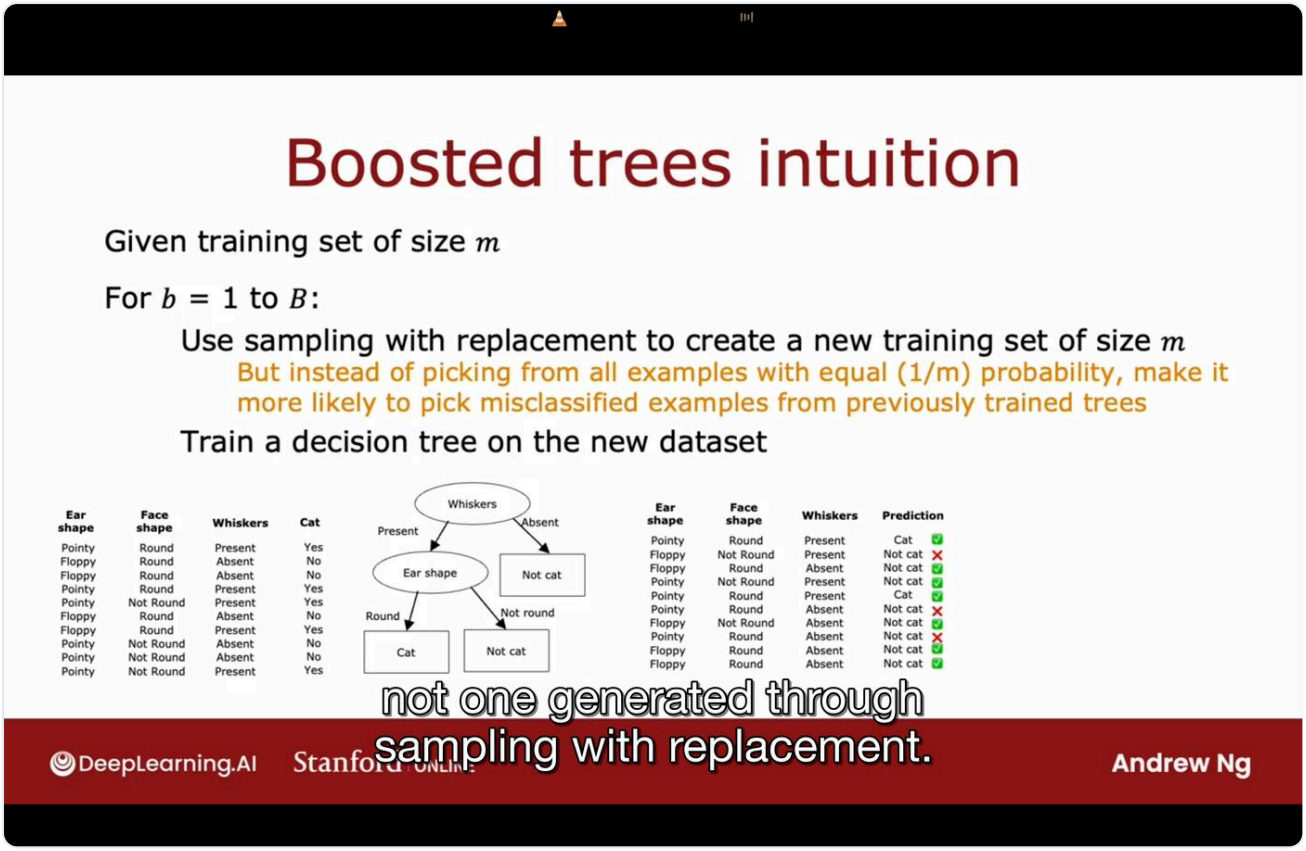


FIGURE 14.2 | Boosted Trees Intuition

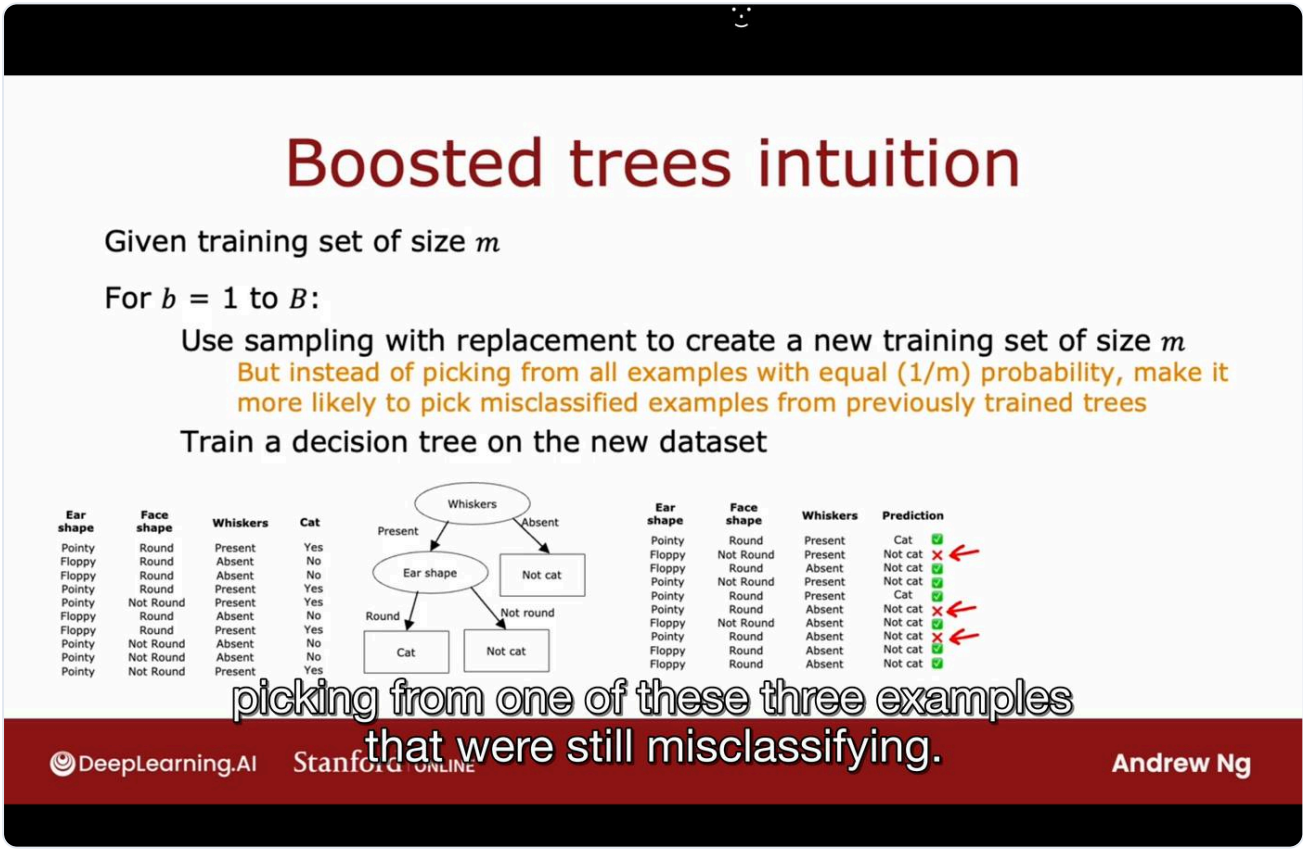


FIGURE 14.3 | Boosted Trees Intuition

Given the training set to size m, you repeat B times, use sampling with replacement to create a new training set of size M and then train the decision tree on the new data set. And so the first time through this loop, we may create a training set like that and train a decision tree like that. But here's where we're going to change the algorithm, which is every time through this loop, other than the first time, that is the second time, third time and so on. When sampling, instead of picking from all m examples of equal probability with one over m probability, let's make it more likely that we'll pick misclassified examples that the previously trained trees do poorly on.

In training and education, there's an idea called deliberate practice. For example, if you're learning to play the piano and you're trying to master a piece on the piano rather than practicing the entire say five minute piece over and over, which is quite time consuming. If you instead play the piece and then focus your attention on just the parts of the piece that you aren't yet playing that well in practice those smaller parts over and over. Then that turns out to be a more efficient way for you to learn to play the piano well. And so this idea of boosting is similar. We're going to look at the decision trees, we've trained so far and look at what we're still not yet doing well on.

And then when building the next decision tree, we're going to focus more attention on the examples that we're not yet doing well. So rather than looking at all the training examples, we focus more attention on the subset of examples is not yet doing well on and get the new decision tree, the next decision tree reporting ensemble to try to do well on them. And this is the idea behind boosting and it turns out to help the learning algorithm learn to do better more quickly. So in detail, we will look at this tree that we have just built and go back to the original training set. Notice that this is the original training set, not one generated through sampling with replacement.

And we'll go through all ten examples and look at what this learned decision tree predicts on all ten examples. So this fourth most column are their predictions and put a checkmark across next to each example, depending on whether the trees classification was correct or incorrect. So what we'll do in the second time through this loop is we will sort of use sampling with replacement to generate another training set of ten examples. But every time we pick an example from these ten will give a higher chance of picking from one of these three examples that were still misclassifying. And so this focuses the second decision trees attention via a process like deliberate practice on the examples that the album is still not yet doing that well.

And the boosting procedure will do this for a total of B times where on each iteration, you look at what the ensemble of trees for trees 1, 2 up through (b- 1), are not yet doing that well on. And when you're building tree number b, you will then have a higher probability of picking examples that the ensemble of the previously sample trees is still not yet doing well on. The mathematical details of exactly how much to increase the probability of picking this versus that example are quite complex, but you don't have to worry about them in order to use boosted tree implementations. And of different ways of implementing boosting the most widely used one today is XGBoost, which stands for extreme gradient boosting, which is an open source implementation of boosted trees that is very fast and efficient.

XGBoost also has a good choice of the default splitting criteria and criteria for when to stop splitting.

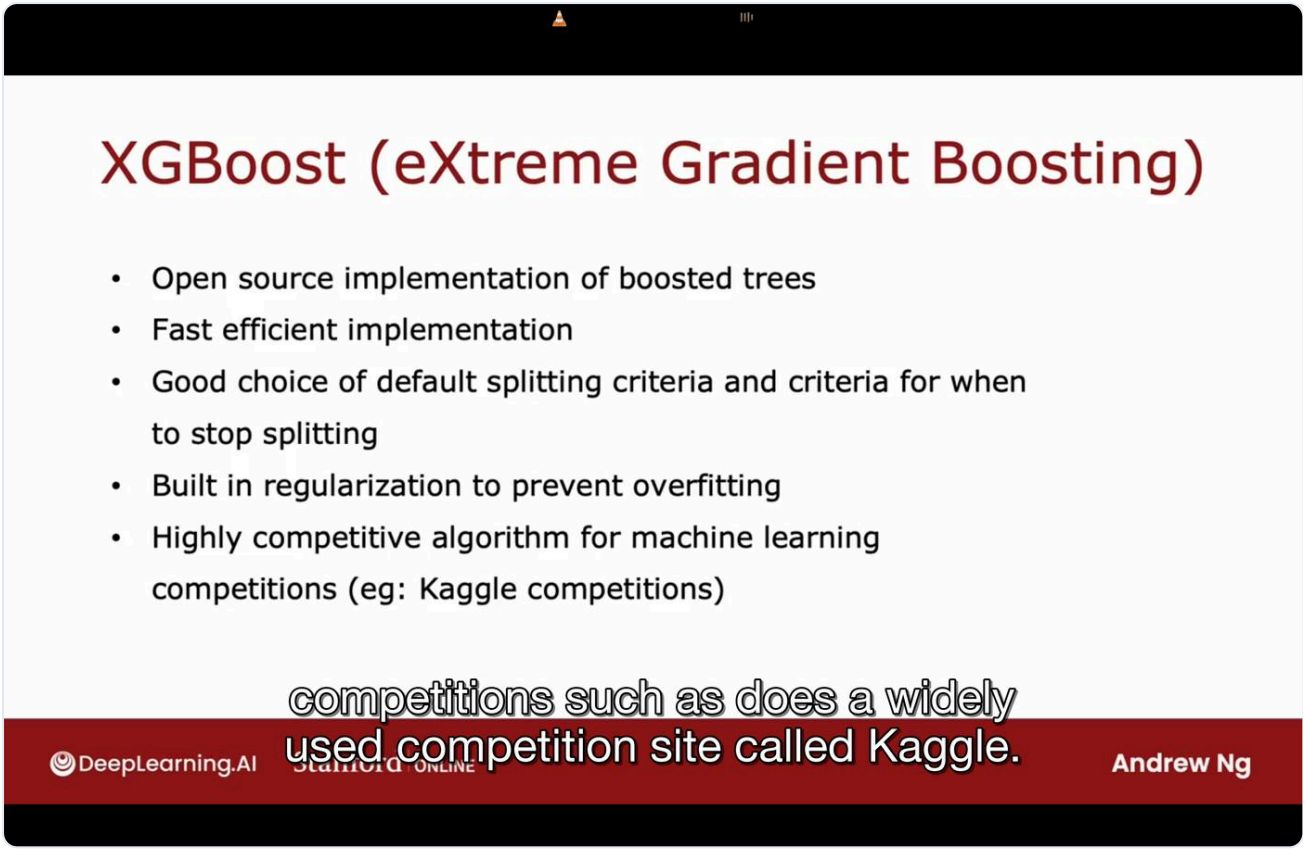


FIGURE 14.4 | XGBoost (eXtreme Gradient Boosting)

And one of the innovations in XGBoost is that it also has built in regularization to prevent overfitting, and in machine learning competitions such as does a widely used competition site called Kaggle. XGBoost is often a highly competitive algorithm. In fact, XGBoost and deep learning algorithms seem to be the two types of algorithms that win a lot of these competitions. And one technical note, rather than doing sampling with replacement XGBoost actually assigns different weights to different training examples. So it doesn't actually need to generate a lot of randomly chosen training sets and this makes it even a little bit more efficient than using a sampling with replacement procedure.

But the intuition that you saw on the previous slide is still correct in terms of how XGBoost is choosing examples to focus on. The details of XGBoost are quite complex to implement, which is why many practitioners will use the open source libraries that implement XGBoost.

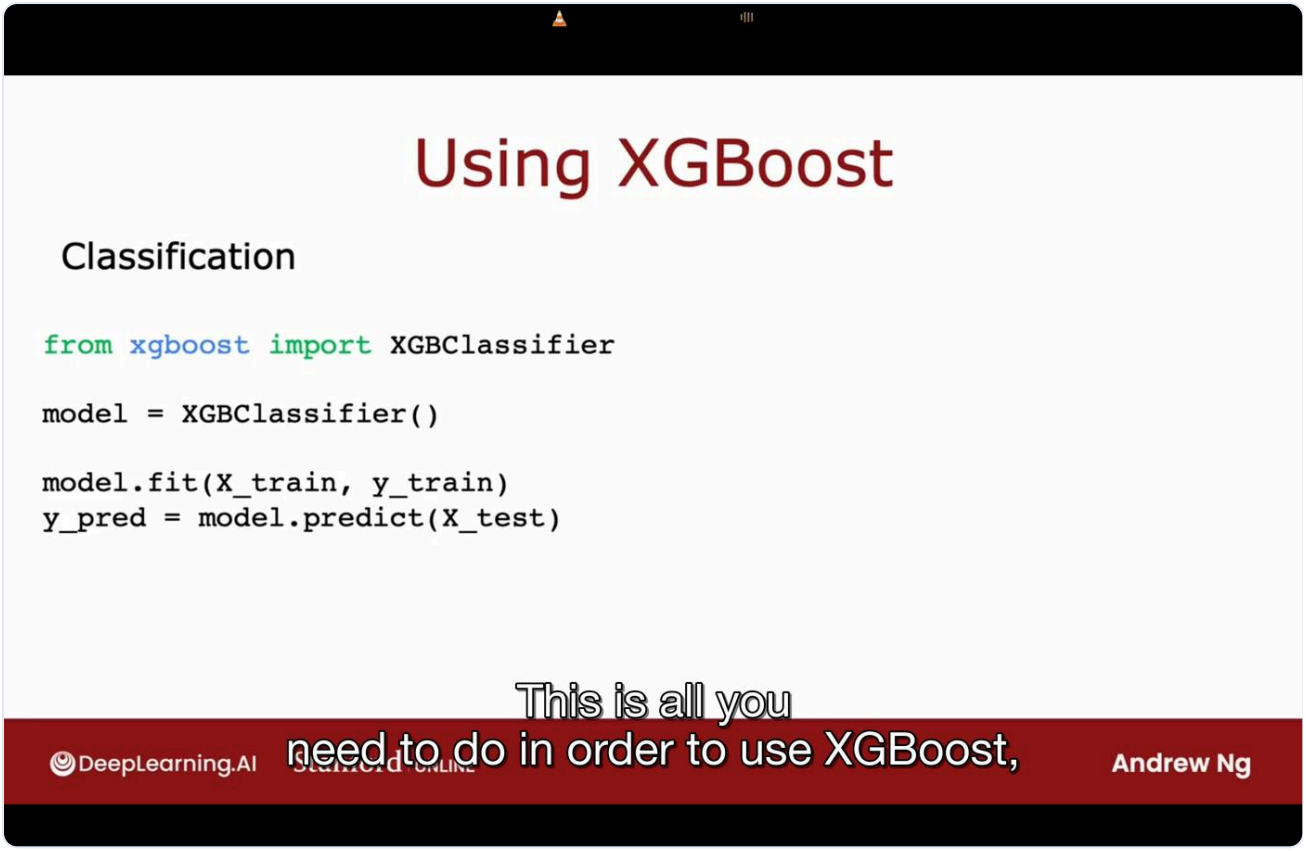


FIGURE 14.5 | Using XGBoost

This is all you need to do in order to use XGBoost, you will import the XGBoost library as follows and initialize a model as an XGBoost classifier. Further model and then finally, this allows you to make predictions using this boosted decision trees algorithm. I hope that you find this algorithm useful for many applications that you may build in the future.

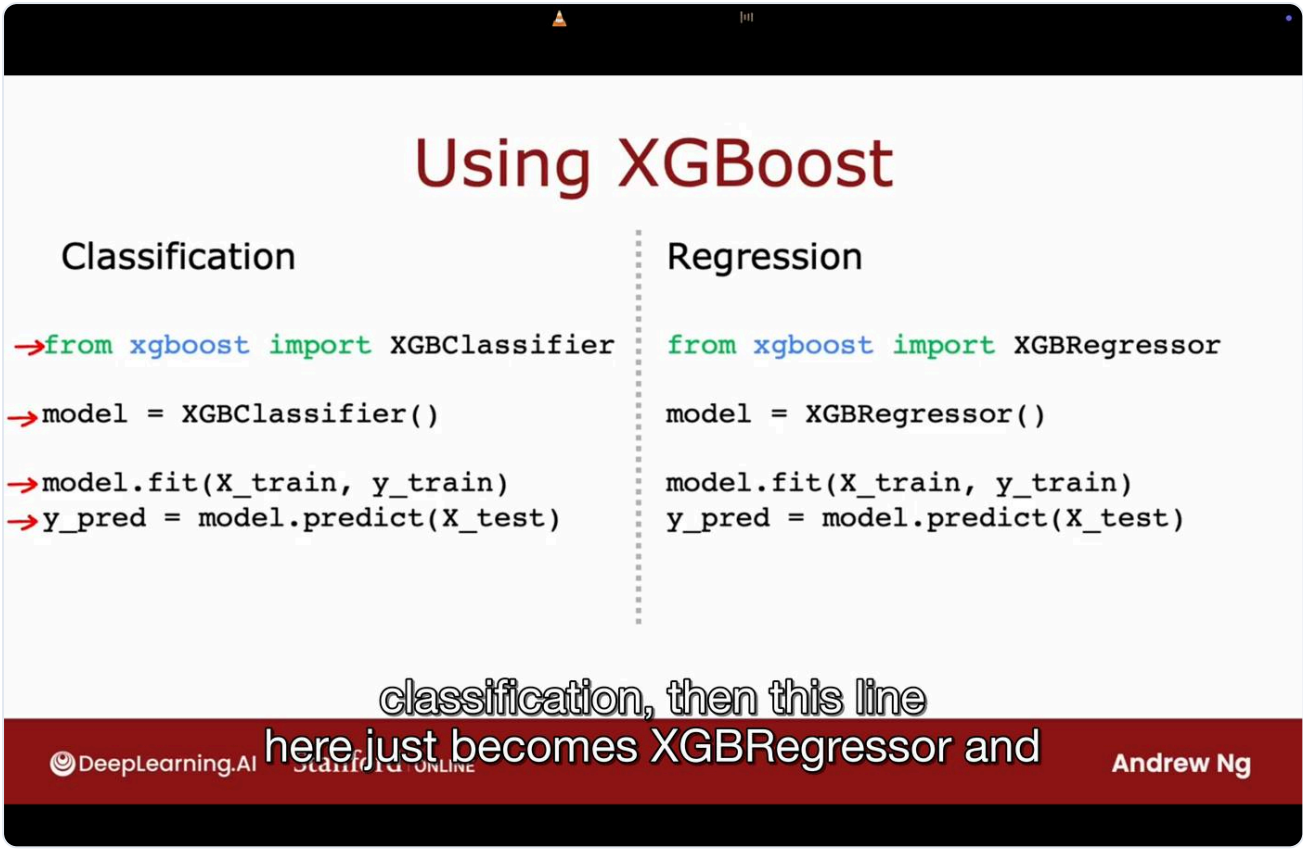


FIGURE 14.6 | Using XGBoost

Or alternatively, if you want to use XGBoost for regression rather than for classification, then this line here just becomes XGBRegressor and the rest of the code works similarly. So that's it for the XGBoost algorithm. We have just one last video for this week and for this course where we'll wrap up and also talk about when should you use a decision tree versus maybe use the neural network. Let's go on to the last and final video of this week.

LECTURE 62

Decision Trees vs Neural Networks

Both decision trees, including tree ensembles as well as neural networks are very powerful, very effective learning algorithms. When should you pick one or the other? Let's look at some of the pros and cons of each. Decision trees and tree ensembles will often work well on tabular data, also called structured data. What that means is if your dataset looks like a giant spreadsheet then decision trees would be worth considering. For example, in the housing price prediction application we had a dataset with features corresponding to the size of the house, the number of bedrooms, the number of floors, and the age at home. That type of data stored in a spreadsheet with either categorical or continuous valued features and both for classification or for regression task where you're trying to predict a discrete category or predict a number.

All of these problems are ones that decision trees can do well on. In contrast, I will not recommend using decision trees and tree ensembles on unstructured data. That's data such as images, video, audio, and texts that you're less likely to store in a spreadsheet format. Neural networks as we'll see in a second will tend to work better for unstructured data task. One huge advantage of decision trees and tree ensembles is that they can be very fast to train. You might remember this diagram from the previous week in which we talked about the iterative loop of machine learning development.

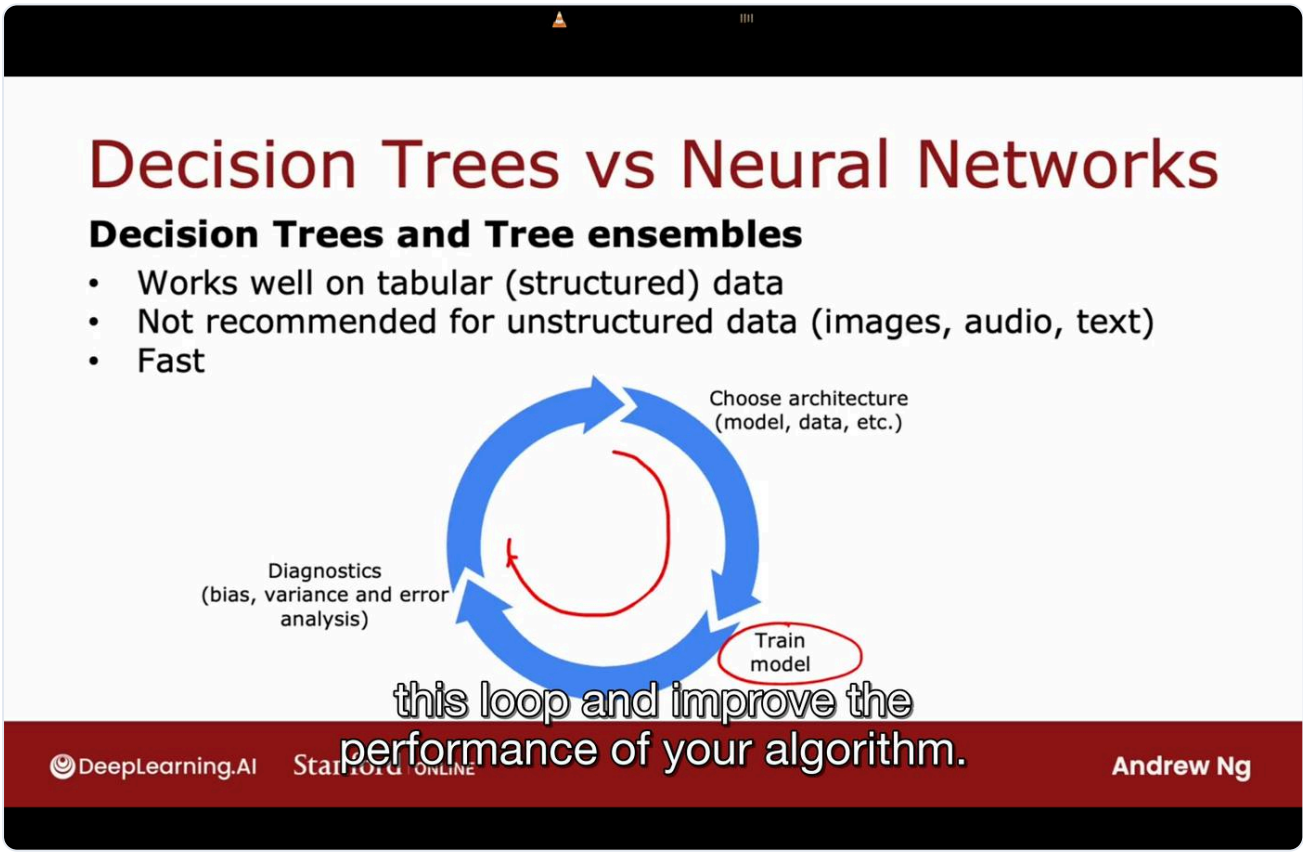


FIGURE 15.1 | Decision Trees vs Neural Networks

If your model takes many hours to train then that limits how quickly you can go through this loop and improve the performance of your algorithm. But because decision trees, including tree ensembles tend to be pretty fast to train, that allows you to go to this loop more quickly and maybe more efficiently improve the performance of your learning algorithm. Finally, small decision trees maybe human interpretable.

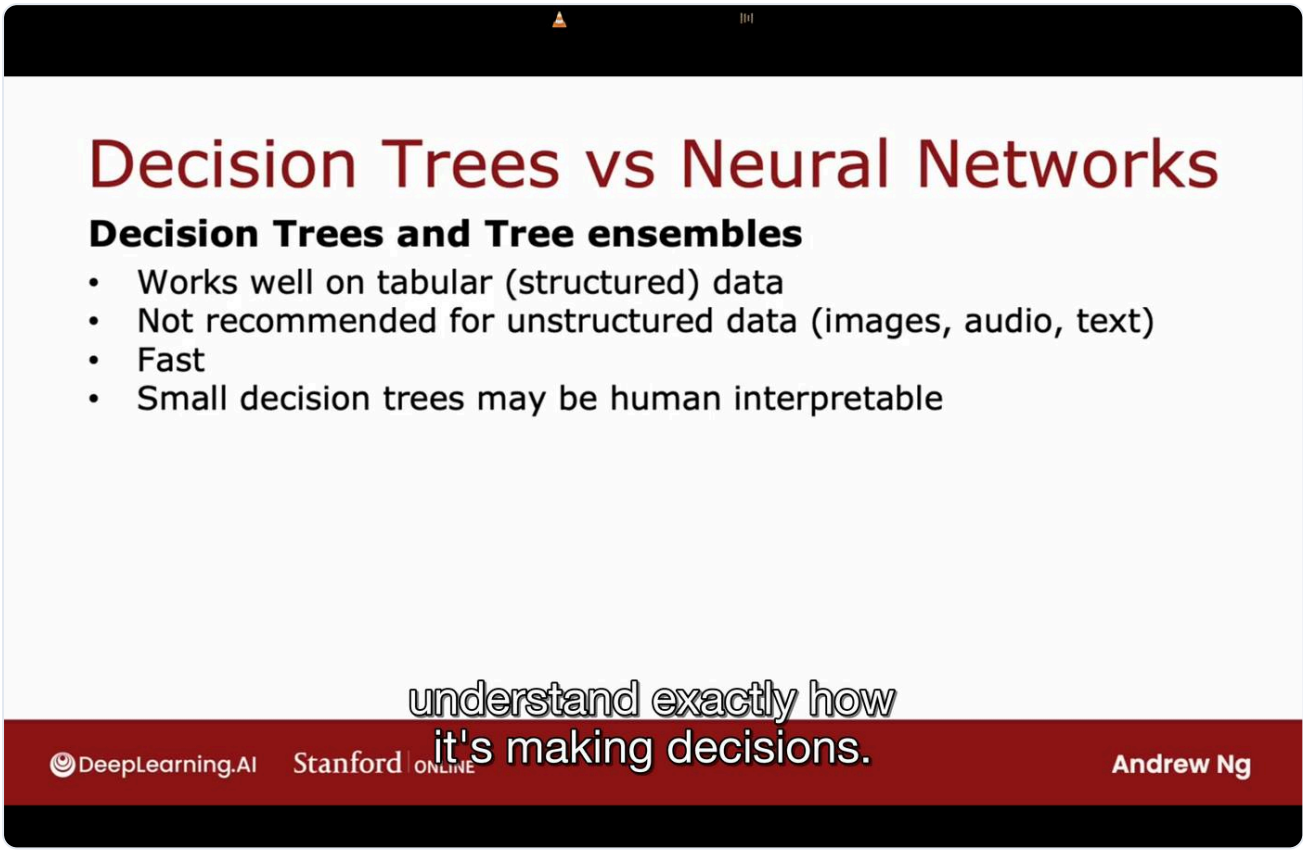


FIGURE 15.2 | Decision Trees vs Neural Networks

If you are training just a single decision tree and that decision tree has only say a few dozen nodes you may be able to print out a decision tree to understand exactly how it's making decisions. I think that the interpretability of decision trees is sometimes a bit overstated because when you build an ensemble of 100 trees and if each of those trees has hundreds of nodes, then looking at that ensemble to figure out what it's doing does become difficult and may need some separate visualization techniques. But if you have a small decision tree you can actually look at it and see, oh, it's classifying whether something is a cat by looking at certain features in certain ways.

If you've decided to use a decision tree or tree ensemble, I would probably use XGBoost for most of the applications I will work on. One slight downside of a tree ensemble is that it is a bit more expensive than a single decision tree. If you had a very constrained computational budget you might use a single decision tree but other than that setting I would almost always use a tree ensemble and use XGBoost in particular. How about neural networks? In contrast to decision trees and tree ensembles, it works well on all types of data, including tabular or structured data as well as unstructured data. As well as mixed data that includes both structured and unstructured components.

Whereas on tabular structured data, neural networks and decision trees are often both competitive on unstructured data, such as images, video, audio, and text, a neural network will really be the preferred algorithm and not the decision tree or a tree ensemble. On the downside though, neural networks may be slower than a decision tree. A large neural network can just take a long time to train. Other benefits of neural networks includes that it works with transfer learning and this is really important because for many applications we have only a small dataset being able to use transfer learning and carry out pre-training on a much larger dataset that is critical to getting competitive performance.

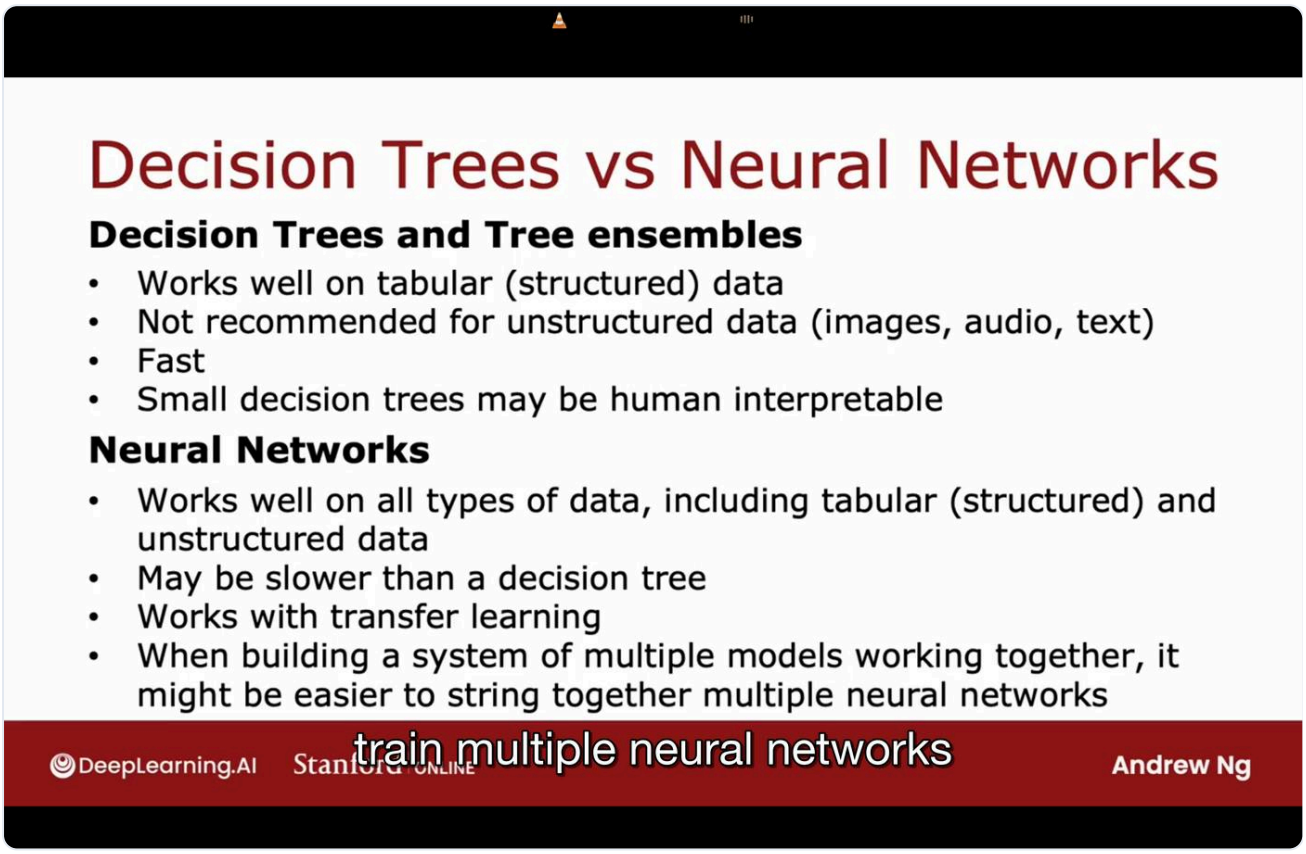


FIGURE 15.3 | Decision Trees vs Neural Networks

Finally, if you're building a system of multiple machine learning models working together, it might be easier to string together and train multiple neural networks than multiple decision trees. The reasons for this are quite technical and you don't need to worry about it for the purpose of this course. But it relates to that even when you string together multiple neural networks you can train them all together using gradient descent. Whereas for decision trees you can only train one decision tree at a time. That's it. You've reached the end of the videos for this course on Advanced Learning Algorithms. Thank you for sticking with me all this way and congratulations on getting to the end of the videos on advanced learning algorithms.

You've now learned how to build and use both neural networks and decision trees and also heard about a variety of tips, practical advice on how to get these algorithms to work well for you. But even if all that you've seen on supervised learning, that's just part of what learning algorithms can do. Supervised learnings need labeled datasets with the labels Y on your training set. There's another set of very powerful algorithms called unsupervised learning algorithms where you

don't even need labels Y for the algorithm to figure out very interesting patterns and to do things with the data that you have. I look forward to seeing you also in the third and final course of this specialization which should be on unsupervised learning.

Now, before you finish up this course I hope you also enjoy practicing the ideas of decision trees in their practice quizzes and in their practice labs. I'd like to wish you the best of luck in the practice labs or for those of you that may be Star Wars fans, let me say, may the forest be with you.

LECTURE 63

Language

[MUSIC] Hi, I'm delighted to be here with my old friend and collaborator, Professor Chris Manning. Chris has a very long and impressive bio but just briefly, he is Professor of Computer Science at Stanford University and also the director of the Stanford AI lab. And he also has the distinction of being the most highly cited researcher in NLP or natural language processing. So, really good to be here with you, Chris. >> Good to get a chance to chat Andrew. >> So we've known each other collaborated for many years and one interesting part of your background I always thought was that even though today you're distinguished researcher in machine learning in NLP, you actually started off in a very different area.

Your PhD if I remember correctly was in linguistics and you were studying the syntax of language. So how did you go from studying syntax to being an NLP researcher? >> So I can certainly tell you about that, but I should also point out that I'm still actually a professor of linguistics as well. I have a joint appointment at Stanford. And once in a blue moon not very often, I do actually still teach some real linguistics as well as computer involved natural language processing. So starting out, I was very interested in human languages and how they work, how people understand them, how they're inquired are acquired. So I had this sort of appeal, I saw this appeal in human languages.

But that equally led me to think about ideas that we now very much think about as machine learning or computational ideas. So two of the central ideas in human language, how do little children acquire human language? And for adults, well, we're just talking to each other now and we pretty much understand each other. And that's actually an amazing thing, how we manage to do that. So what kind of processing allows that? And so that early on got me interested in looking at machine learning. In fact, even before I'd made it to grad school, I started baby steps and learning machine learning coming off of those interests. >> That all human language has learned, we had learned at some point in the lives to speak English and we grown up in a different place, we would have learned a totally different language.

So is amazing to think how humans do that and now maybe machines learn language too. But so just tell us more about your journey. So you had a PhD in linguistics and then how did you? >> So there's some stuff before that as well. So I mean when I was an undergrad, well, officially I actually did three majors. This was in Australia, one in math, one in computer science and one in linguistics. Now people get a slightly exaggerated sense of what that means if you're in an American context because it'd be I think impossible to complete three majors, the undergrad at Stanford. But actually where I was as an undergrad doing, I did an arts degree so I could do whatever I wanted like linguistics, you had to complete two majors to complete the arts degree.

So it was sort of more like double majoring maybe in US terms. >> You probably don't know this about me but Mellon, I actually was a triple major, that was once in the statistics and economics. Okay, we both fellow triple majors. >> Yeah, so anyway, I did have background and interest in doing things with computer science. And so my interests were kind of mixed and I mean actually,

when I applied to grad schools, I mean one of the places I applied to was Carnegie Mellon because they were strong in computational linguistics. And if I'd gone there I would have been enrolled as a CS student but I ended up at as a linguistics student because at that time, there wasn't any natural language processing in the CS Department.

But I was still interested in pursuing ideas in natural language processing. But at that point in the early nineties, things were just starting to change. But the bulk of natural language processing was rule based logical declarative systems. But it was also in those years at the beginning of the nineties, when they first started to be lots of human language material, text and speech available digitally. So this was really actually just before the world wide web exploded. But they already started to be things like legal materials and newspaper articles and parliamentary hand SARS where you could last get your hands on millions of words of human language.

And it just seemed really clear that there had to be exciting things that you could do by working empirically from lots of human language. And that's what really sort of got me involved in a new kind of natural language processing that then led into my subsequent career. >> It sounds like your career was initially more linguistics and of the rise of data and machine learning and empirical methods it shifted to what NLP and machine learning and NLP. >> Yeah, I mean it absolutely certainly shifted and I've certainly sort of shifted much more to doing both natural language processing and machine learning models. But to some extent, the balance has varied.

But I've sort of been with that as a while, actually as an undergrad for my undergrad honors thesis, it was sort of learning the forms of words. So how you can which became a famous problem of sort of learning past tense of English verbs and the early connection literature. And I was trying to sort of learn paradigms of forms of verbs. And I was learning rules for the different forms using the C 4.5 decision tree learning algorithm. [LAUGH] If you remember that. >> Yeah right, good times. Yeah and it's surprisingly non intuitive, right? How going from present tense to past tense from I don't know to and all the other special cases can be. >> Yeah. >> Yeah, so we talked a bunch about NLP natural language processing.

So for some of the learners pick up machine learning for the first time, can you say what is NLP? >> Sure, absolutely, yeah. So NLP stands for natural language processing another word that's or term that's sometimes used for that is computational linguistics, it's the same thing. I mean natural language processing is actually a weird term, right? So it means that we're doing things with human languages. So you have to have the conception that you're enough of a computer scientist that when you say language, you think in your brain programming language. And therefore you need to say natural language to mean that you're talking about the languages that human beings use.

So overall natural language processing is doing anything intelligent with human languages. So in one sense that breaks down into understanding human languages, producing human languages, acquiring human languages, though people also often think about it in terms of different applications. And so then you might think about things like machine translation or doing question answering or generating advertising copy or summarization. There are so many different tasks that

people work on with particular goals in mind where you do things with human language. And there's a lot of natural language processing because so much of what the world works on our human world is dealt with and transmitted in terms of human language material.

So, because of all of these applications or even web search, right? Most of us use NLP. >> Yeah. >> Many, many times a day >> You're right, in some sense, the biggest application of natural language is web search, right? [LAUGH] And that's really the the big one, I mean, traditionally, it was a kind of a simple one, right? That in the good old days, it was, you know, there were various weighting factors and so on, but it was mainly sort of matching keywords, then your search terms and then some factors about the quality of the page. It didn't really feel like language understanding, but that's really been changing over the years. So these days, you'll often, if you ask a question to a search engine, it will give you an answer box where it has extracted a piece of text and puts what it thinks is the answer in bold or color or something like that.

Which is then this task of question answering and then it's really a natural language understanding task. >> Yeah, yeah, and I feel like in addition to web searches, maybe the big one, even when we're going to a online shopping website or a movie website and typing in what we want and doing a website search on a much smaller website than the big search engines. That also increasingly uses sophisticated NLP algorithms, and it is also creating quite a lot of value. Maybe to you is not the real NLP, but it still seems very valuable. >> I agree, it's very valuable. And there are lots of interesting problems in any e-commerce website with search very difficult problems actually when people describe the kind of goods they want.

And you need to be trying to match it to products that are available, that isn't an easy problem at all, it turns out. >> Yeah, that's true, yeah. So over the last, I don't know, couple of decades, NLP has gone through a major shift from more of the rule based techniques that you alluded to just now to using really machine learning much more pervasively. And so you were one of the people that leading parts of that charge and seeing every step of the way, you're creating some of the stems as it happened, can you say a bit about that process and what you saw? >> Sure, absolutely. Yeah, so when I started off as an undergrad in grad student, really most of natural language processing was done by hand built systems which variously used rules and inference procedures to sort of try and build up a path and an understanding of a piece of text. >> What's an example of a rule or inference system?

So, a rule could be part of the structure of human language. Like English sentence normally consists of a subject noun phrase, followed by a verb and an object noun phrase. And that gives you some idea as to how to understand the meaning of the sentence. But it might also be saying something about how to interpret a word so that a lot of words in English are very ambiguous. But if you have something like the word star and it's in the context of a movie, then it's probably referring to a human being not an astronomical object. And in those days people tried to deal with things like that using rules of that sort. That doesn't seem very likely to work to us these days, but, once upon a time that was pretty standard.

And so it was only when lots of digital text and speech started to become available that it really seemed like there was this different way that instead we could start calculating statistics over human language, material and building machine learning models. And so that was the first thing

that I got into, in the sort of mid to late 1990s. And so the first area where I started doing lots of research and publishing papers and getting well known is building what in the early days, we often called statistical natural language processing. But it later merged into in general prob approaches to artificial intelligence and machine learning. And that sort of took us through to approximately 2010, let's say.

And that's roughly when the new interest in deep learning using large artificial neural networks started to take off. For my interest in that I really have you to thank Andrew because at this stage, Andrew is still full time at Stanford and he was in the office next door to me and he was really excited about the new things that were happening in the area of deep learning, I guess. Anyone who walked into his office, he'd tell them, it's sort of exciting for what's happening now and I'm on neural network, you have to start looking at that. And so that was really the impetus that got me pretty early on involved in looking at things in neural networks.

I had actually seen a bit of it before, so while I was a grad student here, actually, Dave Rumelhart was at Stanford in Psych and I'd taken his neural networks class. And so, I'd seen some of that, but it hadn't actually really been what I'd gotten into for my own research. So- >> I didn't know that, thank you, yeah. >> Yeah. >> And then we wound up supervising some students together. >> Yeah, absolutely. >> I'd love to hear the rise of also deep learning and NLP, what did you see since you were in the field? >> Yeah, so starting about 2010, yeah, me, students started to do the first papers and deep learning aimed at NLP conferences. It's always hard when you're trying to do something new.

We had exactly the same experiences that people 15 or so years earlier had had when they start trying to do statistical NLP of when there's an established way of doing things, it's really hard to push out new ideas. So really some of our first papers were rejected from conferences and instead appeared at machine learning conferences or deep learning workshops, but very quickly that started to change and people got super interested in neural network ideas. But I sort of feel like the neural network period which sort of started effectively about 2010, it's self divides in two because for the first period, let's say, basically say it's till 2018. We showed a lot of success at building neural networks for all sorts of tasks.

We built them for syntactic parsing and sentiment analysis. And what else dude? >> Question answering. But it was sort of like we were doing the same thing that we used to do with other kinds of machine learning models, except we now had a better machine learning model. And we were sort of instead of training up a logistic regression or a support vector machine, we were still doing the same kind of sentiment analysis task, but now we are doing it with a neural network. So I think in looking back now in some sense, the bigger change came around 2018. Because that was when the idea of, well, we could just start with a large amount of human language material and build large what are self supervised models.

So that was models then and like BERT and GPTs and successor models to that. And they could just sort of acquire from word prediction over a huge amount of text this amazing knowledge of human languages. And I think really probably that's going to be viewed in retrospect as the bigger kind of cut point where the way Things were done really changed. >> Yeah, I think there is that trend for the large language models learning from massive amount of data. I think even in

the lead up to that, there was one of your research papers that really slightly blew my mind, which is a glove paper. So because with word embeddings where you learn the vector numbers to represent a word using a neural network.

That was quite mind blowing for me. And then the glove work that you did really cleaned up the math, made it so much simpler. And then I remember I said, that's all there is to do. And then you can learn these really surprisingly detailed representations of the computer learns nuances of what words mean. >> Absolutely. Yeah, so I should give a little bit of credit to others. Other people also worked on some similar ideas including and Ja Weston and colleagues at Google. But the glove word vectors is one of the very prominent systems of word vectors. So these word vectors already did. Yeah, you're right, illustrate this idea of using self supervised learning that we just took massive amounts of text.

And then we could build these models that knew an enormous amount about the meaning of words. It's still something I sort of show people every year in the first lecture of my NLP class. Because it's something simple but it actually just works so surprisingly well. You can do this sort of simple modeling of trying to predict a word given the words in the context and simply by sort of running the math of learning to do those predictions. Well, you learn all these things about word meaning and you can do these really nice patterns of similar word meaning or analogies of something pencil is to drawing as paintbrush is to and it'll say painting, right?

That it's sort of already showing just a lot of successful learning. So that was the precursor to what then got developed to the next stage with things like BERT and GPT where it wasn't just meanings of individual words. But meanings of whole pieces of text and context. >> Yeah, so I found it amazing that you can take a small neural network or some model and then give it lots of English sentences or some other language and hide the word. Ask it to predict what is the word that I just hid and that allows it to learn these analogies. And these very deep, what you think are really deep things behind the meaning of the word. And then 2018, maybe this other infection point what happened after that? >> Yeah.

So, in 2018, that was the point in which well, sort of really two things happened. One thing is that people, or really in 2017 had developed this new architecture. Which was much more scalable onto modern parallel GPUs. And so that was the transformer architecture. The second part of it though was maybe people rediscover of it because I was using the same trick as the glove model that if you have the task of just predicting a word given a context. Either a context on both sides of it or the preceding words that that just turns out to be an amazing learning task. And that surprises a lot of people. And a lot of the time you see discussions where people say disparaging things of this is nothing interesting is happening.

And all it's doing is statistics to predict which word is most likely to come after the preceding words. And I think the really interesting thing is that that's true, but it's not true. Because yes what the task is is you're predicting the next word given preceding words. But the really interesting thing is if you want to do that task really as well as possible. Then it actually helps to understand the whole of the rest of the sentence and know who's doing what to who and what's in the

sentence. But more than that, it also helps to understand the world because if your text is going something along the lines of the currency used in Fiji is that. Well, you need to have some world knowledge to know what the right answer to that is.

And so good models at doing this, learn both to follow the structure of sentences and their meaning and to know facts about the world also so that they can predict. And therefore this turns into what's sometimes referred to as an AI complete task, right? That you really need. There's nothing that can't actually be useful in answering this what word comes next sense, right? You can be in the World Cup semifinals the teams and you need to know something about soccer [LAUGH] to be giving the right answer. >> I complete this funny concept, right? Is this idea that you can solve this one problem, you can solve everything in AI or kind of make an analogy to NP complete problems from the theory of computing.

What do you think? Do you think predicting the next word is AI complete, I have very mixed feelings about that myself. I can say, I don't think it's true. I'm curious what you think. I think it's not quite true because I think there are other kind of things that human beings manage to work out. There are human beings that have clever insights in mathematics or there are human beings who are looking at something that's a much more. Three dimensional real world puzzle of sort of figuring out how to do something mechanical or something like that. And that's just not a language problem. But on the other hand, I think language gets closer to universality than some people think as well because we live in this 3D world.

And operate in it with our bodies and our feelings and other creatures and artifacts around it. And you could think, well, not much of that is in language at all. But actually just about all of this stuff we think about, we talk about, we write about it in language. We can describe the positions of things relative to each other in language. So a surprising amount of the other parts of the world are seen in reflection in language. And therefore you're learning about all of them too. When you learn about language use. >> You learn about one aspect of a lot of things even if things how do you ride a bicycle can't really. >> You don't really learn how to ride a bicycle, [LAUGH] but you learn some aspects of what it involves that you need to balance.

And you have to have your feet on the pedals and push them and all of that kind of things. Yeah. >> And so with this trend in NLP, the large language models has been very exciting for the last several years. What are your thoughts on where all this will go? >> Well yeah, so it's just been amazingly. Successful and exciting, right? That so we haven't really explained all the details, right? So there's the first stage of learning these large language models where the task is just to predict the next word. And you do that billions of times over a very large piece of text. And behold, you get this large neural network, which is just a really useful artifact for all sorts of natural language processing tasks.

But then you still actually have to do something with it if you want to do a particular task, whether that's question answering or summarization or detecting toxic content in social media or something like that. And at that point, there's a choice of things that you could do with it. The traditional answer was then you had a particular task, let's say it's detecting toxic comments in social media. And you'd take some supervised data for that and then you'd fine tune the language

model to answer that classification task. But you were enormously helped by having this base of this large self supervised model because it meant that the model had enormous knowledge of language and it could generalize very quickly.

So unlike the sort of the standard old days of, of supervised learning where it was kind of, well, if you give me 10,000 labeled example examples, I might be able to produce a halfway decent model for you. But if you give me 50,000 labeled examples, it will be a lot better. It's sort of turned it into this world of. Well, if you give me 100 labeled examples and I fine tuning a large language model, I'll be able to do great better than I would have been able to do with the 50,000 examples in the old world. Some of the more recent exciting works now, even going beyond that, it's now, well, maybe you don't actually have to fine tune the model at all.

So people have done a lot of work using methods, sometimes referred to as prompting or instruction where you can simply in natural language, perhaps with examples, perhaps with explicit instructions, just tell the model what you want it to do and it does it which even as someone who's been working in natural language processing for 30 years. I mean, it actually just blows my mind how well this works it, I guess I wasn't a decade ago thinking that in now we'd be able to just tell the model, I want you to summarize this piece of text here and it, they will then summarize it. I think that is incredible. Yeah, so we are in this very exciting time where a lot of new natural language capabilities are unfolding.

I think there's just no doubt at all for the next couple of years, the future of that is extremely bright as people work out different things and different ways to do things and people start to apply in different application areas. The kind of capabilities that have been unlocked with recent technological developments. There's always a question in technology as to sort of whether the curve keeps on heading steeply upwards or whether there's then some new things, we have to discover how to do it. >> It's been going up for quite a while. So hopefully extrapolation is always dangerous. But, but we'll see, I I'm just curious, you know, you, you mentioned writing prompts to the NLP system, the large language model, what you want and it seems to magically do it.

I'm curious, do you think prompt engineering is the path of the future where actually when I write these prompts, I sometimes find it works miraculously and sometimes it's frustrating the process of rewording my instructions to tweak the wording to get it just right to generate the result I want. So, do you think prompt engineering is the way of the future or do you think is a intermediate hack until someone invents a better way to control these, control the outputs of these systems. >> I think it's both, I think will be the way of the future but I also think at the moment people are doing, yeah, a lot of hacking around and rewording to try and get things to work better and with any luck with a few more years of development that will start to go away. I mean, one way of to think about the difference is sort of in comparison to the kind of voice assistance or virtual assistance that are available on phones speaker devices like Amazon, Alexa these days, right?

I mean, I think all of us have had the experience that present those devices aren't always great, but if you know the right way to word things, it'll do something. But if you use the wrong wording, it won't and the difference with human beings is by and large, you don't have to think about that. You can say what you want and it doesn't matter what words you choose, they'll the other

human being assuming that someone who knows the same language, et cetera will understand you and do what you want. And I think and would hope that we'll start to see the same kind of progression with these models that at the moment fiddling around with the particular wording, you use can make a very big difference to how well it works.

But hopefully in a few years time, that just won't be true, you'll be able to use different wordings and it'll still work. But the basic idea that we're moving into this age where actually human language will be able to be used as an instruction language to tell your computer what to do. So, instead of having to use menus and radio buttons and things like that or writing Python code, instead of either of those things that you'll be able to say what you want in the computer will do it. I think that age is opening up in front of us that will continue to build and that will be hugely transformative. >> Feels like come a long ways but only a much more to come and much more to go. >> Yeah, absolutely. >> In the development of NLP technology, the only thing I want to ask you and I suspect you and I may have different perspectives on this.

But in the last couple of decades, the trend has been to rely less on rule based engineering and more on machine learning on data. Sometimes lots of data look into the future. Where do you think that mix of hand coded constraints or other constraints, explicit constraints versus let's get a neural network and throw lots of data at it. Where do you think that balance will fall? >> I think that there's no doubt that using learning from data is the way forward and what we're going to continue to do. But I think there's still a space for models that have more structure, more inductive bias that have some kind of basis of exploiting the nature of language.

So in recent years, the model that's been enormously successful is the transformer neural network and the transformer neural networks, essentially this huge association machine. So it'll just suck associations from anywhere. >> And look at two words and figure out which words to which other word. >> Yes. So you use everything to predict anything and do it over and over again and you'll get anything you want. And you know, that's been incredibly, incredibly successful, but it's been incredibly successful in the domain where you have humongous amounts of data. Right, so that these transformer models for these large language models are now being trained on tens of billions of words of text.

When I started off in statistical natural language processing. And some of the traditional linguists used to complain about the fact that I was collecting statistics from 30 million words of Newswire. And building a predictive model and thought that was just not what linguistics was about. I felt I had a perfectly good answer, which is that a human kid as they're learning language. They're exposed to actually, well, more than 30 million words of data. But that kind of amount of data, so the kind of amount of some data we were using were perfectly reasonable amounts of data to be using. To be not exactly trying to model human language acquisition. But to be thinking about how we can learn about language for lots of data.

But these modern transformers are now using already at least two orders of magnitude, more data. And most most people think the way to get things to the next level is to use more still and make it three orders of magnitude. And in one sense that scaling up strategy has been hugely effective. So, I don't blame anybody for saying, let's make another order of magnitude bigger and see what amazing things we can do. But it also shows that human learning is just way, way better

in being able to extract a lot more information out of a quite limited amount of data. And at that point, you can have various hypotheses. But I think it's reasonable to assume that human learning is somewhat structured towards the structure of the world.

And things it sees in the world and that allows it to learn more quickly from less data. >> Right, I'm with you on that. I think better learning algorithms, our current machine learning, our rooms are much less efficient or makes much less efficient use of data. And so there's way more data than any child. And then I think whether the improved learning algorithms will be from linguistic rules or whether it will just be engineers. Engineering much more efficient versions of the transformer or whatever comes after it. That I think will be. >> That will be traditional. I don't think it'll be by people explicitly putting traditional linguistic rules into the system.

I don't think that's the way forward. On the other hand I think what we're starting to see is models like these transformer models are actually discovering the structure of language themselves, right? So the broad effects of human language that English has the subject before the verb and the object afterwards. Whereas in Japanese that the verbs at the end of the sentence and the subject and object are normally in that order before it. But could be in the other order actually transformer models are learning these facts. You can interrogate them and see that even though they were never explicitly told about subjects and objects that they know these notions.

So I think they're discovering a lot else as well about language use and context and the meanings and senses of words and what is and isn't, unpleasant language. But part of what they're learning is the same kind of structure that linguists have laid out as the sort of structure of different human languages. >> So it's as if over many decades, linguists discovered certain things. And by training on billions of words, transformers are discovering the same things that linguists discovered in humans. That's cool. So all this is really exciting progress in NLP, driven by machine learning and by other things. To someone entering the field, entering machine learning or AI or NLP.

There's just a lot going on. What advice would you have for someone wanting to break into machine learning? >> Yeah. Well it's a great time to break in. I think there's just no doubt at all that we're still in the early stages of seeing the impact of this new approach where effectively software computer science is being reinvented on the basis of much more use of machine learning. And the various other things that come up away from that. And then more generally across industries, there are just lots of opportunities for more automation. Making more use of interpretation of human language material for me or in other areas like vision and robotics, the same kind of things.

So, lots of possibilities. So at that point, there's lots to do obviously in that you want to get some kind of good foundation, right. So knowing some of the core technical methods of machine learning, understanding ideas of how to build models from data. Look at losses, do training diagnose errors, all of these core things. That's definitely useful for natural language processing in particular, some of those skills are completely relevant. But then there are particular kind of models that are commonly used, including the the transformer that we've talked about a lot today. You

definitely should know about transformers and indeed they're increasingly being used in every other part of machine learning as well for vision bioinformatics, even robotics is now using transformers.

But beyond that, I think it's also useful to learn something about human language and the nature of the problems that involves. Because even though people aren't directly going to be encoding rules of human language into their computing system. A sensitivity to sort of what kind of things happen in language and what to look out for and what you might want to model that's still a useful skill to have. >> And then in terms of learning the foundations, learning about these concepts. You had entered AI from a linguistic background and we now see people from all walks of life wanting to start doing work in AI. What are your thoughts on the preparation one should have or and your thoughts on how to start from something other than computer science or AI.

So there are lots of places you can come from and vector across and different ways. And we're seeing tons of people doing that that they are people who started off in different areas, whether it was chemistry, physics or even much further in field. And people have history, whatever have started to look at machine learning. I think there are sort of two levels of answer there. One level of answer is one of the amazing transformations is that there's now these very good software packages for doing things with neural network models. This software is really easy to use. You don't actually need to understand a lot of highly technical stuff. You've got need to have some kind of high level conception about what is the idea of machine learning.

And how do I go about training a model and what should I look at and the numbers that are being printed out to see if it's working, right. But you don't actually have to have a higher degree to be able to build these models. I mean, and indeed what we're seeing is, lots of high school students are getting into doing this because it's actually something that if you have some basic computer skills and a bit of programming, you can pick up and do. It's just way more accessible than lots of stuff that proceeded at whether in AI outside of AI and other areas, like operating systems or security. But if you want to get to a deeper level than that and actually want to understand more of what's going on.

I think you can't really get there if you don't have a certain mathematics foundation, like at the end of the day that DeepLearning is based on calculus and you need to be optimising functions. And if you sort of don't have any background in that, I think that sort of ends up as a war at some point. So >> The machines learning in data science. It does come in handy for some of the work we >> Yeah. So I think at some level, if you're at the major in history or non mathematical parts of psychology, I actually have a good friend who, yeah, he learned calculus in grad school because he was a psychologist and he'd never done it before. And decided that he wanted to start learning about these new kinds of models and decided it wasn't too late to be able to go and take a calc course.

And so he did, right. So you do need to know some of that stuff, but for lots of people, if they've seen some of that before, even if you're kind of rusty. I think you can kind of, get back in the zone and it doesn't really matter that you haven't done AI as an undergrad or machine learning and things like that, that you can really start to learn how to build these models and do things and really, that's my own story, right? That despite the fact that they let me sit in the school of engineer-

ing at Stanford these days, my background isn't as an engineer. My PhD is in linguistics, that I've sort of largely vectored across from having some knowledge of mathematics and linguistics and knowing some programming into sort of getting much more into building AI models. >> I about something.

Do you think the improved libraries and abstractions that are now available like coding frameworks, like TensorFlow or PyTorch? Do you think that reduces the need to understand calculus? Because boy it's been a while since I had to actually take a derivative in order to even implement or create a new neural network architecture because of automatic differentiation >> Yeah, I mean, absolutely. I mean, so in the early days, when we were doing things sort of 2010 to 2015, right? For every model we built, we were working out the derivatives by hand and then writing some code and whatever it was. Sometimes it was Python but sometimes it might have been Java or C [LAUGH] to calculate these derivatives and checking that we got them right and so on, where these days you actually don't need to know any of that to build DeepLearning models.

I mean, this is actually something I think about, been thinking about even with respect to my own natural language processing with DeepLearning class that I teach. At the beginning, we do still go through doing, matrix calculus and making sure people know about Jacobian and things like that so that they understand what's being done in back propagation, DeepLearning. But there's sort of this thing in which that means that we just give them hell for two weeks. It's sort of like boot camp or something to make them suffer. And then we say, but you do the rest of the class with pytorch and they sort of never have to know any of that again, right? There's always a question of how deep you want to go on technical foundations you, right.

You can keep on going, right? Like does a computer scientist in the 2020 need to understand, electronics and transistors or what happens in your CPU. Well, it's complicated, I mean, in various ways, it is helpful to know some of that stuff. I mean, I know Andrew, you were one of the pioneers in getting machine learning on to GPU and, well, that sort of means you had to have some sense that there's this new hardware out there. And it has some attributes of parallelism that means there's likely to be able to do something exciting. So it is useful to have some broader knowledge and understanding and sometimes something breaks and if you have some deeper knowledge, you can understand why it broke.

But there's another sense in which most people have to take some things on trust and you can do most of what you want to do in neural network modeling these days without knowing calculus at all. >> Yeah, I think that's a great point. I feel like sometimes the reliability of the abstraction determines how often you need to go in to fix something that's broken. So I'm, actually my understanding of quantum physics is very weak. I barely understand it. So you could argue, I don't understand how computers work because transistors are built in quantum physics. But fortunately, if something went wrong with the transistor, I've never had to go hard to fix it, [CROSSTALK] I think.

And so I think or another example, the sort function, the library to sort things, and sometimes they actually don't work, right, swap in the memory or whatever. And that's when, if you really understand how the sort function works, you can go in and fix it. But then sometimes if we have abstractions, libraries, APIs are reliable enough, then that it's nice to those abstractions then di-

minishes the need to understand some of the things that happen on me. So it's an exciting world. Feels like we have giants building on the shoulders of giants and all of these things are becoming more complex and more exciting every month really. >> Yeah, absolutely. >> So thanks Chris, that was really interesting and inspiring and I hope that to everyone watching this hearing Chris own journey to become a computer scientist.

And to become a leading, maybe the leading NLP computer scientist as well as all of this exciting work happening in NLP right now. I hope that inspires you too to jump into the steam and take a go at it. There's just a lot more work to be done collectively by our community that still, so I think the more of us are working on this, the better off the world will be. So thanks a lot, Chris. It was really great having you here. >> Thanks a lot, Andrew. It's been fun chatting. [MUSIC]